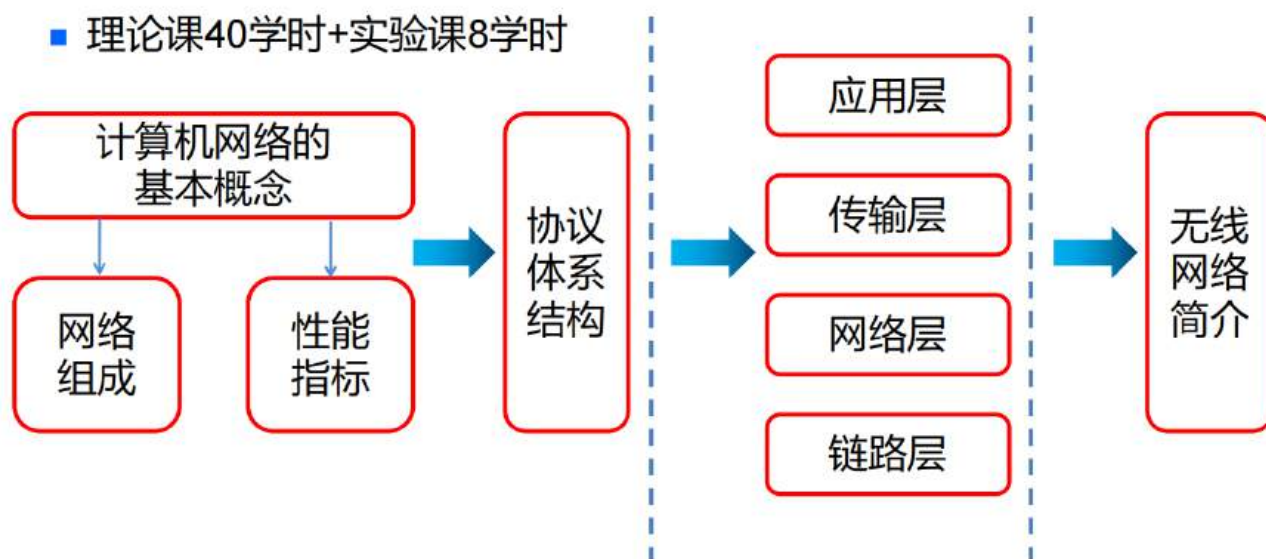


知识结构

3、知识结构

软件学院·计算机网络

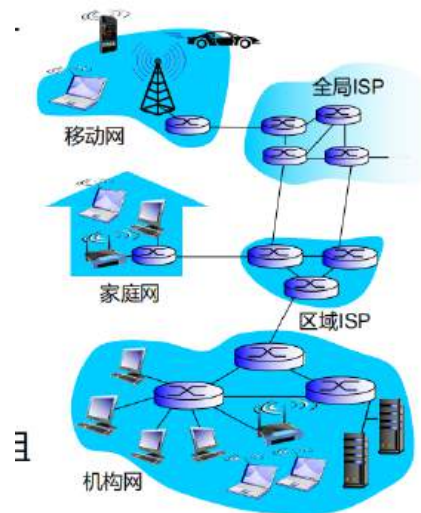
- 理论课40学时+实验课8学时



第一章 计算机网络概述

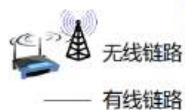


组成元素



□ 海量互联的计算设备

- 主机=端系统
- 运行网络应用程序



□ 通信链路

- 光纤, 同轴电缆, 无线电, 卫星
- 传输速率: 带宽



□ 分组交换: 转发数据分组

- 路由器和交换机

□ 互联网: “万网之网”

- 大量互联的ISP

□ 协议控制消息的发送与接收

- 例如, HTTP、TCP、IP、Skype、802.11等

□ 互联网标准

- IETF: 互联网工程任务工作组
- RFC: 请求评论

- ISP: 联网服务提供商(电信等)
是由交换机和链路组成的网络结构
- 所有的终端都被称为主机(端系统)
- 通信链路分为有线和无线, 其作用相同
- 协议通过从上到下穿过协议栈进行控制消息的发送与接收

当你发送一条消息时，数据会从上到下穿过协议栈。每一层协议都会给原始数据加上一个“信封”（首部/Header）。

- **应用层（定义内容）**：确定消息格式（例如：这是个 HTTP 请求还是个文件）。
- **传输层（加端口号）**：比如 **TCP 协议**，它会给数据打上序号。如果数据太长，它会把数据切成小块。它还负责“流量控制”，确保不会发得太快把对方淹没。
- **网络层（加 IP 地址）**：就像在信封上写下发件人和收件人的地址。它负责规划路径，决定这封信该经过哪个路由器。
- **链路层（加 MAC 地址）**：负责在相邻的两个硬件（如你的电脑到路由器）之间跳跃。
- **IETF (Internet Engineering Task Force, 互联网工程任务组)** 是一个全球性的、开放的非营利组织。
 - **身份**：它是互联网标准的实际制定者。
 - **特点**：它没有正式的会员制。任何感兴趣的人（工程师、学者、甚至是你）都可以参加其会议或加入邮件列表。
 - **格言**：“We believe in rough consensus and running code.”（我们相信**大致的共识和跑得通的代码**。）这意味着他们不玩虚的，标准必须经过实践检验。
- **RFC (Request for Comments, 征求修正意见书)** 是互联网的一系列编号文档。
 - **演变**：起初，它真的只是“征求意见”的小纸条。但现在，虽然名字还叫“征求意见”，但其中最重要的部分已经成为了**事实上的国际标准**。
 - **地位**：每一个我们熟悉的网络技术（如 HTTP、TCP、IP、域名解析 DNS）都对应着一个或多个 RFC 编号。
 - 例如：**RFC 793** 定义了 TCP；**RFC 2616** 定义了 HTTP/1.1。

注：速率就是带宽（仅在该课内容中成立）

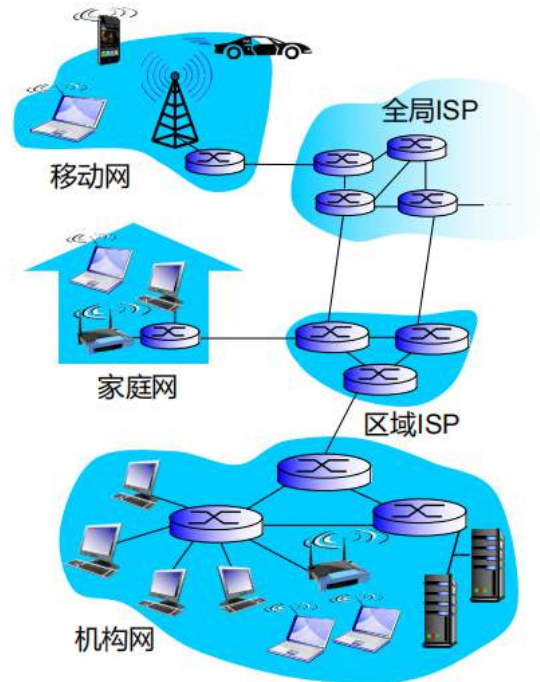
提供服务

▣ 互联网为多种应用提供基础设施服务

- 例如，网页服务、网络电话、电子邮件、网络游戏、电子商务、社交网络...

▣ 互联网为多种应用提供应用编程接口

- 通过应用程序连接互联网的网络钩子
- 面向应用提供类似于生活中邮政业务的选择性服务



什么是应用编程接口(API)，什么是网络钩子(Webhook)?

- 应用编程接口：仅讨论网络相关，其涉及更多的是客户端主动向服务端发送消息，以传统bs架构举例，后端暴露的接口就是API，客户端通过对服务器发起请求，服务器回答发送给客户端
- 网络钩子：其涉及更多的是服务端主动向客户端发送消息，比如客户端给服务端一个url，要求服务端在达成某种条件的时候主动访问该url

网络协议

网络协议其实就是在发送特定消息的时候，决定这种消息应该怎么处理，怎么传输，怎么接收

三要素

- 语义：本身的含义
- 语法：语言的规则
- 时序：时间顺序

定义

协议定义了网络实体之间发送和接收消息的**格式、顺序**，以及发送和接收消息后或发生某些事件时所采取的**行动**。

*网络结构

□ 网络边缘

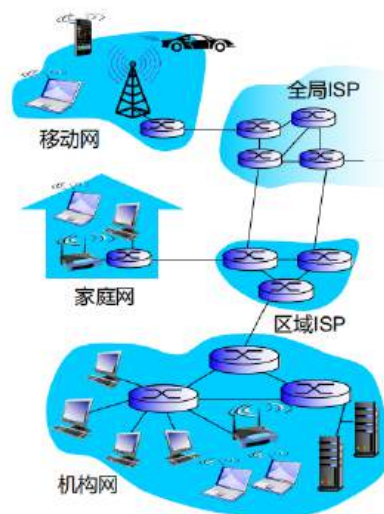
- 主机：客户端和服务端
- 服务器通常位于数据中心

□ 接入网络、物理媒介

- 有线通信链路
- 无线通信链路

□ 网络核心

- 互联的路由器
- 互联网络的网络



边缘路由器：从主机连接到服务器的第一个路由器

网络边缘

网络边缘包含接入网络和物理媒介，详细见下（有时候会认为网络边缘，网络核心，接入网为计算机网络的组成部分）

主机发送数据

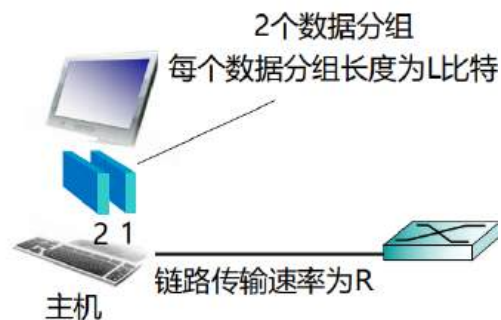
主机：发送数据分组

软件学院·计算机网络

主机发送数据分组：

- 获取应用报文
- 将报文分成小块，即分为**长度L比特**的数据**分组**
- 将分组以**发送速率R**发送至接入网

- 传输速率v.s.链路带宽



分组发送时延，即将L比特分组发往链路的时间

$$\frac{L \text{ (bits)}}{R \text{ (bits/sec)}}$$

应用在发送数据的时候，先分成多个组

链路带宽表示同时最大理论比特流量

传输速率即为传输比特速度

发送时延指的是将L大小的比特数据以R的速率传输所需的时间

物理媒介

双绞线

双绞线其实就是网线，但是有不同的分类

分为电话线和现代网线

双绞线（TP）

□ 两根绝缘铜线

- 5类线：典型应用于100Mbps、1Gbps的以太网中
- 6类线：10G



分为屏蔽双绞线和非屏蔽双绞线或者6类线和5类线

按屏蔽保护分类（物理结构）

这种分类主要看线缆对外部电磁干扰（EMI）的抵抗能力。

分类名称	缩写	特点	应用场景
非屏蔽双绞线	UTP	只有绝缘胶皮包裹，没有金属屏蔽层。直径小、易弯曲、成本低。	家庭、办公楼等普通室内环境。
屏蔽双绞线	STP/FTP	线芯外包裹了铝箔或铜网屏蔽层，能有效减少辐射和干扰。	机房、工厂等强干扰环境，或安全级别要求高的场所。

按性能等级分类（传输标准）

这种分类是根据线缆的材质、绞合度以及支持的带宽频率来划分的，决定了网速的上限。

分类名称	频率范围	最大传输速率	典型特征
超五类线 (Cat 5e)	100 MHz	1000 Mbps (千兆)	目前入门级网线，由于性价比高，在老旧建筑中非常普及。
六类线 (Cat 6)	250 MHz	1 Gbps (满千兆)	内部增加了一个十字骨架用于隔离四对线芯，减少串扰，传输更稳定。

分类名称	频率范围	最大传输速率	典型特征
超六类线 (Cat 6A)	500 MHz	10 Gbps (万兆)	频率翻倍，通常带有更厚的外皮或屏蔽层，支持更长距离的万兆传输。

同轴电缆

同轴电缆

- 2层同心的铜导体
- 信号双向传输
- 支持宽带传输
 - 一电缆多信道
 - HFC



区别于电话线，其是多种信号共用的线缆

同轴电缆长什么样？

“同轴”的意思是：所有的层都围绕同一个中心轴。它的结构比双绞线复杂得多，防干扰能力也极强：

1. **中心铜芯：** 传输信号的主干。
2. **绝缘层：** 塑料材质，包裹铜芯。
3. **金属网屏蔽层：** 这是它的核心优势。这层铝网或铜网能彻底阻挡外界的电磁干扰。
4. **外护套：** 最外层的黑色或白色保护皮。

它为什么比电话线（DSL）强？

电缆网络原本是用来传输**有线电视信号**的。电视信号包含了几百个频道，数据量极大。

- **高带宽：** 同轴电缆的物理带宽非常高，能支持极高的频率。
- **屏蔽性好：** 相比电话线，同轴电缆即便拉几公里长，信号依然非常清晰。
- **混合组网 (HFC)：** 现在运营商通常用光纤传到小区，然后用同轴电缆进家。这种模式叫 **HFC**。

光缆

- 光脉冲在玻璃纤维中传播
 - 1个脉冲就是1bit
- 速度高
 - 点对点间高速传输，例如典型速率10-100Gpbs
- 误码率低
 - 不受电磁干扰
 - 中继器相距远

物理媒介：无线电波

软件学院·计算机网络

- 电磁波信号承载信息
- 没有物理的“线”
- 双向传输
- 传播受环境影响：
 - 反射
 - 障碍物阻隔
 - 干扰

无线链路类型

AI识图

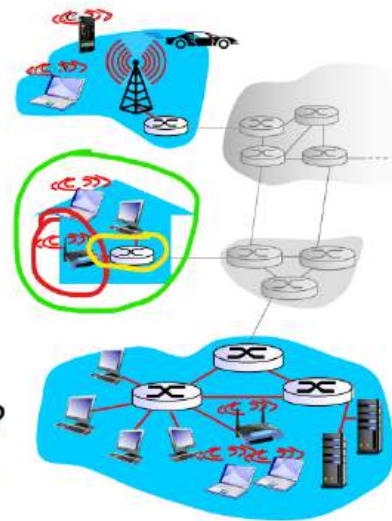
- 地面微波
 - 信道带宽高达45Mbps
- 卫星
 - 例如，低/中/高轨卫星通信，带宽Kbps至45Mbps、典型端到端时延280毫秒
- 无线局域网LAN
 - 例如典型带宽11Mbps、54Mbps的WiFi
- 无线广域网WAN
 - 例如典型带宽Mbps级别的3G、4G、5G蜂窝网络

问题：端系统如何连接边缘路由器？

- 家庭接入网
- 机构接入网，如学校、企业
- 移动接入网

注意：

- 接入网的带宽是多少(bps)？
- 媒介是共享的还是专用的？



- 红色圈的是我们常见的路由器，其不是边缘路由器，称为Access-pointer(接入点)
- 黄色是边缘路由器
- 整个绿色称为接入网，这里是家庭接入网

家庭接入网：数字用户线路（DSL）

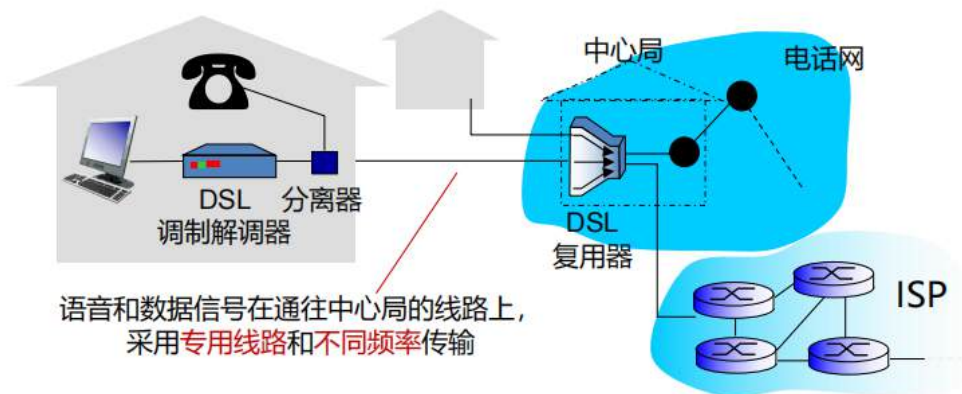
使用电话线（内部通常只有 **1 对**（2 根）或 **2 对**（4 根）互相绞合的铜线，是双绞线的一种）

它是做什么的：它让你家里的电话线在打求救电话的同时，还能高速上网。

最常见的形式：**ADSL** (Asymmetric DSL，非对称数字用户线路)。之所以叫“非对称”，是因为它的**下载速度**远大于**上传速度**（符合当时人们看网页多、发东西少的习惯）。

它是如何工作的：它通过“频分复用”技术，把电话线频率切成几块：

- 低频段：留给语音通话（电话）。
- 高频段：留给数据传输（上网）。
- 中间需要一个：DSL 猫（Modem）和分离器（Splitter）。



- 调制解调器：对主机的某些信号进行处理（比如说调整频率），其实就是猫
- 复用器：将多家的信号汇总，分配到电话网和互联网

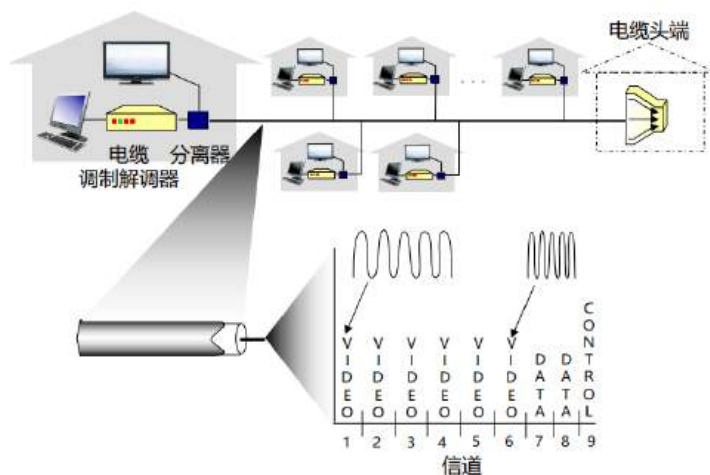
ADSL中的“A”指Asymmetric非对称

Ø 上行传输速率典型值 < 1 Mbps (ITU 2003)

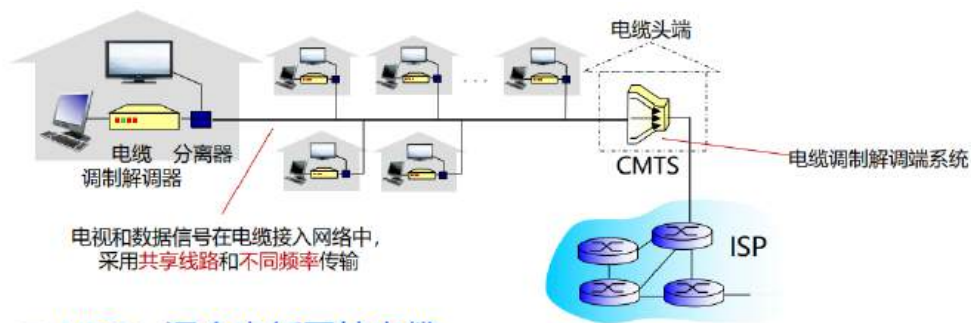
Ø 下行传输速率典型值 < 10 Mbps (ITU 2003)

家庭接入：电缆网络（HFC）

使用同轴电缆，通常是运营商使用光纤将信号接入小区，小区使用同轴电缆去传递信息



频分复用：语音和数据信号在通往中心局的线路上，采用专用线路和不同频率传输，每一条线路是独占的，即只支持1对1，其实就是不同频率代表不同的信道，每一种信息传递占用某一个信道



□ HFC: 混合光纤同轴电缆

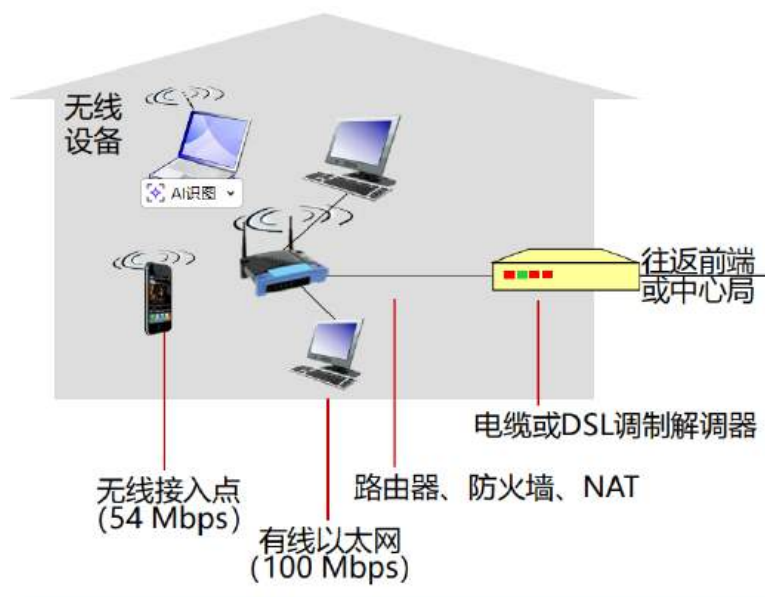
- 非对称: 典型上行速率30.7Mbps下行速率42.8Mbps (DOCSIS2.0)

□ 用户采用电缆和光纤网络连接至ISP路由器

- 家庭采用共享接入网络连接至电缆头端
- 注意此处与DSL采用专用线路连接至中心局不同

家庭接入：典型接入网

全部使用光纤，光纤直接到家（FTTH），连接家中的光猫



机构接入网：以太网

以太网（**Ethernet**）是目前**全球局域网**（LAN）中最通用的一套“交通规则”和“物理接口标准”。

以太网到底是什么？

以太网并不是一种具体的“线”，而是一个由 **IEEE 802.3 标准**定义的体系。它规定了两件事：

1. **物理层**：应该用什么样的线（比如 8 芯双绞线、光纤）和什么样的接头（比如 RJ-45 大方头）。
2. **链路层**：数据包应该长什么样（以太网帧结构），以及多台电脑连在一起时，怎么排队说话才不会撞车（冲突检测机制）。

以 以太网 为核心的“全家桶”

当你提到以太网时，通常涉及以下这三样东西的组合：

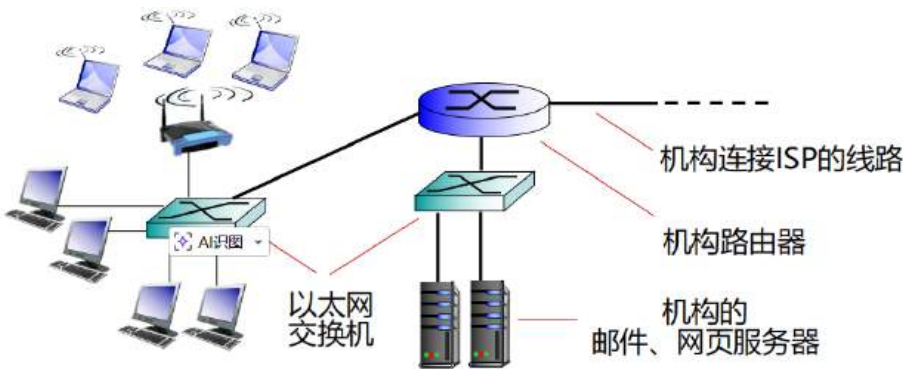
- **介质：**主要是你见过的 **8 芯双绞线（网线）**。
- **接口：**电脑或路由器上的 **RJ-45 网口**。
- **设备：****交换机（Switch）**。它是以太网的核心，负责把数据准确地投递到对应的 MAC 地址。

以太网 vs. Wi-Fi

这是最直观的理解方式。在家里，你通常有两种方式连入局域网：

- **有线连接 = 以太网 (Ethernet)：**插上网线，速度极快（1Gbps 或 10Gbps），延迟极低，非常稳定。
- **无线连接 = Wi-Fi (WLAN)：**通过电磁波传输，灵活但容易受墙壁、微波炉干扰。

底层逻辑：虽然传输介质不同（一个是铜线脉冲，一个是无线电波），但它们最终都会在路由器里汇聚，转换成统一的 IP 数据包发往互联网。



- ▣ 典型应用包括企业、大学等机构
- ▣ 典型速率包括10Mbps、100Mbps、1Gbps、10Gbps等
- ▣ 典型场景是端系统通过以太网交换机接入网络

设备名称	物理本质	逻辑身份	常用连接方式
光猫	光网络终端 (ONT)	边缘路由器 (连接 ISP 与家庭)	光纤入，Wi-Fi/网线出
家用路由器	多功能网络设备	核心网关 + 接入点 (AP)	网线入，Wi-Fi/网线出
电脑/手机	终端节点	客户端	Wi-Fi 或 以太网 (Ethernet)

无线接入网

- 共享的无线接入网，将终端连接至路由器
 - 例如通过基站这个“接入点”

无线局域网:

- 典型范围100英尺
- 典型标准802.11b/g (WiFi):
11.54 Mbps传输速率



至互联网

无线广域网

- 典型范围10-20公里
- 典型速率1到10 Mbps间
- 电信运营商提供服务
如3G、4G: LTE、LTE-A、5G、B5G、6G



至互联网

无线局域网 (WLAN)

LAN是局域网

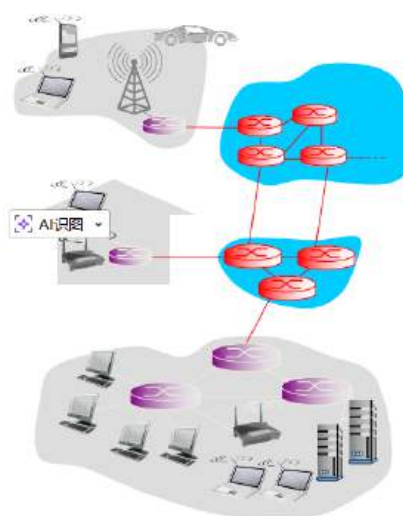
WIFI是技术，WLAN是网络结构

无线广域网 (WRAN)

比如说在户外手机连接基站使用5G网络

网络核心

- 网状互联的路由器
- 分组交换：来自主机应用的消息被分割为分组
 - 分组从源端开始被发送至中转路由器，直到目的端
 - 每个分组发送都占用链路最大带宽



网络核心主要研究途中高亮的部分

通常我们的传输速率接近带宽上限，因此通常说带宽而不说速率

数据交换的发展历史

电路交换->报文交换->分组交换->虚电路，数据报

简单来说，早期使用电路交换，也就是说，假设有十条线路，同时最多可以处理10个主机的数据，每一个主机的线路是固定的，但这样就有一个问题，如果主机数太多，就会出现线路爆炸的问题。传输的数据都不会分成包，而是直接流式传递。

因此出现了报文交换，这时候将数据分为极少的大包传递，但在路由器处必须等待整个包传递完成后才能继续向下传递，延迟非常高

最后出现了分组交换，即共享线路，也就是将每一个主机发出的数据包分为小块，多个主机同时向该线路发送小块数据包，但在请求量极大的时候可能会出现严重阻塞和丢包

因此针对分组交换，又出现了两种对分组交换的特殊实现形式，为虚电路、数据报，用来解决类似的问题

电路交换

与分组交换不同，电路交换指的是一个主机独占一条线路

电路交换

软件学院·计算机网络

针对源端和目的端之间的通信，
分配并保留端到端资源

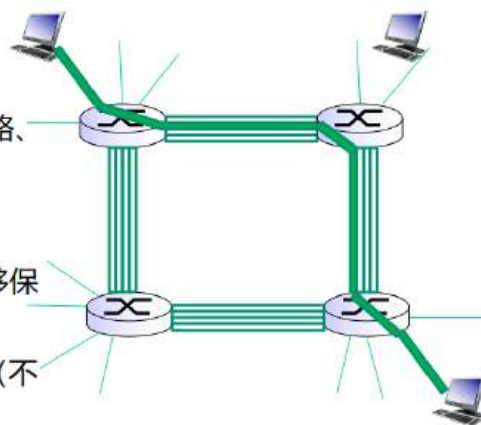
□ 图中每条链路有4条电路

- 通信中，顶部链接用第2条电路、右侧链接用第1条电路。

□ 专用资源：非共享

- 使用时，类似实际电路，能够保证性能
- 未用时，相应的电路段空闲（不进行共享）

□ 常用于传统的电话网络

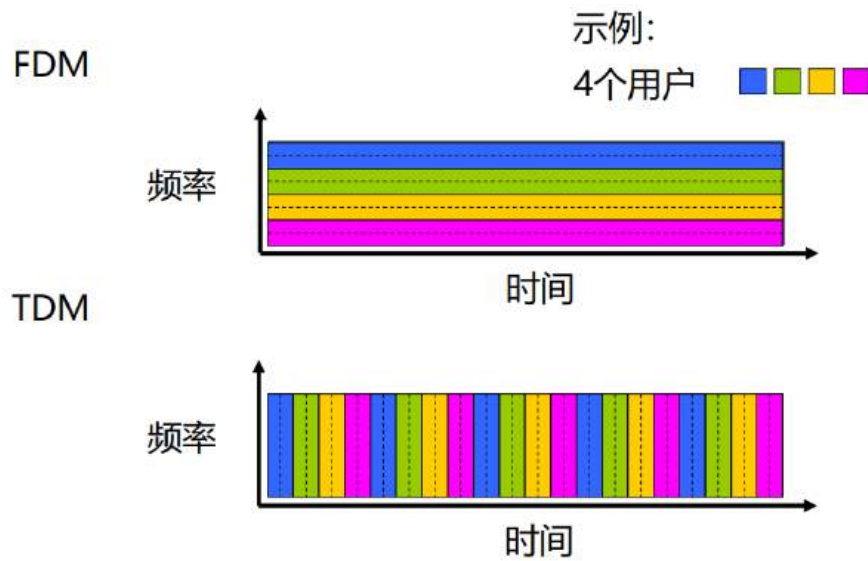


电路交换其实就是预先分配好多条道路，然后给每个流量传递都分配好线路，这条线路不能在这个流量传递结束之前被其他流量使用

其实指的就是打电话的时候的情形，会有占线

FDM与TDM

FDM即频分复用，TDM即时分复用



对于TDM，假设有 n 个用户，那么 n 个用户一个周期的时间称为时系slot

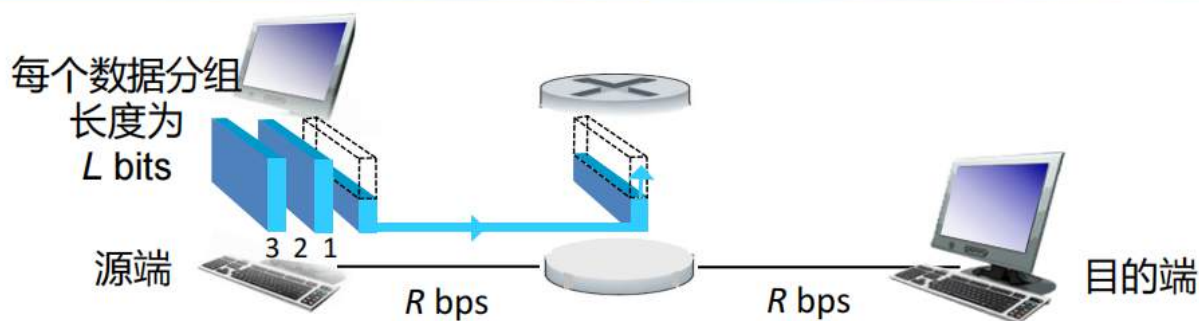
resourceblock 将时间和频率都分成小段

分组交换

本质上是多个主机共用一条线路

存储转发

讨论单一主机传递流量时，流量的行为



- 将 L bit 数据包发送到带宽为 R bps 的链路中, 单跳场景示例:
需要 L/R 秒
 - $L = 7.5$ Mbits
 - $R = 1.5$ Mbps
 - 单跳传输时延 = 5 s
- **存储和转发**: 首先整个数据包全部到达路由器, 然后发送至下一条链路

端到端时延 = $2L/R$
(假设传播延迟为零)

} 其他延迟问题, 以下陆续讲解

如图, 比如说主机要发送一堆数据, 这组数据一共有 $3L$ bits, 然后我们将其分为三组, 每一组有 $1L$ bits

传递有两条原则:

1. **存储转发原则**: 每一组在传递到路由器时, 该组前面已经传到该路由器的数据不能继续向前传递, 必须等待该组全部数据传输到路由器后才可继续向前传递
2. 因为上一条原则, 为了达到速率最大化, 我们通常将某一个线路的全部带宽用来传递某一组的数据, 只有等待上一组的数据全部传输之后再下一组数据的传输

因此我们可以得到这三组流量实现端对端传输的总延时为 $4L/R$ (在前两条原则的前提下)

为什么? (==表示端a到路由器, --表示路由器到端b)

第一组 |==|

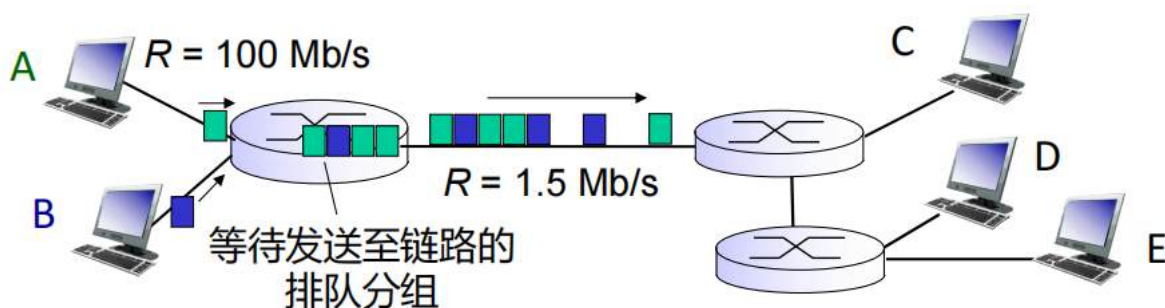
第二组 |==|

第三组 |==|

可以看到总共需要4个 L/R 时间段

补充解释: 如果在三组同时传递下的总延时为多少? 显然为 $6L/R$, 这也证明了原则二的正确性

分组交换：排队和丢包



排队和丢失的原因:

- 分组到达路由器的速率，超过了路由器的转发速率
 - 路由器来不及转发，分组进入缓存队列，开始排队
 - 缓存队列满，发生丢包

截图工具

假设分组到达路由器的速率大于路由器的转发速率，这时候就会排队进入缓存队列。如果缓存溢出，可能会出现丢包。但是现代路由器可能有更多策略，比如按照优先级进行排队处理等等。

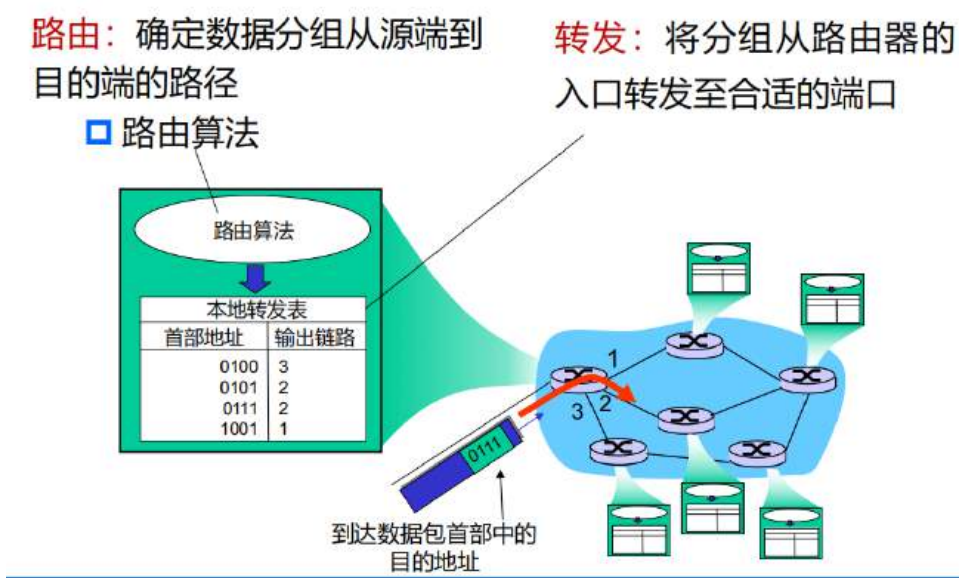
虚电路

虚电路其实是在分组交换的基础上，结合电路交换的思路，给某些需要极低延迟的用户分配专门的线路，保证该用户的数据永远走这条最短线路，减少分配开销，防止绕路，具有“超级优先权”和“保留车道”，但其还是属于分组交换，因为这条线路物理上依然会被别人的包使用。实现原理就是数据包自带完整的线路。

数据报

数据包其实就是纯粹的分组交换，每一个数据包都只带目标的ip地址，具体怎么走取决于路由和当时的网络情况决定。

关键功能



- **路由（控制面）：**像是在指挥部**决策**。任务是根据复杂的地图（路由表），算出一封信该往哪走。
- **转发（数据面）：**像是在中转站**干活**。任务是把信从这边的入口，直接扔到那边的出口。

在**虚电路**和**数据报**这两套逻辑下，这两个功能的体现有着天壤之别：

数据报（Datagram）：每一站都要“动脑子”

在数据报网络里（比如现在的互联网 IP 协议），路由器是一个“忙碌的决策者”。

- **路由（Routing）：**路由器会不断和邻居交换信息，更新地图。比如它发现“西安到重庆”的主路断了，它会重新计算出“西安-成都-重庆”的路线。
- **转发（Forwarding）：**
 - **独立性：**每一个到达的包，路由器都要拆开看一眼**完整的目的 IP 地址**。
 - **查表：**路由器在路由表里通过“最长前缀匹配”来查找。这个包该从 1 号口走，还是 2 号口走？
 - **体现：**转发是“**按包寻址**”。即便 100 个包都是发往同一个地方，路由器也要机械地查 100 次表。

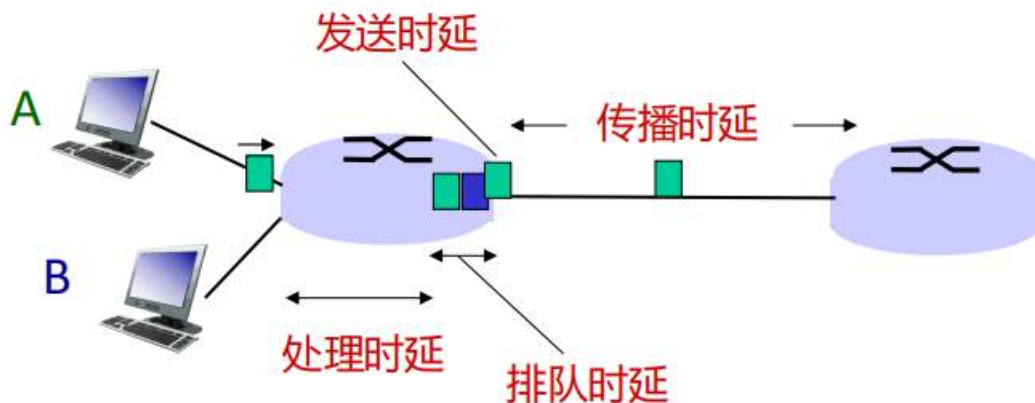
虚电路（Virtual Circuit）：只在开始时“动脑子”

在虚电路网络里（比如 MPLS 或 ATM），路由器在数据传输阶段表现得像个“熟练的搬运工”。

- **路由（Routing）：**路由只发生在**连接建立阶段**（握手）。
 - 当你发起连接时，网络会为你规划一条固定路径。
 - 沿途的所有路由器会达成共识：“凡是贴着**标签 101** 的包，一律转给 2 号口，并换成**标签 202** 发出去。”一旦路径定死，路由计算就停止了。
- **转发（Forwarding）：**
 - **标签交换：**路由器不再看复杂的 IP 地址，只看包头那块小小的**虚电路标识符（VCI/Label）**。
 - **体现：**转发是“**按标签索引**”。这就像查字典，直接翻到第 101 页就知道答案了。这种查表速度极快，不需要复杂的逻辑匹配。

网络中的时延、丢包和吞吐量

时延



d_{proc} : 处理时延

- 检查比特差错
- 确定输出链路
- 典型值为微秒或更低的数量级

d_{queue} : 排队时延

- 分组在缓冲队列中，排队等待的时间
- 取决于路由器中拥塞程度

d_{trans} : 发送时延

- L 表示分组长度，单位bit
- R 表示链路带宽，单位bps
- $d_{trans} = L/R$

d_{prop} : 传播时延

- d 表示物理链路长度
- s 表示信号在媒介中的传播速度 (例如 2×10^8 m/sec)

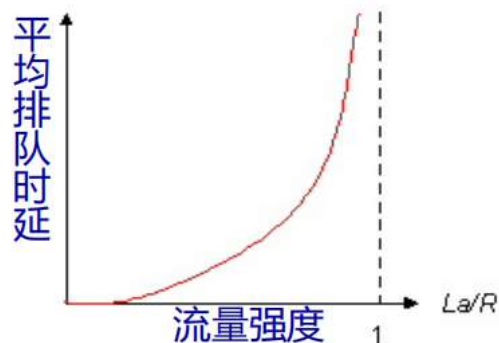
□ $d_{prop} = d/s$

注意区分
发送和传播时延

注意发送时延和传播时延的计算方式的差异

我们注意到，如果当发送的速率和传播的速率差异过大的时候，整个系统的利用率会降低（比如一大堆车都堵在收费站，因为速度足够大，一通过收费站就立马到达下一个了，这样效率很低

- R 表示链路带宽，单位是bps
- L 表示分组长度，单位是bits
- a 表示分组到达的平均速率



- $La/R \sim 0$: 平均排队时延小
- $La/R \rightarrow 1$: 平均排队时延大
- $La/R > 1$: 到达的任务超过了处理能力极限，平均排队时间趋近于无穷



直观来说，公式 La/R 的分子是来到路由器的比特的速率，分母是比特从路由器走出去的速率

但是按理来说，比值接近1应该是最理想的，但实际上因为我们用的 a 是平均速率，而且默认所有数据包均匀到达，但实际上可能会出现某时刻大量数据包堆积而某些时候几乎没有数据包，因此实际上接近1的时候就卡死了

一个关于时延计算的例子

1. 前置参数定义

- 分组数量: $n = \lceil \frac{L}{p} \rceil$
- 单个分组总长度: $l = p + h$ (单位: bits)
- 单段发送时延: $t_{trans} = \frac{l}{b}$
- 链路跳数: 一共有 k 条线路，意味着中间经过 $k - 1$ 个路由器 (节点)。

2. 总时延 T 的分段拆解

我们将总时间划分为三个逻辑阶段：

第一部分：建立连接 (t_1)

由于采用虚电路方式，在正式传输前需要进行路径锁定。

$$t_1 = s$$

第二部分：流水线等待 (t_2)

代入最后一个分组（第 n 个分组）的视角。在最后一组开始从源端 a 的网卡发出之前，它必须等待前面的 $n - 1$ 个分组全部完成首段线路的发送。

$$t_2 = (n - 1) \cdot \frac{l}{b}$$

第三部分：最后一组的全程穿越 (t_3)

这是最后一个分组从源端 a 启程，直到被终端 b 完全接收所需的全部时间。它包含以下分项：

- **发送时延**：经过 k 条线路，产生 k 次发送动作： $k \cdot \frac{l}{b}$
- **传播时延**：在 k 条物理线路上飞行： $k \cdot d$
- **处理时延**：在中间 $k - 1$ 个节点被解析： $(k - 1) \cdot m$

$$t_3 = k \cdot \frac{l}{b} + k \cdot d + (k - 1) \cdot m$$

3. 最终汇总公式

将上述三部分相加，得到整个报文从端 a 到端 b 的总传输时延 T ：

$$T = t_1 + t_2 + t_3$$

$$T = s + (n - 1) \cdot \frac{l}{b} + [k \cdot \frac{l}{b} + k \cdot d + (k - 1) \cdot m]$$

通过合并同类项（关于 $\frac{l}{b}$ 的部分），公式可以进一步简化为：

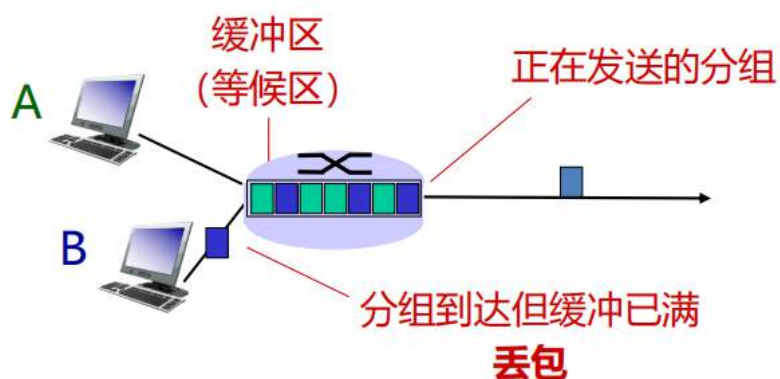
$$T = s + (n + k - 1) \cdot \frac{l}{b} + k \cdot d + (k - 1) \cdot m$$

四大时延在题目中的对应关系

时延类型	定义与在本题中的体现
发送时延 (Transmission Delay)	$\frac{l}{b}$ 。将分组的所有比特推向链路所需的时间。
传播时延 (Propagation Delay)	d 。电磁波在物理媒体中传播一段距离所需的时间。
处理时延 (Processing Delay)	m 。路由器检查分组首部、决定转发方向等操作的时间。
排队时延 (Queuing Delay)	在本题的理想模型中，排队时延体现在 t_2 阶段，即由于带宽限制，后续分组必须等待前序分组腾出信道。

丢包

- 队列（缓冲）容量有限
- 分组到达已满的队列（丢包）
- 丢失的分组可能由源节点或上一个节点重传，也可能不进行重传



- 互联网在现实中的时延和丢包是什么样的？
- **traceroute**程序：测量互联网中从源端到目的端的路径上，从源端到各路由器的时延。对路由器 i ，过程如下：
 - 在通往目的端的路径上，发送方发3个分组到路由器 i
 - 路由器 i 收到分组后，向发送方返回一个响应
 - 发送方记录从发送分组到收到响应的时间间隔



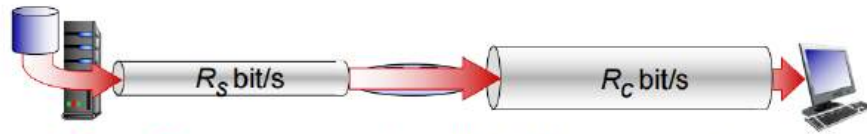
有这么一个情景，假设从主机发出一个数据包到达路由器a，此时路由器a可以到达路由器b，c，d，它希望找到一个时延最低的路线去进行传递，那么它一般会给每个路由器都分别发三个探针数据包去获取最短时间间隔，这个行为叫做刺探

吞吐量

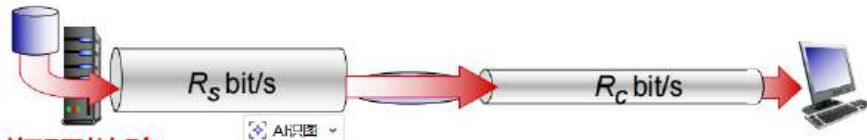
吞吐量强调端对端

- 定义：**发送端和接收端**之间数据比特的传输速率，即比特数/单位时间，bps
 - 瞬时吞吐量：给定时间点的比特传输速率
 - 平均吞吐量：较长时段内的比特传输速率

- 当 $R_s < R_c$ 时，平均端到端吞吐量是多少？



- 当 $R_s < R_c$ 时，平均端到端吞吐量是多少？



瓶颈链路

在端到端路径上，限制端到端吞吐量的链路

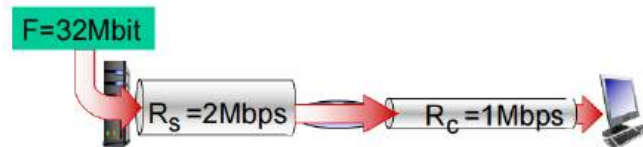
两个例子的吞吐量都是最小的链路的吞吐量，因此这种不对称的链路称为瓶颈链路

- 从源端到目的端需要多长时间？

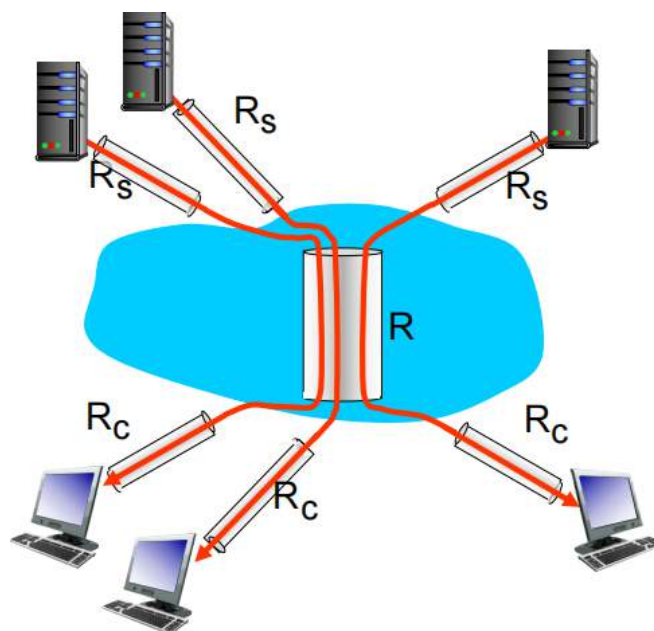
A. 16 s

B. 32 s

C. 不会



通过吞吐量的计算，我们可以计算出需要32秒



假设中心R的带宽足够大，那么端对端的最大速率一般由与主机直接连接的线路决定

假设中心R的带宽不够大，那么有可能会由中心R带宽决定

*协议层次

明确的结构能够识别复杂系统的各部分，以及各部分之间的关系有利于通过层次化参考模型进行讨论分析

协议栈（TCP/IP）

- **应用层：**支持网络应用
 - FTP, SMTP, HTTP
- **传输层：**进程间的数据传输
 - TCP, UDP
- **网络层：**源端到目的端的数据报路由
 - IP, 路由协议
- **链路层：**相邻结点间的数据传输
 - Ethernet, 802.11(WiFi), PPP
- **物理层：**物理媒介上的比特流传送



网络数据的“变名”之旅

在每一层加头部的行为叫做**封装**，反过来叫做**解封装**

- **应用层 (Application Layer)：** 原始信息确实叫做 **报文 (Message)**。这是最完整、最具业务逻辑的数据块。
- **传输层 (Transport Layer)：** 加上传输层首部后：
 - 如果是 **TCP**，称为 **段 (Segment)** 或 **报文段**。
 - 如果是 **UDP**，称为 **用户数据报 (User Datagram)**。
- **网络层 (Network Layer)：** 加上 **IP 首部**后，统一称为 **数据报 (Datagram)** 或 **分组 (Packet)**。
- **链路层 (Data Link Layer)：** 加上**帧头和帧尾**后，称为 **帧 (Frame)**。这一层开始涉及物理寻址（MAC地址）。
- **物理层 (Physical Layer)：** 数据被转换成电信号、光信号或电磁波，称为 **比特流 (Bits)**。

1. 应用层 (Application Layer) —— “你要寄什么东西？”

应用层直接面向用户或应用程序。它定义了数据交换的规则。

- **功能：**支持各种网络应用，解决“我们要聊什么、怎么聊”的问题。
- **常见协议：**
 - **HTTP：**网页浏览。
 - **SMTP：**发送电子邮件。

- **FTP**：文件传输。
- **DNS**：域名解析（把网址翻译成 IP）。
- **通俗理解**：应用层决定了你是在玩游戏、发微信还是看网页。

2. 传输层 (Transport Layer) —— “交给哪个程序？”

正如你刚才问的，传输层负责**进程间**的数据传输。

- **功能**：它通过“端口号”确保数据能准确送到电脑里的某个程序（比如把游戏数据包丢给游戏进程，而不是网页浏览器）。
- **常见协议**：
 - **TCP**：可靠传输，就像挂号信，丢了会重传。
 - **UDP**：极速传输，就像平信，丢了就丢了（追求低延迟，适合你的游戏项目）。
- **通俗理解**：它是你家门上的“收件人姓名”，确保包裹不送错人。

3. 网络层 (Network Layer) —— “快递该往哪条路走？”

网络层负责在复杂的网络中选择最佳路径。

- **功能**：实现源主机到目的主机的数据路由和转发。
- **常见协议**：
 - **IP**：定义了互联网的地址规范。
 - **路由协议**：路由器之间互相“商量”哪条路最近。
- **通俗理解**：它是快递单上的“收货地址”，以及物流中心的路径规划系统。

4. 链路层 (Data Link Layer) —— “相邻的两个点怎么传？”

链路层负责在**相邻结点**（比如你的电脑和路由器之间，或者两台路由器之间）进行可靠的数据传输。

- **功能**：它把数据切成一帧一帧（Frame），处理物理地址（MAC 地址）和差错控制。
- **常见协议**：
 - **Ethernet**：有线以太网。
 - **802.11 (WiFi)**：无线局域网。
 - **PPP**：宽带拨号常用的点对点协议。
- **通俗理解**：它是具体的运输方式。比如这一站是走铁路（有线），下一站换飞机（无线）。

5. 物理层 (Physical Layer) —— “电信号和光信号”

物理层是整个系统的地基。

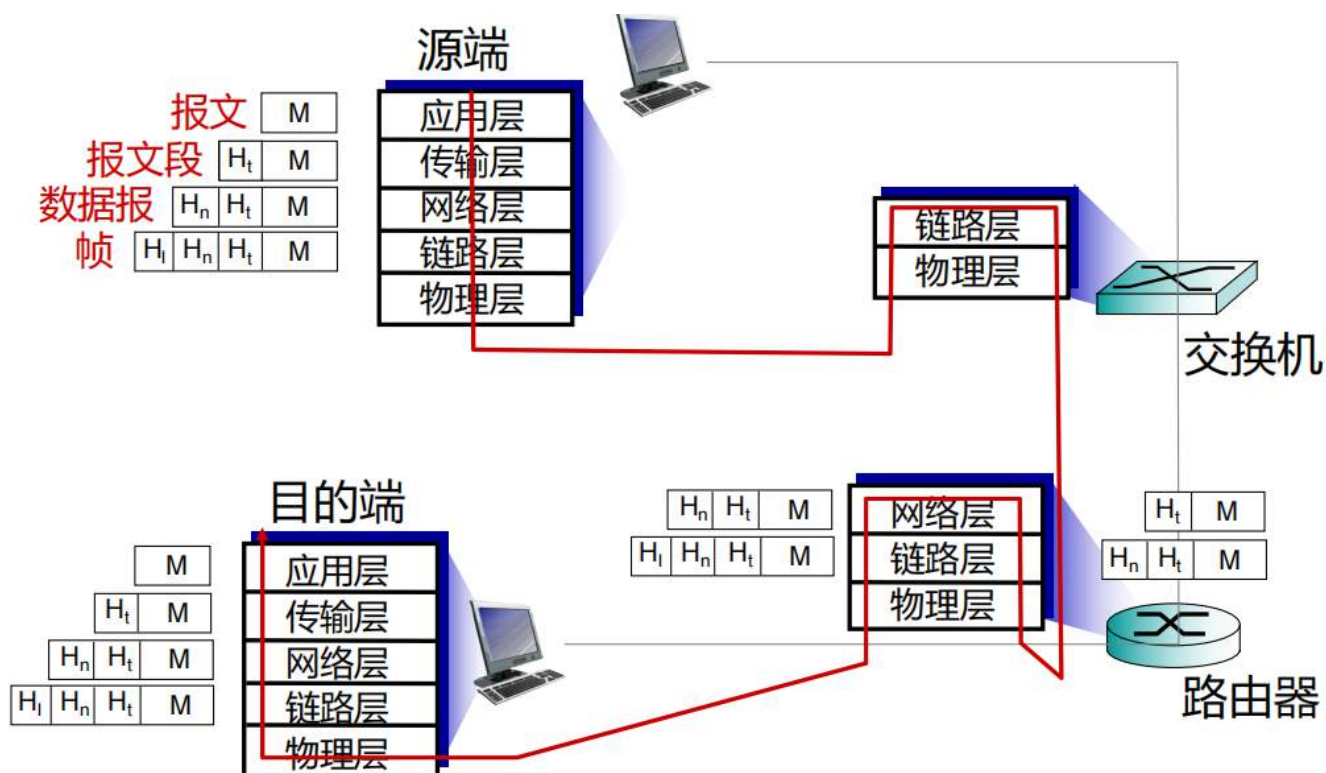
- **功能**：在物理媒介（如电缆、光纤、空气）上透明地传送原始比特流（0 和 1）。
- **关键点**：它规定了插头的形状、光纤的频率、电压的高低等物理特性。
- **通俗理解**：它就是那根看得见摸得着的网线或光纤，是承载一切的物理载体。

ISO/OSI参考模型

- **表示层**：支持数据的可解释性，即使得应用程序能够理解数据的含义。例如，加密、压缩、特定的机器规则等。
- **会话层**：支持数据交互中的同步、校验，（从错误中）恢复等。
- 互联网协议栈没有这两层
 - 考虑实际上是否需要呢？
 - 当网络需要这些层次提供的服务时，在应用层实现实现这些服务



ISO/OSI参考模型是一个国际标准，而互联网协议栈（5层），其他模型，网络应用开发可以进行适当改动



需要记忆源端，交换机，路由器，目的端各自处理的层次

第二章 应用层

从本章开始，每一层的讲解之前会先对其下层的实现有一定的介绍，这是该层实现的基础

我们从应用层的基本概念开始介绍，然后介绍传输层功能支撑，功能支撑按照架构分为C/S和P2P来介绍，最后再介绍其他协议

常见协议有：HTTP,FTP,SMTP/POP3/IMAP,DNS

应用架构

C/S

即Client/Server（客户端/服务端）架构

服务器：

- 常开的主机
- 长期的IP地址
- 支持网络规模扩展的数据中心

客户端：

- 可能进行间歇性连接
- 可能使用动态的IP地址
- 客户端之间不会直接互联，而是与服务器通信

常见协议：http,ftp,邮件协议,DNS

P2P

- 对等方：直接通信，间歇连接
没有始终开启的服务器
- 一个对等方**不仅需要其他对等方提供的服务，也向其他对等方提供服务**
 - 很少甚至不依赖专用服务器
 - 自扩展性：新的对等方加入网络自行扩展，带来新的服务能力和新的需求
- 去中心化架构

管理、安全、可靠性需要深入讨论

进程通信

进程：在主机中运行的那些程序

- 在同一主机中，两个进程通过**进程间通信**交互信息，该过程一般由操作系统定义
- 在不同主机上，不同的进程通过**报文**交换机制交互信息，该过程一般由应用层协议规定

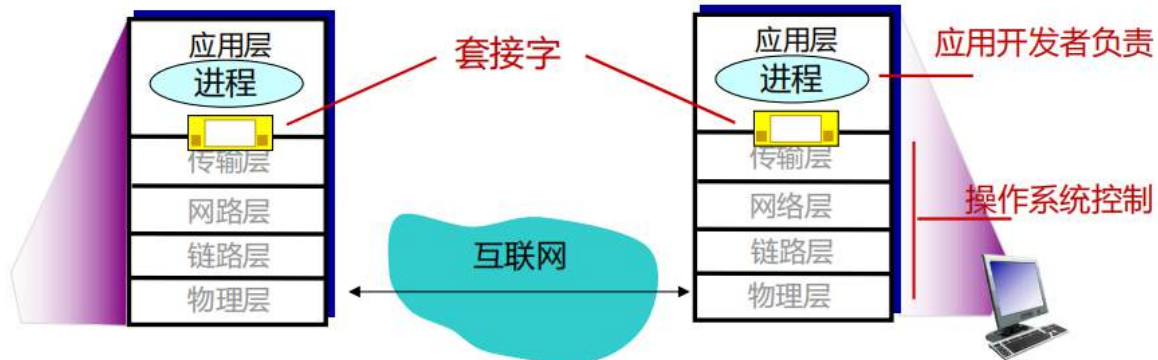
客户端，服务器

客户端进程：发起与服务器通信的进程（例如请求）

服务器进程：等待与客户端连接的进程（例如监听）

- 注意：基于P2P架构的应用同时拥有客户端进程和服务端进程

套接字Socket



相当于把(网络基础设施)

- 源 IP 地址（你是谁）
- 源端口（你哪个进程在说话）
- 目的 IP 地址（你要找谁）
- 目的端口（你找对方哪个进程）
- 传输协议（用 TCP 还是 UDP）

封装起来的一个门，进程通过套接字发送和接收报文，主要行为：

1. 发送进程将报文推出门外
2. 在向接收进程的门外传递报文的过程中，发送进程依赖于门另一端的网络基础设施

进程寻址

- 为了收发报文，进程必须有唯一的**标识符**（ID）
- 主机设备具有唯一的32位IP地址（实际对应的是网络接口）
- **问题**：在运行着进程的主机上，IP地址能否满足唯一标识进程的目标？
 - **回答**：不行。因为同一主机上，可能运行着多个进程。
- 主机上某进程关联的**标识符**包括**IP地址**和**端口号**
- 示例端口号
 - HTTP服务器：80
 - 邮件服务器：25
- 向job.nwpu.edu.cn网页服务器发送HTTP消息时：
 - **IP地址**：61.150.43.9
 - **端口号**：80

进程寻址就是这个根据IP和端口号分发包裹的行为

应用层协议定义的内容

协议定义的核心要素

为了让不同的进程（比如 Chrome 浏览器和 Nginx 服务器）能互相听懂对方的话，协议必须明确定义以下四点：

1. 报文类型（Type）

- 明确这封“信”是来**请求（Request）*数据的**，还是**针对请求给出的*响应（Response）**，亦或是简单的状态宣告。

2. 报文语法（Syntax）

- 规定报文的**格式和结构**。比如信封上有哪些字段（如：URL、状态码、报文头），这些信息是用文本表示还是二进制，每个部分占多少字节。

3. 报文语义（Semantics）

- 规定每个字段的具体**含义**。比如状态码 `404` 代表“找不到资源”，字段 `Content-Length` 代表后面跟着的数据有多大。

4. 操作规则（Rules）

- 定义进程在什么时候发送报文、收到报文后该如何反馈。这关乎**时机与逻辑**（例如：收到请求后必须立即回执，还是等待数据就绪后再回复）。
-

协议的两大阵营

根据协议的“开放程度”，它们通常被分为两类：

- **开放协议（Open Protocols）**
 - **特点：**完全公开透明，定义在 **RFC**（Request for Comments）文档中。
 - **核心价值：互操作性。**只要开发者遵循 RFC 标准，任何公司开发的程序都能互相通信。
 - **典型例子：****HTTP**（网页浏览）、**SMTP**（邮件发送）。
- **专有协议（Proprietary Protocols）**
 - **特点：**由特定公司或组织私有，不公开具体细节，像一个“黑盒”。
 - **核心价值：商业护城河**或针对特定场景的**深度优化**。通常只有该公司的客户端才能互通。
 - **典型例子：**早期 **Skype** 的 P2P 通信协议、**QQ** 的通信协议。

应用层对传输服务的需求

课本上又称为传输层对应用层的支撑服务

可靠性（Reliability）：数据能不能丢？

这决定了传输层必须使用什么样的协议。

- **不容忍丢包（可靠）：**像**文件传输 (FTP)**、**在线交易**，丢掉一个字节（比特位）都会导致程序崩溃或资金错误。
- **容忍丢包（不可靠）：**像**视频通话**或**在线语音**，由于人耳的补偿机制，偶尔丢掉一两个数据包只会产生轻微的电音或卡顿，不影响整体沟通。

实时性（Timing）：速度够不够快？

这关注的是从发送端到接收端的**端到端时延**。

- **低时延要求：**典型的例子是 **交互式游戏 (ThetaForce)**。如果你在 FPS 游戏中开火，指令必须在几十毫秒内传达，否则就会出现“瞬移”或判定失效。
- **时延不敏感：****电子邮件**或**文件下载**。邮件晚到 10 秒钟，用户通常感知不到，也不会影响邮件的内容质量。

吞吐量（Throughput）：带宽够不够大？

这指的是单位时间内传输的数据量。

- **保证带宽：****高清直播**或**多媒体流**（如 4K 视频）。如果带宽低于某个阈值，画面就会因缓冲而中断，这种应用对带宽有“起步价”要求。
- **弹性应用 (Elastic Apps)：****网页浏览**或**Git 提交**。带宽大就传得快点，带宽小就慢点，有多少用多少，不会因为带宽小就彻底无法运行。

安全性（Security）：过程够不够稳？

这属于附加的保护层。

- **加密与机密性：**防止第三方监听（如 HTTPS 交易）。
 - **数据完整性：**防止数据在传输过程中被篡改。
 - **端点鉴别：**确认和你通信的那扇“门”后面真的是你想要找的人。
-

应用需求对比表

应用场景	丢包容忍度	时延要求	吞吐量要求
文件传输 / 文本聊天	0 容忍	高容忍	弹性 (Elastic)
网络电话 (VoIP)	允许少量丢包	极低时延	固定 (低)
流媒体视频	允许少量丢包	中等时延	固定 (高)
实时游戏	允许少量丢包	极低时延	弹性 (中)

应用	数据丢失	吞吐量	时间敏感
文件传输	不允许丢包	弹性的	不
电子邮件	不允许丢包	弹性的	不
网页文档	不允许丢包	弹性的	不
实时的音频/视频	可容忍丢包	音频:5kbps-1Mbps 视频:10kbps-	是, 100毫秒
存储式音频/视频	可容忍丢包	5Mbps	是, 几秒
交互式游戏	可容忍丢包	同上	是, 100毫秒
文本消息	不允许丢包	几kbps级别以上 弹性的	部分是, 部分不是

互联网传输层协议的服务

TCP

tcp是面向连接的，运输之前必须连接！！

TCP服务

- **面向连接**: 客户端和服务
器进程之间需要建立连接
- **可靠传输**: 收发进程间的
数据传输保证可靠
- **拥塞控制**: 当网络超负荷
时, 约束发端的发送行为
- **流量控制**: 发送数据的速
度不超过接收数据的速度
- **不提供的服务**: 实时、安
全、最小带宽等保证

安全TCP

1. 传统的 TCP/UDP: 明文传输的“裸奔”状态

在没有 SSL/TLS 保护的情况下, 数据传输是透明的:

- **安全风险**: 当你把密码或敏感信息推入传统的 Socket 时, 这些信息会以**明文 (Plaintext)** 形式穿过互联网。
- **中间人威胁**: 任何位于路径上的路由器、交换机或恶意节点, 都可以轻易地截获并阅读你的报文 (就像拆看一封没有封口的信)。

2. SSL: 应用层提供的“安全盔甲”

为了解决上述问题, SSL (及其后继者 TLS) 应运而生。它不是替换了 TCP, 而是在 **TCP 之上、应用层之下** 加了一层保护。

- **位置**: SSL 运行在应用层。程序员通常通过调用类似 OpenSSL 的**类库**来使用它, 而不是直接操作原始 TCP。
- **三大核心功能**:
 1. **加密 (Encryption)**: 报文在进入套接字前被搅碎成密文, 即使被截获, 没有密钥也无法还原。
 2. **数据完整性 (Data Integrity)**: 通过校验码确保报文在传输过程中没有被第三方恶意篡改。
 3. **端点认证 (Authentication)**: 通过证书确认对方的身份 (比如: 确保你连接的确实是 NWPU 的官网, 而不是钓鱼网站)。

3. 通信流程的转变: 从明文到加密

- **不带 SSL**: [明文数据] -> [TCP Socket] -> [互联网 (明文飘过)]
- **带有 SSL**: [明文数据] -> [SSL API 加密] -> [加密数据] -> [TCP Socket] -> [互联网 (密文穿梭)]

关键复述: SSL 到底在哪一层?

虽然它被称为“安全套接字层”，但从实现角度看，**SSL 位于应用层**。应用进程不再直接与 TCP 打交道，而是通过 **SSL Socket API** 进行通信。对于应用来说，它还是往 Socket 里丢数据，但 SSL 库在后台默默完成了“穿衣服（加密）”和“脱衣服（解密）”的工作。

UDP

UDP服务

- **无连接**
- **不可靠**：收发进程间的数据传输不保证可靠性
- **不提供**：连接、可靠、拥塞控制、流量控制、实时、安全、最小带宽等保证
- **问题**：为什么存在两种传输层协议？TCP看起来不错，为什么还要有UDP？

TCP与UDP的对比

1. 开销与速度

- **TCP 像是在通长途电话**：通话前要先确认“喂，你在吗？”（建立连接），通话结束要说“再见”（释放连接）。每一句话都要对方回应“听到了”（确认机制）。这些握手和确认占据了大量的带宽和时间。
- **UDP 像是在扔明信片**：没有连接开销，头部空间极小（仅 8 字节，而 TCP 至少 20 字节）。对于高频、小量的报文发送，UDP 极其高效。

2. 时延控制

这是 UDP 存在的**最大理由**。

- **TCP 的重传机制**：如果中间丢了一个包，TCP 会停下来直到补发成功，后续的数据都会在缓冲区排队（队头阻塞）。对于**实时游戏或语音通话**，这一秒的卡顿比丢掉几个像素点致命得多。
- **UDP 的果断**：丢了就丢了，它会继续发送最新的数据。在直播中，你宁愿画面闪一下绿屏，也不想看到画面卡住 5 秒钟后突然倍速播放。

3. 控制权的归属

- **TCP 是“黑盒”算法**：拥塞控制和流量控制是由操作系统内核硬编码的。如果网络稍微拥堵，TCP 会为了“大局”主动减慢速度。
- **UDP 提供了“空白画布”**：它把控制权完全交给**应用层**。开发者可以根据需求自己实现“部分可靠性”。
 - 例如：你之前关注的 **ENet** 库或 **KCP** 协议，就是跑在 UDP 之上的。它们允许开发者自定义：哪些重要指令（如玩家死亡）必须送达，哪些不重要信息（如玩家移动）丢了也没关系。

总结对比表

特性	TCP (稳重型)	UDP (速度型)
首要目标	万无一失	快、省、简
实时性	差（受重传和拥塞控制影响）	极好
适用场景	网页 (HTTP)、文件传输 (FTP)、邮件	实时游戏、视频会议、DNS 查询
就像是	挂号信（必须本人签字回执）	普通平信（扔进邮箱就不管了）

应用	应用层协议	传输层协议
电子邮件	SMTP [RFC 2821]	TCP
远程终端访问	Telnet [RFC 854]	TCP
网页服务	HTTP [RFC 2616]	TCP
文件传输	FTP [RFC 959]	TCP
流媒体	HTTP (例如YouTube), RTP [RFC 1889]	TCP或UDP
互联网电话	SIP、RTP, 专用协议(例如Skype)	TCP或UDP

Web和HTTP

Web

Web是一个网页服务，HTTP协议为其提供协议

网页服务概览：

- 一个网页（web page）由一些对象（object）组成
 - 对象包括HTML文件、JPEG图片、Java小程序、音频文件等
- 总的来讲，网页包括一个基本 HTML 文件，该文件包括多个被引用的对象，每个对象URL寻址。

www.someschool.edu:80/someDept/pic.gif

主机名

路径名

HTTP

超文本传输协议

基于TCP的HTTP过程

- 客户端创建socket，通过80端口发起到服务器的TCP连接；
- 服务器接受来自客户端的TCP 连接；
- 应用层协议的报文（HTTP 报文）在浏览器（HTTP客户端）和网页服务器（HTTP 服务器）之间进行交互；
- 关闭TCP 连接。

HTTP是“无状态”的

- 服务器不保存客户端以前的请求信息

补充

保留“状态”的协议很复杂

- 需要保留和维护历史状态信息；
- 如果服务器和/或客户端崩溃，它们维护的“状态”可能不一致，需要协调机制。

- HTTP无状态
- 传输层使用TCP

HTTP连接

非持续HTTP连接

- 每个TCP连接，最多只能发送一个对象
 - 发送完成，即关闭连接
- 当需要下载多个对象时，意味着需要多个连接

持续HTTP连接

- 在客户端和服务器之间，仅需建立一个连接，就可以发送多个对象

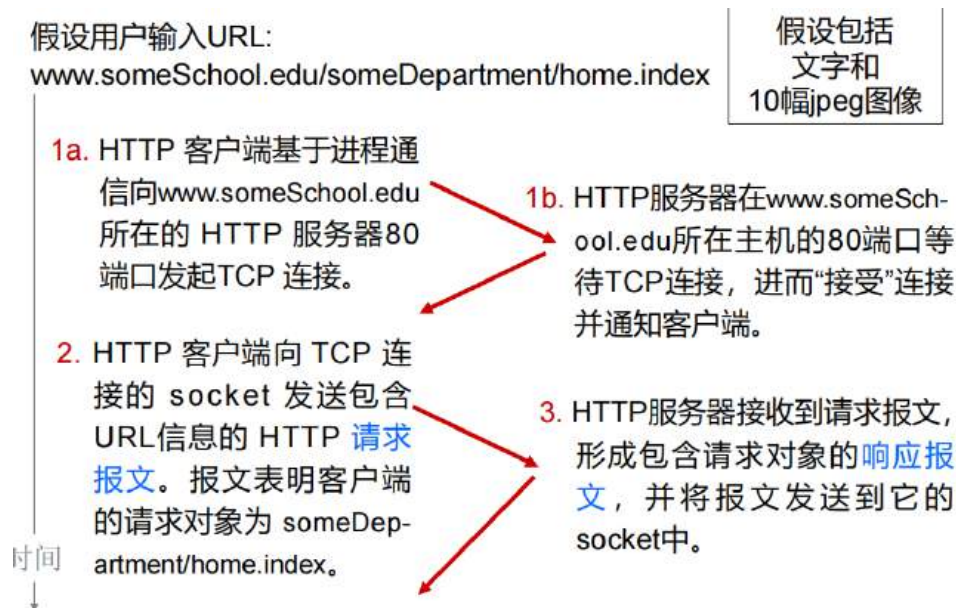
非持续HTTP连接

假设一共请求10个资源，要进行1次确认连接的HTTP连接和10次HTTP连接来分别获取10个资源，每次HTTP连接会进行一次TCP连接，因此一共进行11次

- HTTP连接数：11

- TCP连接数：11
- 总时间：44RTT（单次发送时间）

一次HTTP非持续连接的过程



刚开始先进行一次request-respond来接受连接，接下来建立一次TCP连接来获取和发送资源，这一次HTTP连接会进行三次握手

司

4. HTTP服务器关闭TCP连接。
5. HTTP客户端接收包含html文件的响应报文，显示html页面。通过解析html文件，定位10个引用的jpeg对象。
6. 针对10个jpeg对象中的每一个，重复步骤1-5。

问题：这个场景中需要进行多少个TCP连接？

- A. 1 B. 10 C. 11 D. 不知道

答案为C

持续HTTP

- ▣ 服务器发送响应后，保持TCP连接的打开状态
- ▣ 在该对客户端/服务器间，后续的HTTP报文，直接通过已开启的TCP连接交互客户端遇到引用对象时，立即向服务器发送请求
- ▣ 连接空闲一段时间后关闭
- ▣ 所有对象仅需1连接、1 RTT

持续连接的非流水线式

假设一共请求10个资源，我们这种方式只需要先进行一次request-respond来接受连接，然后连续进行10次TCP连接获取资源即可，也就是说一共只进行了一次HTTP连接，这里每一次TCP连接需要等待上一次TCP连接结束之后才能进行，因此称为非流水线

- HTTP连接数：1
- TCP连接数：10
- 总时间：22RTT

持续连接的流水线式

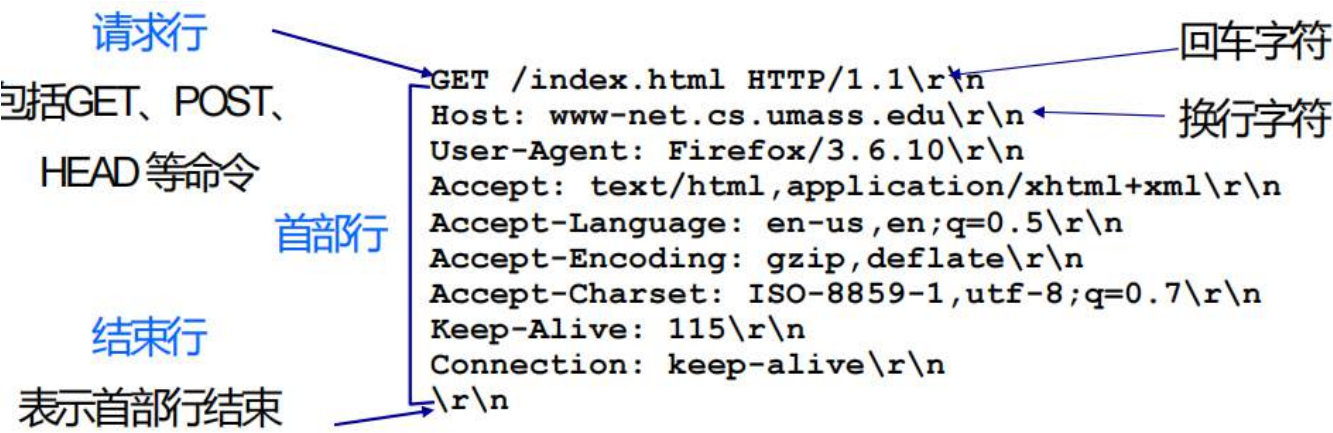
在持续连接的非流水线式的基础上，我们允许上一次TCP连接还没完成的时候开始下一次TCP连接，这样时间可以大大缩短

- HTTP连接数：1
- TCP连接数：10
- 总时间：远小于22RTT

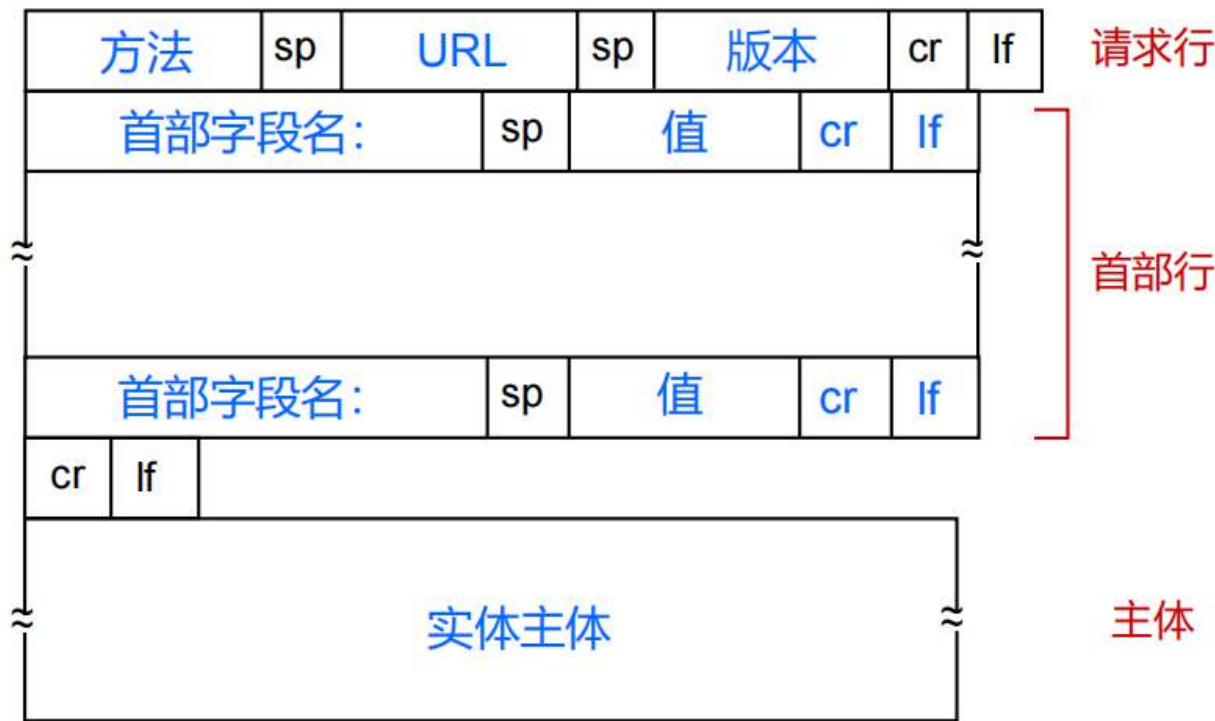
HTTP报文

HTTP报文包括两种类型：请求request、响应response

HTTP请求报文



keep-alive表示持续连接



如何请求？

GET 方法：

- 网页通过统一资源定位（URL），实现信息的获取
- 输入的信息，通过请求行的 URL 字段，上传至服务器

POST 方法：

- 网页通过提交表单功能，实现信息的输入
- 输入的信息，通过实体主体，上传至服务器

GET直接去通过URL获取资源，而POST需要提交一个表单，用来给服务器上传信息并获取回复和资源

HTTP响应报文

状态行：包括协议规定的状态码，以及描述状态的短语

首部行

```
HTTP/1.1 200 OK\r\n
Date: Sun, 26 Sep 2010 20:09:20 GMT\r\n
Server: Apache/2.0.52 (CentOS)\r\n
Last-Modified: Tue, 30 Oct 2007 17:00:02 GMT\r\n
ETag: "17dc6-a5c-bf716880"\r\n
Accept-Ranges: bytes\r\n
Content-Length: 2652\r\n
Keep-Alive: timeout=10, max=100\r\n
Connection: Keep-Alive\r\n
Content-Type: text/html; charset=ISO-8859-1\r\n
\r\n
data data data data data ...
```

数据：例如请求的HTML文件

响应状态码

200 OK

- 请求成功，请求的对象将在后续报文中传输

301 永久移动

- 请求的对象已被永久移动，新位置将在后续报文中指明

400 错误请求

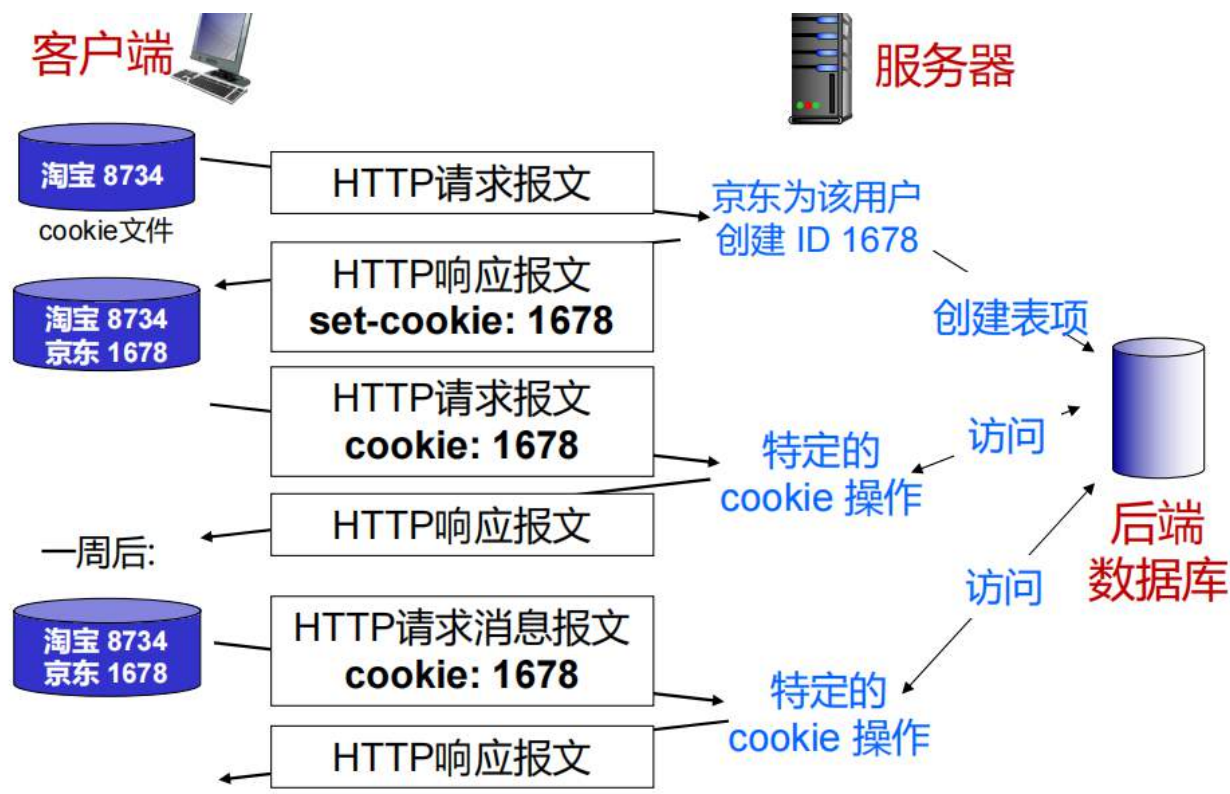
- 服务器无法理解请求信息

404 未找到

- 服务器无法找到请求的文件

505：服务器不支持请求报文的HTTP协议版本

Cookie



核心组件

- 响应头部 (Set-Cookie)：服务器在返回页面时，顺便塞给你一个 ID。
- 请求头部 (Cookie)：你下次访问该网站时，浏览器会自动带上这个 ID。
- 本地存储：你的电脑（浏览器）会把这些 ID 存在一个本地文件中。
- 后端数据库：服务器端记录着 ID 1678 对应的是“韩梅梅”，以及她的购物车里有什么。

交互流程

1. 初次访问：你（客户端）向京东发送请求。此时你没有 Cookie。
2. 分配 ID：京东服务器发现你是新客，生成一个唯一 ID（如 1678），并在 HTTP 响应头里写上 set-cookie: 1678。
3. 本地保存：你的浏览器收到响应，把 京东: 1678 存进本地小本本。
4. 后续识别：之后你再点击京东的其他页面，浏览器会自动在请求头里加上 cookie: 1678。
5. 状态保持：即使过了一周（如最后一步所示），只要 Cookie 没过期，服务器一看 ID 就知道：“哦，是那个 ID 1678 的老客户回来了。”

实际作用与代价

- 会话管理：实现免登录。你不用每点一个页面就输一次密码。
- 个性化：根据你的 ID 推荐你可能喜欢的商品（个性化推荐）。
- 购物车：记录你临时挑选的商品。

但是，它也有隐患：

- 隐私追踪：由于 Cookie 能记录你的行为，站点可以“拼凑”出你的个人画像（姓名、偏好等）。

暮云的随想：

这里我很想把cookie和jwt对比来讨论，这样才能看出来cookie的优劣在哪

先来说一下什么是jwt吧

jwt就是用户在登录之后，服务器会根据算法生成一个token给用户，用户拿到这个token，手动将其放入Header中，将自己的http报文（包含了整个json化的token）传给服务器后，服务器会根据这个token先判断是否有效，如果无效就拒绝请求，这体现了安全性。然后这个token中也包含着用户的id信息来判断用户是谁，然后再进行业务操作

维度	Cookie	JWT (JSON Web Token)
本质	存储机制（HTTP 协议自带的一个本地存储“口袋”）	数据格式（一种经过加密/签名的 JSON “票据”）
状态	通常有状态：服务器存 Session，Cookie 只存一个 ID	无状态：所有用户信息都塞在 Token 里，服务器不存
自动发送	是：浏览器在访问对应域名时会自动带上	否：通常需要代码手动放入 Header（除非存进 Cookie）
大小限制	很小（通常限制在 4KB 以内）	较大（因为它包含了用户数据和签名信息）

在用户第一次登录之后，

如果使用cookie，那么服务器会给用户的浏览器分配一个cookie id，然后服务器会将这个cookie id与账号相绑定存到数据库中（有状态），假设用户在多个浏览器上都进行过登录，那么数据库当中一个用户可能有多个cookie id，这是允许的

如果使用jwt，那么服务器会给用户返回一个token，这个token不会在服务器中与账号绑定（无状态），用户每一次发送任何http请求的时候都需要发送token

维度	传统 Cookie + Session	JWT (可存放在 Cookie 或 Header)
存储压力	服务器要存成千上万个 ID（压力大）	服务器不存，只管校验（压力小）
多端登录	数据库里一条账号对应多个 ID	一个账号可以生成无数个有效的 JWT
安全性控制	强：服务器可以随时踢人下线	弱：一旦发出，除非过期否则很难作废
适用场景	强状态控制（如银行、管理后台）	跨域、移动端、分布式微服务

有无状态的差异体现在，如果用户的cookie泄露，那么用户可以告诉服务器清空该账号对应的所有cookie，因为有数据库，而jwt没有办法撤回，如果别人一旦拥有了jwt，那么他就拥有了你的全部权限

Web缓存

Web 缓存/代理服务器的基本原理

目标：在不重复访问远端“源服务器”的情况下，直接在本地满足用户的请求。

- **角色切换：**代理服务器非常特殊，它既是**服务器**（对客户端而言），又是**客户端**（对源服务器而言）。
 - **核心逻辑：**
 1. 客户端把请求发给代理服务器。
 2. 如果代理服务器缓存了该对象，直接秒回（命中）。
 3. 如果没有，代理服务器去源服务器拿，存一份副本后再给客户端。
 - **为什么要它？**降低响应时延、减少机构接入链路的昂贵流量、分担源服务器压力。
-

缓存性能与链路利用率计算

展示了一个经典的工程计算案例，解释了**为什么“本地代理”比“单纯增加带宽”更划算**。

场景假设：

- 对象大小 100K bits，请求率 15/sec。
- 接入链路带宽：1.54 Mbps。
- 局域网（LAN）带宽：100 Mbps。

计算结论对比：

- **不加缓存：**接入链路的流量强度接近 **1** ($100K \times 15 / 1.54M \approx 0.97$)。由于排队论效应，当利用率接近 1 时，时延会无限增大。
 - **增加带宽：**把链路升级到更高倍率。虽然解决了问题，但经济成本极高**。
 - ****搭建本地代理：**
 - **假设缓存命中率为 0.4：**意味着只有 60% 的请求需要走昂贵的接入链路。
 - **计算利用率：**新的流量强度变为 $0.6 \times 0.97 \approx 0.58$ 。
 - **总时延：** $0.6 \times (\text{源站延迟 } 2s) + 0.4 \times (\text{极小的局域网延迟}) \approx 1.2s$ 。
 - **结论：**低成本实现了比高昂带宽更好的性能。
-

条件 GET

这是为了解决“缓存一致性”问题：**如果源服务器上的文件更新了，缓存服务器怎么知道？**

- **核心首部行：**`If-modified-since: <date>`。
- **交互流程：**
 1. **询问：**代理服务器在请求时带上自己手里副本的日期。
 2. **未修改 (304 Not Modified)：**如果源服务器发现文件没变，只回一个状态码，**不发数据主体**。这节省了大量的带宽。
 3. **已修改 (200 OK)：**如果文件变了，源服务器直接发送新的数据。

FTP

是一种文件传输协议，实现本地到远程主机间文件的上传和下载

核心特征：带外控制

控制信息（如“停止传输”命令）和数据流不在同一个通道里。这就像你在修水管（数据连接）的同时，手里还有一个对讲机（控制连接）指挥开关。即使水管堵了，你的对讲机依然能传达指令。



- ftp标准：RFC 959
- FTP客户端基于TCP协议，控制连接FTP服务器的21端口
 - 由控制连接获得授权
 - 在控制连接发送控制命令，浏览远程主机的目录
- 服务器收到文件传输命令后开启第2个TCP连接，即数据连接，控制20端口，用于文件传输。文件传输结束后，服务器关闭该连接

FTP命令和响应

命令示例：

- ▣ 在控制信道上，以ASCII文本形式发送

- `USER username`
- `PASS password`
- `LIST`
返回当前目录的文件列表
- `RETR filename`
获取文件
- `STOR filename`
将文件存到远程主机

返回码示例：

- ▣ 状态码和状态短语

- 331 Username OK, password required
- 125 data connection already open; transfer starting
- 425 Can't open data connection
- 452 Error writing file

两种模式

A. 主动模式 (Active Mode / PORT 模式)

1. 客户端开启一个随机端口，通过 21 端口告诉服务器：“我准备好了，请连我的 X 端口”。
 2. 服务器主动从自己的 20 端口发起连接，连向客户端的 X 端口。
- **痛点：**如果客户端在局域网内或有防火墙，服务器“主动”连进来的包会被防火墙拦截。

B. 被动模式 (Passive Mode / PASV 模式)

1. 客户端发送 `PASV` 命令给服务器。
 2. 服务器开启一个随机端口（如 30001），回传给客户端。
 3. 客户端主动发起连接，连向服务器的 30001 端口。
- **优点：**解决了防火墙拦截入站连接的问题，因为现在是客户端“由内向外”连服务器。

电子邮件

组成部分

- **用户代理 (UA, User Agent)：**你手机里的 Outlook、Foxmail 或网页版 Gmail。功能：撰写、阅读、显示。
- **邮件服务器 (Mail Server)：**核心大脑。每个服务器都有一个**发送队列**（存待发的信）和**用户邮箱**（存收到的信）。
- **简单邮件传输协议：**连接上述组件的语言。比如SMTP

用户代理

- 也称“邮件阅读器”
- 撰写、编辑、阅读邮件
- 例如：Outlook、Thunderbird、iPhone mail client
- 收发的报文存在邮件服务器

邮件服务器

- 作用：
 - 邮箱包含用户收到的邮件
 - 即将发送出去的邮件报文形成报文队列

SMTP协议用于邮件服务器之间传送邮件报文

客户端：本质上是一个发送邮件的服务器

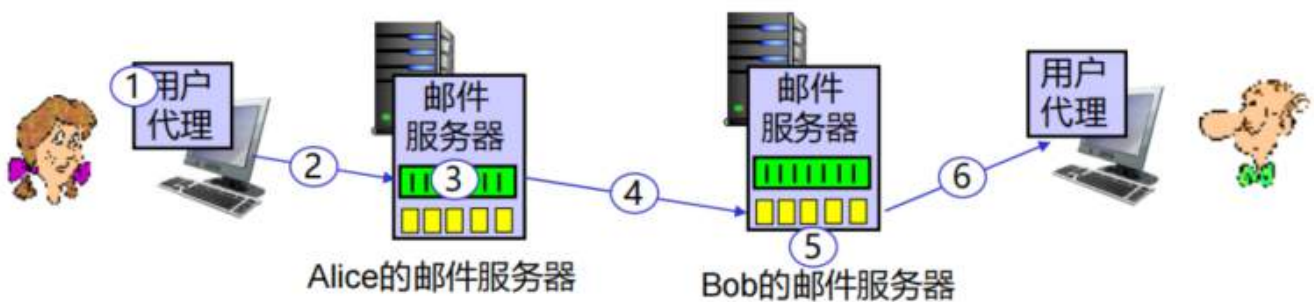
服务端：是一个接收邮件的服务器

SMTP协议

- 使用 TCP 协议将电子邮件从客户端可靠地传输到服务器，典型端口号 25
- 传输的三个阶段：
 - 基于握手建立连接（打招呼）
 - 传输报文
 - 关闭连接
- 直接传输：发送服务器→接收服务器
- 命令/响应交互（类似于HTTP、FTP中）
 - 命令: ASCII文本
 - 响应: 状态码和状态短语
- 报文必须是7位 ASCII码

SMTP邮件推送协议（发）

邮件收发步骤



- 1) Alice使用用户代理编写将发往 bob@some school.edu 的邮件。
- 2) Alice的用户代理向其邮件服务器发送邮件报文，邮件报文被置于报文队列中。
- 3) 在客户端处，SMTP打开与Bob邮件服务器的TCP连接。
- 4) SMTP客户端通过TCP连接发送Alice的邮件报文。
- 5) Bob的邮件服务器将邮件报文置于Bob的邮箱中。
- 6) Bob调用他的用户代理来读取邮件。

注意这里的客户端就是Alice的邮件服务器

SMTP交互

```
S: 220 hamburger.edu
C: HELO crepes.fr
S: 250 Hello crepes.fr, pleased to meet you
C: MAIL FROM: <alice@crepes.fr>
S: 250 alice@crepes.fr... Sender ok
C: RCPT TO: <bob@hamburger.edu>
S: 250 bob@hamburger.edu ... Recipient ok
C: DATA
S: 354 Enter mail, end with "." on a line by itself
C: Do you like ketchup?
C: How about pickles?
C: .
S: 250 Message accepted for delivery
C: QUIT
S: 221 hamburger.edu closing connection
```

SMTP vs HTTP

相似:

- 主机间交互
 - HTTP客户端: 浏览器
 - SMTP客户端: 邮件服务器
- SMTP使用持续连接而HTTP可以选择使用持续连接
- 两者都有 ASCII 命令/响应交互和状态码

不同:

- HTTP: pull (拉取)
SMTP: push (推送)
- HTTP: 每个对象封装在各自的响应报文中
- SMTP: 多个对象通过报文的不同部分发送
- SMTP 要求报文 (头部和主体) 都使用 7bit 的 ASCII 码格式

SMTP 是推协议 (发件人主动把信塞给服务器); HTTP 是拉协议 (客户端主动请求资源)。

报文格式

SMTP：交换邮件报文的协议

RFC 822→5322：定义了文本报文的格式标准

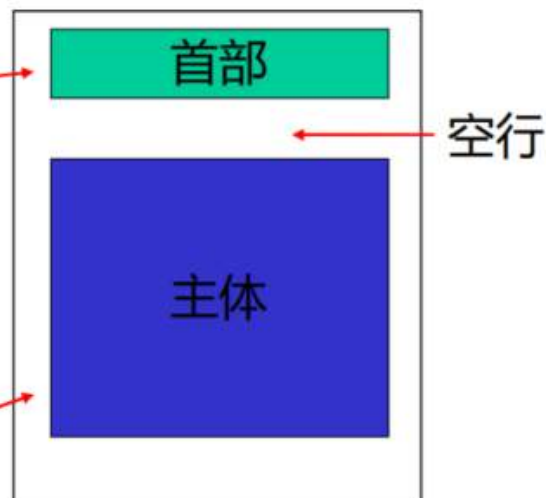
□ 首部行，例如：

- To:
- From:
- Subject:

注意：此处与 SMTP MAIL FROM、RCPT TO 的命令是**不同的**！

□ 主体即 “报文”

- 仅限 ASCII 字符



提升体验：MIME

为什么要 MIME？ 因为早期的 SMTP 只能传文字。

作用：它在邮件头部增加了 `Content-Type`（告诉接收方这是图片、音频还是 PDF）和 `Content-Transfer-Encoding`（通常是 **Base64** 编码，将二进制转为 ASCII）。

```
From: alice@crepes.fr
To: bob@hamburger.edu
Subject: Picture of yummy crepe.
MIME-Version: 1.0
Content-Transfer-Encoding: base64
Content-Type: image/jpeg

base64 encoded data .....
.....
.....base64 encoded data
```

邮件访问协议（收）

了解即可

POP3

- POP3 的“下载并删除”模式
 - 用户如果更换了客户端，那么无法重新阅读邮件（例如，上页B）
- POP3 的“下载并保留”模式
 - 用户在不同客户端，有邮件的副本
- POP3 是跨会话无状态的

IMAP

- 将所有的报文保留在邮件服务器这一个地方
- 允许用户在文件夹中管理邮件
- 跨会话时可以保存用户状态：
 - 文件夹名称，及邮件报文的 ID 和文件夹名称之间的映射关系

POP3

- **模式：**下载并删除（默认）。
- **特点：无状态。**服务器不记得你之前下载过哪些信，也不支持在服务器上创建文件夹同步。

IMAP

- **模式：**在线同步。
- **特点：有状态。**它允许你在服务器上创建文件夹、移动邮件。你在手机上读了信，电脑上也会显示“已读”。

HTTP

- 现在绝大多数人（如 Gmail, QQ邮箱）直接通过浏览器收发邮件，这种情况下，UA 到服务器之间跑的是 **HTTP**，而服务器之间转发依然跑的是 **SMTP**。

DNS

DNS完成的就是IP地址和域名之间的映射服务

DNS核心服务

主机名与 IP 地址间的映射

这是最基本的功能。将人类可读的 FQDN（全限定域名）转换为机器可读的 IP。

主机别名

- **规范名（Canonical Name）：**一个服务器真正的、长长的名字。
- **别名：**为了方便好记起的绰号。例如，一个复杂的后台服务器名字可以映射成简单的 `www.example.com`。
- **邮件服务器别名：**DNS 可以让邮件服务器拥有专门的名字（MX 记录），确保邮件能精准投递到目标邮箱主机。

负载均衡

这是大型网站（如淘宝、京东）保持流畅的关键：

- **冗余机制**：一个域名可以对应 **多个不同的 IP 地址**。
- **原理**：当成千上万的人访问同一个域名时，DNS 会轮流给出不同的 IP。这样压力就会被分散到不同的服务器上，而不是全堆在一台机器上。

DNS的实现

实现方式

DNS是一个应用层协议

- **网络“边缘”的复杂性**：DNS 被设计在**应用层**，这意味着它不依赖于网络底层的改变，而是通过运行在终端主机上的软件（如浏览器、DNS 解析器）来交互。
- **核心功能**：实现地址与域名的“翻译”。它就像互联网的地图导航，虽然它不是路，但没它你哪儿也去不了。

为什么不用中心式系统？

图中给出的结论是“**可扩展能力欠佳**”。如果全世界的域名查询都涌向同一台（或同一组）中心服务器，会发生什么？

单点故障 (Single Point of Failure)

- 如果中心服务器宕机，**全球互联网都会瘫痪**。你无法打开任何网页，因为你的电脑找不到它们的 IP 地址。

通信容量 (Traffic Volume)

- 全世界每秒有数以亿计的 DNS 请求。没有任何单一的带宽和处理器能承受这种级别的瞬间压力。

中心式数据库距离远 (Distant Centralized Database)

- 光速是有上限的。如果你在中国访问西工大官网，却要跨越太平洋去美国的中心服务器查询 IP，延迟会非常高。

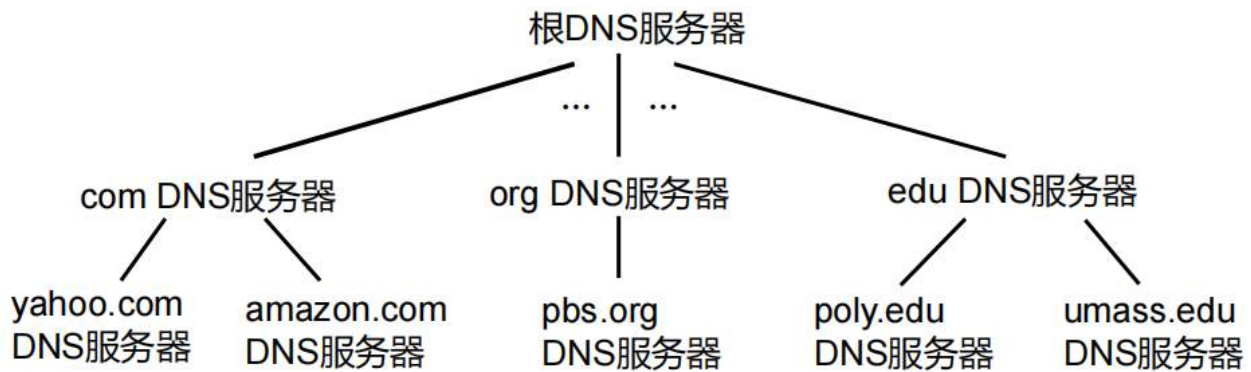
易于维护 (Maintenance)

- 中心式系统维护起来极其臃肿。如果每天有几百万个新域名要注册，中心数据库的更新压力和验证逻辑将变得不可控。

分布式层次化数据库

为了解决上述问题，DNS 采用了**按层次构成**的分布式架构（如前一张图提到的根服务器 -> TLD 服务器 -> 权威服务器）。

- **本地化**：你学校或小区的 DNS 服务器会**缓存**常用的记录。比如，第一个同学查了百度，第二个同学再查时，本地服务器直接给答案，根本不需要去问外面的服务器。
- **高可用**：即便某个地方的子服务器挂了，只会影响局部，而不会让整个网络瘫痪。



客户端要获取www.amazon.com的IP地址:

- ▣ 客户端查询根DNS服务器，得到com的DNS服务器地址
- ▣ 客户端查询顶级域DNS服务器，得到amazon.com的DNS服务器地址
- ▣ 客户端查询权威DNS服务器，得到www.amazon.com的IP地址

我们在查找某个域名的时候，通常会直接访问根DNS服务器，然后自顶向下

比如说，我们的nwpua335.online的域名挂在cloudflare下，那么最终查找到我们的域名就需要到这个cloudflare的权威DNS服务器下面才能找到

下面是分布式层次化的具体结构：

根 DNS 服务器 (Root)

▣ 当名字映射未知时，根域名服务器：

- 查询权威域名服务器中的信息
- 获取名字映射
- 返回映射信息至本地域名服务器



遍布世界各地的13个根域名服务器
(标号从A到M)

- **动作：**用户的电脑（通过本地域名服务器）首先去问全球只有 13 组的“根”。

- **回答：**根服务器并不知道你的具体 IP，但它看你的后缀是 `.online`，于是它说：“去问负责 `.online` 的顶级域服务器吧，地址是 `x.x.x.x`。”

顶级域 DNS 服务器 (TLD)

一般由大型域名服务器维护，比如.com之类的

- **动作：**去问 `.online` 的服务器。
- **回答：**服务器查表发现 `nwpua335.online` 这个域名已经交给了 **Cloudflare** 托管。
- **关键点：**它会返回 Cloudflare 分配给你的那两个专属 DNS 地址（例如：`dash.ns.cloudflare.com`）。

Cloudflare 名称服务器 Cloudflare 上的每个 DNS 区域都会被分配一组 Cloudflare 品牌名称服务器。	
类型	值
NS	javier.ns.cloudflare.com
NS	tina.ns.cloudflare.com

比如说，cloudflare就给我分配了两个专属的DNS地址

权威 DNS 服务器 (Cloudflare)

一般由大型的运营服务提供商维护

- **动作：**最后去问 Cloudflare 的服务器：“`www.example.com` 对应的 IP 是多少？”
- **回答：**Cloudflare 检查你在后台设置的 **A 记录**，给出最终的 IP 地址（或者 Cloudflare 节点的 IP）。

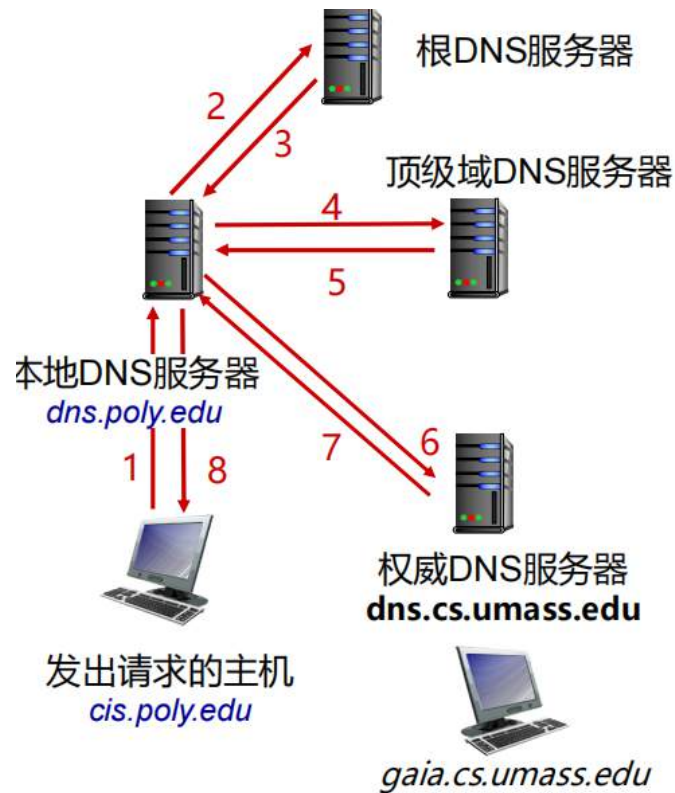
本地DNS服务器

- 严格来讲，不属于DNS服务器的层级结构
- 每个互联网服务提供商（住宅的、公司的、大学的）都有一个本地DNS服务器
也被称为“默认域名服务器”
- 当主机查询DNS时，查询请求先到它的本地域名服务器
 - 本地缓存中有域名-地址的映射（但是可能已经过时）
 - 作为代理将查询请求转发到层次结构中的DNS服务器

报文交互

迭代查询

- 直连的服务器回复需要联系的服务器域名
- “我不知道这个域名, 但是可以问这个服务器”

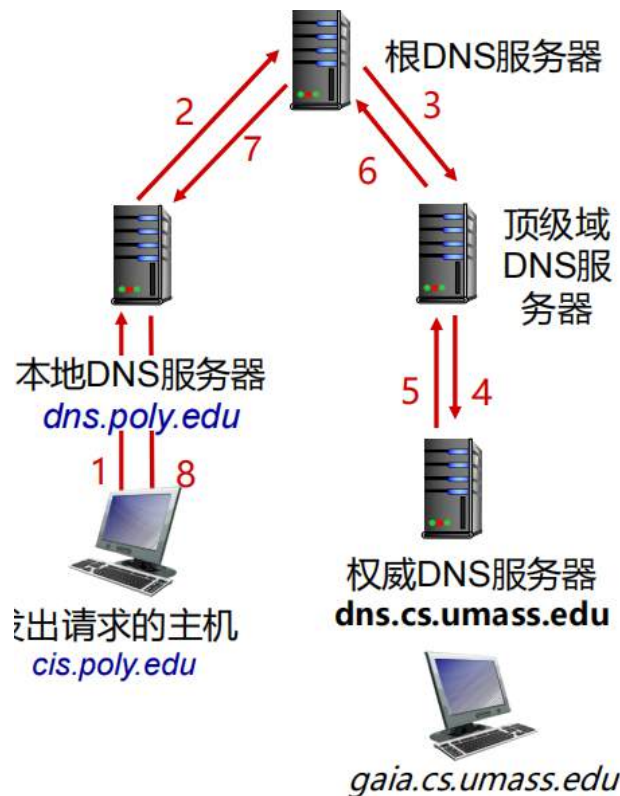


当你输入一个域名时, 你的电脑并不会直接知道 IP, 而是会经历一个“逐级打听”的过程:

- **本地缓存/本地域名服务器**: 首先检查你自己电脑里的记录。如果没有, 就问学校或运营商的 本地DNS 服务器。
- **根域名服务器 (Root)**: 如果本地域名服务器不知道, 它会去问“根”, 根会告诉它去哪里找 `.cn`。
- **顶级域名服务器 (TLD)**: 去 `.cn` 服务器打听, 它会指引你找到 `.edu.cn` 的管理机构。
- **权威域名服务器**: 最后, 西工大 (NWPU) 自己的服务器会给出最终答案: `www.nwpu.edu.cn` 对应的 32位 IP 地址。

递归查询

- 将域名解析的任务交给直连的域名服务器
- 分层结构中上层的域名服务器**负担加重**



当你的电脑向 DNS 服务器发起递归查询时，它表达的意思是：“我只问你这一次，你必须直接告诉我最终 IP，中间怎么打听我不管。”

1. 用户主机 → 本地域名服务器：

你对本地域名服务器（如学校或运营商的 DNS）说：“帮我查一下 `www.nwpu.edu.cn` 的 IP，不许给我中间地址，必须给我结果！”

2. 本地域名服务器 → 根域名服务器（Root）：

本地域名服务器此时承担了全部压力。它转头问“根”：“大哥，`www.nwpu.edu.cn` 是多少？”（在纯递归环境下，根服务器不能只给线索，它得替本地服务器去问下一级）。

3. 根域名服务器 → 顶级域名服务器（TLD）：

“根”发现自己也不知道，于是它替你去问 `.cn` 的服务器：“兄弟，帮我查查这个西工大的地址。”

4. 顶级域名服务器 → 权威域名服务器：

`.cn` 服务器再替根服务器去问西工大（NWPU）的权威服务器：“老弟，你家 `www` 主机的 IP 是多少？赶紧报上来。”

5. 最终答案的原路返回：

权威服务器把 IP 给 TLD，TLD 给根，根给本地 DNS，最后本地 DNS 把这个 32 位 IP 拍在你的电脑桌上：“呐，你要的答案，拿走不谢。”

缓存、更新记录

□ 任何域名服务器一旦得知了某个映射, 就将其缓存

- 如果有相同的查询请求, 则立即返回IP地址
- 本地域名服务器通常缓存着顶级域名服务器地址, 这样的话, 根域名服务器就不会被频繁访问

□ 时效性

- 缓存的表项在一段时间(TTL)后超时或消失
- 缓存的表项虽然尽力进行名称、地址的互译, 但是仍然可能已经过期
 - 如果某域名的主机更改了IP地址, 则所有表项的TTL过期前, 互联网内仍然可能有设备不知情

□ IETF标准组提出更新/通知机制

- RFC 2136

我们在更新或添加DNS记录的时候, 有时候需要等几分钟, 才能正常查询该域名的ip, 原因就是有TTL (也就是缓存存活时间), 没有死之前我们是找不到现在的新记录的, 还没有同步

关于查询时间

完成一次简单的访问单一web资源需要花费多少RTT(Round-Trip Time,往返时延)?

1. 假设本地服务器 (Recursive Resolver) 有缓存

总计: 2 个 RTT

- DNS 阶段 (1 RTT):

- RR: (nwpua335.online、61.128.194.38、A、自动)
- 解释: 客户端向本地 DNS 发起递归查询。因为本地服务器有缓存, 它直接返回 A 记录。由于 UDP 通常是非连接的, 这一步消耗 1 个 RTT。

- Web 服务阶段 (1 RTT):

- 解释: 客户端拿到 IP 后, 发起 HTTP 请求。在持久连接 (Persistent Connection) 或已握手的前提下, 获取单一资源需要 1 个 RTT。
- 注: 如果是首次建立连接, 通常还需要额外的 TCP 三次握手 RTT。

2. 假设本地服务器没有缓存 (迭代查询)

总计: 5 个 RTT (4 个给 DNS, 1 个给 Web)

在这种情况下, 本地 DNS 需要代替客户端跑腿, 依次询问各个层级的权威服务器。

DNS 迭代阶段 (4 个 RTT)

- **Root Server (RTT 1):**
 - **NS 记录:** `RR:(online、a.nic.online、NS、86400)`
 - **解释:** 本地 DNS 询问根服务器, 获取 `.online` 顶级域服务器的地址。
- **TLD Server (RTT 2):**
 - **NS 记录:** `RR:(nwpua335.online、nice.dnspod.net、NS、86400)`
 - **解释:** 本地 DNS 询问顶级域服务器, 获取负责 `nwpua335.online` 的权威服务器 (DNSPod) 地址。
- **Authoritative Server (RTT 3):**
 - **CNAME 记录:** `RR:(alist、nwpua335.online、CNAME、自动)`
 - **解释:** 询问权威服务器。服务器发现 `alist` 是别名, 返回**规范名 (Value)**。
- **Authoritative Server (RTT 4):**
 - **A 记录:** `RR:(nwpua335.online、61.128.194.38、A、自动)`
 - **解释:** 本地 DNS 拿着规范名再次请求 (或在同级缓存寻找), 最终获取 **A 记录** 的物理 IP 并返回给客户端。

Web 服务阶段 (1 个 RTT)

- **HTTP GET (RTT 5):**
 - **解释:** 客户端终于拿到了 IP, 发送请求并接收 Web 资源 (假设 TCP 连接已复用或不计入握手)。

3. 如果不考虑“简单”, 实际可能更多

在真实的现代网络环境中, RTT 往往比 5 个更多:

- **TCP 握手 (+1 RTT):** 如果是冷启动, 必须先进行三次握手。
- **TLS 握手 (+1~2 RTT):** 如果是 HTTPS 访问, 加密协议的握手会增加额外的往返。
- **重定向 (+N RTT):** 如果 Web 服务器返回 301/302 重定向, 流程会再次循环。

DNS记录

DNS是存储着资源记录 (RR) 的分布式数据库

`RR:(name、value、type、TTL)`

DNS：存储着资源记录（RR）的分布式数据库

RR格式:(name、value、type、TTL)

type=A

- name是主机名
- value是IP地址

type=NS

- name是域名
 - 例如，baidu.com
- value是该域的权威域名服务器的主机名

type=CNAME

- name是别名
- value是规范名称
 - 例如www.ibm.com其实是servereast.backup2.ibm.com

type=MX

- 与以上类似，但用于邮件服务器

把 **A 记录** 比作“名字 → 家庭住址”，把 **CNAME** 就是“绰号 → 名字”

管理 nwpua335.online 的 DNS				DNS 设置: 完全 ① 导入和导出 ▼ 仪表盘显示设置		
查看、添加和编辑 DNS 记录。编辑将在保存后生效。						
搜索 DNS 记录						
添加筛选器	Q		搜索	添加记录		
☐	类型 ①	名称 ①	内容 ①	代理状态 ①	TTL ①	操作
☐	A	nwpua335.online	61.128.194.38	仅 DNS	自动	编辑 ▶
☐	A	store	192.0.2.1	已代理	自动	编辑 ▶
☐	CNAME	alist	nwpua335.online	仅 DNS	自动	编辑 ▶
☐	Tunnel	mc	muyunisland	已代理	自动	编辑 ▶
☐	Tunnel	penguin	muyunisland	已代理	自动	编辑 ▶
☐	NS	nwpua335.online	nice.dnspod.net	仅 DNS	自动	编辑 ▶
☐	NS	nwpua335.online	karate.dnspod.net	仅 DNS	自动	编辑 ▶
☐	SRV	_minecraft_tcp.be...	0 5 55555 nwpua335.... 0	仅 DNS	自动	编辑 ▶
☐	SRV	_minecraft_tcp.mc	0 5 45448 nwpua335.... 0	仅 DNS	自动	编辑 ▶

这里来解释一下每个记录，按照RR:(name、value、type、TTL)的格式

A 记录： RR:(nwpua335.online、61.128.194.38、A、自动)

- 解释：**这是根域名解析。它直接将主域名指向服务器的公网 IP 地址。当你直接输入 nwpua335.online 访问时，DNS 协议会直接返回这个 IPv4 地址。

CNAME 记录： RR:(alist、nwpua335.online、CNAME、自动)

- 解释：**这是别名记录。在这里，alist 是别名，而 nwpua335.online 就是教材里说的规范名 (Value)。它的逻辑是：访问 alist 时，先跳转到 nwpua335.online，再由后者带路去找最终 IP。这样当你修改主域名 IP 时，alist 会自动跟随。

NS 记录： RR:(nwpua335.online、nice.dnspod.net、NS、自动)

- 解释：**这是域名服务器记录。它告诉全球互联网：nwpua335.online 这个域名的解析权力交给了 DNSPod 的服务器 nice.dnspod.net。它是整个解析链条的“总管”。

NS 记录： RR:(nwpua335.online、karate.dnspod.net、NS、自动)

- **解释：** 这是**备用域名服务器记录**。为了防止单点故障，NS 记录通常成对出现。如果第一个 nice 没响应，系统会请求 karate 节点来确保你的网站和游戏服务能被搜到。

这里以我们的nwpua335.online举例

在一个完整的 DNS **递归查询**过程中，会出现什么RR记录？

1. Root Server

当本地 DNS 询问根服务器“.online 在哪？”时：

- **NS 记录：** RR:(online、a.nic.online、NS、86400)
 - **解释：** 根服务器不直接给 IP，而是返回负责 .online 顶级域的**权威服务器地址**。
- **A/AAAA 记录 (Glue Record)：** RR:(a.nic.online、37.209.192.7、A、86400)
 - **解释：** 所谓的“粘合记录”。为了防止循环查询（如果你不知道服务器的 IP 就没法访问服务器），根委会直接给出这些 NS 服务器的 IP。

2. TLD Server

当本地 DNS 询问 .online 服务器“nwpua335.online 在哪？”时：

- **NS 记录：** RR:(nwpua335.online、nice.dnspod.net、NS、86400)
- **NS 记录：** RR:(nwpua335.online、karate.dnspod.net、NS、86400)

3. Authoritative Server

当本地 DNS 询问 nice.dnspod.net “alist.nwpua335.online 的 IP 是多少？”时：

- **CNAME 记录：** RR:(alist、nwpua335.online、CNAME、自动)
 - **解释：** 权威服务器发现这是个别名，返回**规范名 (Value)**。此时本地 DNS 会拿着这个规范名**重新开始**一轮查询（或在同级查找）。
- **A 记录：** RR:(nwpua335.online、61.128.194.38、A、自动)
 - **解释：** 最终目标！返回对应的 **IPv4 地址**。

在一个完整的 DNS **迭代查询**过程中，会出现什么RR记录？

1. Root Server

当本地 DNS 询问根服务器 “.online 在哪？” 时：

- **NS 记录：** RR:(online、a.nic.online、NS、86400)
 - **解释：** 根服务器不直接提供具体 IP，而是返回负责管理 .online 顶级域（TLD）的权威服务器域名。
- **A/AAAA 记录 (Glue Record)：** RR:(a.nic.online、37.209.192.7、A、86400)
 - **解释：** 所谓的“粘合记录”。为了打破“找 IP 需要域名，找域名需要 IP”的死循环，根委会直接给出这些 TLD 服务器的物理 IP 地址。

2. TLD Server

当本地 DNS 询问 `.online` 服务器 “`nwpua335.online` 在哪？” 时：

- **NS 记录：** `RR:(nwpua335.online、nice.dnspod.net、NS、86400)`
- **NS 记录：** `RR:(nwpua335.online、karate.dnspod.net、NS、86400)`
 - **解释：** 顶级域服务器告知该域名的解析权已经委派（Delegate）给了第三方权威解析商（如 DNSPod）。

3. Authoritative Server

当本地 DNS 询问 `nice.dnspod.net` “`alist.nwpua335.online` 的 IP 是多少？” 时：

- **CNAME 记录：** `RR:(alist、nwpua335.online、CNAME、自动)`
 - **解释：** 权威服务器发现这是一个别名，返回**规范名（Value）**。此时本地 DNS 会识别出这不是最终 IP，并转而去查询该规范名对应的记录。
- **A 记录：** `RR:(nwpua335.online、61.128.194.38、A、自动)`
 - **解释：** 最终目标！返回对应的 IPv4 地址。至此，递归查询的“找人”环节结束。

报文格式

分为查询和应答报文，两者的格式相同

报文首部

- **标识号：** 16比特数字#
， 查询报文和相应的应答报文有相同的#

- **标志位：**
 - 查询还是应答
 - 期望递归
 - 可以递归
 - 授权应答

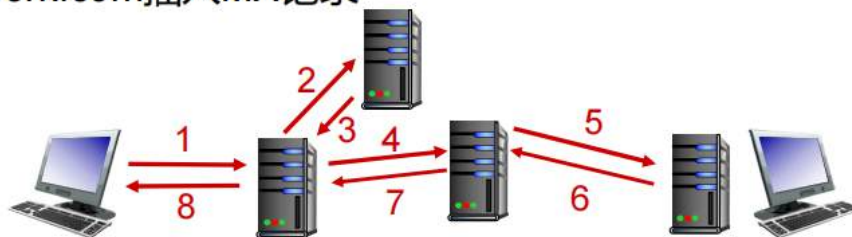
2 字节		2 字节	
标识号		标志位	
问题的数量		回答RR的数量	
授权RR的数量		附加RR的数量	
问题信息（可变数量的问题）			
回答信息（可变数量的RR）			
授权信息（可变数量的RR）			
附加信息（可变数量的RR）			

上面的三行都是首部



怎么注册一个域名？

- 新成立了“计算机网络”公司
- 在**DNS注册商** (由ICANN认证) 处注册名字computernetwork.com
 - 提供主要、次要权威域名服务器的名称、IP地址
 - 注册商将以下2个RR插入到.com顶级域服务器
(computernetwork.com, dns1. computernetwork.com, NS)
(dns1. computernetwork.com, 212.212.212.1, A)
- 创建权威域名服务器记录
 - 为www.computernetwork.com插入A记录;
 - 为computernetwork.com插入MX记录
- 解析域名



先向DNS注册商申请后，顶级域服务器会把你申请成功的名字插入两条记录(NS,A),然后你就可以去权威域名服务器添加新纪录，让权威域名服务器为你分配该服务器的两个NS记录，这样就可以查找到了

传输层协议

DNS（域名系统）主要使用的传输层协议是 **UDP**，但在特定情况下也会切换到 **TCP**。

简单来说，DNS 默认使用 **UDP 53 端口**，因为它追求的是“快”；但在处理大数据或保证可靠性时，它会改用 **TCP 53 端口**。

P2P体系结构

概览

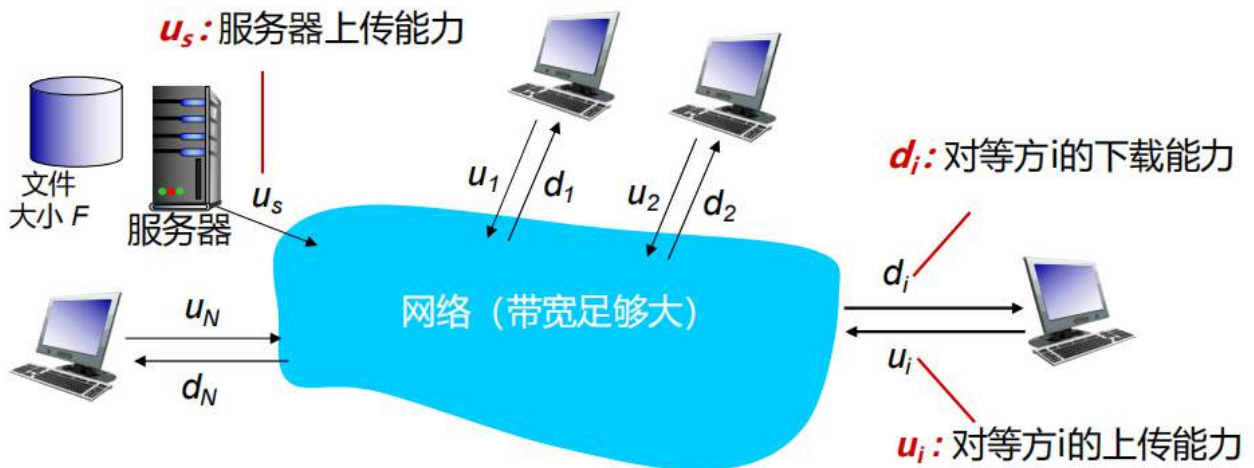
- 没有始终在线的服务器
- 任意终端系统之间直接通信
- 对等方之间进行间歇性连接，IP地址可能发生变动
- 自扩展性

优势

我们来举一个例子，对比C/S架构和P2P架构

问题：将大小为 F 的文件从一台服务器分发到 N 个对等方需要多少时间？

- ▣ 对等方上传/下载带宽是资源受限的



C/S架构

- **服务器发送**：必须依次发送（上传）N 份文件副本

- 发送一份副本的时间： F/u_s
- 发送N份副本的时间： NF/u_s

- **客户端**：每个客户端都需要下载文件副本

- $d_{\min}$ = 客户端下载的最小速率
- 客户端下载最大时间： $F/d_{\min}$



使用C/S方式将文件F分发到N个客户端的时间：

$$D_{C-S} \geq \max\{NF/u_s, F/d_{\min}\}$$

随着N线性增加

P2P架构

- **服务器发送**：必须上传至少一个文件副本

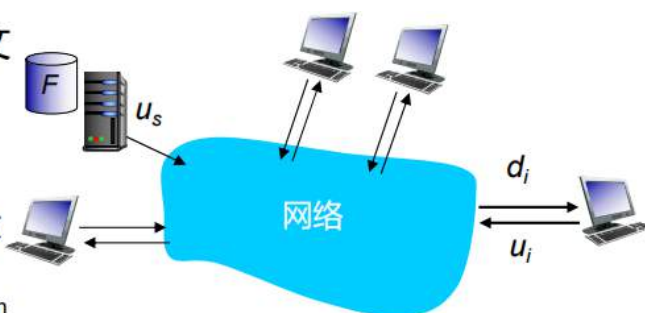
- 发送一个副本的时间： F/u_s

- **客户端**：每个客户端下载文件副本

- 客户端下载的最大时间： $F/d_{\min}$

- **数据量**：全部客户端总的下载量为 NF 比特。

- 总上传速率（限制了下载速率）为 $u_s + \sum u_i$



使用P2P方式将文件F分发到N个客户端的时间：

$$D_{P2P} \geq \max\{F/u_s, F/d_{\min}\}?$$

随着N线性增加

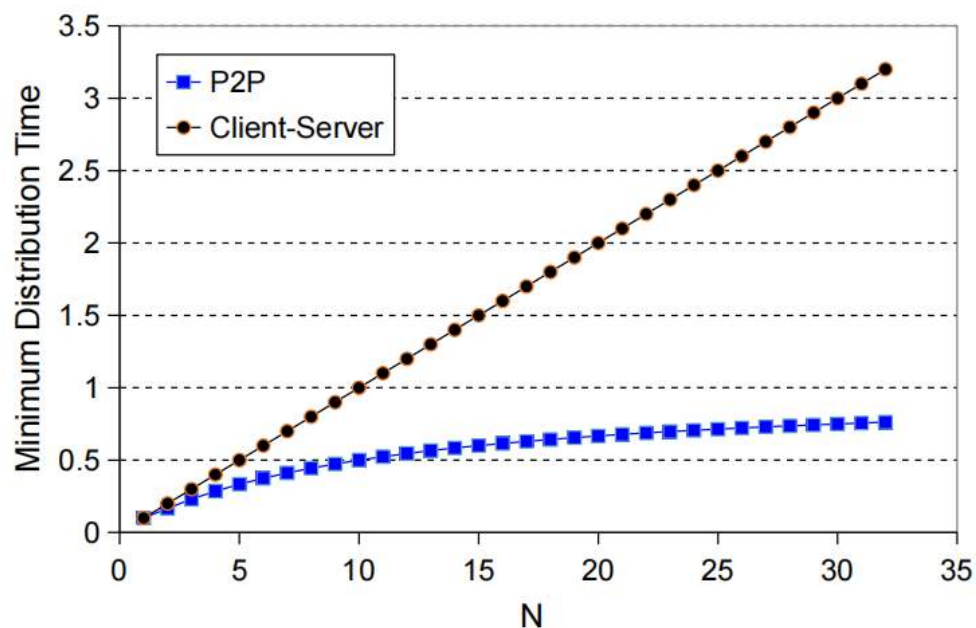
因为每个对等网络都带来了服务能力

$$: \max \left(\frac{NF}{u_s}, \frac{F}{d_{\min}} \right)$$

$$: \max \left(\frac{F}{u_s}, \frac{NF}{u_s + \sum u_i}, \frac{F}{d_{\min}} \right)$$

通常红色下划线的都是最大的，也就是限制其最短时间的部分，我们根据这两个公式计算来比较使用不同方式将文件F分发到N个客户端的时间：

客户端上传速率= u , $F/u = 1$ 小时, $u_s = 10u$, $d_{\min} \geq u_s$



这里假设下载速率大于上传速率

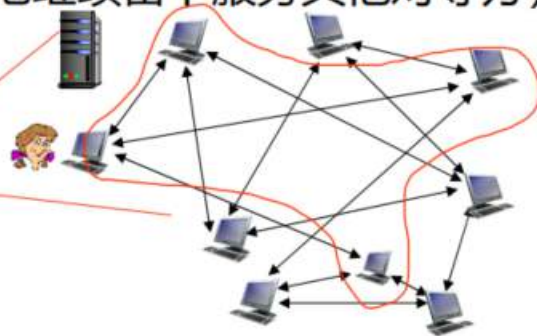
可以发现用户数量越多，C-S架构的时延以线性增长

P2P文件分发：BitTorrent

- 文件被分割为大小为256Kb的文件块
- 对等方间彼此发送/接收这些文件块
 - 对等方在下载的同时，也向其他对等方上传文件块
 - 对等方可能会改变与其交换文件的对等方
 - 对等方可能加入或退出（获得全部文件后，它可能较为自私地离开或者慷慨地继续留下服务其他对等方）

tracker: 跟踪参与torrent的对等方

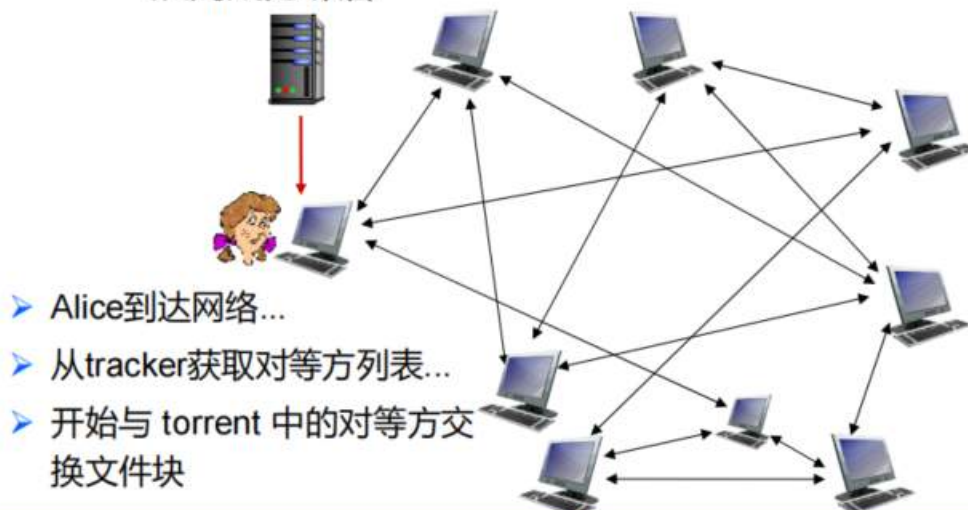
torrent: 交换文件块的对等方小组



本质上来说，就是每个用户在下载的时候，会下载一些文件块，这些文件块可能并不是你需要的东西，但你下载之后，别人可能需要，这时候你就将这些文件块传输给别人，别人也会把你想要的文件块发给你

□ 对等方加入torrent

- 开始时没有文件块，但很快会从其他对等方积累到文件块
- 注册tracker，获得对等方列表，与对等方的一个子集建立连接，从而成为邻居



请求文件块

- 在任何时间，不同的对等方拥有文件块不同的子集
- Alice会周期性地询问每个对等方，索要其拥有的文件块列表
- Alice向其他对等方请求自己没有的文件块，最稀有的优先请求

稀缺优先原则（rarest first）：tracker会实时跟踪每个host拥有的文件包列表，当我所需要的某个文件包被拥有的数量少的时候，这些文件包会优先被传输给我，保证该文件包不会因为某个主机的突然下线在整个网络中丢失

发送文件块

- Alice从向她发送文件块的对等方中，选取速率最高的4个，并将自己的文件块发给它们
 - Alice将禁用其他对等方，不会向它们发送文件块
 - 每10秒重新评估4个对等方
- 每30秒随机选择另一个对等方，并开始向其发送文件块
 - 将该对等方“乐观解禁”
 - 新选的对等方可能成为4个之一

投桃报李原则（tit-for-tat）：Tracker 或对等方会实时评估每个 host 的上传贡献，当 Alice 持续向 Bob 提供数据时，Bob 会将其视为主选对等方并予以等价回传。这种机制确保了网络中的带宽资源优先向贡献者倾斜，通过周期性的“解禁”测试与互利交换，防止“只拿不给”的搭便车行为导致集群传输效率坍塌。

传输层

传输层服务

传输层服务和协议

服务

为运行在不同主机（端系统）上的**应用进程**间提供**逻辑通信**

协议

传输协议在终端系统上运行

- 发送端：将应用层报文分解成报文段，并传至网络层
- 接收端：将报文段重组为应用层报文，并传至应用层

应用程序可用的传输层协议不止一种

互联网：TCP和UDP

传输层和网络层的依赖关系

- 网络层:主机之间的逻辑通信
- 传输层:进程之间的逻辑通信
 - 依赖网络层服务,也是其功能增强

以生活为例

Ann家的12个孩子给Bill家的12个孩子送礼物

- 应用层报文=包装的礼物
- 进程=孩子
- 主机=家庭
- 传输层协议（复用/解复用）= Ann和Bob收集并分发自家兄弟姐妹的礼物
- 网络层协议=邮政服务

网络层协议(IP协议)

- 网络层的协议为IP协议（又称为互联网协议）
- IP协议是一个**不可靠**的协议
 - 不可靠的服务
 - 不保证报文段交付（可能导致丢包）
 - 不保证有序交付（可能导致乱序）
 - 不保证数据的完整性（可能导致出错）
- **数据报**：网络层数据分组的名称

复用和解复用

数据报和报文段的关系

报文段 (Segment)： 传输层发现“你好”太长了（假设），把它切成几块，并在每块前面贴上 **TCP 头**（写明从哪个端口发到哪个端口）。

数据报 (Datagram/Packet)： 网络层把这个 TCP 报文段装进一个更大的信封，贴上 **IP 头**（写明从哪个 IP 发到哪个 IP）。

复用

- **动作：**发生在**源主机（发送端）**。
- **过程：**传输层处理来自多个套接字的数据，为每个数据块打上传输层首部（包括源端口、目的端口等），然后将这些**报文段**统一交给网络层（IP层）发送出去，但其含义并不是把它们打包起来，而是一种表示把所有不同端口号的报文段放到一起统一处理的意思
- **直观理解：**就像一个快递点，把不同街区寄来的包裹全部收集然后添加网络层首部

解复用

- **动作：**发生在**目的主机（接收端）**。
- **过程：**传输层接收到网络层传上来的报文段，根据报文头部的**端口号**等信息，将数据准确地交付到对应的套接字（Socket），从而送达特定的应用进程。
- **直观理解：**货车到达目的地卸货后，快递员根据包裹上的门牌号，把快递准确送到每一户人家。

□ 主机发送报文段：使用**IP地址和端口号**将报文段发送到合适的套接字

□ 主机接收IP数据报

- 每个数据报都有源IP地址、目的IP地址
- 每个数据报携带一个传输层报文段
- 每个报文段都有源端口号、目的端口号



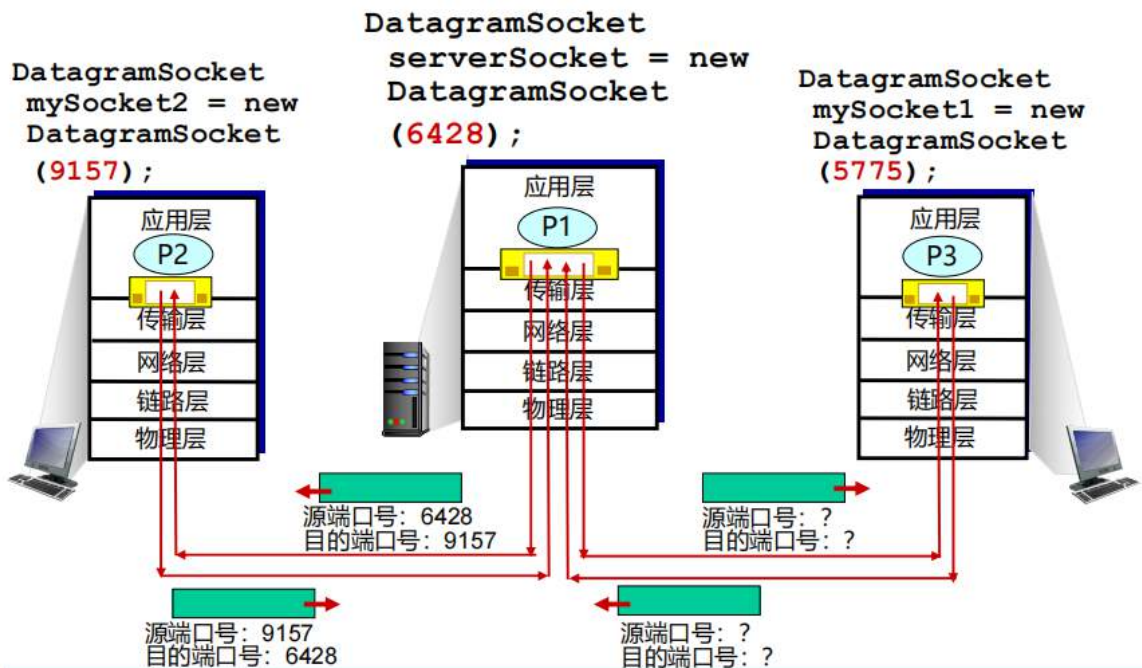
暮云的随想：其实说通俗一点，比如说我要发送一句话，传输层会把这一段数据拆分成多个小段，每一段只添加上源端口和目的端口，这样的小包裹就是报文段，所有的传输层都以报文段作为处理的基本单位，然后这个网络层要收集这些不同端口的全部报文段，这时会把每一个小包裹再放到一个大包裹（数据报）里，在这个大包裹上写上对应的IP地址，然后网络层不需要管里面的端口，只要负责把这个大包裹给对应的快递站就可以了，对方的快递站拿到了之后把大包裹拆开，它能确定这个包裹的目的地一定是这里，因此只需要自己的传输层去处理端口来把快递分配到对应端口就可以了

无连接解复用 (UDP)

“认门不认人”

UDP 的解复用只依赖于**二元组：（目的 IP, 目的端口号）**。

- **多对一的特性：**假设服务器 A 在端口 8888 监听。客户端 B 从端口 1001 发来数据，客户端 C 从端口 2002 发来数据。
 - 对于服务器 A 的传输层来说，只要看到报文头的**目的端口号**是 8888，它就会把这两个报文段都丢给同一个 UDP 套接字。
- **源信息的处理：**虽然解复用时不看源 IP 和源端口，但 UDP 报文头里依然带着它们。应用程序可以通过 `recvfrom()` 等接口获取这些信息，以便知道“这封信是谁寄来的”，从而回信。

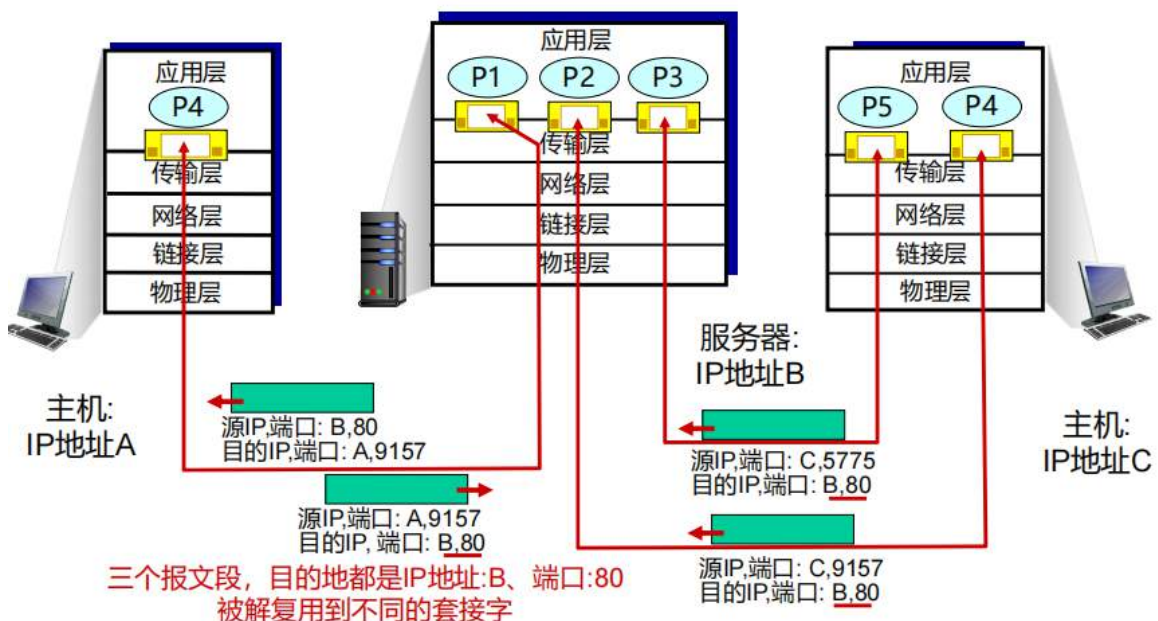


面向连接的解复用 (TCP)

“精准对接”

TCP 的解复用极其严格，它依赖于四元组：(源 IP, 源端口号, 目的 IP, 目的端口号)。

- **一对一的隔离性：** 即便目的 IP 和目的端口完全一样（例如都是访问 Web 服务器的 80 端口），只要源 IP 或源端口中有一个不同，TCP 就会将它们引导至不同的套接字。
- **欢迎套接字与连接套接字：**
 1. **欢迎套接字 (Listening Socket)：** 像酒店大门的迎宾，只负责监听（比如 80 端口）。
 2. **连接套接字 (Connected Socket)：** 一旦三次握手成功，服务器会创建一个新的、专属的套接字来服务这个特定的客户端。
- **结果：** 现代 Web 服务器（如 Nginx/Apache）可以同时与成千上万个客户端通信，每个客户端在服务器内核中都对应一个独立的“私密通道”(Socket)。



传输层区分这两种解复用方式，本质上是在“通信可靠性”与“系统效率”之间做权衡：

TCP 使用“源IP、源端口、目的IP、目的端口”的**四元组**进行解复用，是为了给每个连接创建独立的、互不干扰的逻辑通道，从而实现精准的状态追踪和可靠传输，为了实现重传、流量控制和拥塞控制，服务器必须记住每一个客户端当前的进度；而 **UDP** 仅依赖“目的IP、目的端口”的**二元组**，允许大量客户端共享同一个套接字，省去了维护连接状态的开销，从而支撑极速、高并发的简单交互，数据丢了就丢了。

线程化服务器 (Threaded Server) 是一种并发处理机制，它允许服务器通过创建多个**线程 (Thread)** 来同时处理多个客户端的请求，而不是一次只处理一个。

UDP：无连接的传输

UDP用户数据报协议

UDP: 用户数据报协议[RFC 768]

软件学院 · 计算机网络

- 简单协议：
 - 基本功能
 - 差错检测
 - 提供“尽力而为”服务，UDP报文段可能会：
 - 丢包、错误、无序交付
 - 无连接：
 - UDP发送方、接收方之间没有握手
 - 每个UDP报文段独立处理
 - UDP用于：
 - 流媒体应用程序（实时、容损）
 - DNS, SNMP
- 为什么会有UDP?**
- 不需要建立连接
 - 无建立连接可能增加的时延
 - 无收发方要维持的连接状态
 - 无较长首部带来的复杂分析
 - 无拥塞控制：UDP可以尽快爆炸式传输

SNMP (Simple Network Management Protocol, 简单网络管理协议) 是专门用于 **网络管理** 的应用层协议。

简单来说，它是管理员用来“监控”和“控制”网络设备（如路由器、交换机、服务器、甚至打印机）的通用语言。无论设备是什么品牌，只要支持 SNMP，管理员就可以在办公室里看到它们的实时运行状态。

角色组成

一个 SNMP 管理系统通常由三个核心部分组成：

- **管理站 (NMS, Network Management Station):** 运行管理软件的控制中心（比如你的电脑或监控服务器）。它负责向设备发出查询请求或接收报警。
- **代理 (Agent):** 运行在网络设备（如 NPU 的核心交换机）上的软件模块。它负责收集本设备的 CPU 温度、流量、接口状态等数据。
- **管理信息库 (MIB, Management Information Base):** 一份“数据字典”。它定义了设备中哪些信息是可以被查询的（比如：1.3.6.1.2.1.1.1 代表设备描述）。

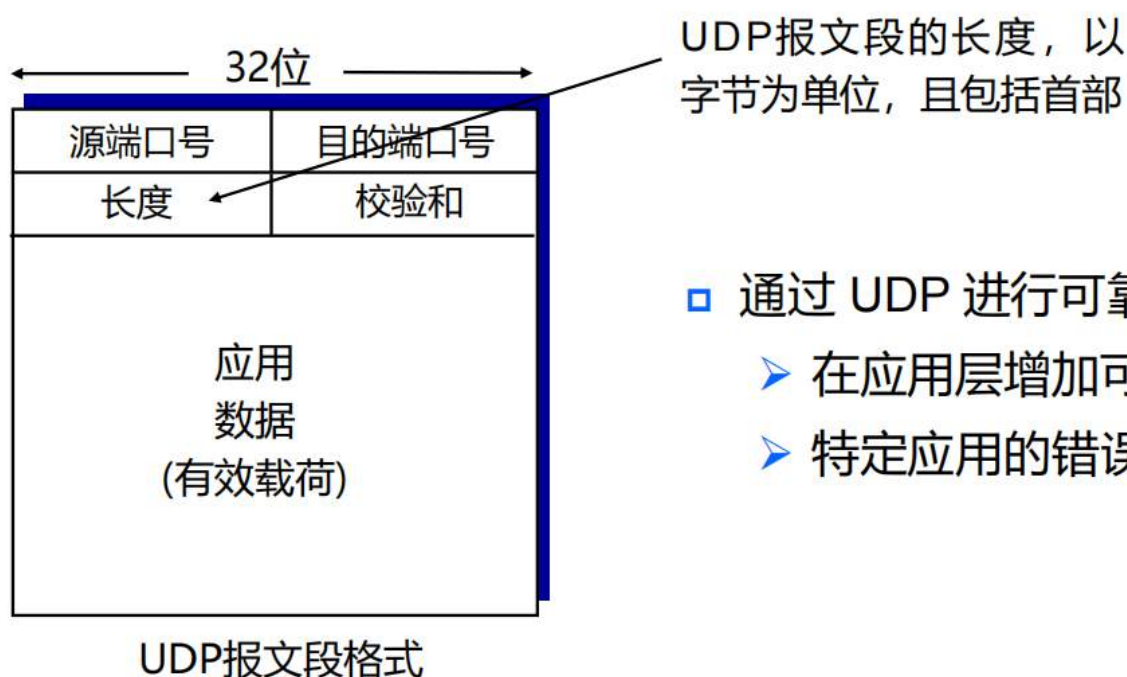
核心操作

SNMP 的操作非常简洁，主要有以下几种：

1. **Get (查询):** NMS 问：“你的 1 号端口流量是多少？”
2. **Set (配置):** NMS 说：“把你的 2 号端口关闭。”
3. **Trap (告警):** 设备主动报信：“不好了！我的风扇停转了！”（这是**非请求**的，设备发现异常立即上报）。
4. **Inform:** 加强版的 Trap，需要 NMS 确认收到，否则设备会重发。

报文段格式

报文段首部



- 通过 UDP 进行可靠传输：
 - 在应用层增加可靠性
 - 特定应用的错误恢复

目标：检测传输报文段中的错误（例如比特翻转）

发送方：

- 将包括首部在内的报文段内容，视为以16比特为一组的序列
- 校验和：报文段的附加内容（实际是补码和）
- 发送方将校验和的值放在UDP的校验和字段

接收方：

- 计算接收报文段的校验和
- 检查计算的校验和是否等于校验和字段的值：
 - 不等：检测到错误。
 - 相等：未检测到错误。（思考：说明没有错误吗？）

实际上来说，我们不能说相等的时候就是没错，只是在一定的范围内未检测到错误

计算

怎么计算两个16bit的加法？

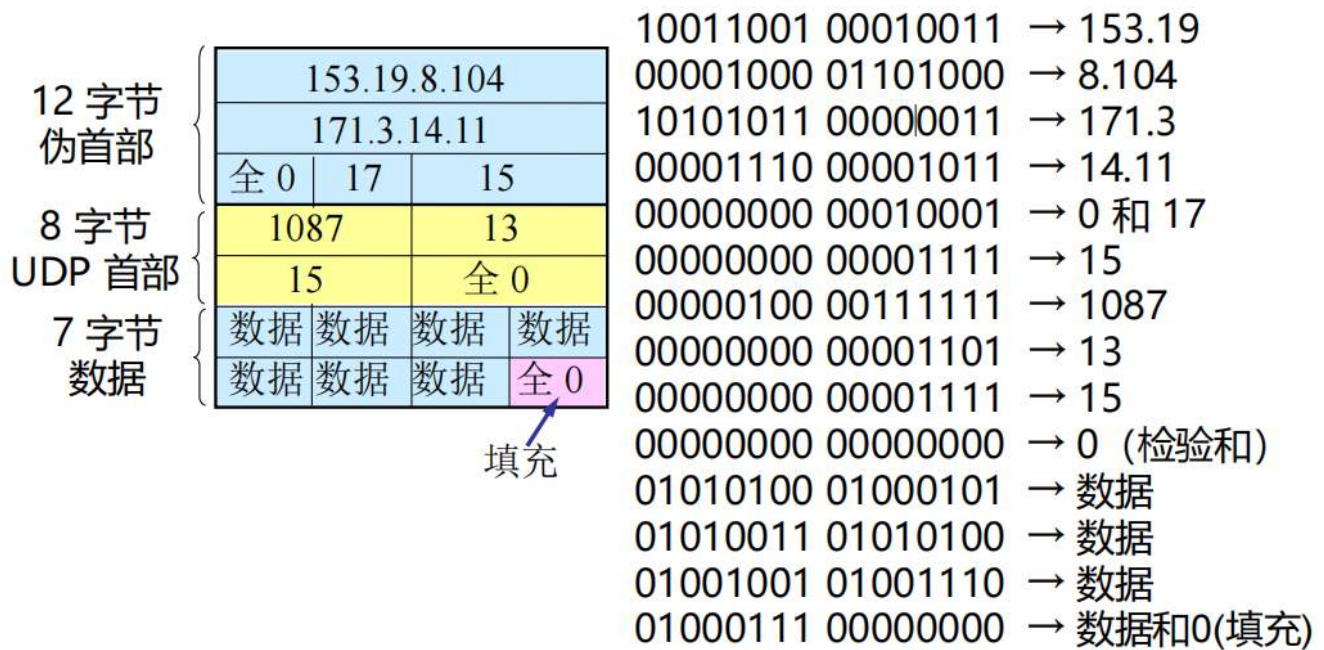
	1	1	1	0	0	1	1	0	0	1	1	0	0	1	1	0
	1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
溢出回卷	1	1	0	1	1	1	0	1	1	1	0	1	1	1	0	1
和	1	0	1	1	1	0	1	1	1	0	1	1	1	1	0	0
校验和	0	1	0	0	0	1	0	0	0	1	0	0	0	0	1	1

溢出回卷就是给最低位加个1即可

校验和是和的反码，校验和是16bit

校验和和和的和是 $2^{16}-1$

实际上的校验和是计算多个16bit的和



按二进制反码运算求和 10010110 11101101 → 求和的结果
 将得出的结果求反码 01101001 00010010 → 校验和

口诀：加反加判溢出回卷

1. 发送方：制作校验和（“加、反”）

发送方需要生成一个“防伪标记”填入报文头。

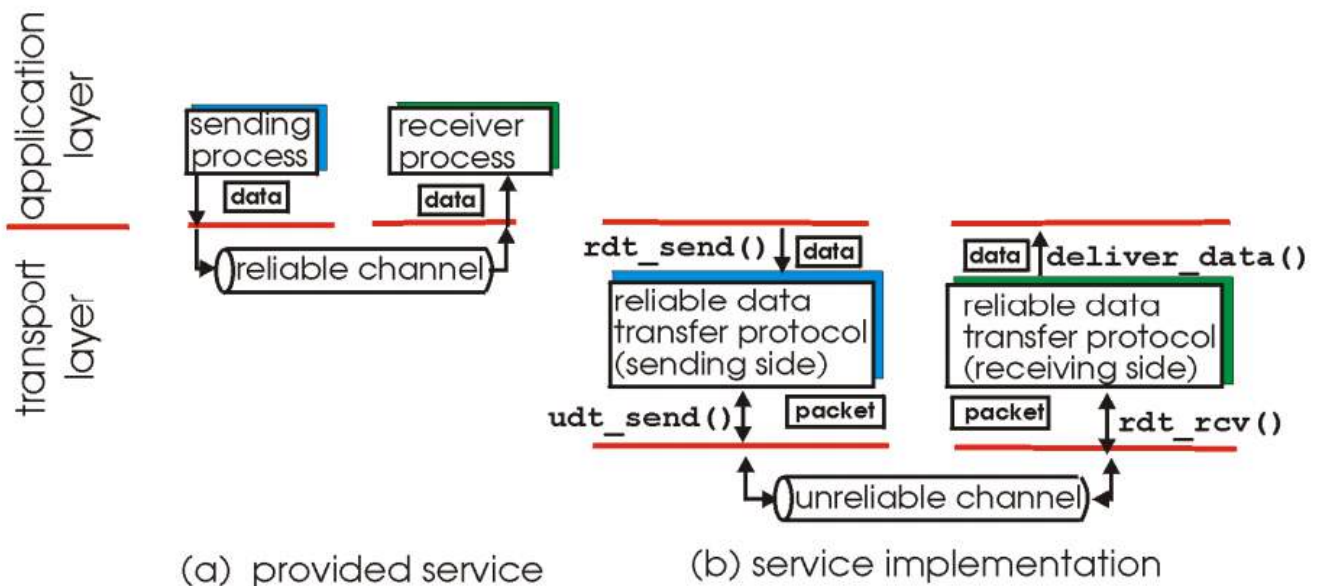
- **加（累加）**：发送方把要传的数据（比如 100 字节）按 16 位一组，全部加起来。如果中间产生了溢出，就执行“回卷”（把最高位进位加到最低位）。
- **反（取反）**：把累加最终的结果按位取反。这个取反后的值，就是最终的 **Checksum**。
- **发送**：数据 + Checksum 一起发走。

2. 接收方：查验真伪（“加、判”）

接收方收到一堆数据和一个 Checksum，它需要确认路上一位都没错。

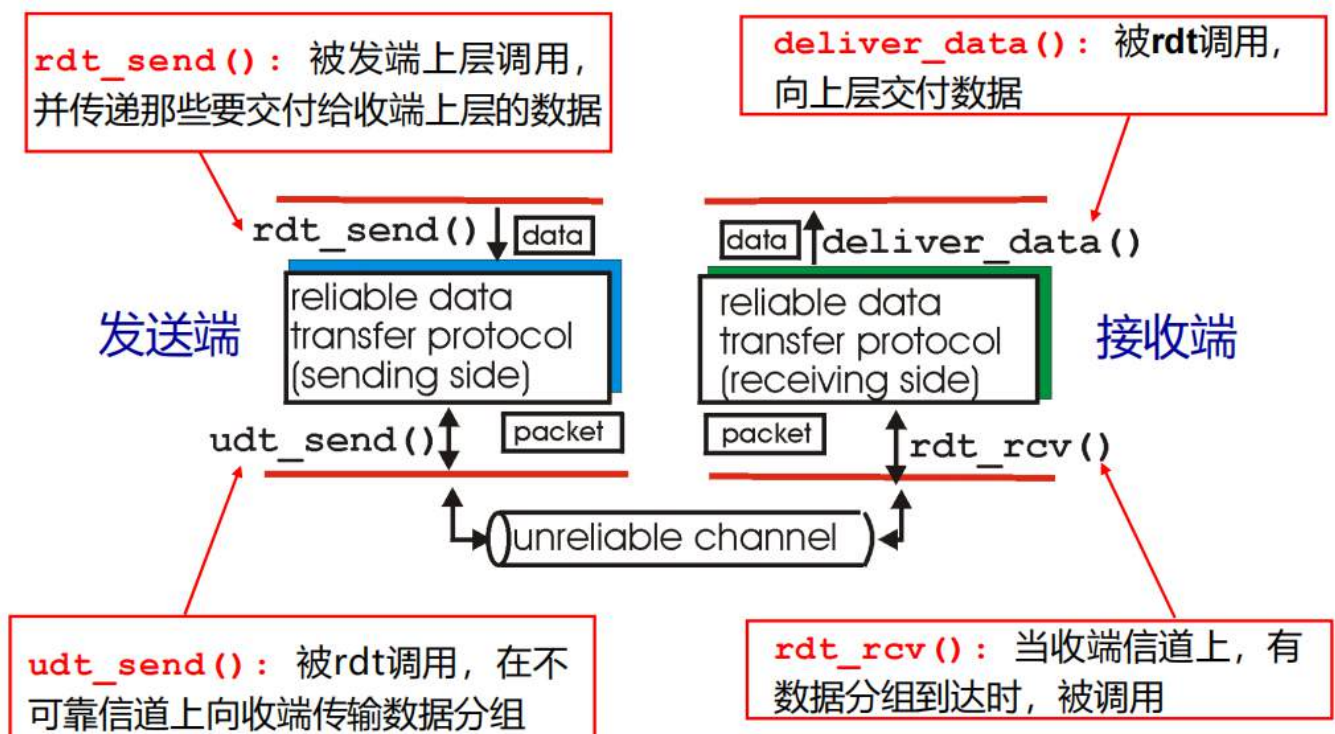
- **加（全家桶累加）**：接收方把收到的**所有数据**（包括数据部分和那个 Checksum 字段）**通通加在一起**。
 - **逻辑**：因为 Checksum 是发送方取反得来的，数学上 $A + \text{not } A = \text{全 } 1$ 。
- **判（判断结果）**：* 如果结果是**全 1**（即 0xFFFF）：说明数据在传输过程中大概率没有变化，校验通过。
 - 如果结果**不是全 1**：说明数据或者 Checksum 本身在路上被电磁干扰“踢”了一脚，变位了。
- **溢出回卷**：接收方在做加法时，同样要遵循“溢出就回卷”的原则，否则算出来的结果对不上。

可靠数据传输的原理



不可靠信道的特性决定了可靠数据传输协议（rdt）的复杂性

因为本质上网络层是不可靠的，因此想实现可靠TCP需要一个rdt协议来保证其可靠性



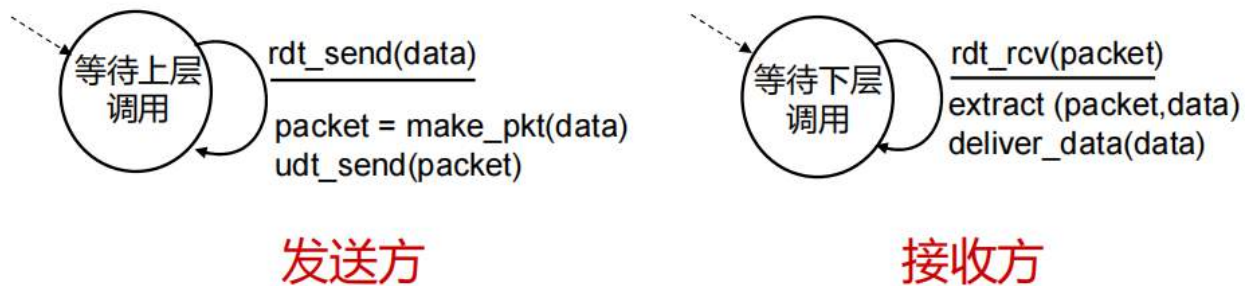
- 有限状态机(FSM): 某一个状态机的状态数量是固定的，不是无限的。其当处于此状态时，下一个状态由下一个事件唯一确定。

停等式协议ARQ

以下rdt1.0到3.0都为停等式

rdt1.0:假设信道完全可靠

- ▣ 底层信道完全可靠
 - 没有比特错误
 - 没有分组丢失
- ▣ 发送方、接收方独立的FSM:



发送方： 只有一个状态“等待上层调用”。一旦有数据（`rdt_send`），它就打包（`make_pkt`）发走（`udt_send`），然后瞬间回到等待状态。

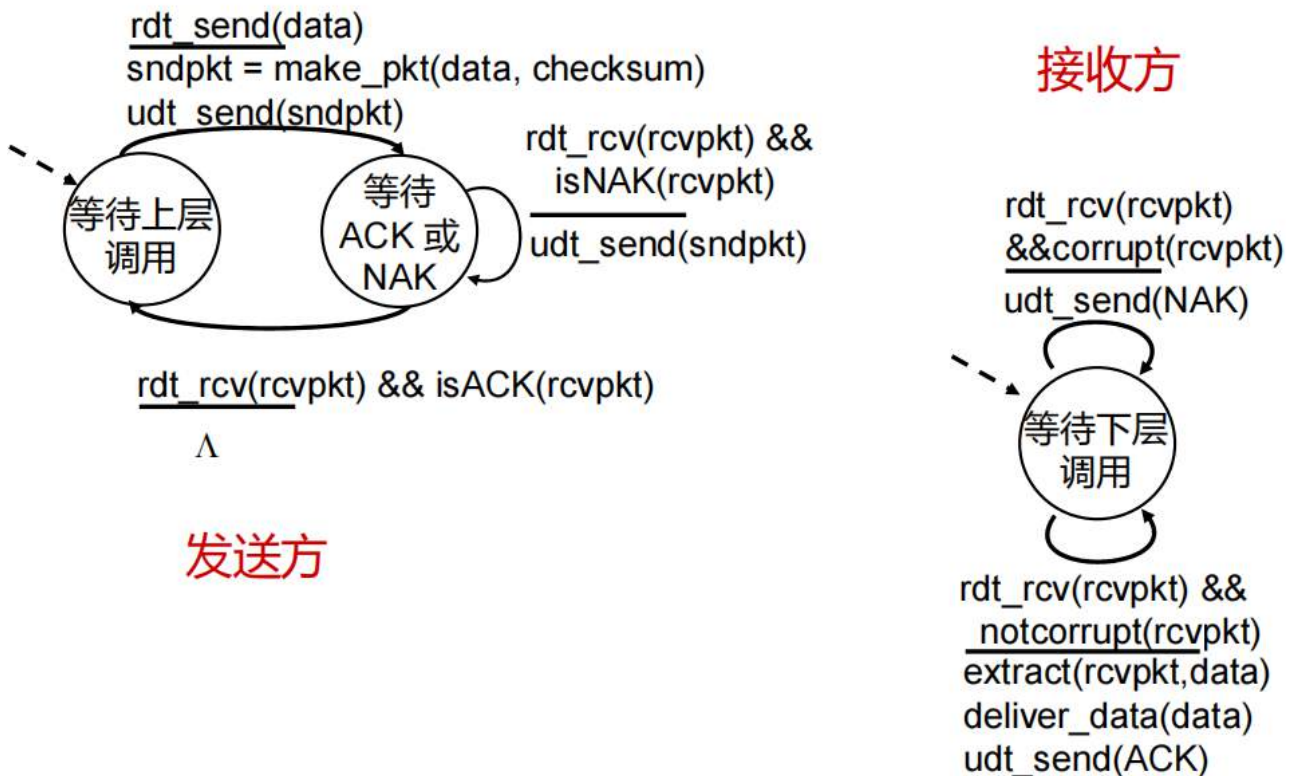
接收方： 只有一个状态“等待下层调用”。一旦包到了（`rdt_rcv`），它就拆包（`extract`）给上层（`deliver_data`）。

rdt2.0:假设信道可能有比特错误

- 底层信道可能会使数据分组发生比特翻转
 - 校验和可以检测比特错误
- 问题：如何修复错误
 - 肯定确认 (ACKs)：接收方明确告知发送方数据分组**成功接收**
 - 否定确认 (NAKs)：接收方明确告知发送方数据分组**发生错误**
 - 发送方在收到NAK时重传
- rdt 2.0的新机制（超越rdt 1.0）
 - 差错检测
 - 收方反馈：通过从接收方到发送方的控制信息 (ACK、NAK)
 - 重传

最重要的就是重传

rdt2.0中对于FSM的描述



如果网络状态非常差，一直传不过去，那么就会一直在等待ACK或NAK的地方忙等待锁死，因此必须有相应的机制解决（比如说限制重传次数等）

rdt2.0的缺陷

如果ACK/NAK出错会怎样？

可能接收方会多次接收，产生冗余，所以不能仅仅重传

怎么解决？

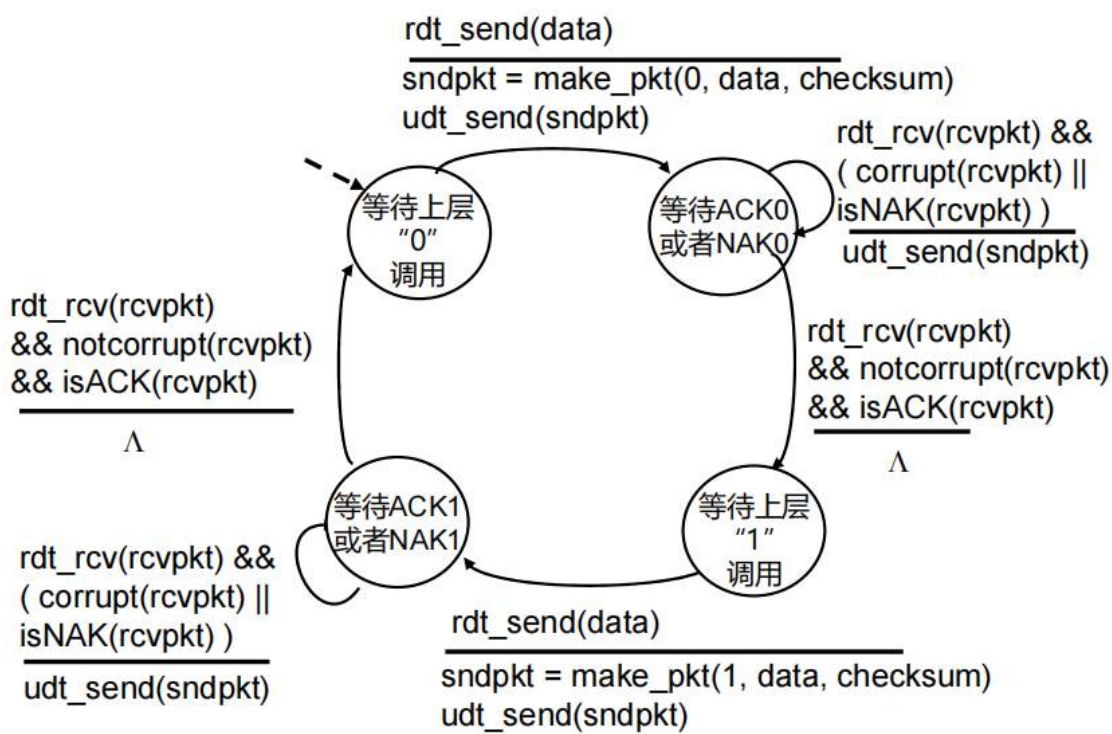
如果ACK/NAK出错，发送方重传当前分组

发送方给每个分组增加序号信息

接收方丢弃冗余分组，而不向上交付

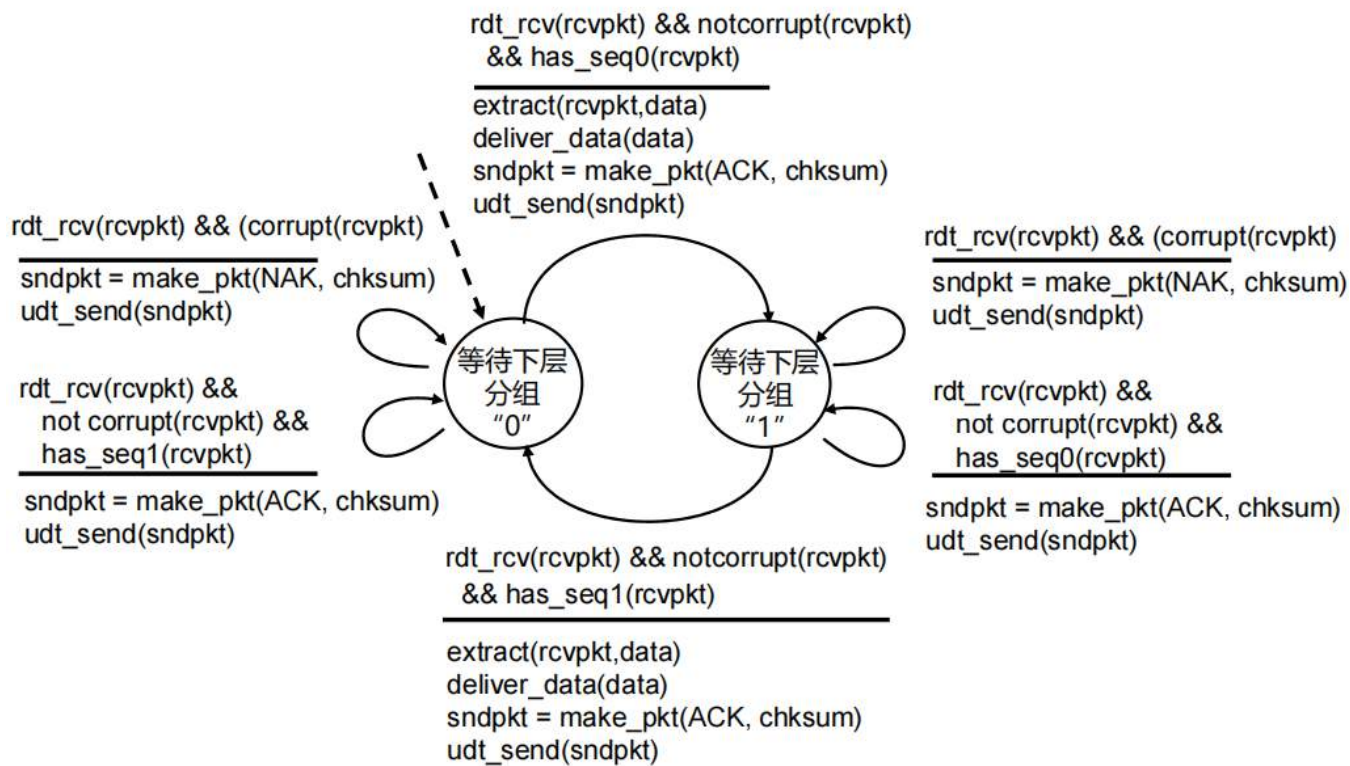
rdt2.1：解决了冗余

发送方



无论是发送方接收到的反馈包本身有错误或者是NAK，发送方都会重传，这个版本验证了反馈包本身的正确性

接收方



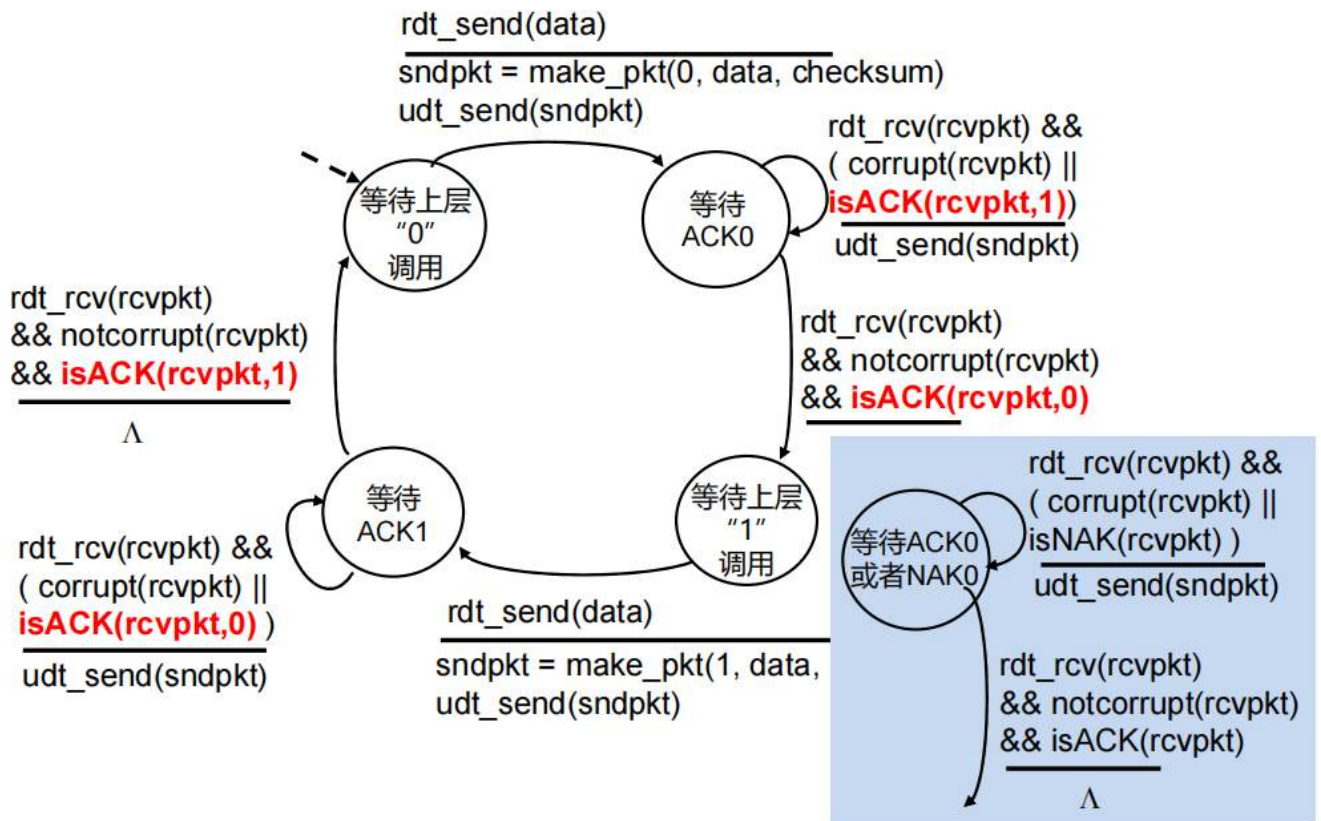
如果接收方已经接受了0的数据并且验证通过，但是ACK没发过去，那么两者的0和1不再对齐，就可以判断出后面传过来的数据是不是冗余的

rdt2.2: 无NAK的协议简化

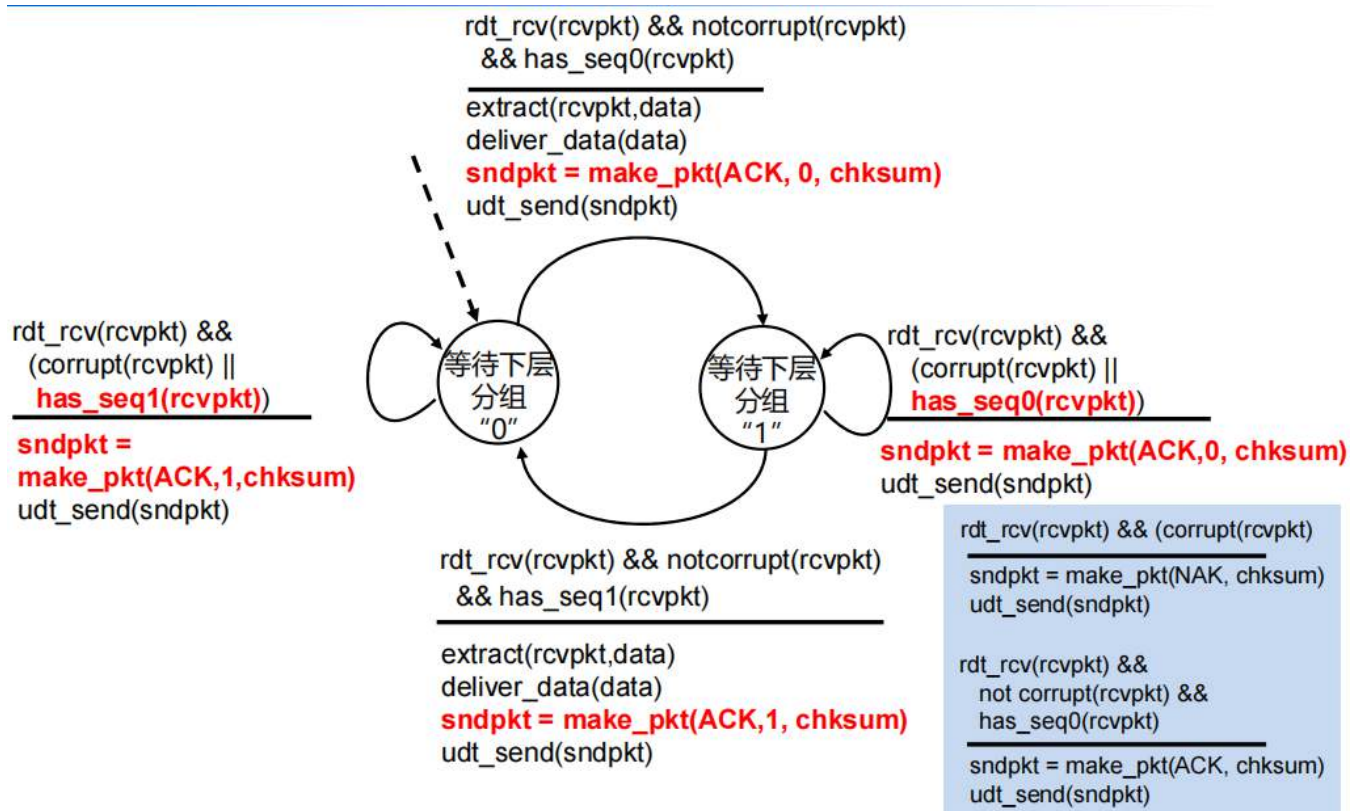
用ACK以及序号的方式来替代NAK这个含义

因为假设我要ACK1，结果传来了ACK0，那么就说明需要重传，不需要NAK

发送方



接收方



新的假设:

底层信道可能丢失分组，
包括数据和ACK

- 可能的应对措施：校验和、ACK (NAK)、序号、重传...

➤ 有用，但是不够

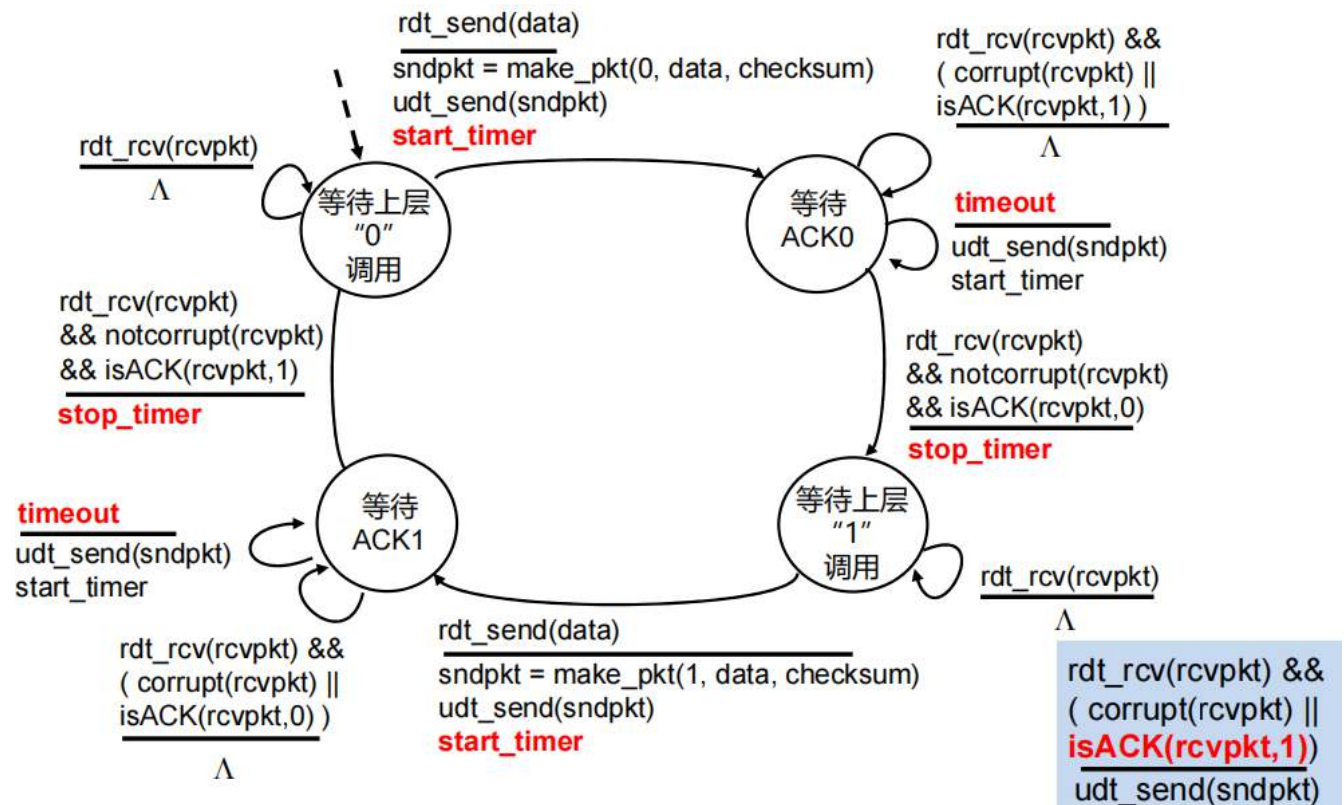
方法:

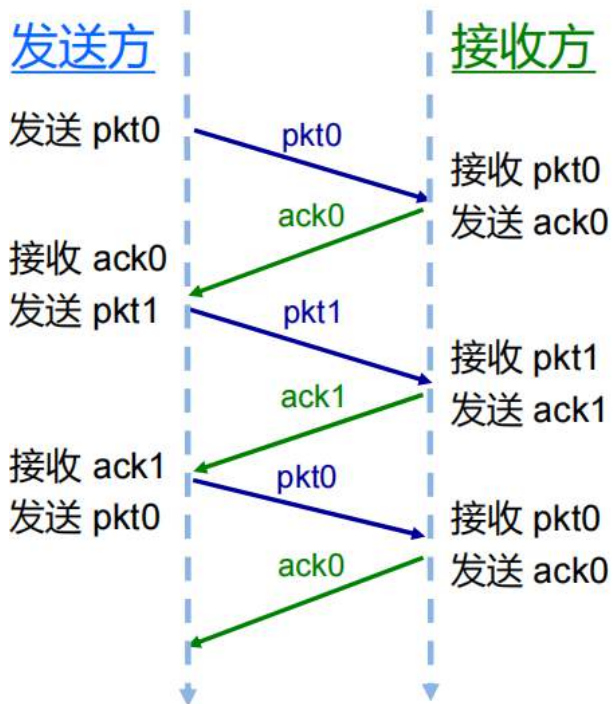
发送方花费“合理”的时间等待接收ACK

- 需要倒计时器
- 倒计时结束仍未收到ACK，则重传数据分组
- 如果数据分组或ACK只是延迟，并没有丢失：
 - 重传会导致冗余，但序号可以应对该问题
 - 接收方必须说明确认分组的ACK序号

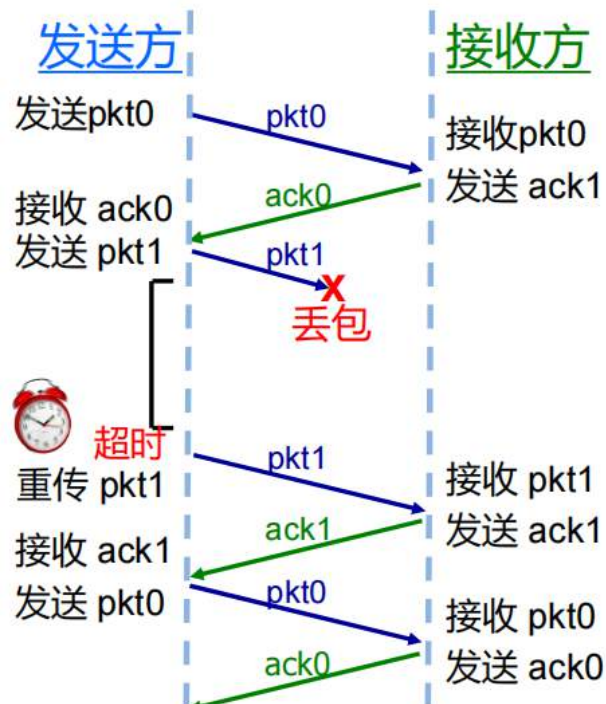
假设发送方一直没拿到反馈包，超过一段时间之后自动重传

发送方

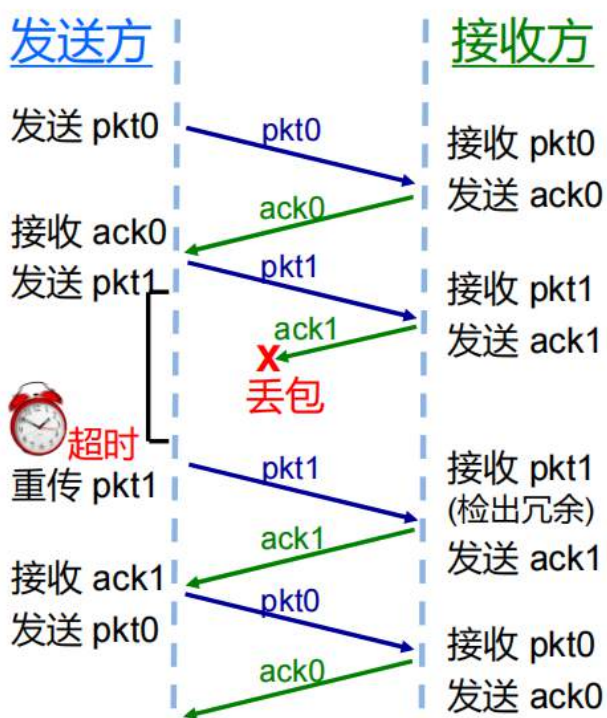




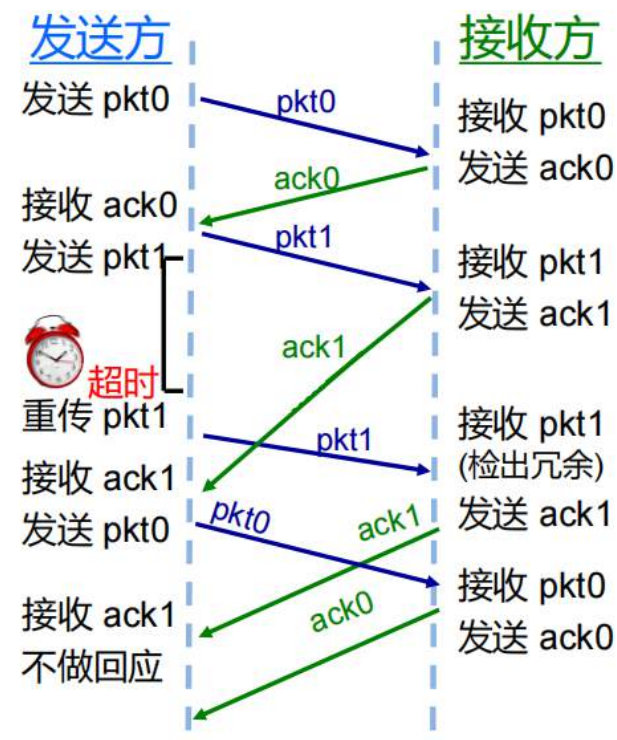
(a) 无丢包



(b) 数据丢失



(c) ACK丢包



(d) 超时 (过早) / ACK延误

针对d来说，假设发送方发送了一个pkt0，它期望接收到一个ack0

- rdt2.2中，如果接收到了一个ack1，那么就会立马重发一个pkt0
- rdt3.0中，如果接收到了一个ack1，那么不予理会，计时器继续等待，rdt3.0重发的条件永远是计时器达到额定计时，只有在接收到ack0的时候才重置计时器

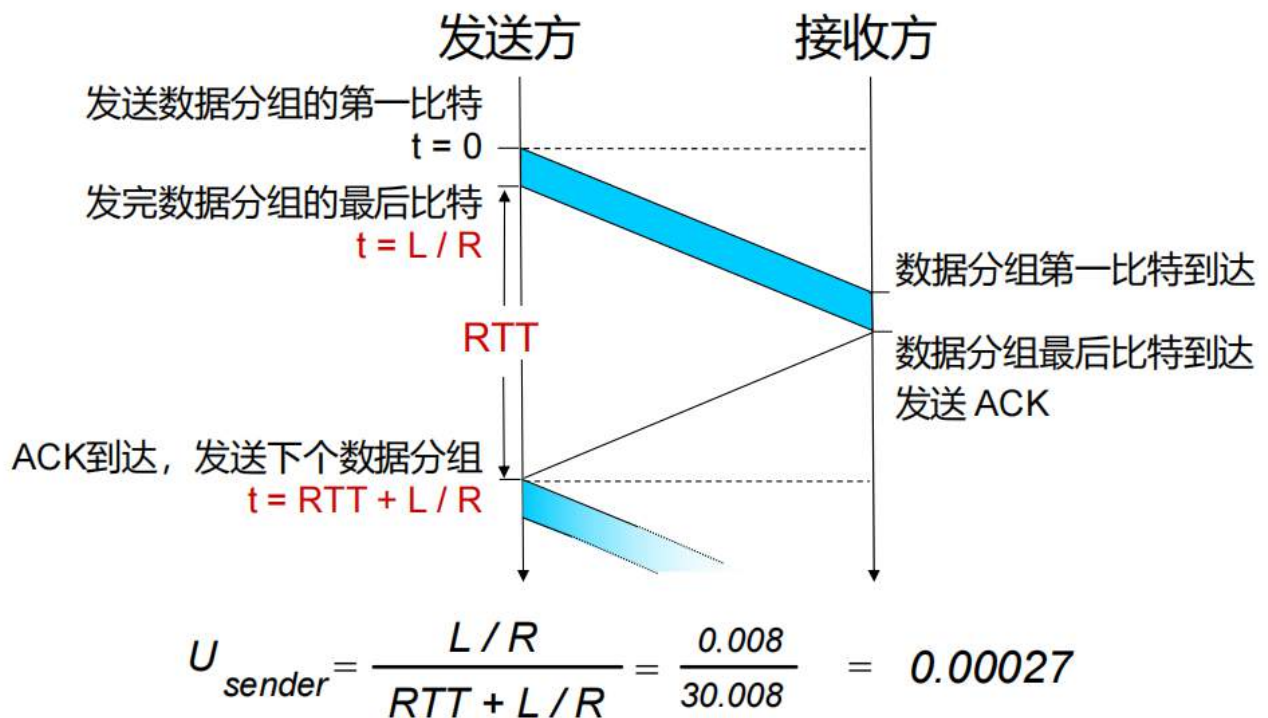
- rdt3.0使数据分组正确传输，但是性能（效率）较差
- 例如，在1 Gbps的链路下，当往返时延为30毫秒，且数据分组的大小为8000比特时：

$$D_{trans} = \frac{L}{R} = \frac{8000 \text{ bits/pkt}}{10^9 \text{ bits/sec}} = 8 \text{ 微秒}$$

- U_{sender} ：利用率——发送方发送数据分组的时间比例

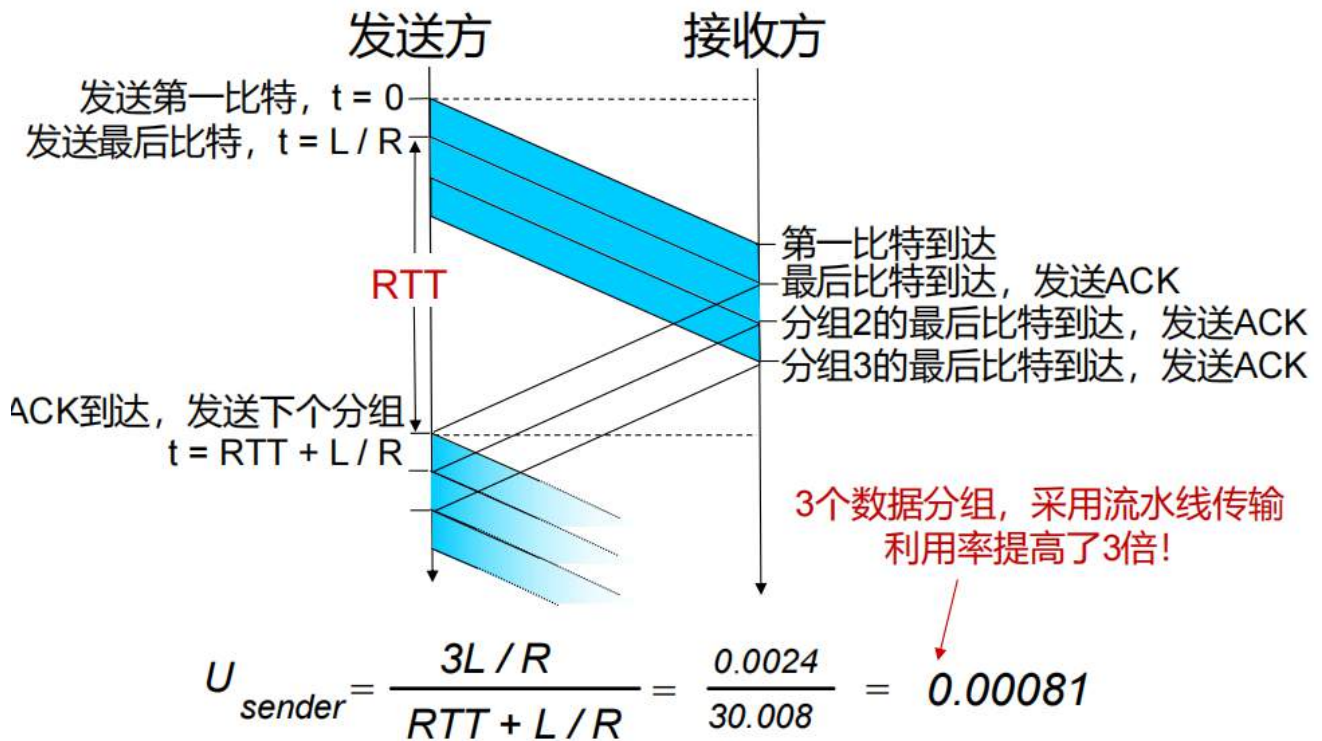
$$U_{sender} = \frac{L/R}{RTT + L/R} = \frac{0.008}{30.008} = 0.00027$$

- 每30.008ms用0.008ms，来传输8Kb的数据分组
- 在1Gbps链路上，吞吐量为267Kb/s
- 网络协议限制了物理资源的使用！



流水线式

流水线式协议的最大优势是提高利用率



按理来说应该计算在接收到第三个分组的时间为最终的时间，但是我们在接受到第一个的时候就开始发下一组了，因此只计算一个

发送方允许同时发送多个正在传输中、尚未被确认的数据分组，但必须要满足：

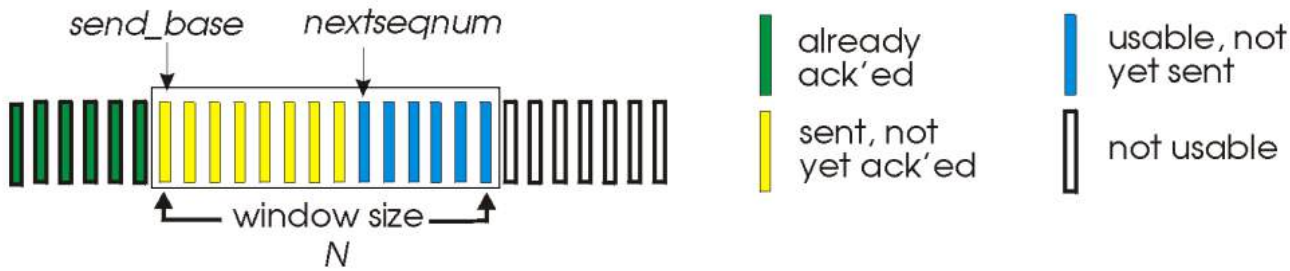
- 必须增加序号范围（不能只使用0和1来进行标记）
- 发送方和/或接收方缓冲（缓存那些还没有确认的分组序号和数据）
- 处理丢包、差错、超时的机制

后退N(GBN)协议

- 发送方最多允许N个未确认的数据分组
- 接收方仅仅发送累积确认
 - 如果数据分组不连续，则不进行确认
- 发送方对未确认分组中最早的那个，设置计时器
 - 当超时后，重传所有未确认的数据分组

累计确认：发送的ACK2表示1，2都接收成功

- 在分组的首部中，有k比特的序号
- 长度为N的“滑动窗口”最多允许N个连续未被确认的分组



假设我设置我的滑动窗口大小为N，那么我就根据当前的send_base指针为首创建一个滑动窗口（即大小为N）

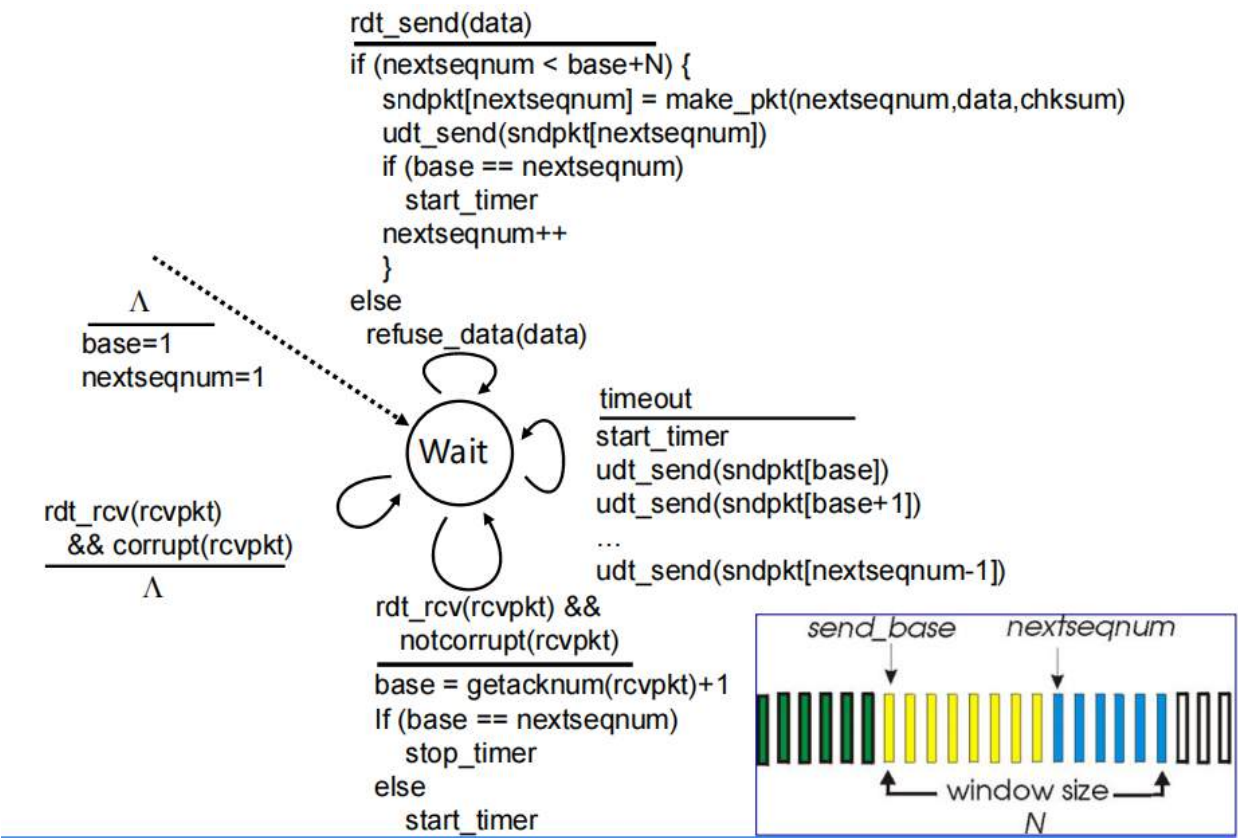
先判断当前的nextseq有没有到达滑动窗口的右边界（如果到了就得等，不能发送）（因为我们这次发送可能是从中途进入的，并不是从滑动窗口初始化开始计算的）

如果没有到，我们开始从前往后开始发，发一个就移动一下nextseq

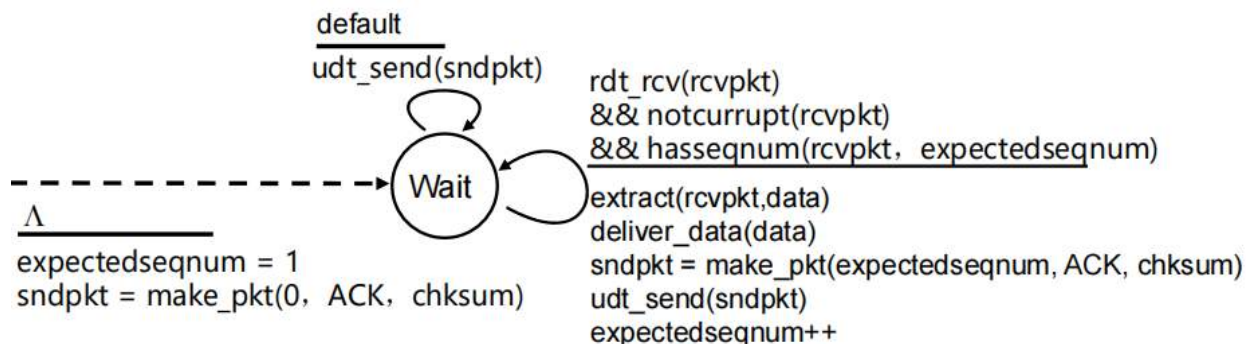
然后开始确认，从发第一个的时候开始计时，然后一旦接收到任何ACK就重置计时器，一直重复（如果有一次超时，比如接收到ACK3但是没接收到ACK4，就从pkt4开始将已经发送但未确认的分组进行重传），然后一旦接收到ACK(n)就将base往后移动到n+1去追nextseq（哪怕反馈丢了某个包，只要找到最大的就可以，这就是累积反馈）

当base追上nextseq的时候，说明数据传完了，销毁窗口

发送方



接收方

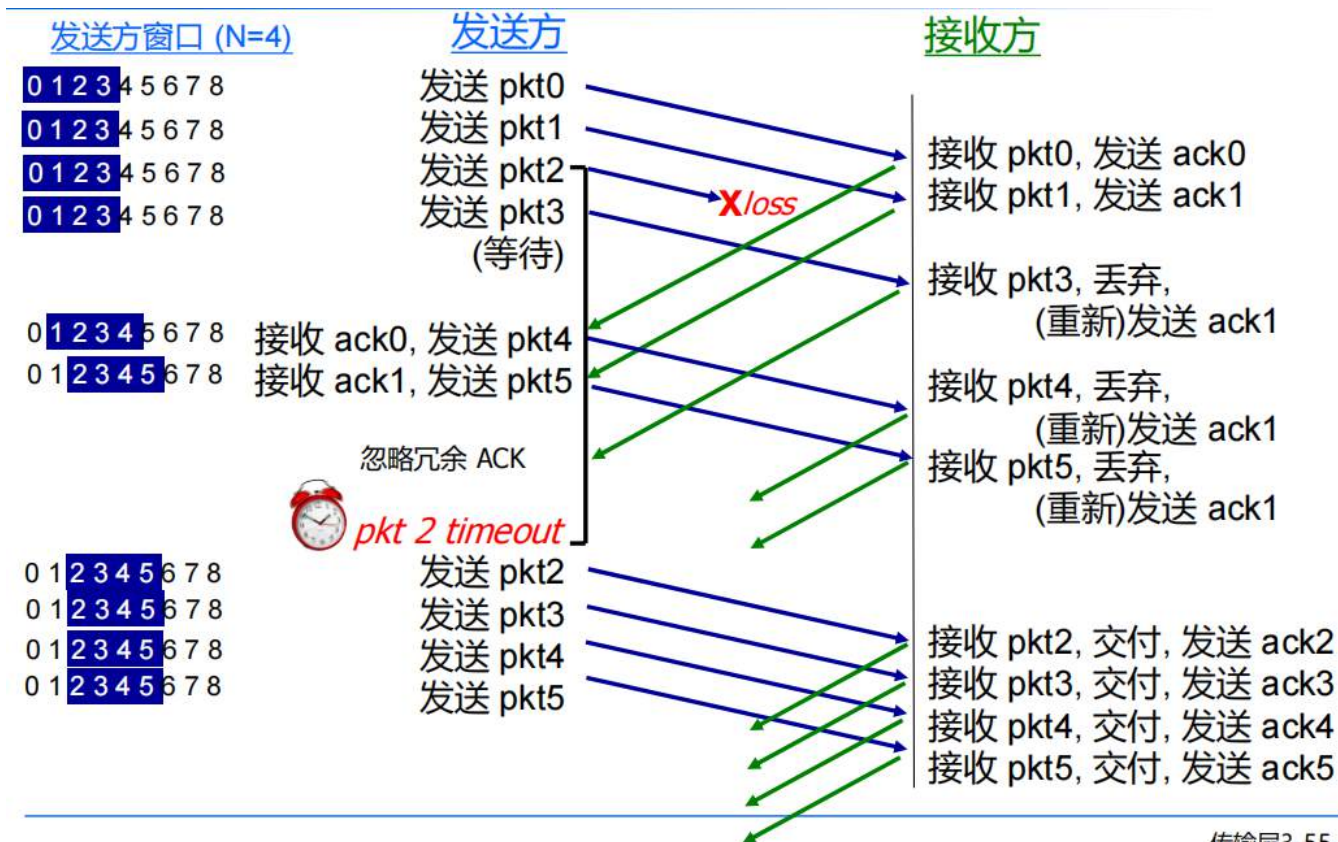


确认信息仅针对：正确接收的有序分组中，拥有最高序号的分组，发送相应的ACK

- 可能生成冗余ACK
- 只需记清期望序号 `expectedseqnum`。

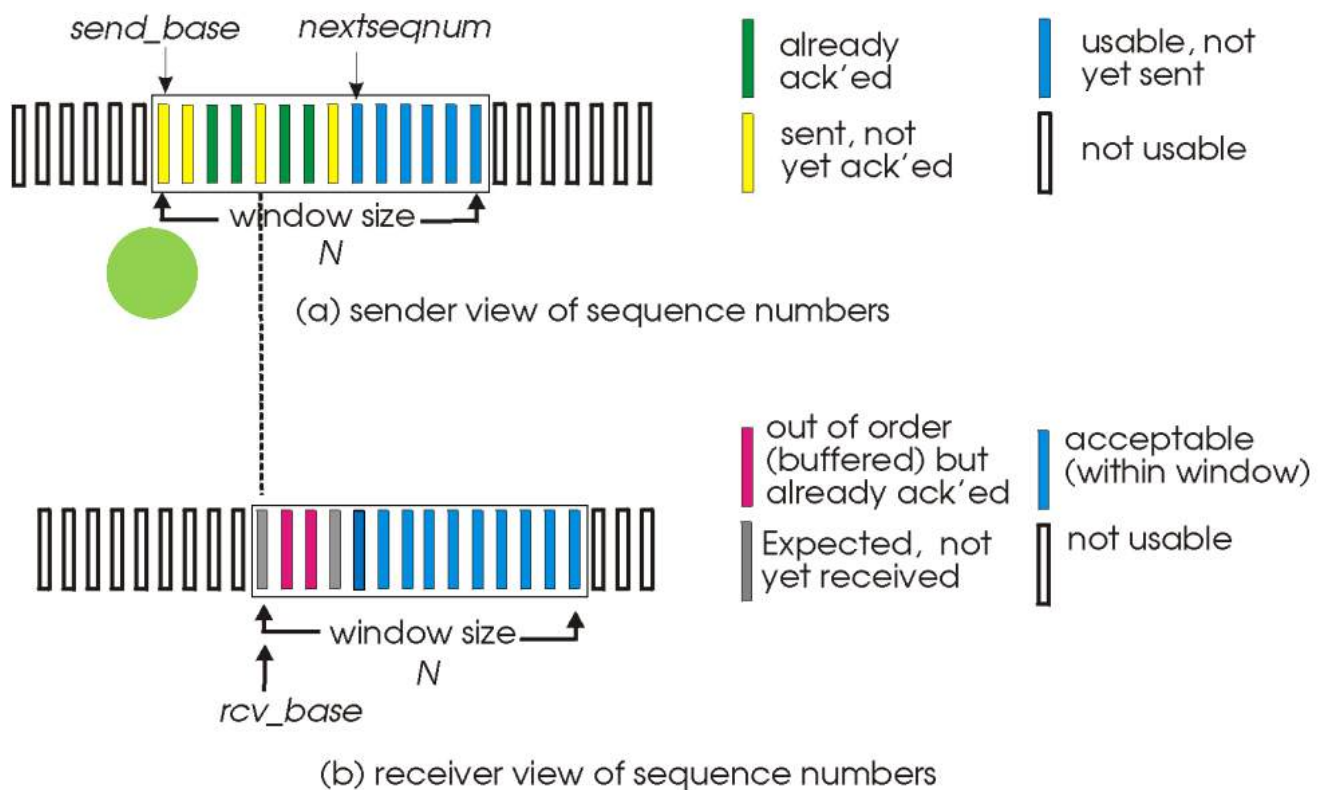
失序分组：

- 丢弃(不缓冲)：接收方不设置缓冲区
- 针对有序分组的最高序号，重发ACK



选择重传(SR)协议

- 发送方最多允许N个未确认的数据分组
- 接收方针对每个分组，分别发送对应的ACK
- 发送方为每个未确认的分组维护一个计时器
 - 当超时后，仅重传对应的未确认分组



发送方

□ 上层数据到达:

- 当窗口中下一序号可用时, 发送分组

□ timeout(n):

- 重发 pkt n, 重启计时器

□ ACK(n)

位于[sendbase, sendbase+N]时:

- 标记分组n为已接收
- 当n为已发送且未确认分组中的最小序号时, 窗口的基序号推至下一个未确认分组的序号

接收方

□ pkt n

位于[rcvbase, rcvbase+N-1]时:

- 发送ACK(n)
- 失序分组: 置于缓冲区
- 有序分组: 与缓冲区的有序分组一起交付, 窗口的基序号推至下一个未收到分组的序号

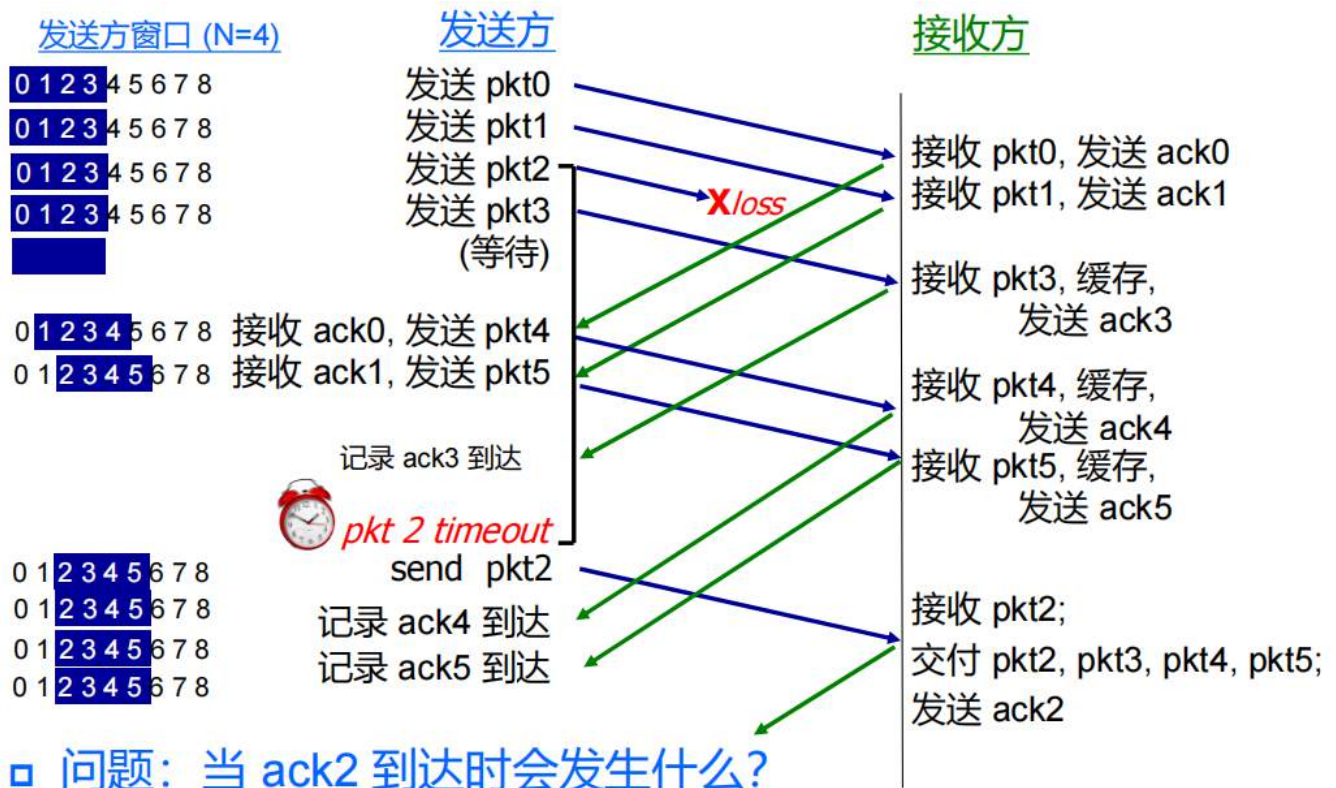
□ pkt n

位于[rcvbase-N, rcvbase-1]时:

- 发送ACK(n)

□ 其他情况: 忽略

对于发送方和接收方的滑动窗口来说, 核心原则都是, 只有当前面的几个发送(接收)都成功了, 才移动窗口到靠左没成功最近的那一个, 否则不动窗口, 各自有各自的反馈和计时器即可, 各自管理

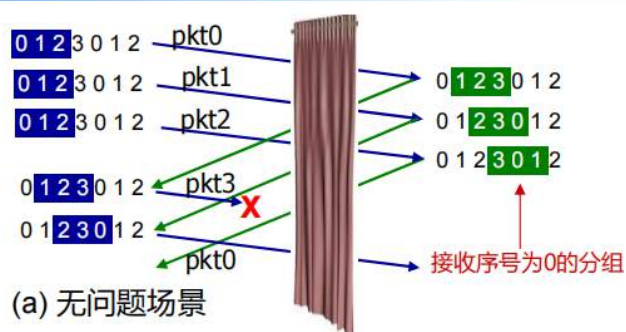


这里不同的是, 接收方需要缓存窗口内的所有pkt (GBN接收方不用缓存), 不能立马扔, 比如这里接收到重传的pkt2之后, 把四个pkt才一起交付给上层(应用层), 否则一个不交付

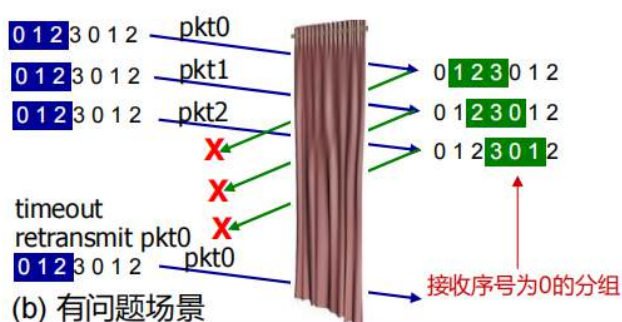
示例:

- 序号为0、1、2、3
- 窗口大小=3
- 接收方认为右图两种情况没有区别
- 在(b)中，冗余数据被当作新数据接收

问题：为了避免 (b) 中的问题，序号大小和窗口大小应具有什么关系？



接收方看不到发送方，两种情况相同处理！
出了大问题！



传输层3-60

序号一般要大于或等于两倍的窗口大小，否则会出现有问题的场景b，即第一组的序号0传给接收方的滑动窗口，但是没地方存

TCP：面向连接的传输

这里不依赖GBN和SR，只是有些思路相似，完全是另外一个协议，后面要摆脱原型算法

概述

□ 面向连接：

- 握手
- 缓冲、变量、套接字
- 单播(一个发送方，一个接收方)

□ 可靠数据传输：

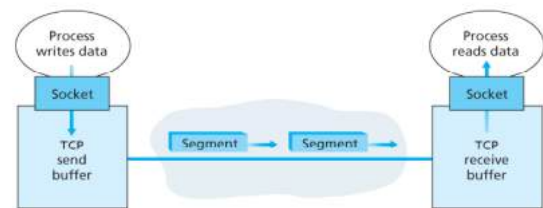
- 有序的字节流
- 流水线（拥塞和流量控制影响着窗口大小）

□ 流量和拥塞控制：

- 发送方的发送速度不得超过接收方的处理速度

□ 全双工：

- 同一时间、同一连接的数据流，是双向传输的
- MSS：最大报文段长度
maximum segment size, 由 MTU 决定



TCP面向连接，与单播息息相关。与广播息息相关的通常是无连接的

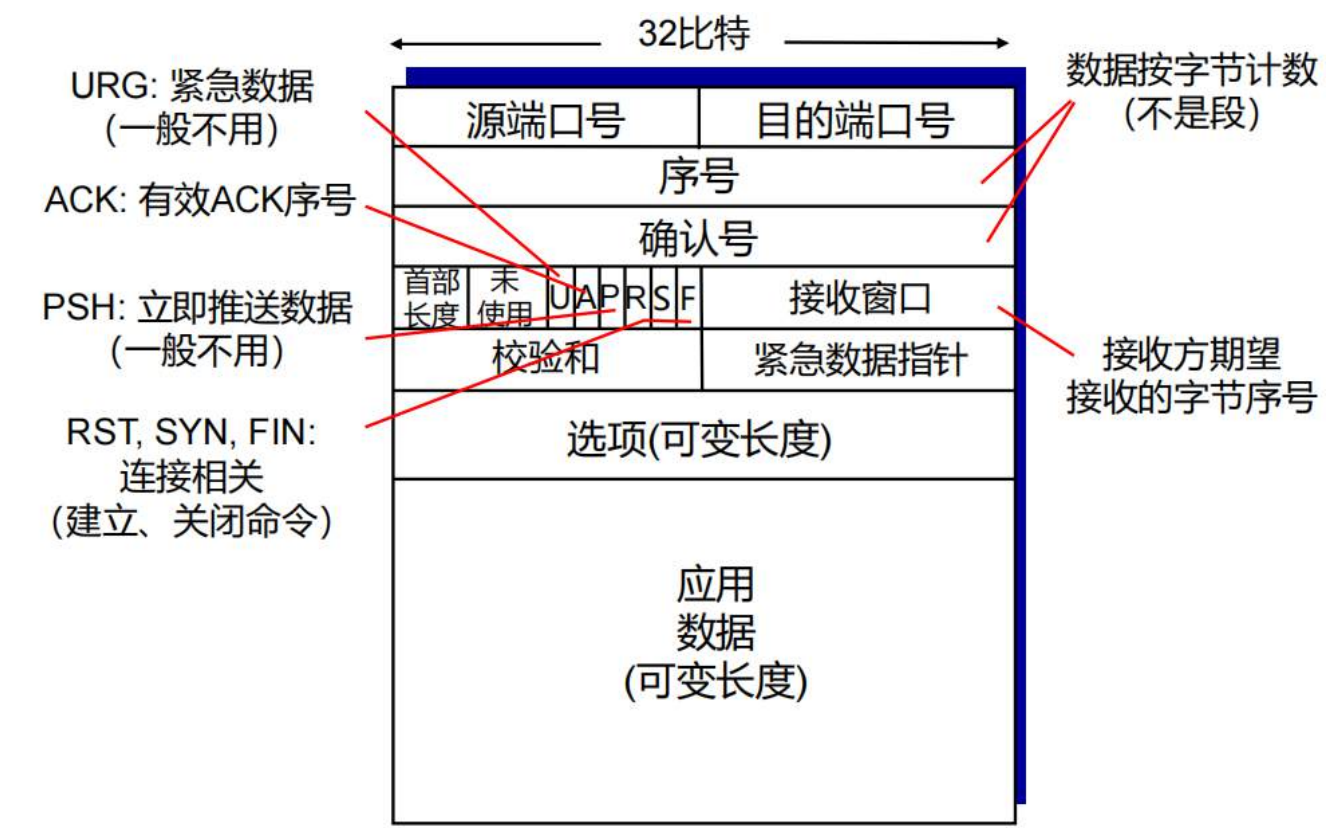
单工：A只发向B

半双工：AB都可以互相发送，但不能同时

全双工：AB可以同时相互发送

MTU是网络层的，MSS被下层决定

报文段结构



注意序号和确认号的数据按字节技术

ACK：置位的时候，表示当前报文段是反馈报文

RST,SYN,FIN与连接相关

接收窗口：告诉网络自己还有多少的接收能力

序号与确认

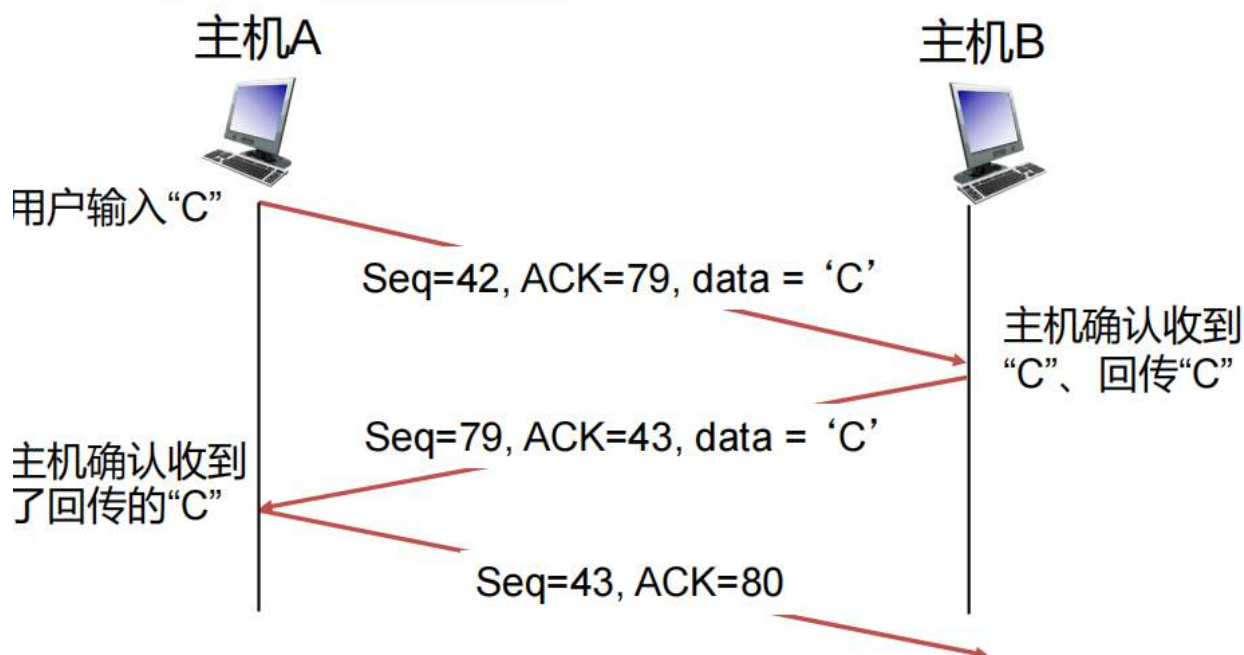
- 序号:**
- 字节流的 "编号", 指向报文段的第一个字节
 - 初始化为随机值
- 确认号:**
- 预期从对方发来的下一个字节的序号
 - 累积确认
- 问题:** 接收方如何处理失序报文段?
- 回答: TCP 标准中未定义, 由开发者决定。



序号指的是一整个报文段（分组）第一个字节的编号，并且避免编号冲突，初始化为随机值

ACK0在这里就是确认号为0，下面的A置位

针对接收方如何处理失序报文段的问题？我们可以采用GBN无缓存直接丢弃也可以采用SR缓存的方式，这个由用户决定



口诀：抄A加S转

将对方的S和A拿过来反转，再给ACK+1

流程是什么？

首先我们要记住这里是双向传递的，因此不能再带入一个发一个收的思路！！

Seq表示我当前的，**ACK**表示我想要的

1. 第一步：主机 A 发送数据 'C'

- **Seq = 42:** 这是主机 A 当前发送序列号的起始值。它告诉主机 B：“这是我发送的第 42 字节数据。”
- **ACK = 79:** 这表示主机 A 期望从主机 B 收到的下一个字节编号是 79。也就是说，主机 A 已经成功收到了主机 B 发送的前 78 个字节。
- **data = 'C':** 实际载荷是一个字符，占据 1 个字节。

2. 第二步：主机 B 回传数据并确认（关键点）

主机 B 收到 'C' 后，执行了两件事：一是确认收到了 A 的数据，二是将 'C' 回显（echo）给 A。

- **Seq = 79:** 因为 A 在上一步说“我想要 79”，所以 B 就从 79 开始发。
- **ACK = 43:** 这个数字来源于 $42 + 1$ 。B 收到 A 的第 42 字节后，告诉 A：“我已经收到第 42 字节了，我下一条想要第 43 字节。”
- **data = 'C':** B 把收到的字符又发还给了 A。

3. 第三步：主机 A 最后确认

- **Seq = 43:** 因为 B 在上一步说“我想要 43”，所以 A 发送序列号为 43 的包。

- **ACK = 80:** 这个数字来源于 $79 + 1$ 。A 收到了 B 发来的 1 字节 (Seq 79)，于是告诉 B：“我已经收到第 79 字节了，下次请从第 80 字节开始发。”
- 此包通常不再携带 data，只是一个纯粹的 ACK 确认包。

往返和超时

这里研究怎么设置超时时长的问题，服务于TCP协议

问题： 如何设置TCP的超时时长？

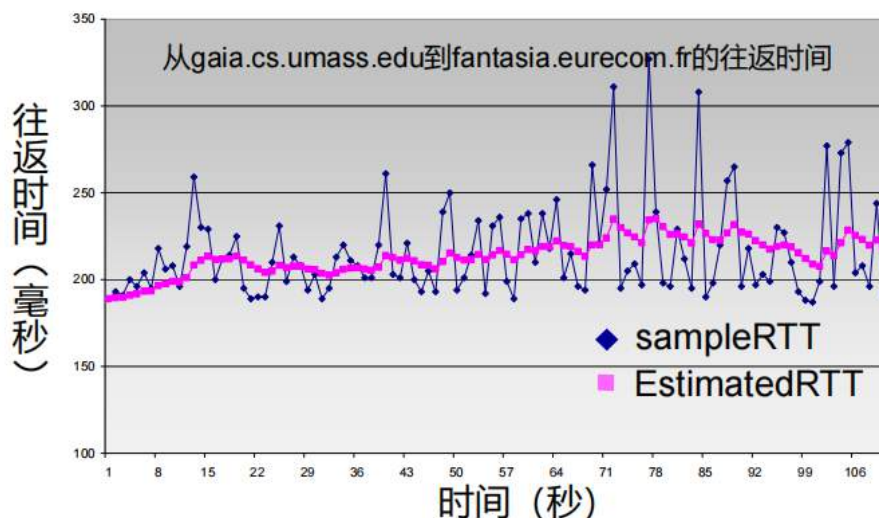
- 长于往返时间RTT
 - 但是RTT是变化的
- 过短：超时太早，产生不必要的重传
- 过长：超时太晚，应对报文段丢失时反应太慢

问题： 如何估计RTT？

- **SampleRTT**：从发送报文段到接收ACK的测量时长
 - 忽略重传
- **SampleRTT**是时变的，期望有更“平滑”的RTT估计值
 - 应该取最近几个测量值的平均值，而不只是当前的SampleRTT

$$\text{EstimatedRTT} = (1 - \alpha) * \text{EstimatedRTT} + \alpha * \text{SampleRTT}$$

- 指数移动加权平均
- 过去样本值的影响，以指数形式快速下降
- 典型值： $\alpha = 0.125$



□ 从 **EstimatedRTT** 看 **SampleRTT** 的偏差:

$$\text{DevRTT} = (1-\beta) \times \text{DevRTT} + \beta \times |\text{SampleRTT} - \text{EstimatedRTT}|$$

(通常, $\beta=0.25$)

□ **超时间隔**: **EstimatedRTT** 加上 “安全裕度”

➤ **EstimatedRTT** 较大的变化 → 较大的裕度

$$\text{TimeoutInterval} = \text{EstimatedRTT} + 4 \times \text{DevRTT}$$



↑
RTT估计值

↑
裕度

➤ 初始值建议1秒

➤ 超时的时候加倍

可靠数据传输

TCP在IP不可靠的服务之上创建rdt服务

- 流水线式的报文段
- 累积确认
- 分别计时和重传

重传操作的触发:

- 超时事件
- 冗余ACK事件

先考虑一个简化场景的TCP发送方:

- 忽略冗余ACK
- 忽略流量控制
- 忽略拥塞控制

从应用收到数据

□ 创建带序号的报文段

➤ 序号是字节流的数量,
指向报文段的第一字节

□ 如果计时器尚未运行, 那么启动计时器

➤ 计时器指向未确认报文段中最早的那个

➤ 超时时间间隔: 后面用
TimeOutInterval表示

超时:

□ 重传引起超时的报文段

□ 重启计时器

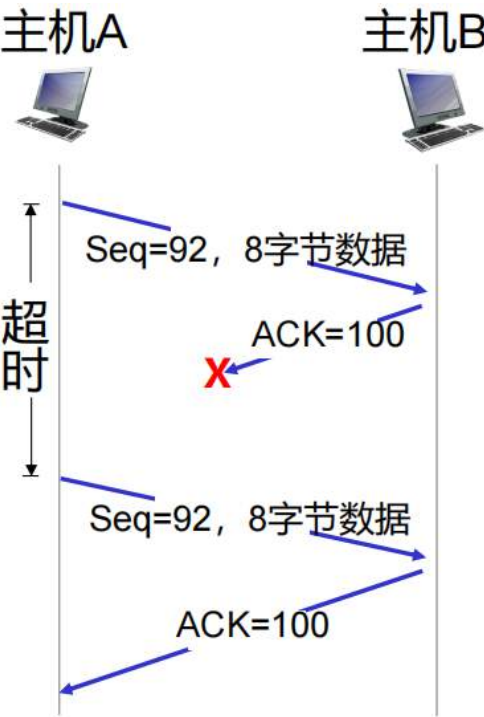
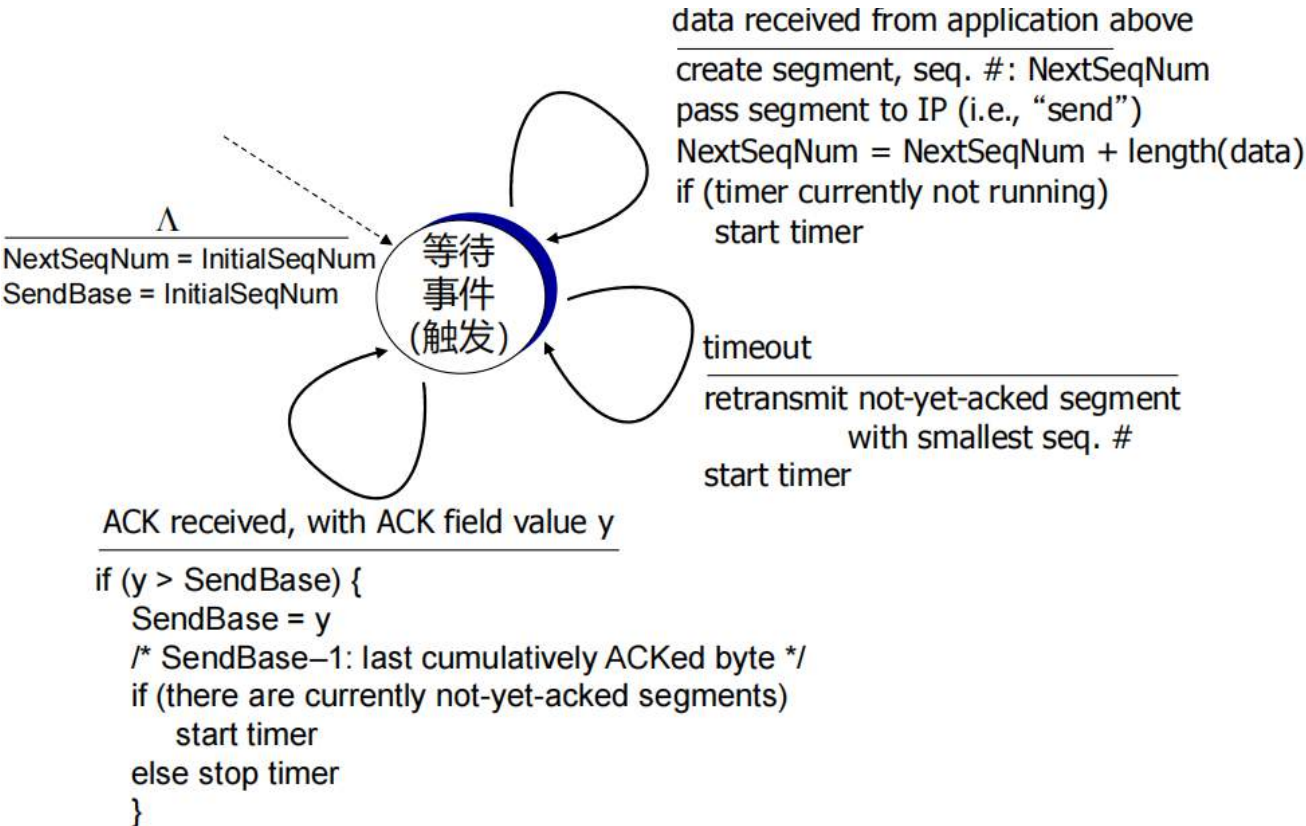
收到ACK:

□ 如果ACK是对此前还未确认报文段的正反馈

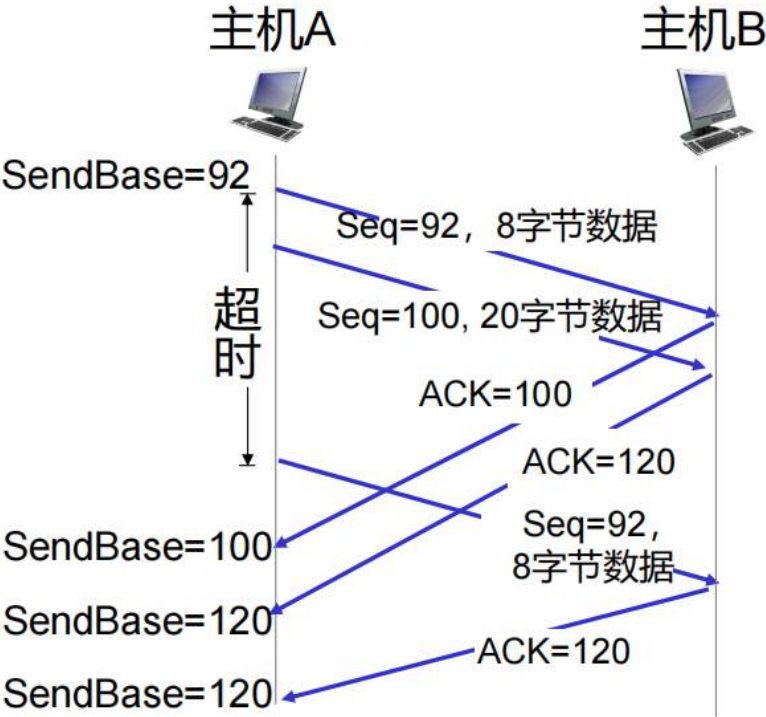
➤ 更新已确认报文段信息

➤ 如果仍有未确认的报文段, 那么启动新计时器

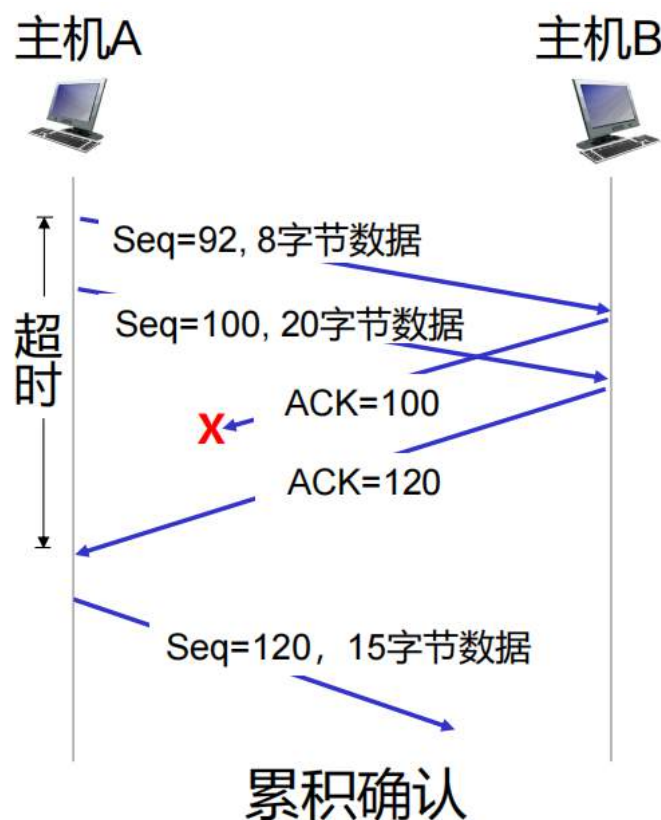
注意这里重传的不是所有未确认的报文段，只重传引起超时的报文段



ACK丢失场景



超时时间间隔较短



假如第一次的S=92,没有被B接收到,那么主机B根据累计确认,哪怕收到了S=100,这个时候也不会返回120,一旦返回了120,说明之前的都传过去了,这就是累计确认

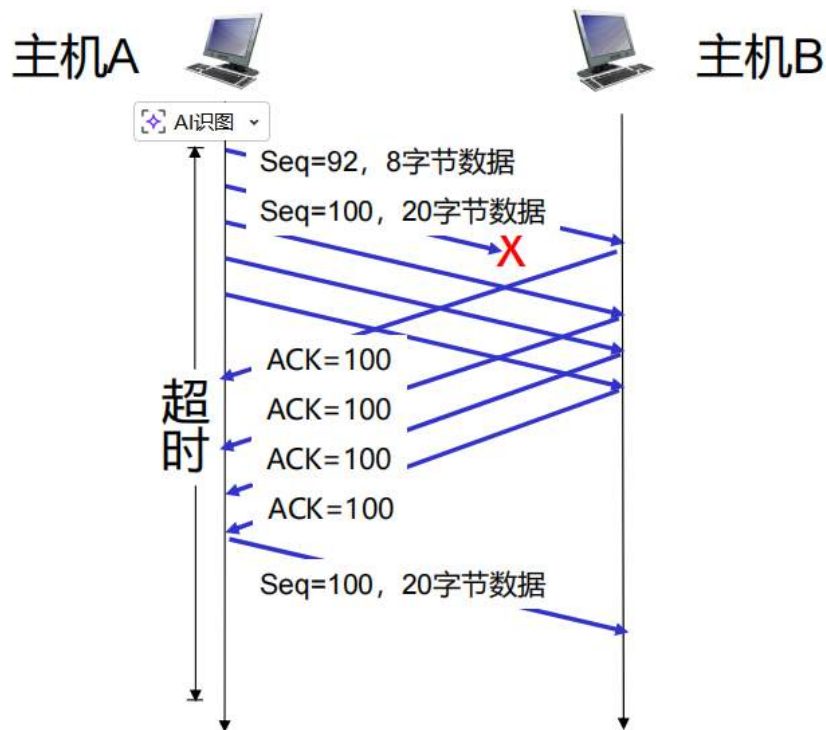
接收方发生的事件	接收方进行的操作
报文段按序到达, 并且是期望序号 → 期望序号之前的所有数据都已经确认	延迟确认。等待500毫秒用于接收下一个报文段, 如果没有下一个报文段, 那么发送ACK
报文段按序到达, 并且是期望序号; 同时, 另一个报文段在等待确认	立即发送单个的累积ACK, 完成对两个按顺序到达报文段的确认
报文段到达, 部分或完全填补空缺	如果报文段是空缺部分中较早的那些, 那么立即发送更新的累积ACK
报文段乱序到达, 并且序号大于期望序号 → 检测到空缺	立即发送冗余ACK, 指出下一个预期字节的序号

如果接收方没有接收到某个包, 就会一直发送想要的那个包的ACK, 哪怕新包来了也不会发新的, 就会产生大量冗余ACK, 必须等发送方超时重发, 接收到之后才根据目前接收到的序号最早的立即发ACK

因为超时时间间隔通常较长, 因此我们可以通过冗余ACK来检测丢失的报文段, 这叫做快重传

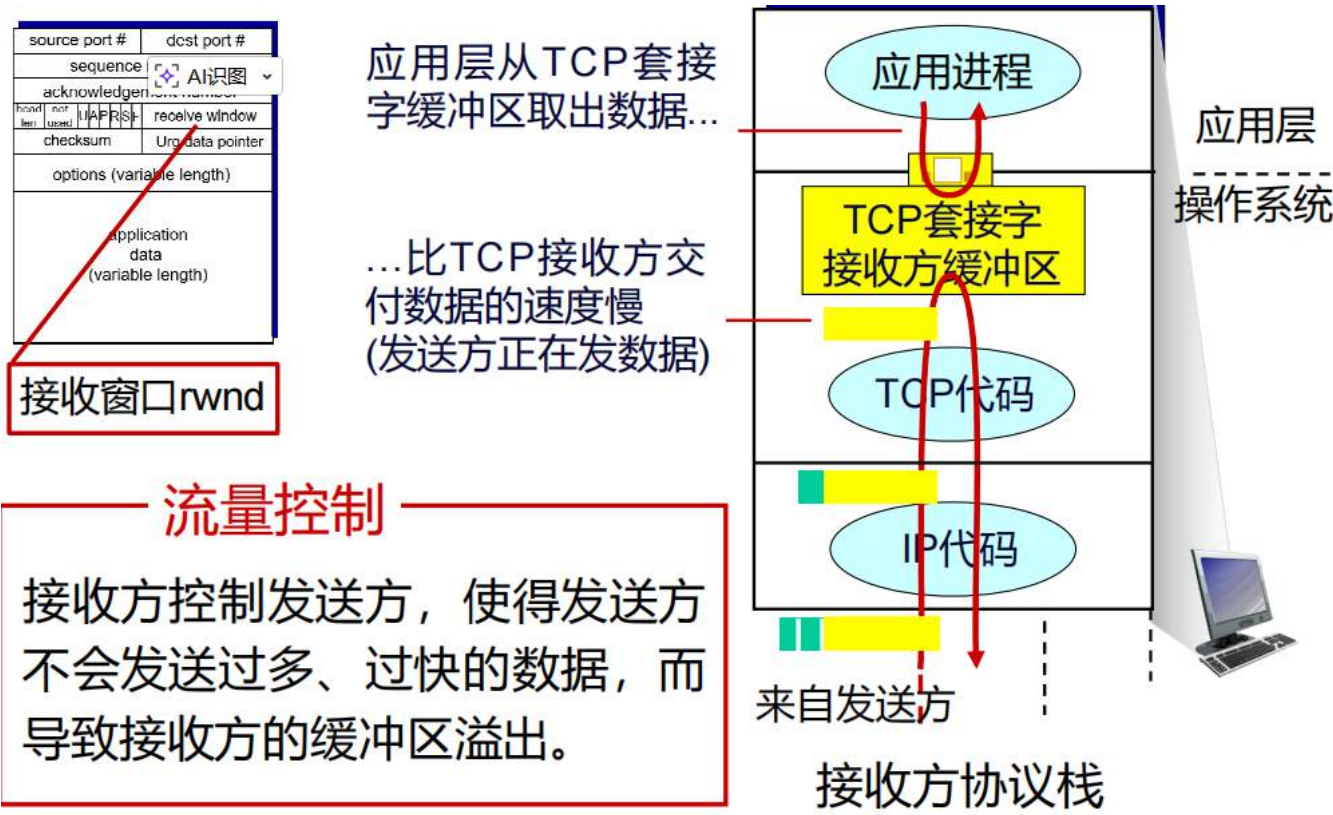
TCP的快重传

- 如果发送方收到相同数据的3个ACK（“三个冗余ACK”），那么重传序号最小的未确认报文段
 - 因为此时的未确认报文段很可能丢失了，所以不需要等到超时



发送方收到三个冗余ACK后，进行快速重传

流量控制

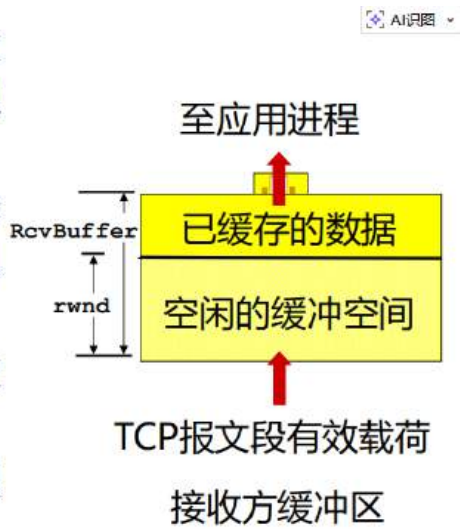


流量控制

接收方控制发送方，使得发送方不会发送过多、过快的数据，而导致接收方的缓冲区溢出。

怎么解决缓冲区（其实是滑动窗口）可能溢出的问题？

- 接收方在收方到发方的TCP报文段首部中，指明 **rwnd** 值，用以“通告”接收方空闲的缓冲区空间
 - RcvBuffer**的大小是通过套接字选项设置的（典型的默认值是4096字节）
 - 许多操作系统会自动调整接收方缓冲区，即**RcvBuffer**值
- 发送方通过接收方 **rwnd** 值，限制未确认（“在途”）的数据量，进而保证接收方缓冲区不溢出

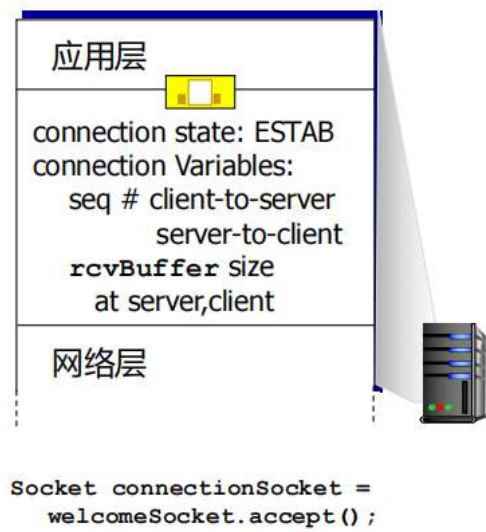
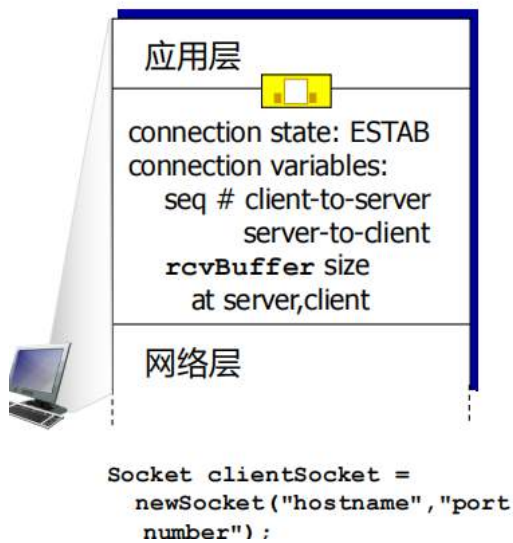


rwnd就是receive window，表示接收方还能接收多少数据，一旦为0，发送方不敢再发，就没有下文了，很明显不合理。因此设计者通常会让发送方再隔一段时间刺探一下

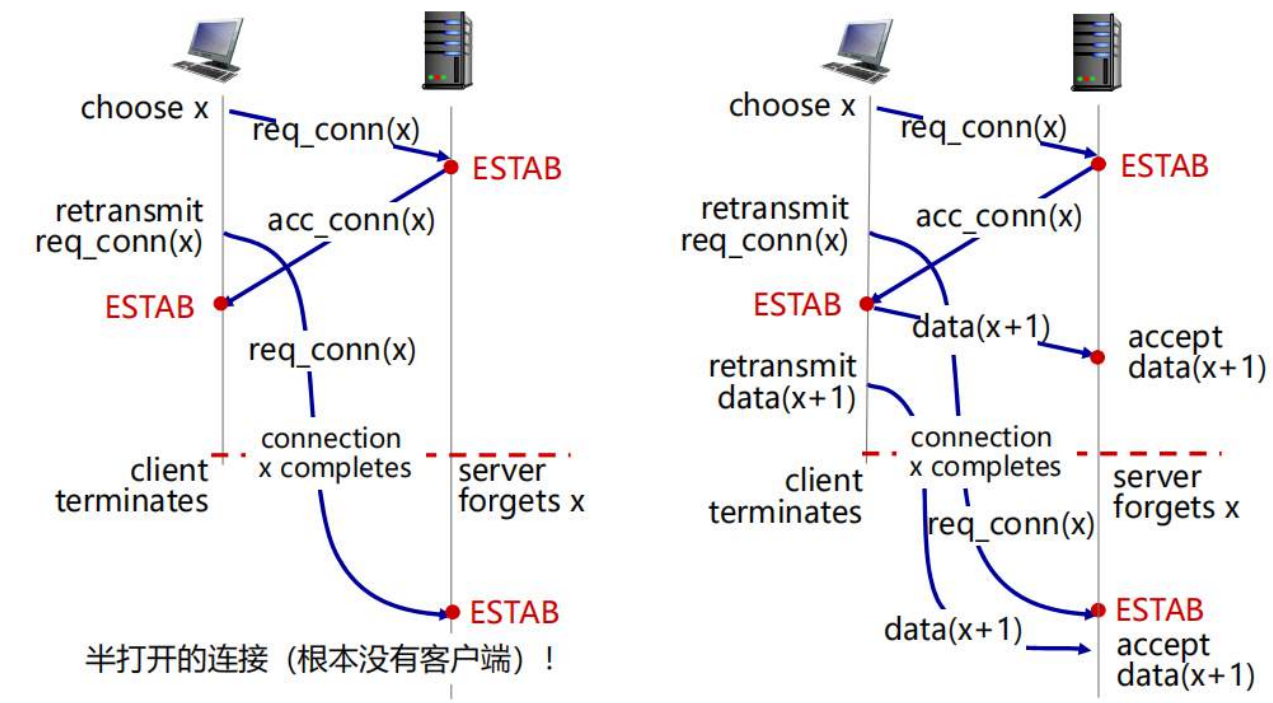
连接管理

在数据交互之前，发送方和接收方的“握手”：

- 同意建立连接（双方都知道对方建立连接的意愿）
- 就连接的参数达成一致



两次握手可能导致的问题



左图：已失效报文段导致的“半打开”连接

这一侧展示的是最基础的资源浪费问题。

1. 发送与阻塞：客户端发送了第一个连接请求 req_conn(x)，但这个报文在网络节点中滞留了（比如走了一条很绕的路），没有及时到达服务器。

2. **重传与成功**：客户端等太久没收到回应，触发超时重传，发送了第二个 `req_conn(x)`。这次很顺利，服务器回应 `acc_conn(x)`，双方建立连接，完成通信，最后正常释放了连接。
3. **“亡灵”复活**：就在连接已经结束、客户端已经关闭（Client terminates）后，**第一个滞留的请求报文突然到达了服务器。**
4. **服务器误判**：
 - 在**两次握手**协议下，服务器收到请求就认为客户端想连，于是立刻进入 `ESTAB`（已建立连接）状态。
 - **后果**：服务器在那傻傻地等客户端发数据，但客户端根本不知道这事儿（它早就结束任务了）。这产生了一个**“半打开连接”**，白白浪费了服务器的内存和 CPU 资源。

右图：旧连接请求引发的数据混乱

这一侧展示了更严重的情况：**旧的数据报文被错误接收。**

1. **背景**：和左图一样，一个旧的连接请求报文在路上堵了很久。
2. **数据滞留**：不仅连接请求堵了，连那个旧连接之前发送的数据 `data(x+1)` 也在网络里打转。
3. **错误建立**：旧连接请求终于到了服务器，服务器在“两次握手”规则下直接进入 `ESTAB` 状态。
4. **脏数据注入**：
 - 随后，那个**旧的数据报文** `data(x+1)` 也飘到了服务器。
 - 服务器一看：“序列号对得上（x+1）！正好是我在等的第一个字节。”于是它欣然接受（Accept data）。
 - **后果**：服务器接收了一堆来自“过去”的、早已失效的数据，导致应用程序处理的数据完全错误，造成逻辑崩溃或安全风险。

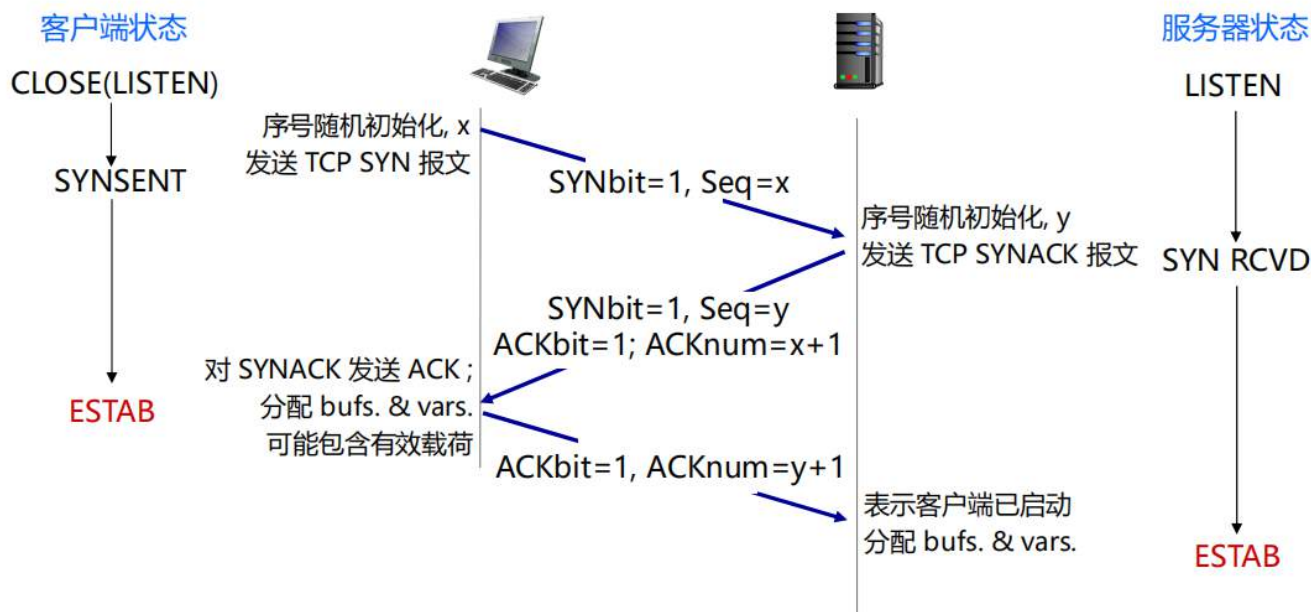
核心结论：为什么三次握手能解决？

如果引入**第三次握手**（客户端最后的 ACK 确认）：

- **防止误连接**：在左图中，当旧的 `req_conn(x)` 到达服务器，服务器回发确认给客户端时，客户端会发现自己并没有发起这个序号为 x 的请求。
- **客户端拒绝**：客户端会发送一个 `RST`（重置）报文给服务器，告诉它：“别等了，那是过时的包。”
- **状态同步**：服务器收到 `RST` 就不再建立连接，从而避免了资源浪费和接收脏数据。

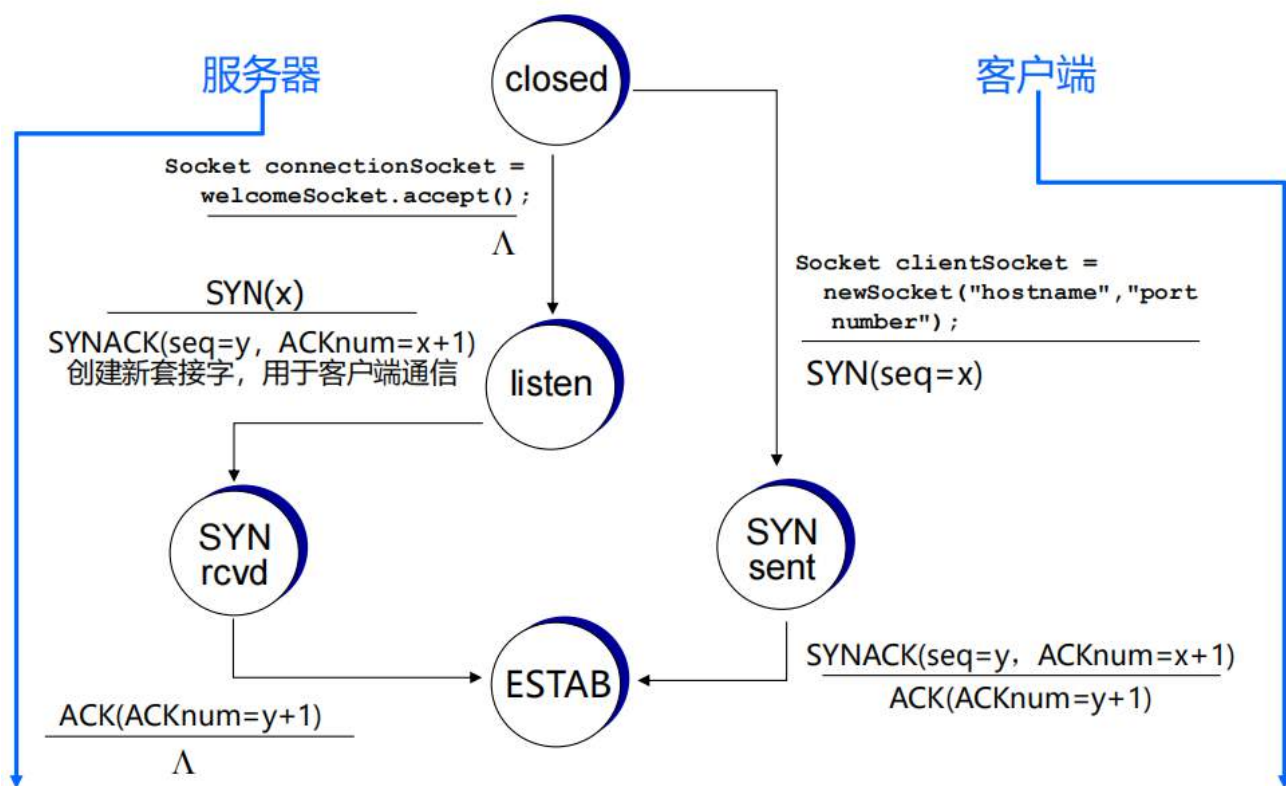
建立连接

因此我们引入**三次握手**



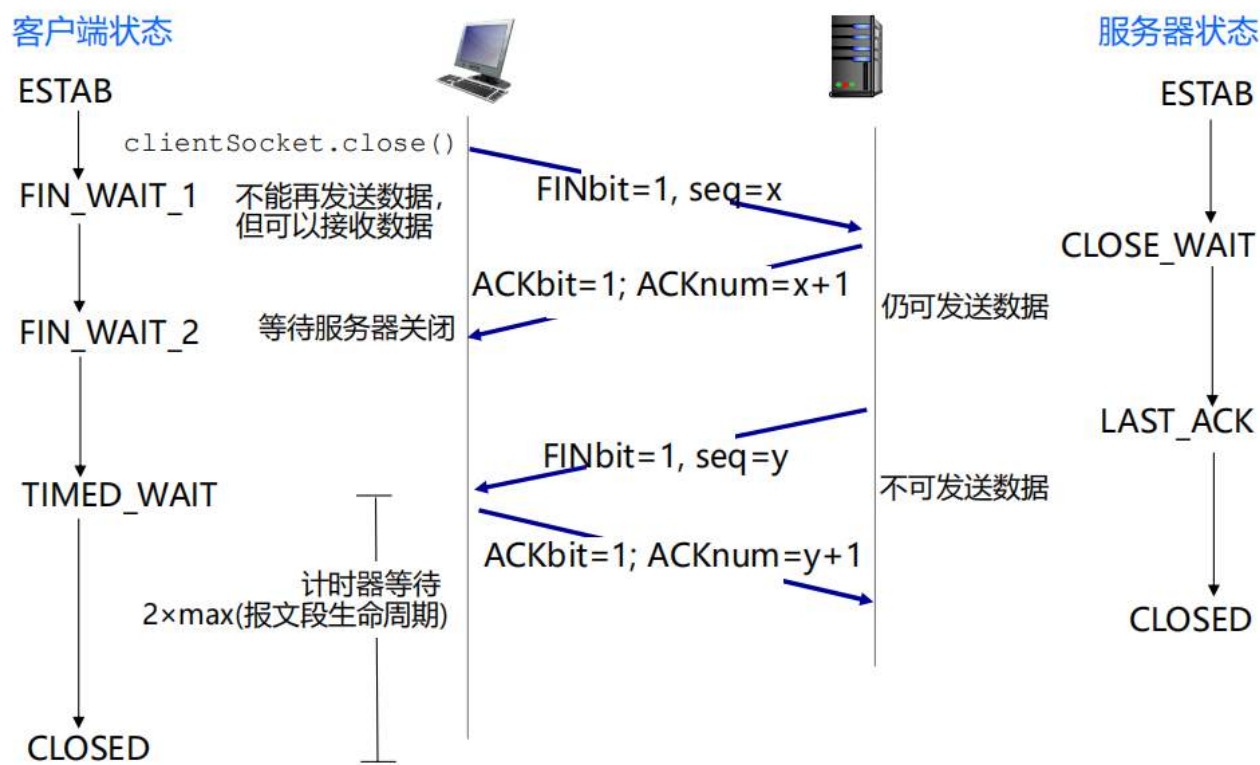
ESTAB表示某一方成功建立连接，开始分配资源

状态机



关闭连接

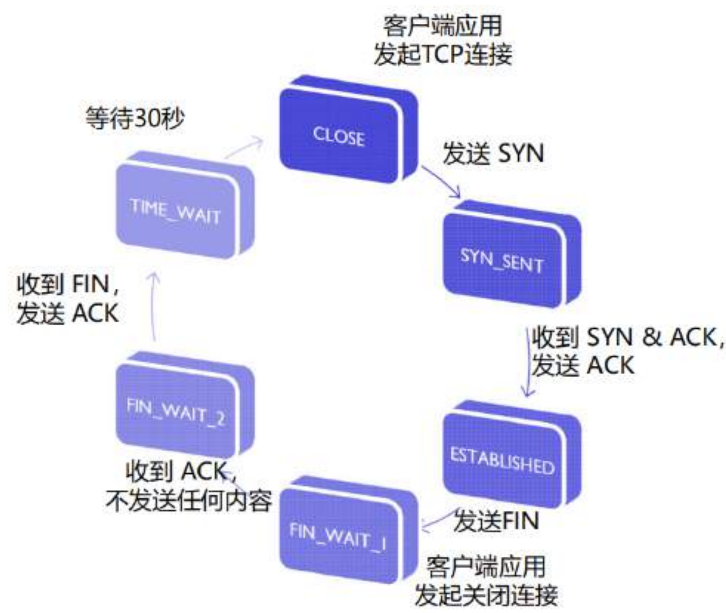
四次挥手



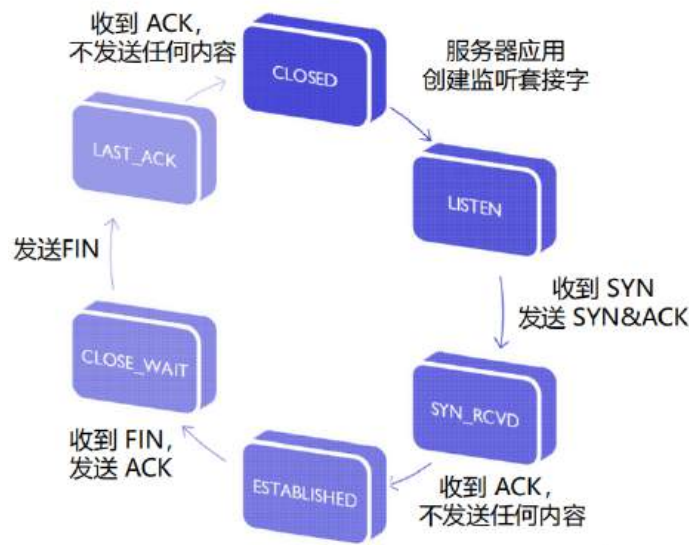
无论对于客户端还是服务端，在发完FINbit之后，这一方就不能再发数据，只能发一些挥手的报文段

客户端在FIN_WAIT_2的阶段，还可能接收到服务器的很多数据，在等到服务器发送请求关闭的FINbit之前，客户端还要处理这些数据

客户端的典型状态



服务端的典型状态



拥塞控制

原理

不同于流量控制：流量控制指的是在端内部的堵塞情况，而拥塞控制指的是包括整个链路的整个网络的堵塞情况

简单来说：过快的速率、过量的数据、过多的源端，以致于网络无法处理

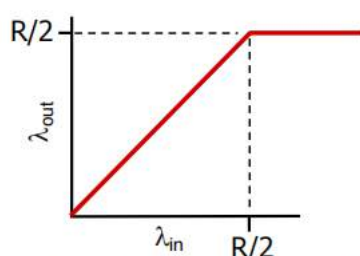
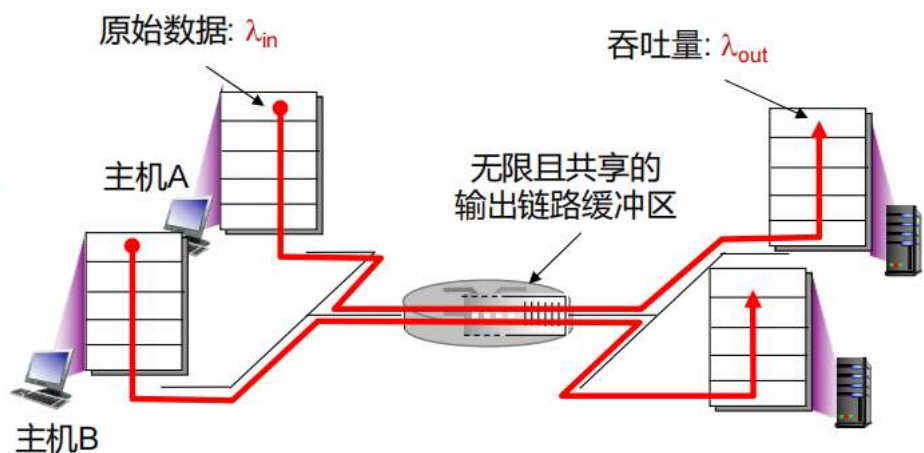
表现形式：

- 丢分组（路由器的缓冲区溢出）
- 长时延（路由器缓冲区的排队）

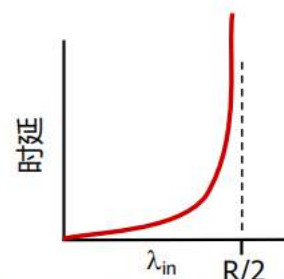
场景1

过快的速率导致，代价是延时高

- 两个发送方，两个接收方
- 一个路由器，无限缓冲区
- 输出链路带宽为R
- 不涉及重传



□ 每连接最大吞吐量：R/2



□ 随着到达率 λ_{in} 接近上限，时延越来越大

链路带宽会决定吞吐量的上限

时延的计算 = 进入路由器的带宽 / 流出路由器的带宽 = $\lambda_{in} / (R/2)$

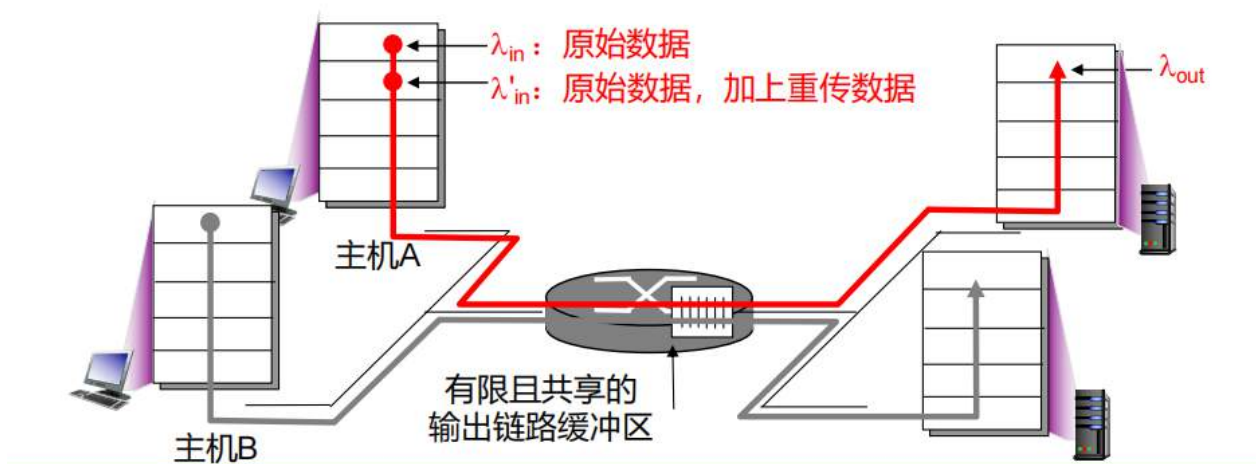
当接近的时候，因为突发业务时延直线上升

场景2

过量的数据导致，代价是多次重传

当路由器是有限缓冲区

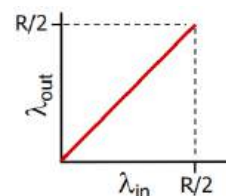
- 一个路由器，有限缓冲区
- 发送方重传超时的数据分组
 - 传输层输入，包括重传部分，即 $\lambda'_{in} \geq \lambda_{in}$
 - 应用层输出 = 应用层输入，即 $\lambda_{out} = \lambda_{in}$



假设1:

理想假设：缓冲信息完全可知

- 发送方仅在路由器缓冲区有空闲时进行发送



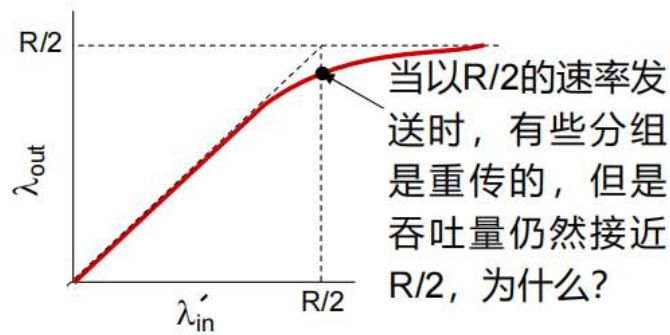
此时不会发生丢包，那么也不会发生数据重传， $\lambda_{in} = \lambda'_{in}$ ，因为已知缓冲区信息，因此 λ_{in} 可以保证在 $R/2$ 之内

假设2:

理想假设：丢包信息可知

- 数据分组由于缓冲区满，可能发生丢包或被丢弃
- 发送方仅重传，那些已知发生了丢包的数据分组

缓冲信息不知道的时候，可能发生丢包， $\lambda_{in} \leq \lambda'_{in}$



因为我知道丢了哪些包，因此总共传输的还是那么多我需要的包，不会重复传已经传过去的

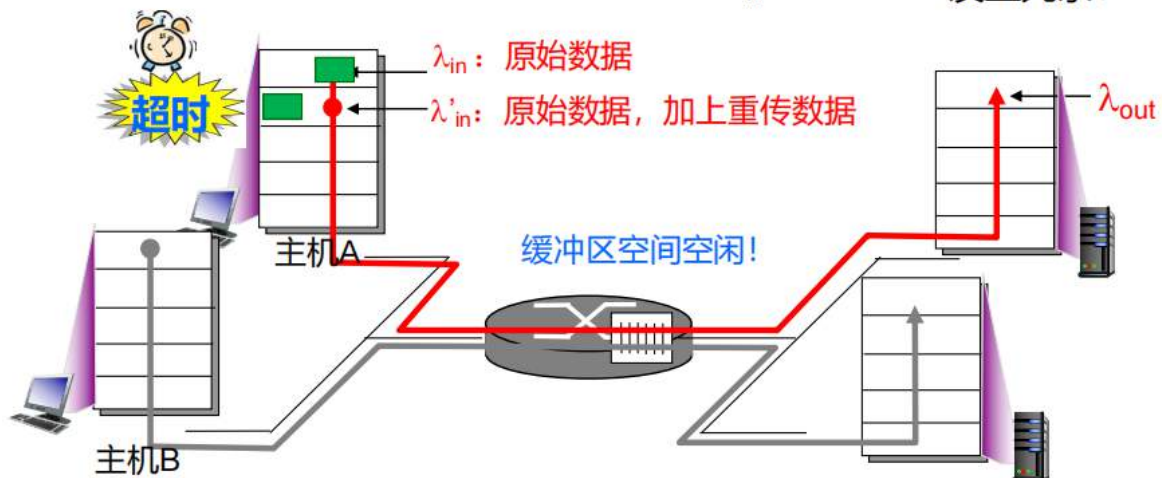
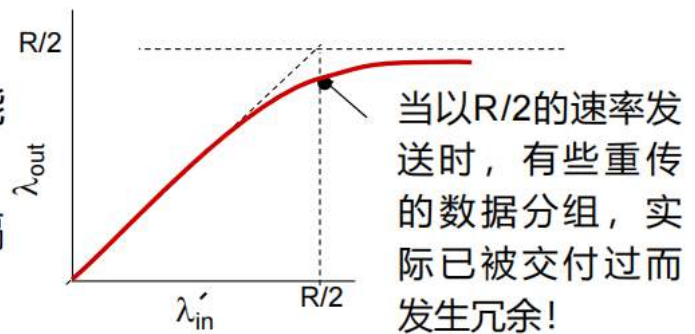
实际上：

代价是带宽

我们并不知道哪些包被丢了，只能根据超时情况进行重传，会出现冗余

实际情况：冗余

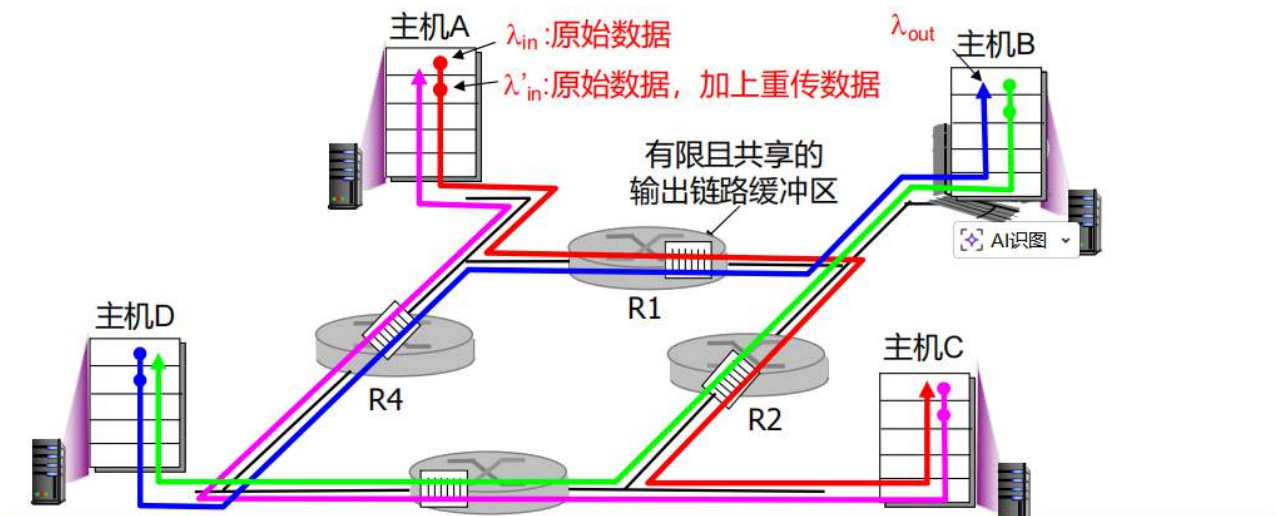
- 数据分组由于缓冲区满，可能发生丢包或被丢弃
- 发送方可能由于超时时间间隔过短，发送了分组及其副本



场景3

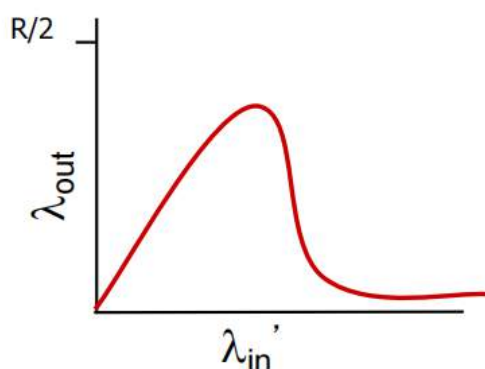
过多的源端导致的，代价是上游源端

- 四个发送方 问题：当 λ_{in} 和 λ'_{in} 增长时会发生什么？
- 多跳路径 回答：随着红色 λ'_{in} 的增加，所有到达上层队列的蓝色
- 超时/重传 分组都被丢弃，蓝色部分的吞吐量 $\rightarrow 0$



多跳路径指从起点到终点之间，数据包或信号需要经过一个或多个中间节点转发才能到达目标的传输路径。

我们可以看A to C 和 D to B都经过R1，A的更近，因此会占据R1更多的缓冲区，会将蓝色的那条路径挤出，蓝色那条路上的上游R4处理数据的很快，但最后都在R1丢包，最终都被浪费了



后续吞吐量越大被挤出去了

当数据分组被丢弃时，用于该分组的“上游”传输带宽都被浪费了！

拥塞的代价：

- 场景1：排队时延
- 场景2：给定“有效吞吐量”下的重传
 - 意味着额外的工作
- 场景2：冗余分组下的非必要链路带宽
 - 实际由重传引发
 - 降低有效吞吐量
- 场景3：上游传输

两种控制策略

端到端的拥塞控制

- 控制机制没有来自网络层的明确反馈
- 网络拥塞是从端系统观察到的丢包、延时等推断出来的
- 典型代表：TCP

网络辅助的拥塞控制

- 路由器向端系统反馈
 - 指示拥塞的单个比特，例如SNA、DECbit、TCP/IP ECN、ATM
 - 显式指定发送方的发送速率

端对端：一端根据当前的丢包，延时状态来调整接收和发送策略（TCP）

辅助控制：路由器向端系统反馈（ATM）

快速分组交换网ATM的ABR拥塞控制

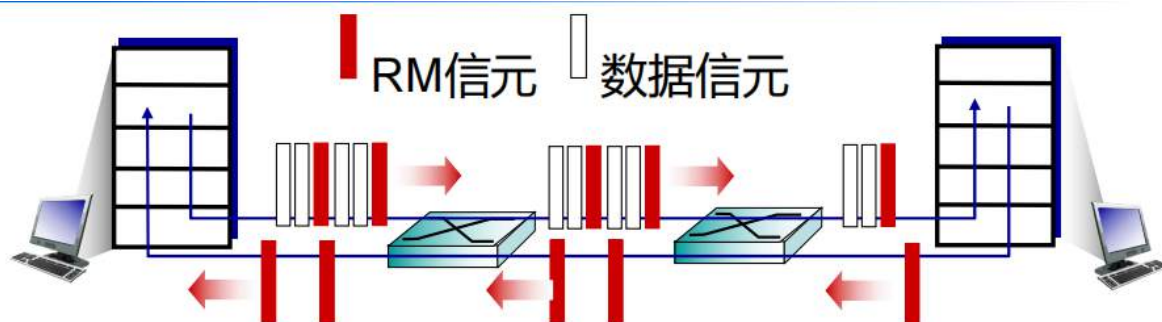
可用比特率 (Available Bit Rate, ABR) :

- 一种“弹性服务”
- 若发送方路径“轻载”:
 - 发送方应当尽可能使用可用带宽
- 若发送方路径“拥塞”:
 - 发送方应当减小至最小保障速率

资源管理 (Resource Management, RM) 信元:

- 发送方在数据信元中, 分散发送RM信元
- 交换机设置RM信元的比特位, 即“网络辅助”策略
 - NI位: 轻拥塞, 速率不增加
 - CI位: 指示拥塞
- 接收方将RM信元, 原封不动地反馈给发送方

上面的资源管理需要一个RTT来完成RM信元的反馈, 有些慢



- 数据信元中的EFCI (显式前向拥塞指示) 位: 由发生拥塞的交换机设置为1
 - 如果数据信元在RM信元之前, 进行了指示拥塞的EFCI置位, 那么接收方将在反馈的RM信元中进行CI置位
- RM信元有2字节的ER (显式速率) 字段
 - 拥塞的交换机可能降低信元中的ER值
 - 发送方的发送速率, 由此受到路径上最大支持速率的限制

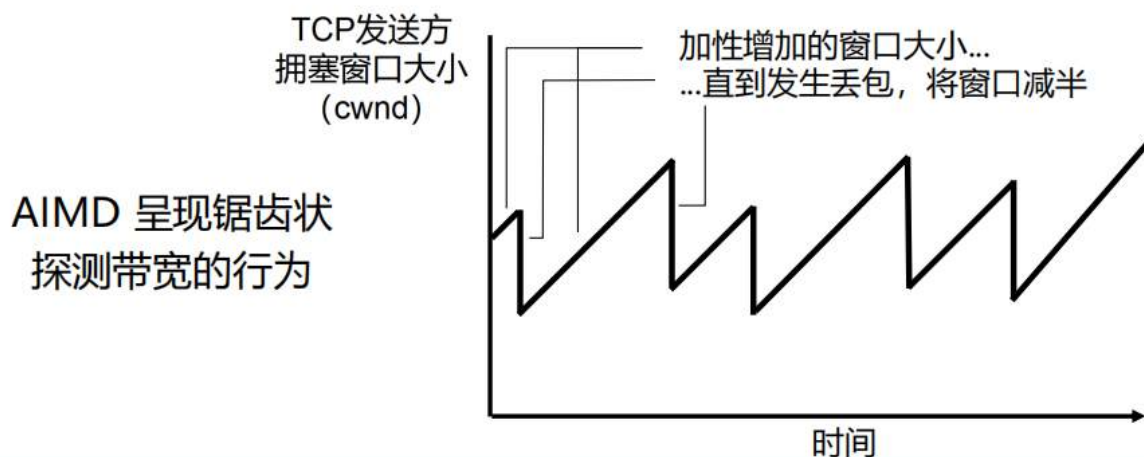
TCP拥塞控制

方法

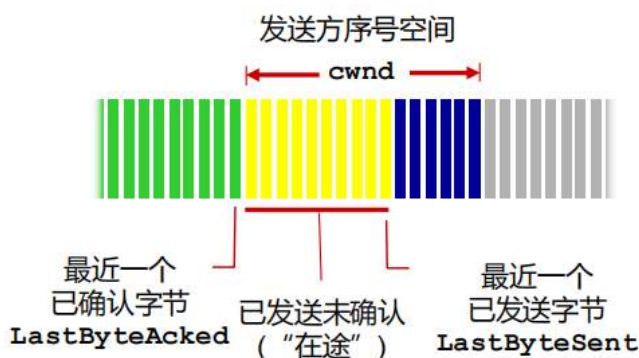
AIMD（拥塞减免）：加性增加，乘性减小

方法：发送方（通过窗口大小）增加发送速率，探测可用带宽，直至发生丢包

- **加性增加：**每过一个RTT，窗口cwnd增加1MSS，直至检测到丢包
- **乘性减小：**发生丢包后，拥塞窗口cwnd减半



本质上通过这种方式来调整滑动窗口大小



- 窗口 cwnd 动态更新，是感知到网络拥塞程度的函数
- 发送方传输限制

$$\text{LastByteSent} - \text{LastByteAcked} \leq \text{cwnd}$$

拥塞窗口大于等于已发送未确认的窗口大小

TCP发送速率

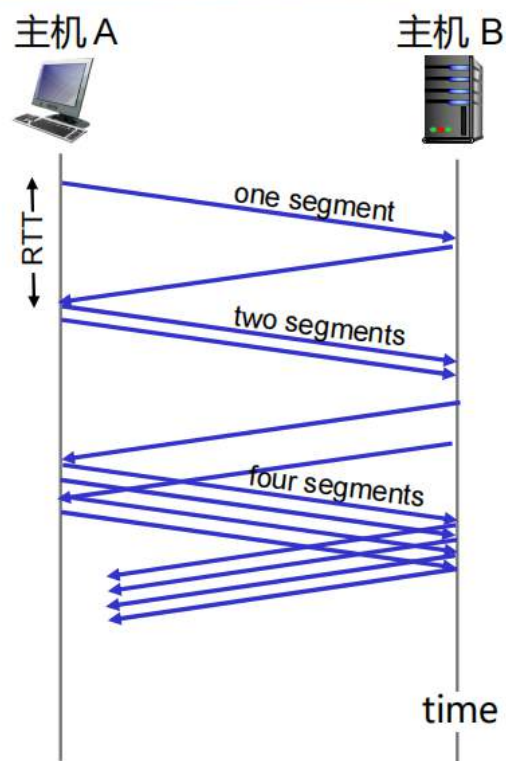
- 简单来讲：发送方首先发送cwnd个字节的数据，然后在一个RTT时长内，等待ACK确认，最后发送更多的字节

$$\text{rate} \approx \frac{\text{cwnd}}{\text{RTT}} \text{ 字节/秒}$$

- 当连接开始时，以指数形式增加速率，直至发生第一次丢包事件

- 初始化 $cwnd = 1 \text{ MSS}$
- 每经过一个 RTT, $cwnd$ 翻倍
- 每收到一个 ACK, $cwnd$ 递增

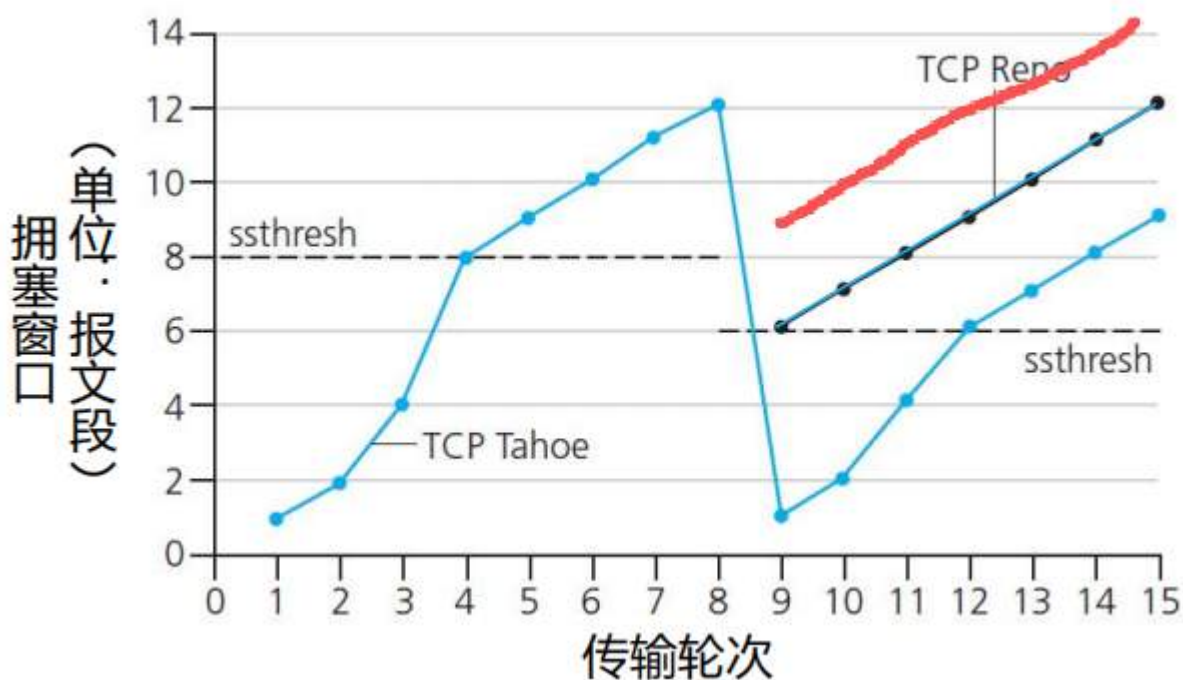
- **小结：**初始速度慢，但是会以指数形式快速上升



丢包的检测与应对

主动的应对策略：我们引入一个 **ssthresh** 值，它作为 $cwnd$ 从指数增长，切换到线性增长的临界点

被动的应对策略：



红色的才表示RENO的真实曲线

注意虽然展示曲线是这样的，但实际上在传输的每一个轮次都会进行超时和冗余ACK

- **Tahoe：**慢启动

丢包——TCP Tahoe (无论是超时, 还是3个冗余ACK)

- **cwnd** 设置为1 MSS;
- **ssthresh** 设置为丢包前 **cwnd** 的 1/2;
- 窗口先指数增长 (同慢启动过程), 到阈值后再切换为线性增长

• **RENO**: 快恢复

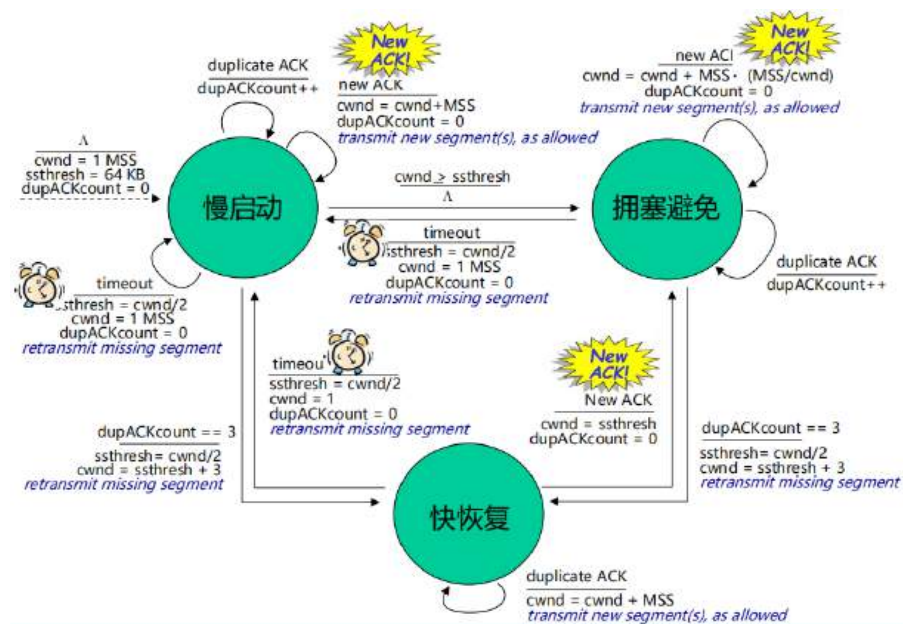
丢包——TCP RENO (当3个冗余ACK时)

- 冗余ACK表示网络虽然拥塞, 但是还能传送一些报文段
- **cwnd** 先减半 (+3), 再线性增长
- **ssthresh** 设置为丢包前 **cwnd** 的 1/2

cwnd = 先减半再加3

Reno的超时处理过程与Tahoe一致

状态机



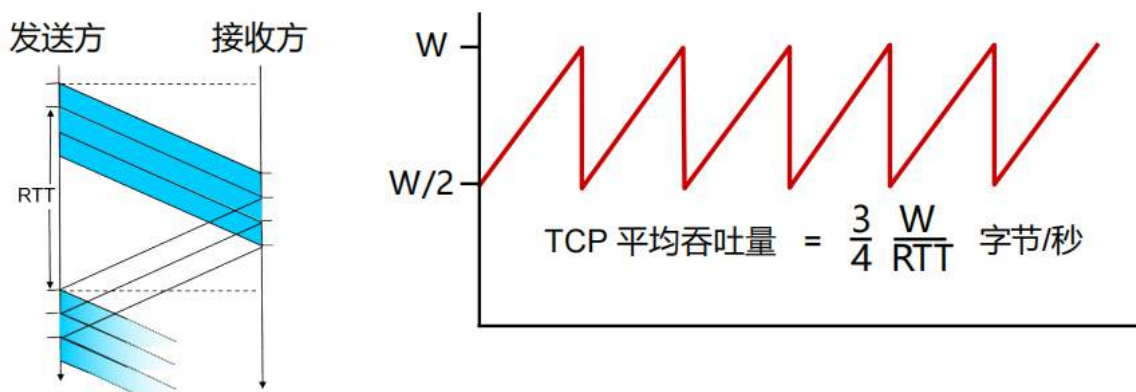
TCP吞吐量

□ TCP平均吞吐量与窗口大小、RTT 有何关系？

- 忽略慢启动过程，假设一直有数据发送

□ W: 发生丢包时，窗口大小（以字节为单位）

- 平均窗口大小为 $\frac{3}{4}W$ （在途数据的字节数）
- 平均吞吐量为 $\frac{3}{4}W$ （每RTT时间）



1. 为什么平均窗口大小是 $\frac{3}{4}W$ ？

在拥塞避免阶段，TCP 窗口的变化遵循 **AIMD（加性增、乘性减）** 原则：

- **线性增长**：窗口从 $\frac{W}{2}$ 开始线性增加，直到达到网络容量临界点 W 。
- **减半重置**：一旦检测到丢包，窗口立即减半，回到 $\frac{W}{2}$ 。

由于窗口在 $[\frac{W}{2}, W]$ 之间均匀线性变化，其**算术平均值**自然就是：

$$\text{Average Window} = \frac{\frac{W}{2} + W}{2} = \frac{3}{4}W$$

2. 吞吐量公式的推导

吞吐量（Throughput）的定义是单位时间内传输的数据量。

- 在一个 RTT 时间内，发送方发送的数据量等于当前的**窗口大小**。
- 既然长期来看平均窗口大小是 $\frac{3}{4}W$ ，那么平均每个 RTT 发送的数据量就是 $\frac{3}{4}W$ （以字节为单位）。
- 因此，**平均吞吐量**的计算公式为：

$$\text{Average Throughput} = \frac{3}{4} \cdot \frac{W}{RTT} \text{ (Bytes/sec)}$$

已知窗口大小W与丢包率L相关，将W写为关于L的函数，可以得到下方的吞吐量和丢包率的关系

- 示例（字节流经过“又长又粗”的管道）：报文段长度为1500字节，往返时间 RTT 为100ms，期望达到 10 Gbps 的吞吐量
- 用报文段丢失概率L表示的吞吐量[Mathis 1997]

$$\text{TCP 吞吐量} = \frac{1.22 \times \text{MSS}}{\text{RTT} \sqrt{L}}$$

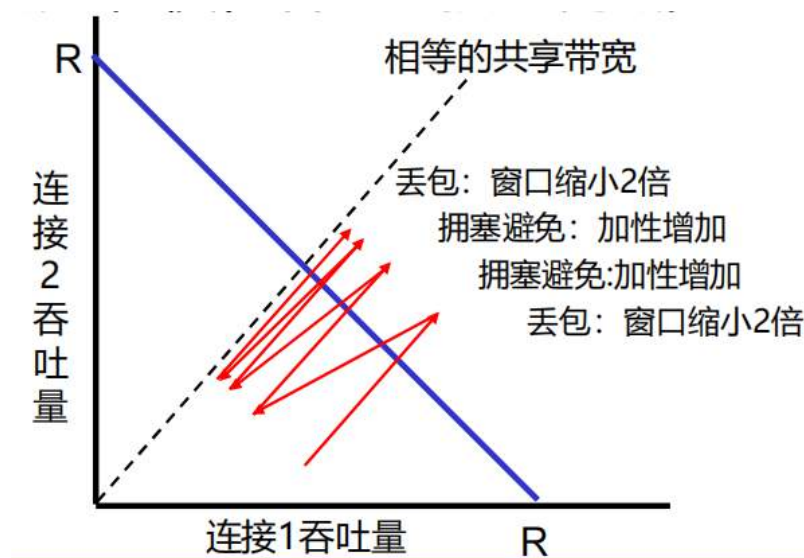
以上结论意味着：想要实现10 Gbps的吞吐量，要求丢包率 $L = 2 \cdot 10^{-10}$ —— **非常小的丢包率！**

- 要求 $W = 83,333$ 在途报文段——支持高速的TCP新版本

TCP公平性

假设有多个连接，那么这些连接在传输数据的时候公平吗？

是公平的



在整个发送过程中，两个连接的最终的吞吐量会收敛于蓝色实线和虚线的交点

TCP的公平和UDP的丢包

以多媒体应用为例：

- 通常不用TCP
 - 不希望拥塞控制限制了速率
- 而是使用UDP
 - 以恒定速率发送音频/视频，并允许丢包

公平性和TCP并行连接

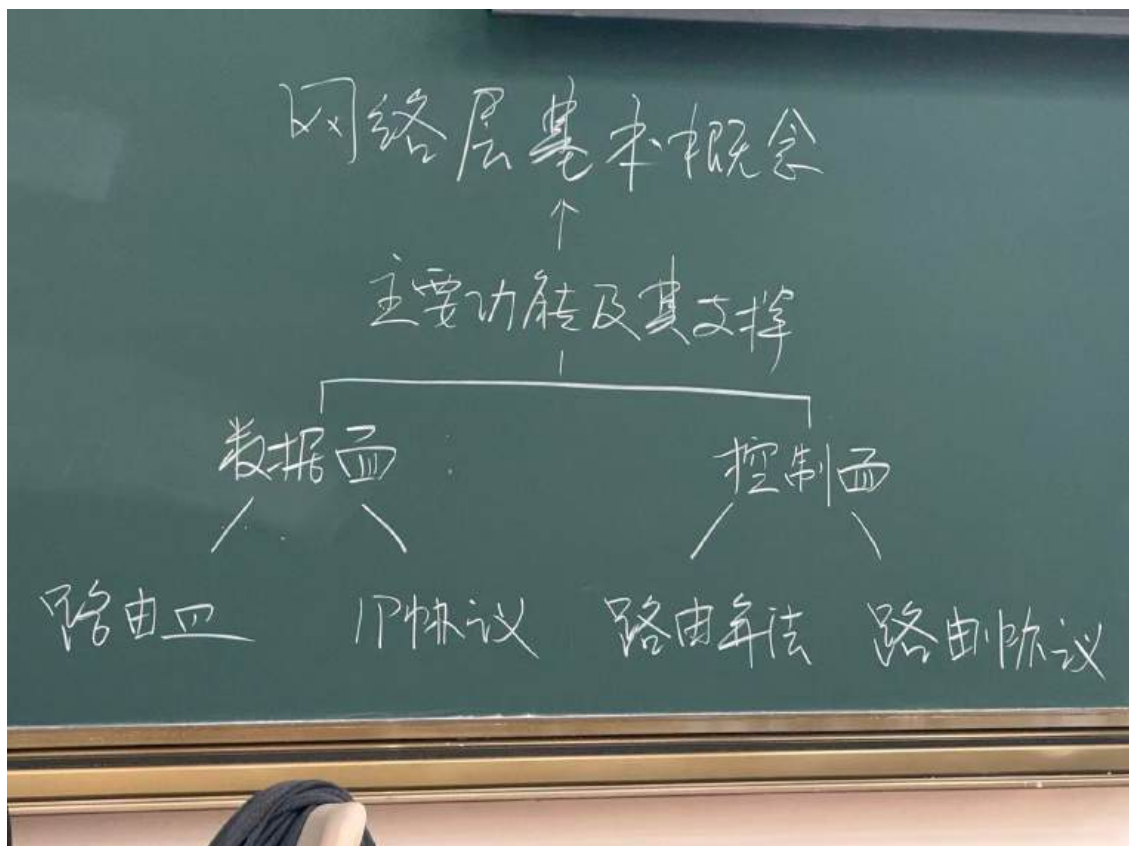
以网页浏览器为例：

- 应用可以在两台主机之间打开多个并行的连接
- 设想，一个速率为 R 的链路现有 9 个连接，一个新的应用请求连接
 - 新应用要求1个TCP，获得速率为 $R/10$
 - 新应用要求11个TCP，获得速率为 $R/2$

所谓的公平指的是每个连接之间是公平，并不代表每个应用之间是公平的

网络层

概述



数据面的讨论站在路由器的角度上，讨论数据的处理

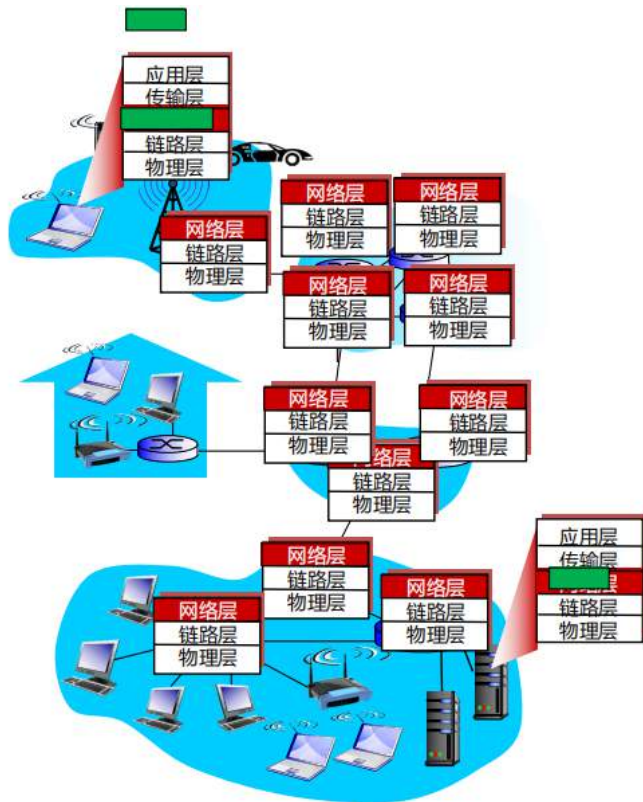
控制面的讨论站在整个网络上，讨论网络的路由，怎么从一个端到另一个端

从发送端到接收端，承担报文段的传送任务

- 在发送端，将报文段封装成数据报
- 在接收端，将报文段交付至传输层

网络中的每个主机、路由器，都有网络层协议

路由器对所有途径此处的IP数据报，检查首部字段



主要功能

转发：将数据报从路由器的入口移动到适当的路由器出口

路由：确定分组从源端到目的端的路径

- 路由算法
- 路由协议

类比：

- **转发：**指分组通过单个立交桥的过程
- **路由：**指规划从出发地到目的地行程的过程

- 转发指的是路由器内部的端口映射策略
- 路由指的是整个网路上数据报的路径

建立连接

在传送数据报之前，源端、目的端与中间经过的路由器，会建立虚拟连接

- 路由器参与连接

网络层与传输层的连接服务

- **网络层**：为两个主机提供服务
(在虚拟连接的情况下，可能涉及途径的路由器)
- **传输层**：为两个进程提供服务

服务模型

问题：从发送方到接收方传输数据报，“信道”采用了什么样的**服务模型**？

单数据报的服务举例：

- 保证交付
- 保证时延小于40ms的交付

数据报流的服务举例：

- 数据报的有序交付
- 确保流的最小带宽
- 限制分组间时间间隔的变化
- 安全

其实意思就是有什么要求，能做什么

网络架构	服务模型	是否能保证?				拥塞反馈
		带宽	(不)丢包	有序	实时	
互联网	尽力而为	否	否	否	否	否 (由丢包推断)
ATM	CBR	恒定速率	是	是	是	无拥塞
ATM	VBR	保证速率	是	是	是	无拥塞
ATM	ABR	保证最小速率	否	是	否	是
ATM	UBR	否	否	是	否	否

这个表就体现了不同的网络架构提供的服务，不同的服务模型有规定的能提供的全部服务

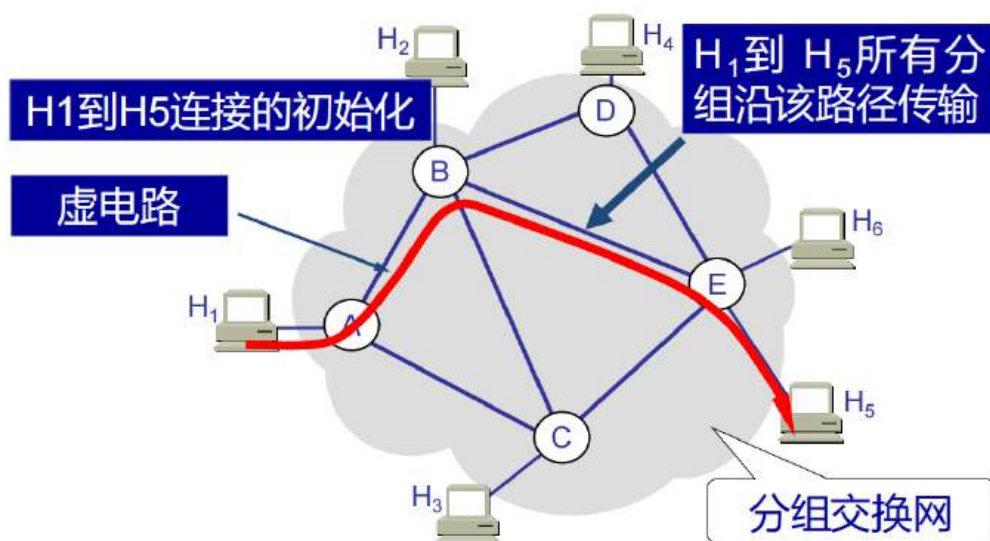
面向连接和无连接服务

- 虚电路提供了网络层面向连接的服务
- 数据报提供了网络层无连接服务
- 与传输层TCP的面向连接服务和UDP的无连接服务类似，不过
 - 服务角度：网络为主机到主机提供服务
 - 选择角度：网络选择其中之一提供服务
 - 实现角度：位于网络核心

虚电路是建立连接的，因此需要按照连接好确定的路径走

数据报没有链接，可以根据当时的网络环境进行传输，因此比较灵活

虚电路



源端到目的端之间的路径，表现如同电话线路一般

- 性能方面的保障
- 源端到目的端路径上，网络的具体操作

- 在传输数据之前，每条连接都需要建立和释放
- 源端到目的端路径上的每个路由器，为每个途径的连接维护着连接的“状态”
- 虚电路可能分配相应的链路带宽、路由器缓冲等资源，这些专用的资源意味着可预测的服务
- 每个分组携带虚电路标识符（而不是目的主机地址）

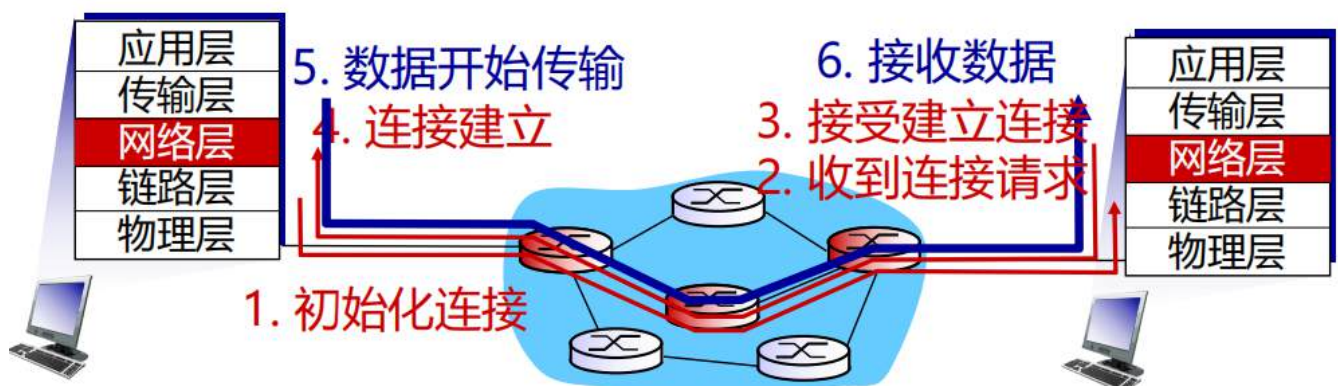
一条虚电路包括：

1. 从源端到目的端的**路径**
 2. 指示路径上沿途链路的**虚电路号**
 3. 指示路由器中基于虚电路号的**转发表项**
- 属于某条虚电路的分组，携带着虚电路号（而不是目的地址）
 - 每条链路上的虚电路号，可能发生变化
 - 新的虚电路号来自转发表

虚电路有整个完整路径，具体包括虚电路号（每一条小路的标识），还有在路由器内部怎么转发

虚电路称为信令协议

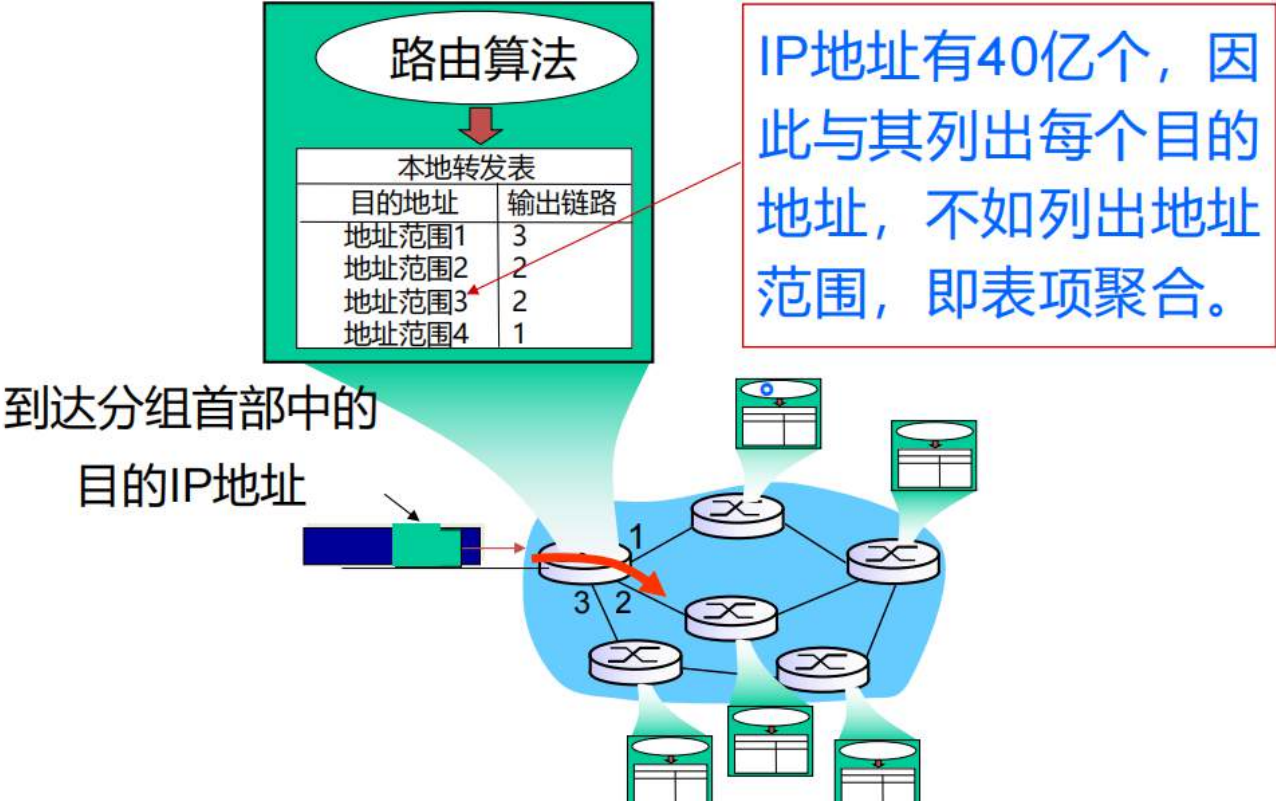
- 用于建立、维护、关闭虚电路
- 用于ATM、帧中继、X.25网络
- 在当今的互联网中不使用



数据报

每一个包的路径都可能不同

数据报的本地转发表



如果每个IP都分配一个输出链路，那么数量太大了

因此我们采用一个范围给一个链路，这样的话后面通过夹逼能找到对应的

但这么做这个范围内的都会走这条路，可能有点挤，但这是值得的

目的地址范围	链路接口
11001000 00010111 00010000 00000000 到 11001000 00010111 00010111 11111111	0
11001000 00010111 00011000 00000000 到 11001000 00010111 00011000 11111111	1
11001000 00010111 00011000 00000000 到 11001000 00010111 00011111 11111111	2
其它	3

最长前缀匹配

最长前缀匹配

针对指定的目的地址，当查找转发表项时，使用匹配目的地址的表项中，具有**最长**地址前缀的那个。

目的地址范围	链路接口
11001000 00010111 00010*** *****	0
11001000 00010111 00011000 *****	1
11001000 00010111 00011*** *****	2
其它	3

例如：

目的地址: 11001000 00010111 00010**110 10100001** 哪一个接口？

目的地址: 11001000 00010111 00011**000 10101010** 哪一个接口？

假设一个地址被多个范围所包括，找那个能匹配最长前缀的的链路去转发

选择数据报还是虚电路？

互联网（数据报）：

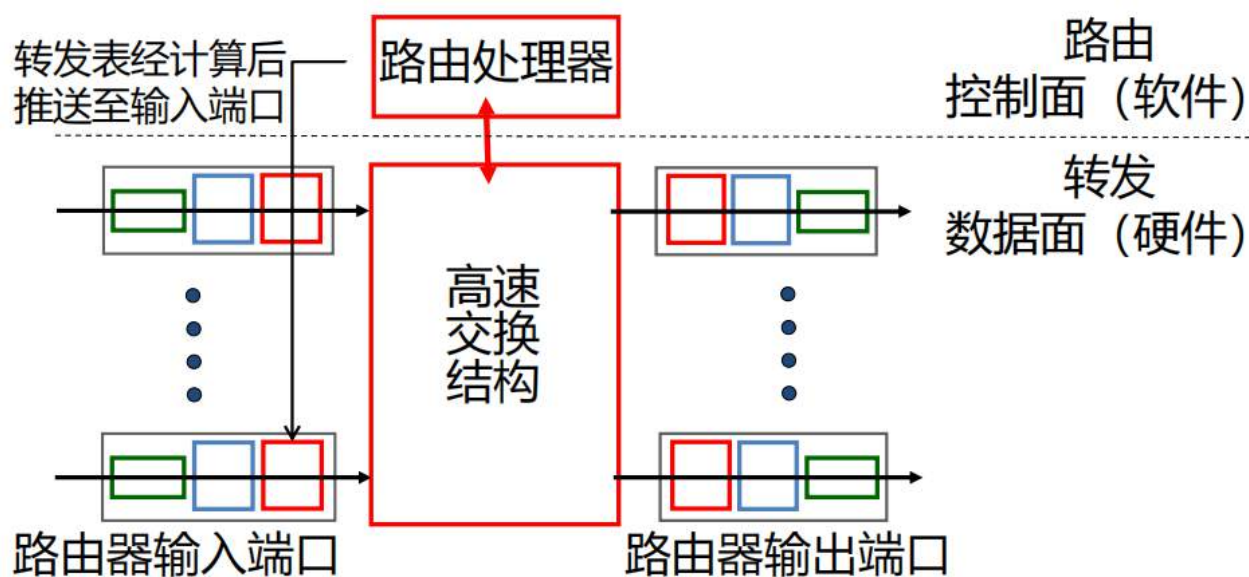
- 计算机之间交互数据
 - 弹性服务，没有严格的实时性要求
- 多种链路类型
 - 不同链路特性
 - 难以统一服务
- 计算机等“高级”的端系统
 - 能够实现控制、自适应、错误恢复等操作
 - 网络内部简单、边缘复杂

ATM网（虚电路）：

- 起源于电话网
- 人类对话
 - 严格的实时性、可靠性要求
 - 需要保障性服务
- 电话等“简单”的端系统
 - 网络内部较为复杂

对比内容	虚电路服务	数据报服务
可靠传输的保证	可靠通信由网络保证	可靠通信由主机保证
连接的建立	必须要	不需要
地址	每个分组含有一个短的虚电路号	每个分组需要有源地址和目的地址
状态信息	建立好的虚电路要占用子网表空间	子网不存储状态信息
路由选择	分组必须经过建立好的路由发送	每个分组独立选择路由
分组顺序	总是按序到达	可能乱序
路由器失效	所有经过失效路由器的虚电路都要终止	失效结点可能丢失分组
差错处理和流量控制	网络或用户主机负责	用户主机负责
拥塞控制	容易控制	难控制

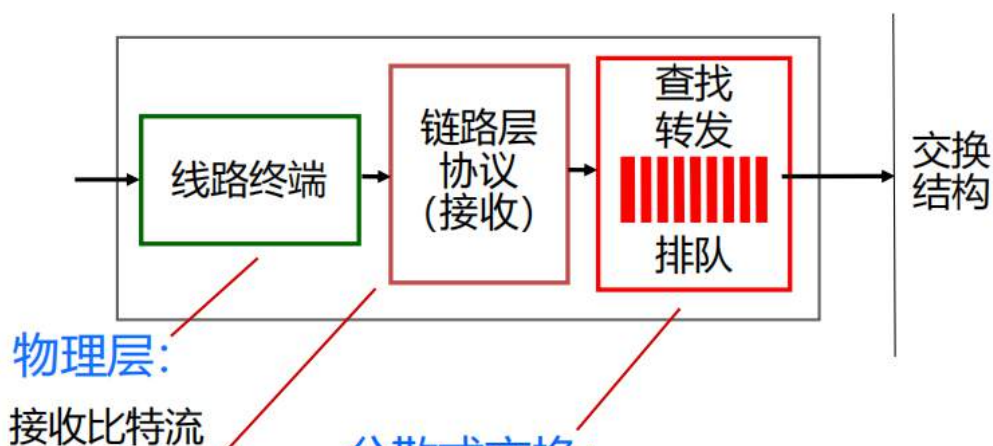
路由器工作原理



路由器的两个关键功能：

- 运行路由算法/协议，例如RIP、OSPF、BGP等
- 将数据报从输入链路转发到输出链路

输入端口

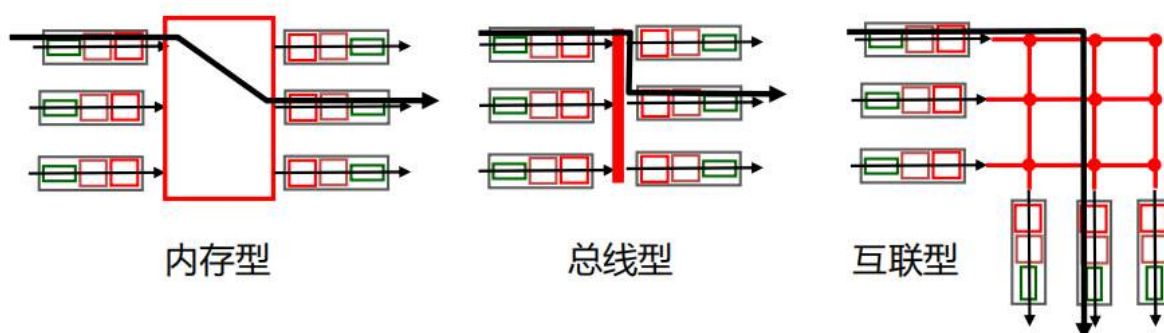


分散式交换:

- 针对给定的数据报目的地址，根据输入端口内存中的转发表，查找相应的输出端口
- 目标：以“线路速度”完成输入端口的处理
- 排队：如果数据报到达的速度大于转发至交换结构的速度，那么可能排队等待

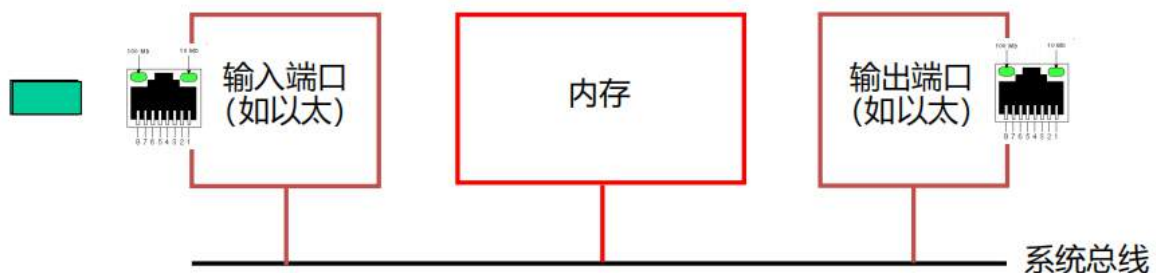
交换结构

- 将数据报从输入缓冲区传送到相应的输出缓冲区
- 交换速度：数据报从输入端口到输出端口的传送速度
 - 通常以输入/输出线路速率的倍数来衡量
 - N 输入：交换速率 N 倍于理想线路速率
- 三种类型的交换结构：



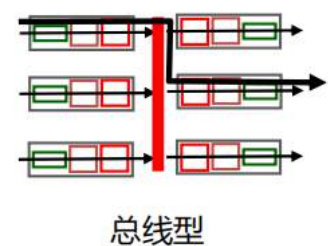
内存型

- ❑ 传统计算机由CPU直接控制完成交换
- ❑ 数据分组拷贝到系统的内存中
- ❑ 交换速率受限于内存带宽（每个分组跨越2次总线）

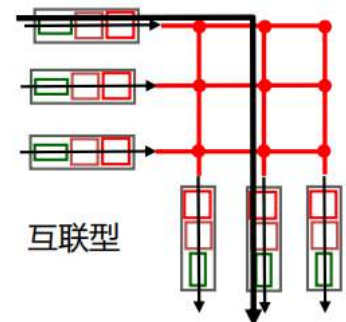


总线型

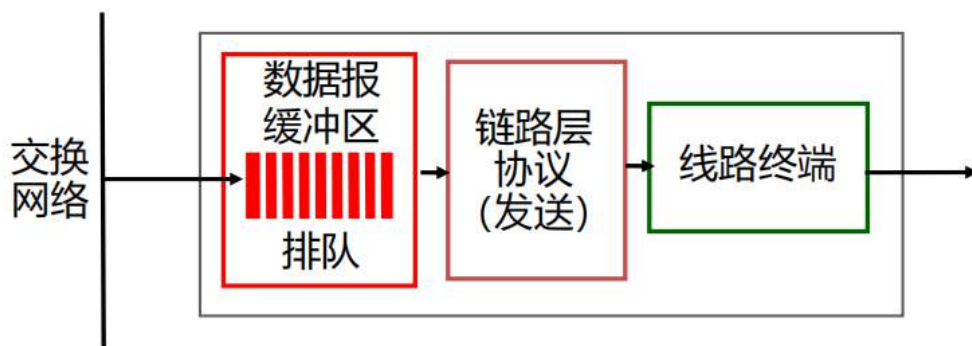
- ❑ 数据报通过共享总线从输入端口的内存转发到输出端口的内存
- ❑ **总线竞争：**
交换速率受限于总线带宽
- ❑ 以思科5600路由器为例：它具有32Gbps总线，速度能够满足接入网和企业级路由器的需求



- ❑ 克服了总线带宽的限制
- ❑ 多处理器早期采用Banyan网络、交叉开关矩阵、其他互联网络等，连接多个处理器
- ❑ 高级设计：
先将数据报分割为固定长度的信元，再通过交换结构传输信元
- ❑ Cisco 12000：通过互联网络其交换带宽达到60Gbps。



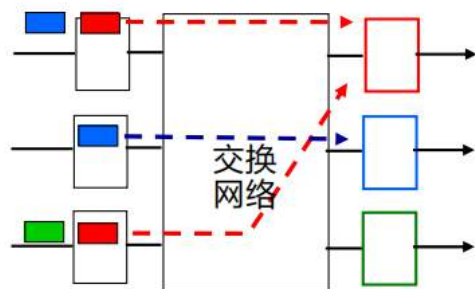
输出端口



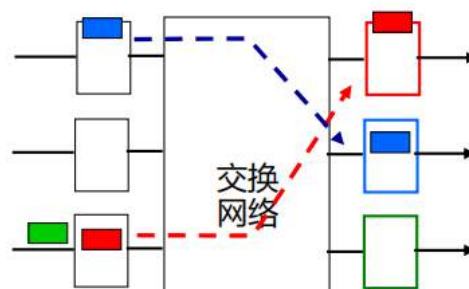
- ❑ **缓冲**：当数据报从交换结构中到达的速度大于发送速度时，数据报需要在缓冲区排队
数据报可能因拥塞、缓冲区不足等而丢失
- ❑ **调度**：依据调度规则，从队列中选要传的数据报
基于网络性能、公平性等进行的优先调度

端口处的排队

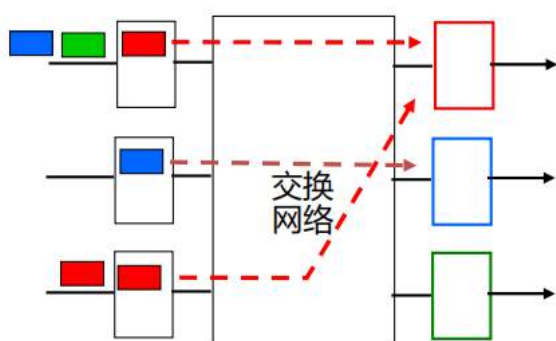
- 当输入端口的速度大于交换结构的速度时，分组可能在输入端口处发生排队
 - 分组可能由于输入端口缓冲溢出，产生排队时延甚至丢包!
- 线路头端（HOL）阻塞：处在队首的数据报阻碍了队列中其他数据报的转发



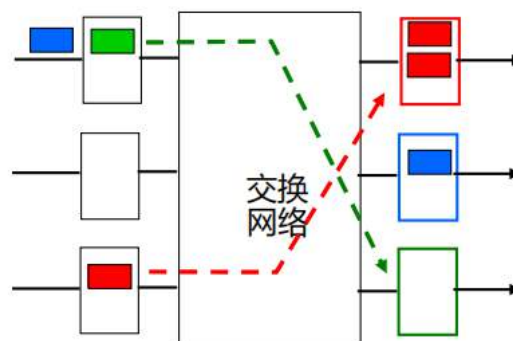
输出端口争用：只能传输一个红色数据报，下方红色数据报被阻塞



一个分组时间后：绿色数据报经历线路头端阻塞



在t时刻，更多分组从输入端口传送到输出端口



一个分组时间后

- 当经交换网的到达速度大于输出线路的速度时进行缓存
- 分组可能由于输出端口缓冲溢出，产生排队时延甚至丢包

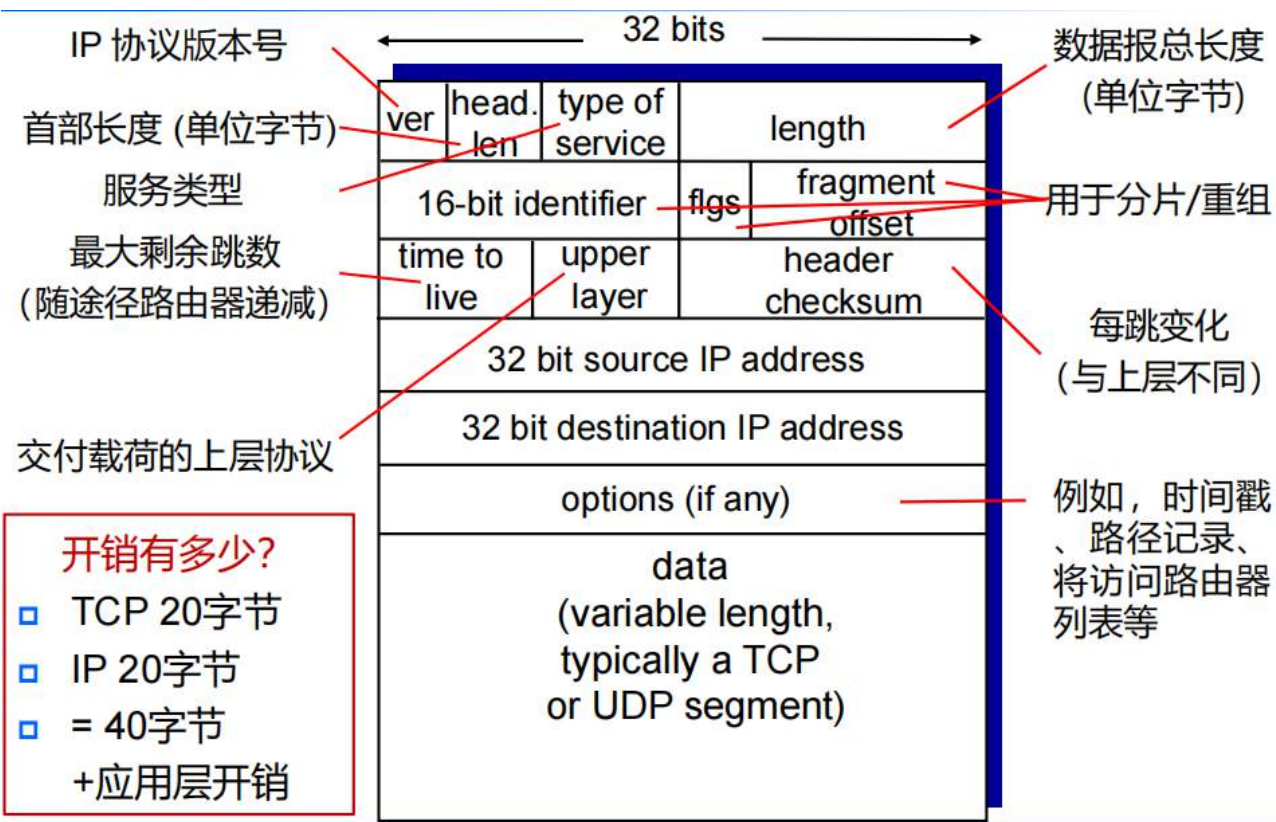
缓冲区的计算

- RFC-3439的经验法则：平均缓冲区大小等于典型的往返时间RTT（例如 250 毫秒）乘以链路容量C
 - 缓冲区大小= $RTT \cdot C$
 - 例如，一条C=10Gpbs的链路，缓冲区为2.5Gbit
- 目前的推荐规则：当有N个流时，缓存大小为

$$\frac{RTT \cdot C}{\sqrt{N}}$$

互联网网际协议IP

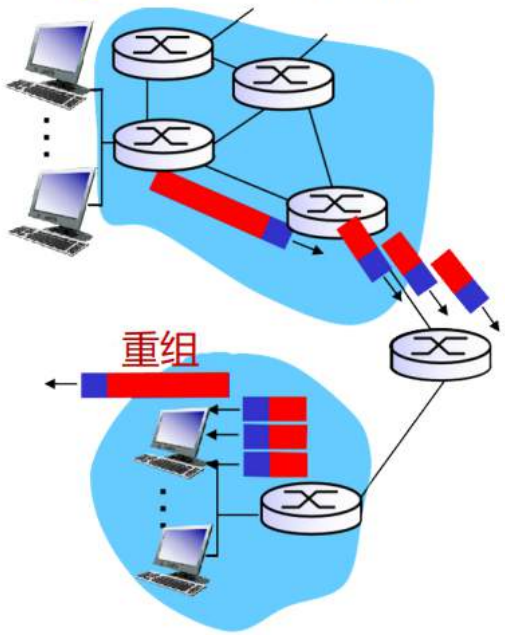
数据报格式



IP分片、重组

示例:

- 4000字节数据报
- MTU = 1500字节



length	ID	fragflag	offset
=4000	=x	DF=0	=0

一个较长的数据报成为几个较短的数据报

length	ID	fragflag	offset
=1500	=x	MF=1	=0
=1500	=x	MF=1	=185
=1040	=x	MF=0	=370

数据字段1480 字节 偏移量=1480/8

假设我们一共有4000字节的数据报（有效位数为3980位）

我们现在以1500字节为我们单个分组的最大长度

也就是每个分组的有效长度为1480

因此我们需要1480+1480+1040这样的分组形式

但是这三个数据异步发送，有可能会出现乱序问题

我们就通过MF标志位告诉接收端是不是最后一个，通过offset来判断顺序

但是如果这个数据在整个原始数据的偏移量很大，那么这个offset这个字节本身的长度就记录不下

因此我们规定offset = 对应原本数据的偏移量 / 8，接收端只需要乘8就可以知道它在原本数据的哪个位置

因此我们在分片的时候，必须要保证每个分片的长度必须是8的倍数

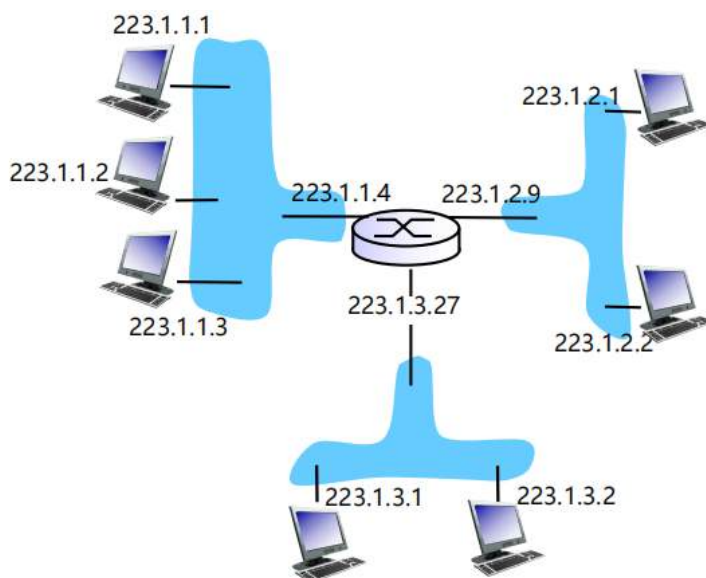
IP地址

同一个局域网下，网络号相同，主机号不同

□ **IP地址**：指示主机接口、路由器接口的32位标识符

□ **接口**：主机/路由器与物理链路之间的连接

- 路由器通常有多个接口
- 主机通常有一个或两个接口，例如有线的以太网、无线的802.11



$$223.1.1.1 = \underbrace{11011111}_{223} \underbrace{00000001}_1 \underbrace{00000001}_1 \underbrace{00000001}_1$$

□ **IP地址关联每个接口**

类别	起始位	网络号 字段大小	剩余字 段大小	网络数量	每个网络中 地址数量	IP地址范围	私有地址
A类	0	8	24	128 (2^7-2)	16,777,216 ($2^{24}-2$)		
B类	10	16	16	16,384 ($2^{14}-2$)	65,536 ($2^{16}-2$)		
C类	110	24	8	2,097,152 ($2^{21}-2$)	256 (2^8-2)		
D类(多播)	1110	未定义	未定义	未定义	未定义		
E类(保留)	1111	未定义	未定义	未定义	未定义		

1. 为什么是 $2^{(n-1)}$? (识别码占位)

在分类 IP 地址中，前几个位是用来告诉路由器“我是哪一类”的。

- **A类**：规定起始位必须是 0。虽然它的网络号字段大小是 8 位，但第一位被固定成了 0。所以真正能变动的只有 $8 - 1 = 7$ 位。因此网络数量是 2^7 。
- **B类**：规定起始位必须是 10。网络号总长度 16 位，固定了 2 位，剩下的变动位是 $16 - 2 = 14$ 位。因此网络数量是 2^{14} 。
- **C类**：规定起始位必须是 110。网络号总长度 24 位，固定了 3 位，剩下的变动位是 $24 - 3 = 21$ 位。因此网络数量是 2^{21} 。

总结：识别码占据了 m 位，剩下的可选范围就是 $2^{(n-m)}$ 。

2. 为什么还要 -2? (特殊地址预留)

在计算机网络原理中，通常会减去两个特殊的网络号（注意：这在不同版本的教材和标准中可能略有差异，通常指的是 A 类）：

1. **全 0 的网络号**：表示“本网络 (This Network)”，不能分配。
2. **全 1 的网络号 (127)**：被保留用于本地软件环回测试 (Loopback Test)，即我们常用的 127.0.0.1。

小贴士：实际上在 B 类和 C 类中，现代标准通常不再严格要求 -2（因为全 0 或全 1 的网络号在子网划分中已经可以利用），但在考试或传统教材里，为了严谨，通常会统一标注 -2。

子网

为什么需要子网？如果一有非常多个设备连在同一台路由器上，那么该路由器要记住全部的映射关系

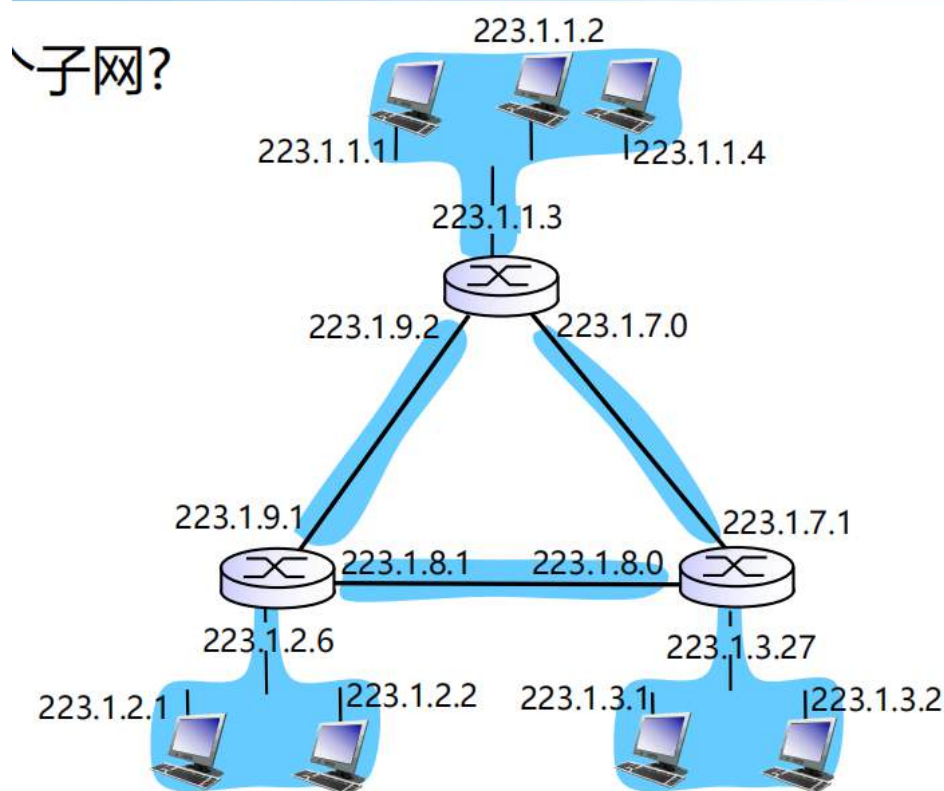
我们将多台设备打包，每个包有一个共同长度的高位，这样上一层路由器就只需要知道往哪一个包发就可以了，具体这个包内怎么处理，由这个包内的路由器决定

我们将ip地址的高位放子网部分（用于标识哪一个子网），低位放主机部分（用于标识访问这个子网的哪一台主机）

假设我们想要这个ip地址的前n位标识是哪一个子网，那么后面的位数就标识是哪一台主机

这个前n位我们全部放1，后面全部放0，这个就是**子网掩码**

我们通过子网掩码和ip地址作按位与，就可以得到对应的子网标识



假设我们的子网掩码是255.255.255.0

那么我们只要看xxx.xxx.xxx这样的ip地址一共有几种，就代表有几个子网

这里就是6个子网

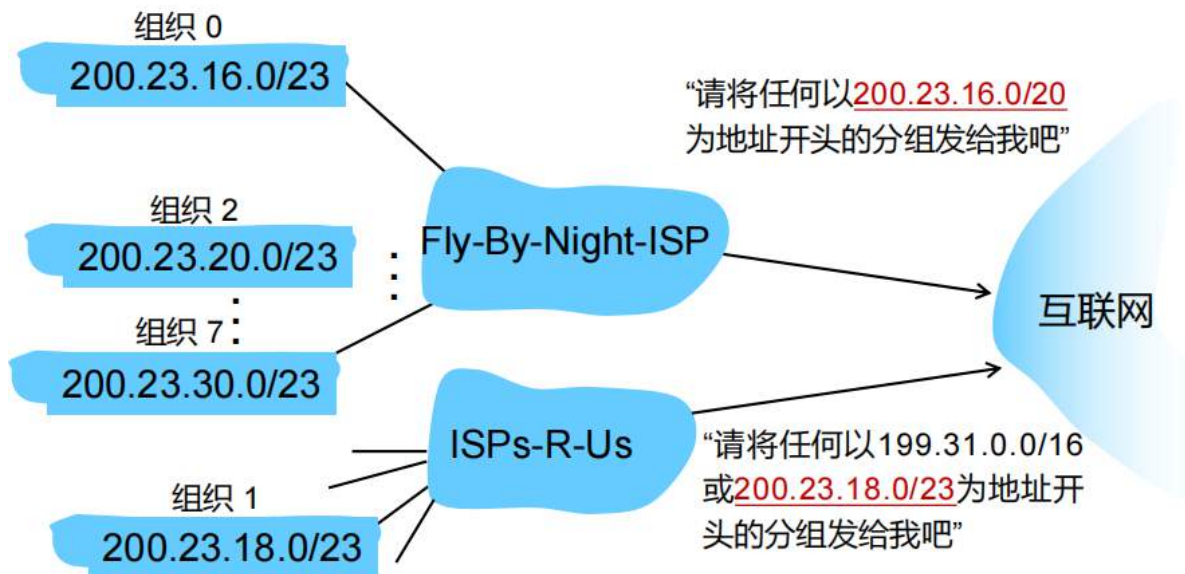
CIDR

无类域间路由 (Classless InterDomain Routing, CIDR)

- IP地址中的网络部分：任意长度
- 地址格式：a.b.c.d/x，其中x是IP地址网络部分的位数



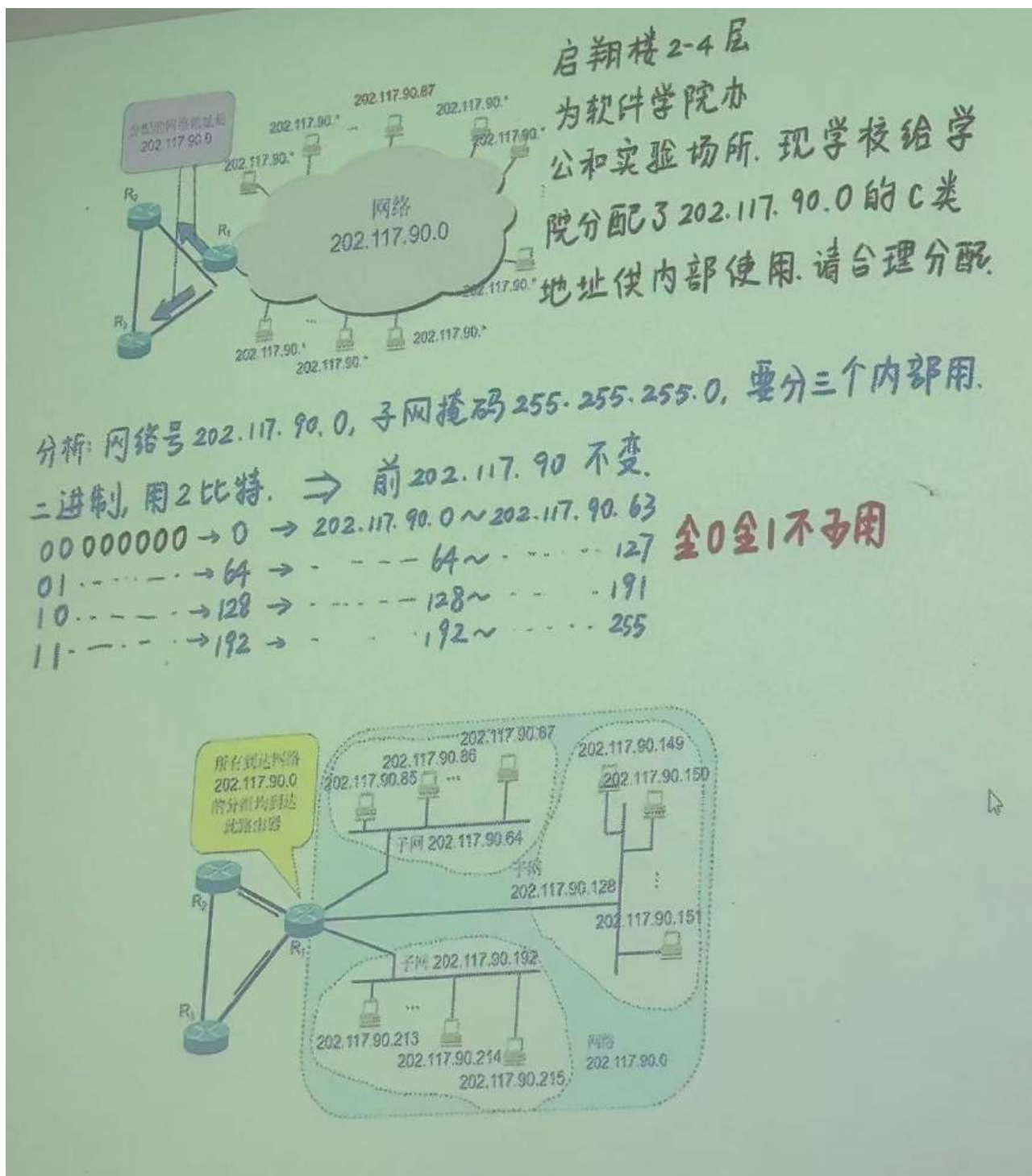
分层编址



ISP块	<u>11001000</u>	<u>00010111</u>	<u>00010000</u>	00000000	200.23.16.0/20
组织 0	<u>11001000</u>	<u>00010111</u>	<u>00010000</u>	00000000	200.23.16.0/23
组织 1	<u>11001000</u>	<u>00010111</u>	<u>00010010</u>	00000000	200.23.18.0/23
组织 2	<u>11001000</u>	<u>00010111</u>	<u>00010100</u>	00000000	200.23.20.0/23
...					
组织 7	<u>11001000</u>	<u>00010111</u>	<u>00011110</u>	00000000	200.23.30.0/23

通过一层层的最长前缀匹配进行路由

如果同时匹配多个ISP块，遵循最长前缀匹配，越具体越好



假设我们每一层要有一个子网，那么子网本身从给整个子网分的所有ip地址的的第一个开始，因此其中的主机不能占用编码位（就是为了区分子网留出来的位）为全0（网关地址）和全1的ip地址（广播地址：所有子网设备都会接收这个包）（202.117.90.0-202.117.90.63和202.117.90.192-202.117.90.255不能用）

现在要将202.100.192.0/18 分为30个子网

因此我们需要32个编码，也就是需要五位编码

因此子网的网络号为：202.100.192.0/23

对应的子网掩码为：255.255.254.0

如果要将202.100.192.0/18 分为32个子网

因此至少需要34个编码，也就是需要六位编码

因此子网的网络号为：202.100.192.0/24

对应的子网掩码为：255.255.255.0

如果要将192.168.1.0/24 分为五个子网

这五个子网各自要给120，30，30，30，30个节点分配

我们按照分层分配，如果120太多，我们就把它的编码位让其独有

$120+2 < 2^7$

192.168.1.0XXX.XXXX 分给120的这个子网,这样就给其子节点留了7位用来存放120个编码

$30+2 < 2^5$

保证0XXX.XXXX不再出现，剩下的再分就可以

192.168.1.100X.XXXX

192.168.1.101X.XXXX

192.168.1.110X.XXXX

192.168.1.111X.XXXX

DHCP

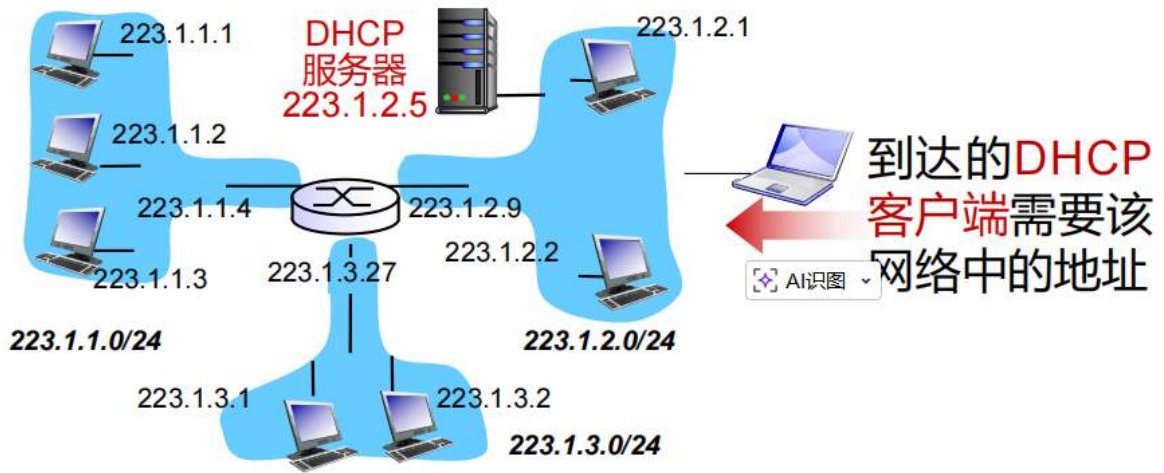
目标：在主机加入网络时，允许其从网络服务器动态获取IP地址

- ▣ 支持移动用户加入网络
- ▣ 可以更新在用地址的租期
- ▣ 允许复用地址（仅在连接“打开”时保留地址）

DHCP不仅分配IP地址，还提供更多子网服务：

- ▣ 客户端第一跳路由器的地址
- ▣ DNS服务器的名称和IP地址
- ▣ 子网掩码

DHCP服务器不仅给主机分配IP地址，还有IP地址资源



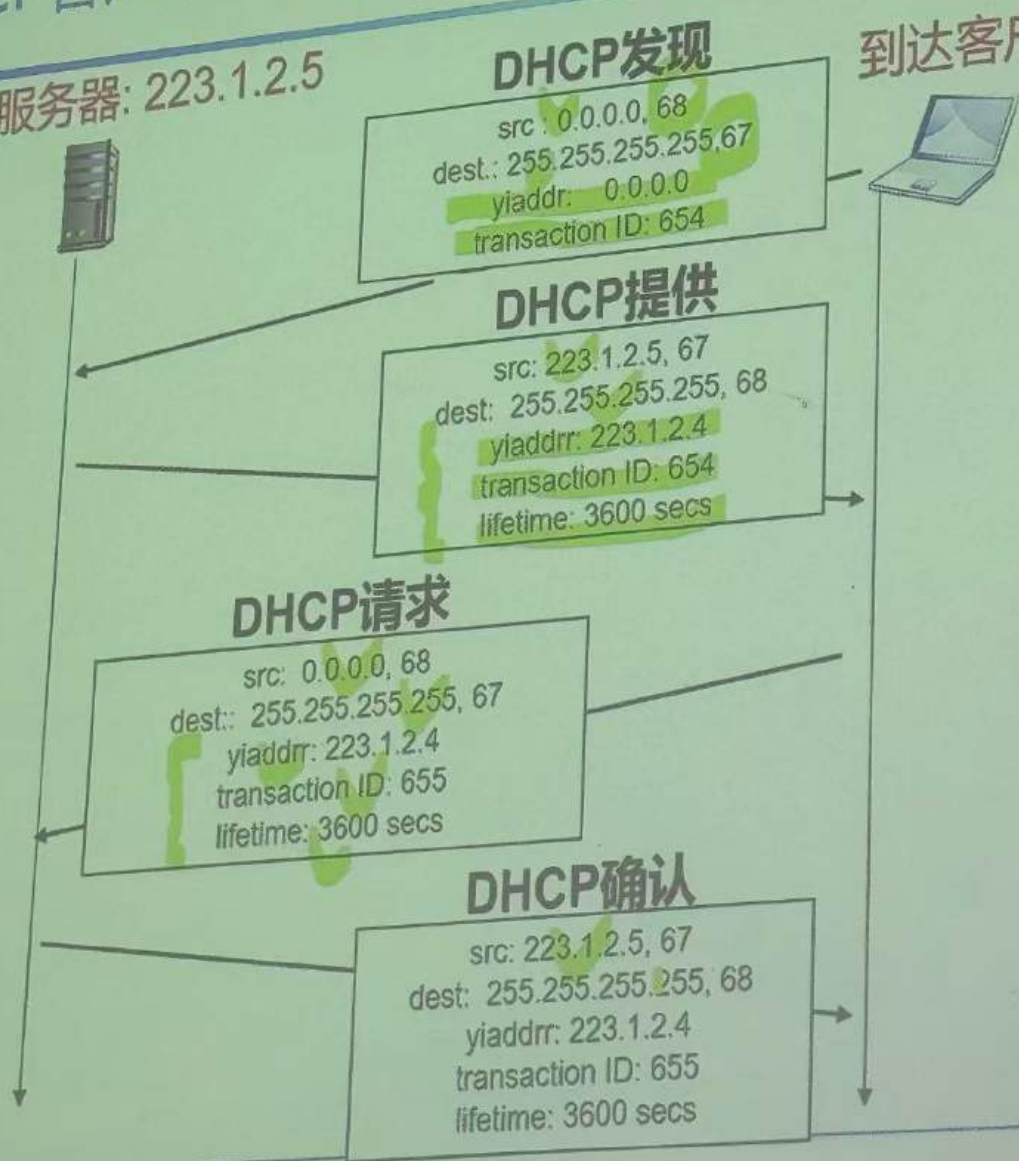
- 主机广播 “DHCP发现” 报文
- DHCP服务器响应 “DHCP提供” 报文
- 主机发送 “DHCP请求” 报文请求IP地址
- DHCP服务器响应 “DHCP确认” 报文发送地址信息

DHCP本质上是一个C/S架构下的应用层协议

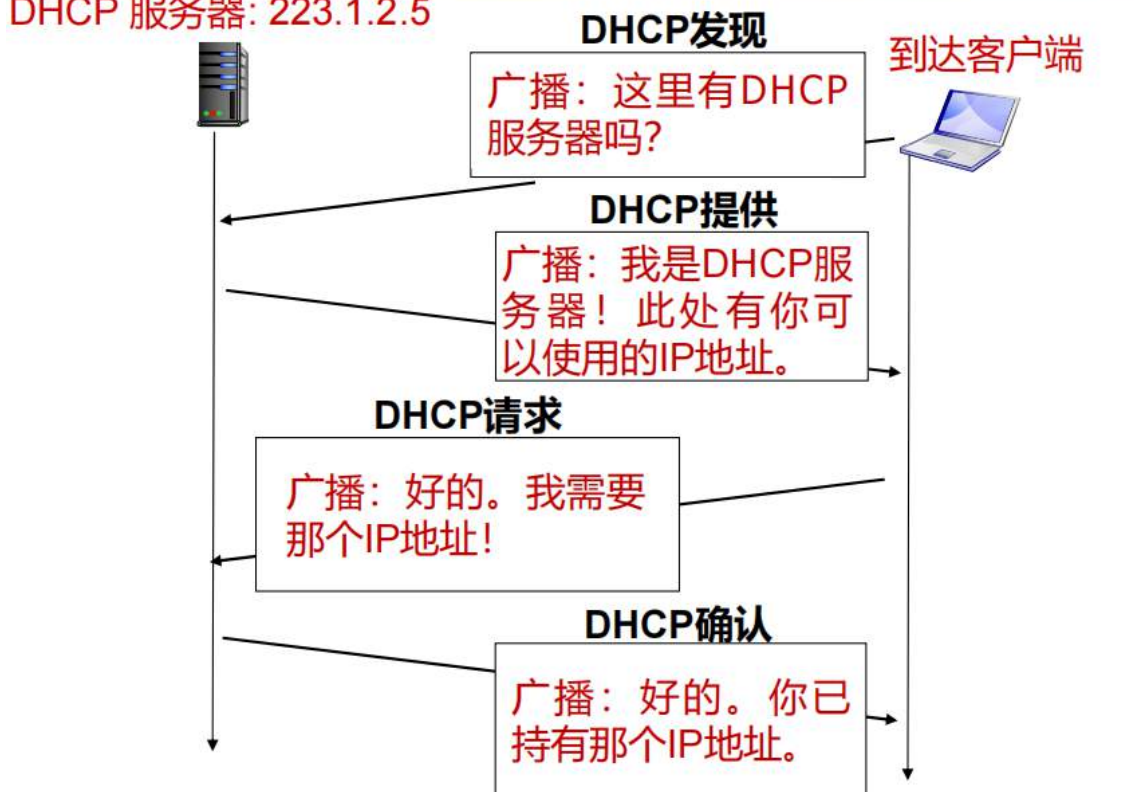
DHCP客户端-服务器场景

DHCP 服务器: 223.1.2.5

到达客户端



DHCP 服务器: 223.1.2.5



NAT

network address transformation

NAT: 网络地址转换

软件学院 · 计算机网络

目的: 对外部网络而言, 本地网络仅用一个IP地址

- 所有设备只需一个IP地址, 而无需ISP地址的一个区段
- 设备地址可以在本地网络变更, 而无需通知外部网络
- 本地网络可以变更ISP, 而无需变更设备的本网地址
- 出于增强的安全性考虑, 本地网络设备没有明确的公网地址, 因此外网无法直接访问

在中国, 每一家的所有设备通过ipconfig看到的ip地址都是NAT下的一个内网ip地址

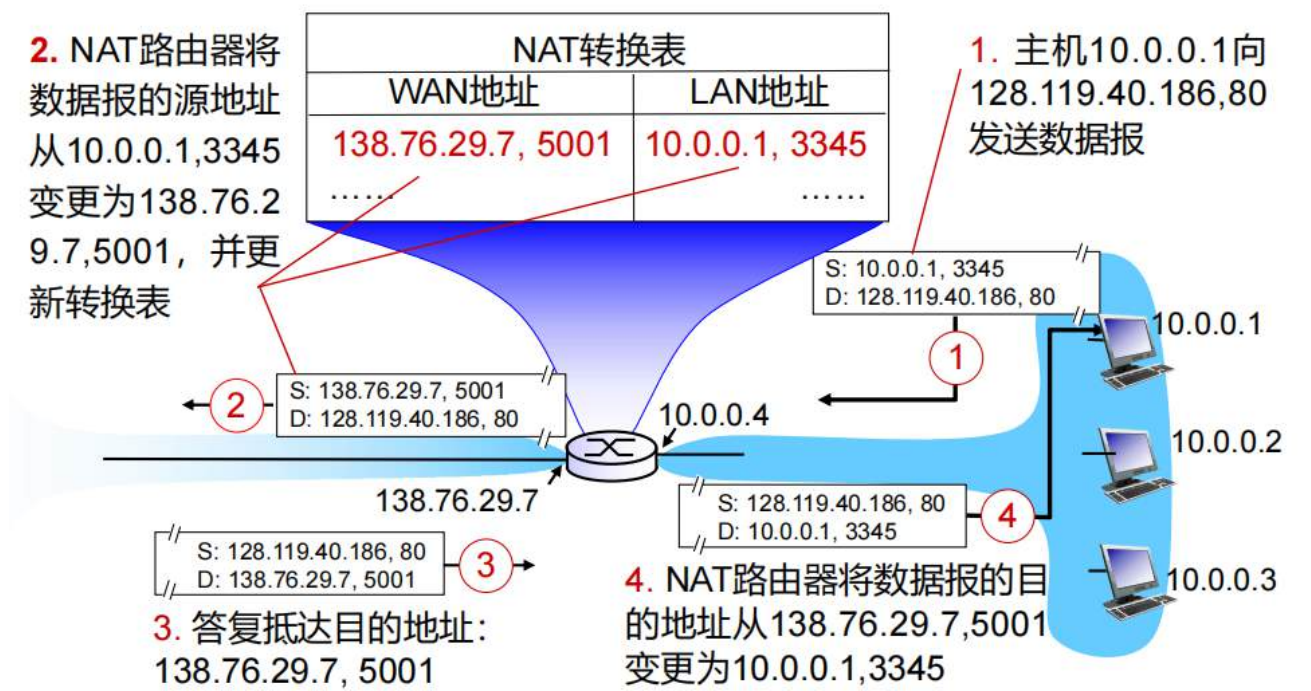
当我想让别人访问我的主机的时候, 如果我的设备属于一个NAT, 那么别人是不能直接访问我的设备的

因为我看到的内网ip地址是虚拟的, 不是真实的ipv4地址

这么做安全性很高, 但是外部网络难以访问我们的主机

那怎么访问我们的主机？

我们可以通过NAT网关的NAT转换表，通过端口进行映射



将IP应用、隔离变化、安全诉求等，作为一个整体

外部想访问我的主机，只需要访问139.76.29.7的5001端口号就可以

理论上我想让别访问我的主机，我只要知道我的NAT映射地址就可以

怎么做？

NAT穿透

方案1：配置静态NAT，转发指定端口的连接请求，至服务器

例如，总是将请求(138.76.29.7,2500)转发到10.0.0.1端口25000



方案2：通用即插即用（UniversalPlug and Play，UPnP）互联网网关设备（Internet Gateway Device，IGD）协议，允许NAT主机：

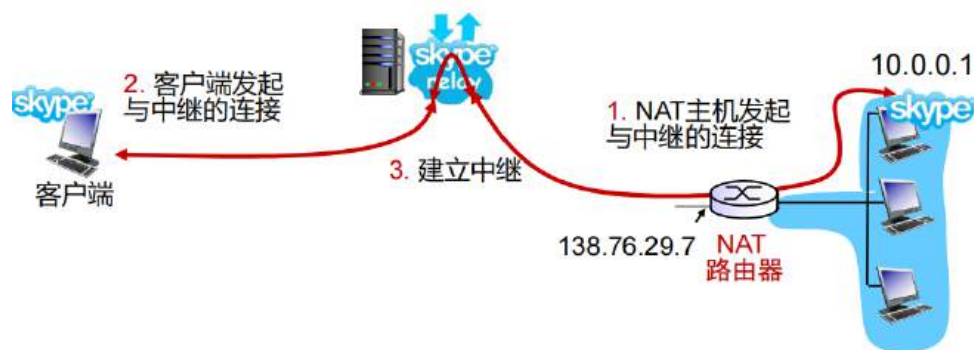
学习公网IP地址（138.76.29.7）添加/移除端口映射（包括租期）

实际上就是，静态NAT端口映射的自动化配置



方案3：中继（在Skype中使用）

- NAT客户端建立与中继的连接
- 外部客户端连接到中继
- 中继在两个连接之间完成分组的桥接



比如说常用的frp内网穿透，cloudflare代理都是这个原理

IPv6

将原本IPv4的32位扩展为128位

- **原始动力：** 32位的地址空间，即将分配完毕
- **其他动力：**
 - 首部格式加快处理/转发的速度
 - 首部优化适应服务质量（QoS）需求
- **IPv6数据报格式：**
 - 首部长度固定为40字节
 - 不允许分片

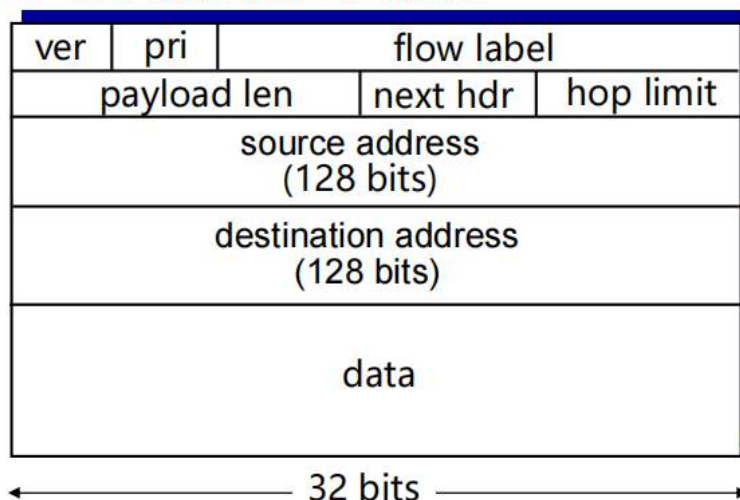
数据格式

priority： 标识流中数据报的优先级

flow label： 标识同一“流”中的数据报

(实际上，标准并未明确定义“流”的概念)

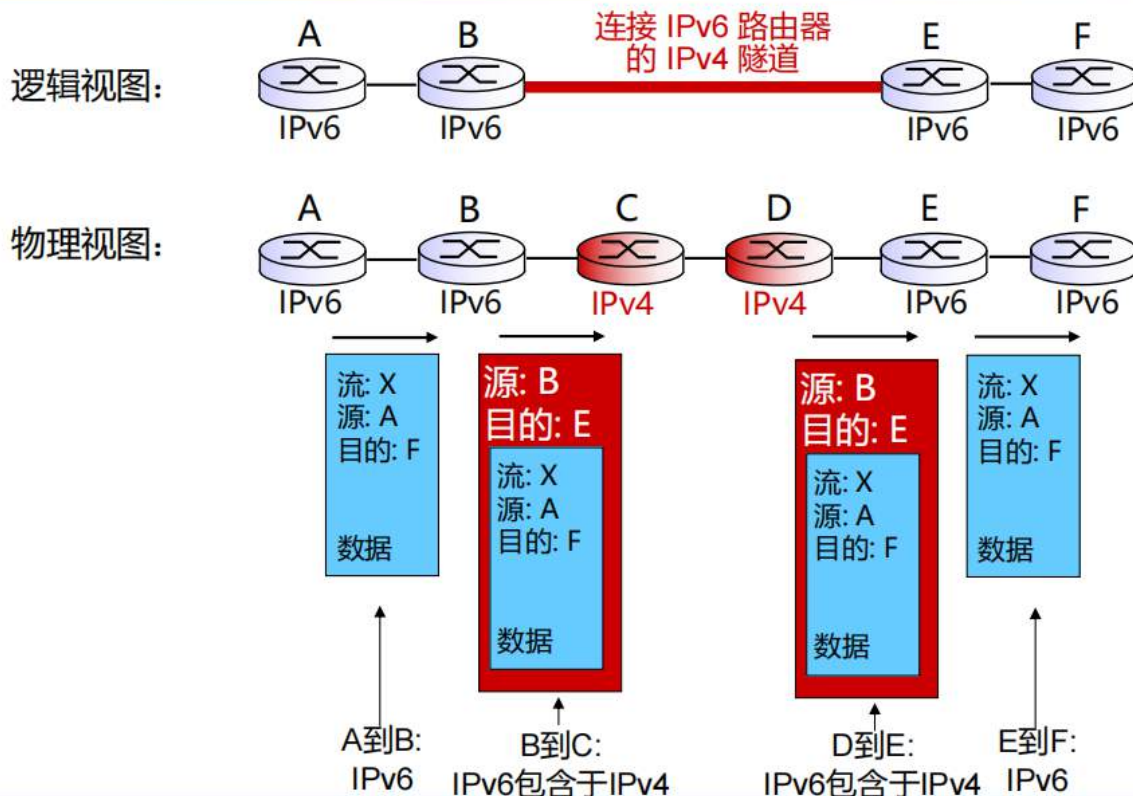
next header： 标识数据的上层协议



- ❑ **identifier, flags, offset**: 删除。出于首部复杂度的考虑, IPv6不允许分片/重组
- ❑ **checksum**: 删除。出于首部变化的考虑, IPv6删除校验和, 以减少每跳的处理时间
- ❑ **options**: 允许。在首部外, 由 “Next Header” 字段指示
- ❑ **ICMPv6**: 新版本的ICMP
 - 增加了报文类型, 例如 “Packet Too Big”
 - 增加了多播 (组播) 中的组管理功能



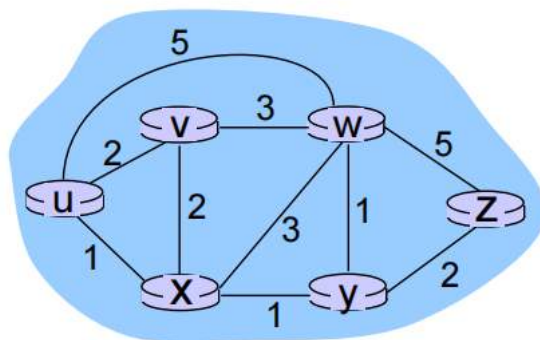
隧道



通过IPv4隧道, 在之上套上一层IPv4首部用于通过IPv4路由器传输

路由

开销



$c(x, x')$ = 链路(x, x')的开销

例如, $c(w, z) = 5$

链路开销可以恒为1, 也可以与带宽、拥塞程度等相关

路径 $(x_1, x_2, x_3, \dots, x_p)$ 开销 = $c(x_1, x_2) + c(x_2, x_3) + \dots + c(x_{p-1}, x_p)$

关键问题: 从u到z的最小开销路径是什么?

路由算法: 寻找最小开销路径的算法

分类

问题: 中心式信息/分散式信息?

中心式:

- 所有路由器都有完美的网络拓扑、链路开销信息
- 链路状态算法

分散式:

- 路由器知道与其物理相连的邻接路由器, 及其到这些邻接路由器的链路开销
- 迭代执行路由算法, 并与邻接路由器交换信息
- 距离矢量算法

问题: 静态/动态?

静态:

- 路由在较长时间内缓慢变化

动态:

- 路由变化较快
 - 周期性更新
 - 链路开销变化

问题: 是否对负载敏感?

Dijkstra算法

- 所有节点都知道网络拓扑与链路开销信息
 - 通过“链路状态广播”实现
 - 所有节点拥有相同的信息
- 计算从一个节点（“源”）到其他所有节点的最小开销路径
 - 获得该点的转发表
- 迭代：经k次迭代，得k个目的节点的最小开销路径

符号说明：

- $c(x, y)$ ：从节点x到y的链路开销；若非直连，则为无穷大
- $D(v)$ ：从源点到目的节点v的当前开销值
- $p(v)$ ：从源点到目的节点v的路径上前置节点
- N' ：已知最小开销路径的节点集合

1 **Initialization:**

2 $N' = \{u\}$

3 for 所有节点v

4 if v与u相邻

5 then $D(v) = c(u, v)$

6 else $D(v) = \infty$

7

8 **Loop**

9 寻找节点w，既不在 N' 中，又使 $D(w)$ 最小

10 将节点w加入点集 N'

11 针对w的所有邻节点v，如果不在 N' 中，那么更新 $D(v)$ ：

12 **$D(v) = \min(D(v), D(w) + c(w, v))$**

13 /* 到节点v的新开销，要么是到节点v的旧开销，

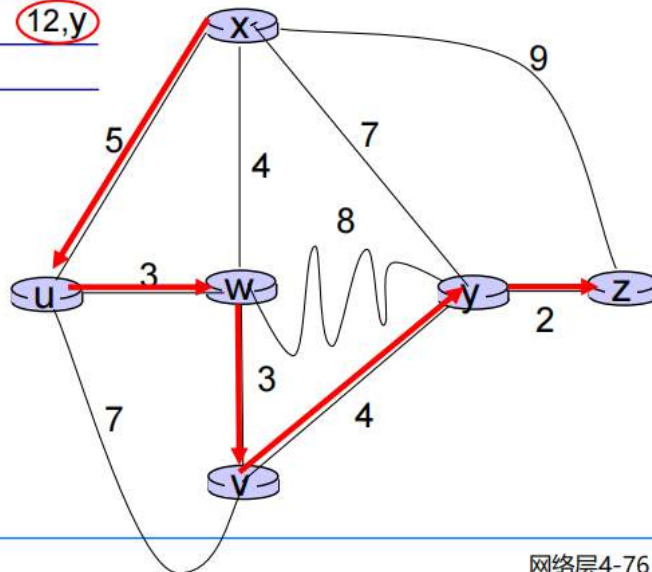
14 要么是到节点w的已知最短路径开销+w到v的开销 */

15 **until all nodes in N'**

步骤	N'	D(v) p(v)	D(w) p(w)	D(x) p(x)	D(y) p(y)	D(z) p(z)
0	u	7,u	3,u	5,u	∞	∞
1	uw	6,w		5,u	11,w	∞
2	uwvx	6,w			11,w	14,x
3	uwxv				10,v	14,x
4	uwxyv				12,y	
5	uwxyvz					

注意：

- 通过追踪前置节点，可以构造最短路径树
- 两点间的联系，可能一直存在，也可能被打破



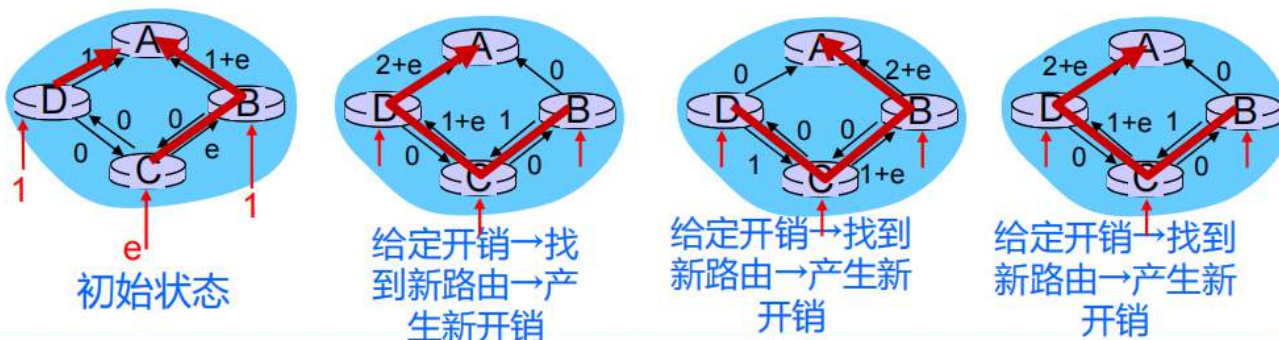
网络层4-76

算法复杂度：n个节点

- 每次迭代：检查不在N'中的所有节点w
- 共计比较 $n(n+1)/2$ 次： $O(n^2)$
- 更加高效的实现： $O(n \log n)$

可能的震荡问题：

- 例如：链路开销等于承担的负载量



某种情况下，开销改变，出现突然都走左边又都走右边的情况，会造成拥堵，这就是震荡问题

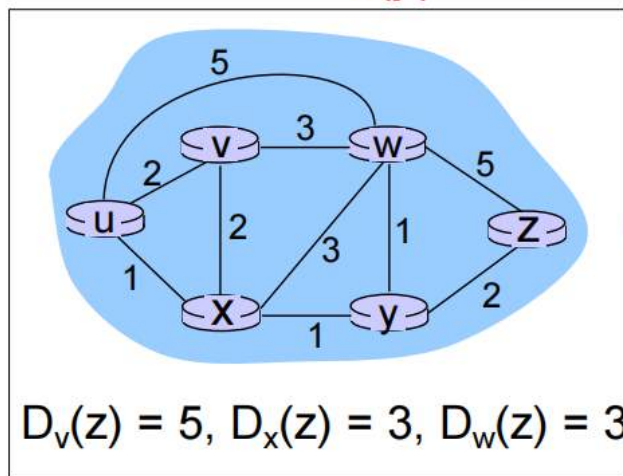
距离矢量路由算法

它只知道临界点的状态

Bellman-Ford方程

Bellman-Ford 方程 (动态规划)

$$D_x(y) = \min \{c(x,v) + D_v(y)\}$$



B-F equation says:

$$\begin{aligned} D_u(z) &= \min \{c(u,v) + D_v(z), \\ &\quad c(u,x) + D_x(z), \\ &\quad c(u,w) + D_w(z)\} \\ &= \min \{2 + 5, \\ &\quad 1 + 3, \\ &\quad 5 + 3\} = 4 \end{aligned}$$

实践中的重要事项:

- 达到min值的节点即为下一跳, 该信息直接用于转发表
- 邻居间通信, 天然地适用于分布式系统

每个节点只关心和了解临界点

参数:

□ $D_x(y)$ = 从 x 到 y 的最小开销

➤ x 维护距离矢量 $\mathbf{D}_x = [D_x(y): y \in N]$

□ 节点x

➤ 知晓到每个邻节点v的开销: $c(x,v)$

➤ 维护邻节点的距离矢量。也就是说, 对于每个邻节点v, 节点x维护

$$\mathbf{D}_v = [D_v(y): y \in N]$$

主要思想:

□ 发送 → 接收 → 更新, 最终收敛



迭代, 异步:

每次迭代的根源

- 本地链路开销变化
- 邻点距离矢量更新

分布式:

- 每个节点当且仅当自己的距离矢量发生变化时, 通知邻居节点
 - 如有必要 (即新变化), 邻节点接着通知它们各自的邻节点, 以此类推

每个节点:

等待 (本地链接开销变化或邻节点的更新报文)

重算开销值

$$D_x(y) = \min \{c(x,v) + D_v(y)\}$$

如果到任何目的节点的 DV 发生变化, 通知邻节点

node x table

		cost to		
		x	y	z
from	x	0	2	7
	y	∞	∞	∞
	z	∞	∞	∞

node y table

		cost to		
		x	y	z
from	x	∞	∞	∞
	y	2	0	1
	z	∞	∞	∞

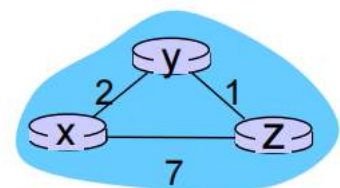
node z table

		cost to		
		x	y	z
from	x	∞	∞	∞
	y	∞	∞	∞
	z	7	1	0

		cost to		
		x	y	z
from	x	0	2	3
	y	2	0	1
	z	7	1	0

$$D_x(y) = \min \{c(x,y) + D_y(y), c(x,z) + D_z(y)\} \\ = \min \{2+0, 7+1\} = 2$$

$$D_x(z) = \min \{c(x,y) + D_y(z), c(x,z) + D_z(z)\} \\ = \min \{2+1, 7+0\} = 3$$



time

以x举例, 其相邻节点把自己的信息给x得到右上角的表, 然后我们计算x->y

两种策略:

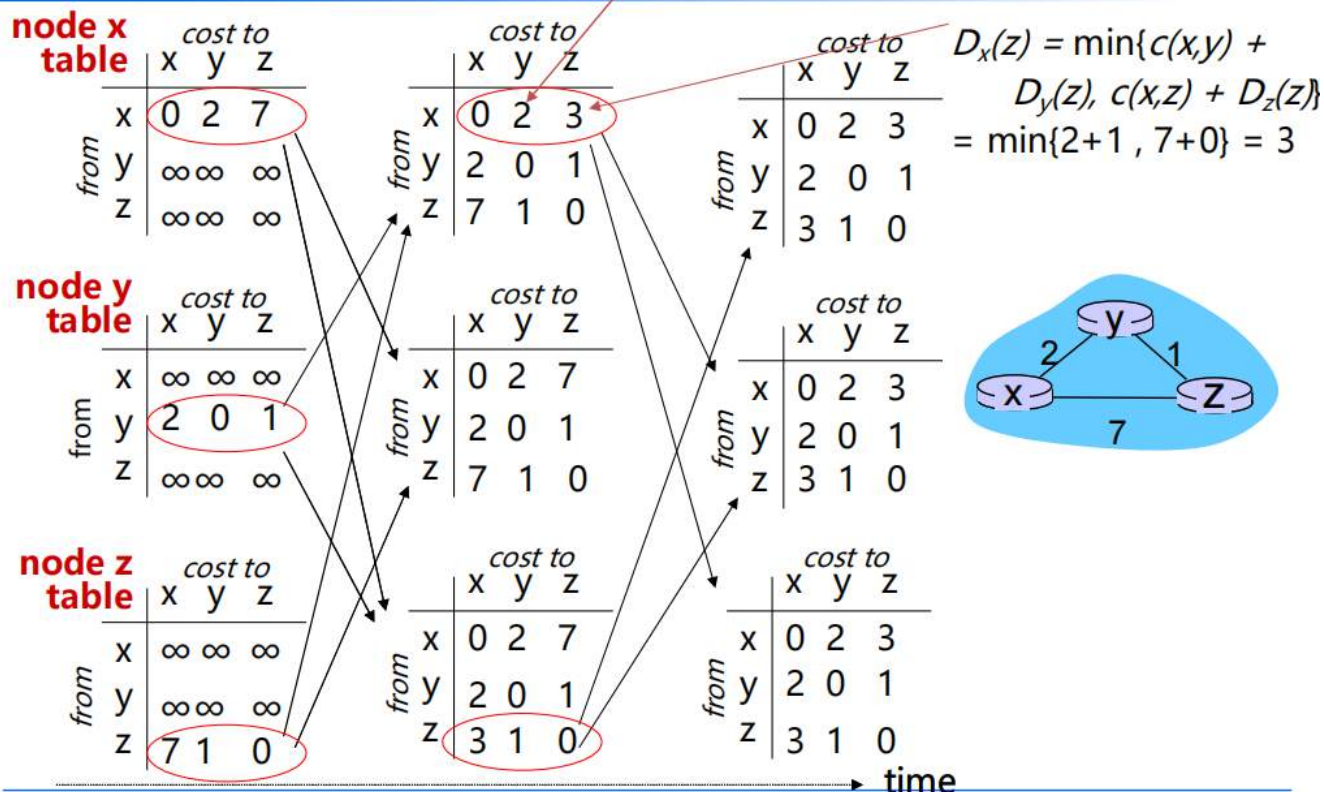
- x -> y -> y
- x -> z -> y

取最小值

x → z 同理，然后以此类推推出y和z的新表，然后再不断相互通知不断迭代直到所有表都不变，即收敛

$$D_x(y) = \min\{c(x,y) + D_y(y), c(x,z) + D_z(y)\} \\ = \min\{2+0, 7+1\} = 2$$

软件学院 · 计算机网络



分析

假如某一开销改变，会发生什么？

如果开销减小，是好事，那么会加快收敛速度

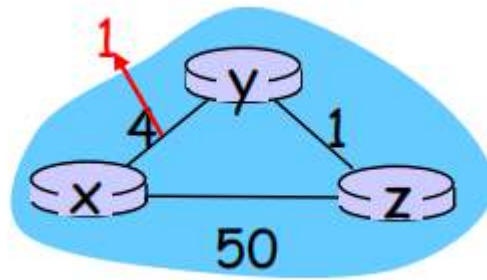
好消息
传得快

t1: y检测到链路开销变化，更新距离矢量，通知邻居节点

t2: z收到y的更新，更新距离矢量表，计算到x新的最小开销，向邻节点发送本地的距离矢量

t3: y收到z的更新，更新距离矢量表，y的最小开销不变，因此y并不会向z发送更新报文

过程：

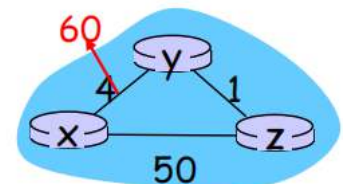


状态	x-y	z-x
未变化	4; ∞ (直接的距离; 通知后的最短距离)	50; ∞
未变化 (第一次更新)	4;51	50;5
更新	1;6	50;2
收敛	1;6	50;2

如果是坏事，那么会减慢收敛速度

链路开销变化：

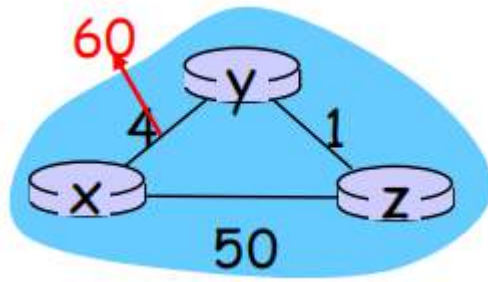
- ▣ 节点检测到本地链路开销变化
- ▣ **坏消息传得慢**——产生“计数至无穷”问题！
- ▣ 算法在稳定之前，历经44次迭代



毒性逆转：

- ▣ 如果Z通过Y到达X：
 - Z告诉Y, “Z到X的距离无穷大”
 - 这样Y就不会通过Z到X了
- ▣ 这会彻底解决“计数至无穷”问题吗？（三个以上环路...）

过程：



状态	x-y	z-x
未变化	4; ∞ （直接的距离；通知后的最短距离）	50; ∞
未变化（第一次更新）	4;51	50;5
更新	60;6	50;5
	60;6	50;7
	60;8	50;7
		40次
	60,52	50,51
	60,51	50,53
	60,51	50,52
收敛	60,51	50,52

为什么会这样？因为报喜不报忧

怎么解决？

毒性逆转

假设A告诉B说A到C的最短的距离是2m，B现在想去C，它可能知道最短距离为3m（A-B的距离为1m），但是A问我去C的距离是多少，我不能告诉他是3m，因为这个是他告诉我的，我要告诉他是无穷的

LS和DV的算法比较

报文复杂度

□ **链路状态(LS)**:在n个节点、E条链路的场景下, 发送 $O(nE)$ 条报文

□ **距离矢量(DV)**:仅与邻节点间交换

- 收敛时间变化

收敛速度

□ **链路状态(LS)**:发送报文量 $O(nE)$
算法复杂度 $O(n^2)$

- 可能存在振荡

□ **距离矢量(DV)**:收敛时间变化

- 可能存在路由环路
- 可能存在“计数至无穷”问题

稳健性: 若路由器出现故障, 会发生什么呢?

□ **链路状态(LS)**:

- 节点会广播错误的**链路**开销
- 每个节点计算自己的开销表

□ **距离矢量(DV)**:

- 节点会广播错误的**路径**开销
- 每个节点的距离矢量表, 被其他节点使用
 - 错误通过网络蔓延

分层路由

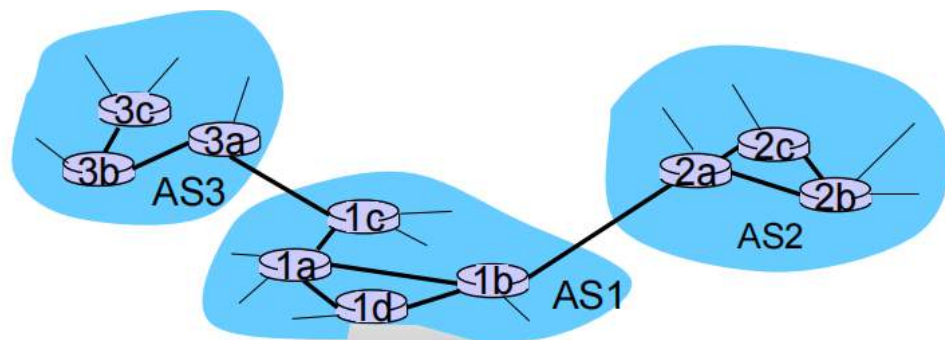
从理想到现实: **自治系统(autonomous systems, AS)**

□ **将路由器按区域划分**

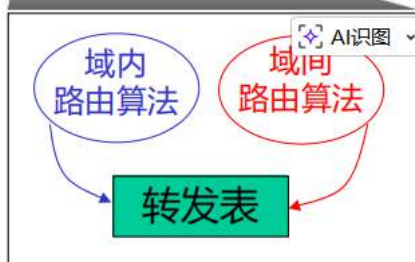
- 同一自治系统内, 路由器运行相同的路由协议——**“域内”**路由协议
- 不同自治系统中, 路由器可以运行不同的域内路由协议

□ **网关路由器**:

- 在其所属自治系统的“边缘”
- 与另一个自治系统的路由器, 有链路相连



- 转发表由域内路由算法、域间路由算法共同配置
 - 域内路由算法为域内目的地设置转发表
 - 域间路由算法和域内路由算法，共同为域外目的地设置转发表



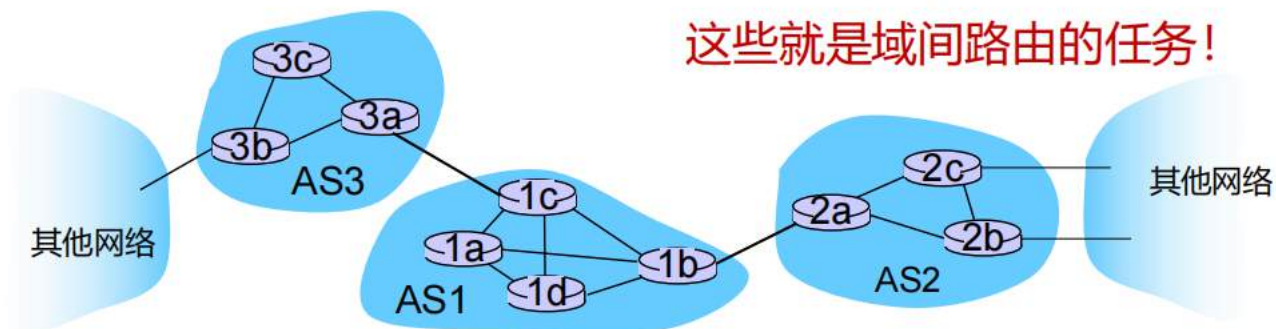
域间路由任务

- 假设AS1中的路由器，收到目的地址在AS1外的一个数据报：
 - 路由器应将分组转发至网关路由器，应该选哪个合适呢？

AS1必须完成以下任务：

1. 学习通过AS2可以到达哪些目的地址、通过AS3可以到达哪些目的地址...
2. 传播这些可达性信息，至AS1中所有路由器

这些就是域间路由的任务！



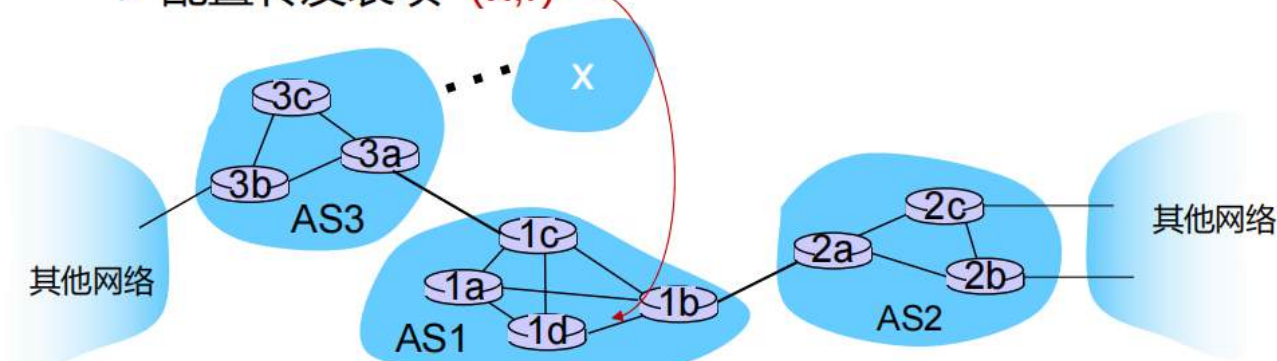
示例

- 假设AS1基于域间路由协议学习到：可以通过AS3到达子网 x （实际经网关1c），不能通过AS2到达 x

- 域间路由协议将可达信息传播至所有内部路由器

- 路由器1d根据域内路由信息，确定其到网关1c的最小开销路径，途径接口 l

- 配置转发表项 (x, l)

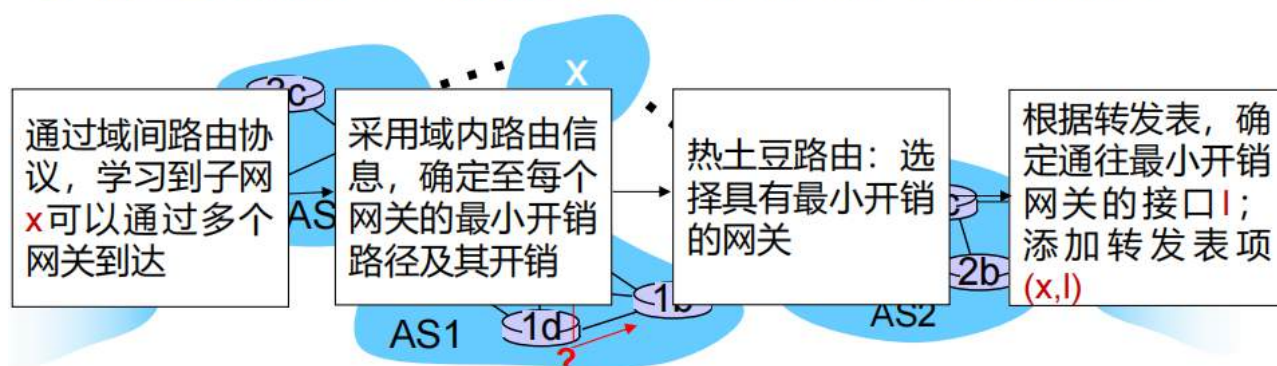


- 假设AS1基于域间路由协议学习到：既可以通过AS3到达子网 x ，也可以通过AS2到达。

- 路由器1d必须确定转发分组至目的 x 的途径网关，进而配置转发表

- 这也是域间路由协议的任务！

- 热土豆路由：**分组发送给这两个路由器中最近的那个



互联网路由

这里主要研究路由协议，强调多个路由器的交互方式

例如RIP、OSPF、BGP

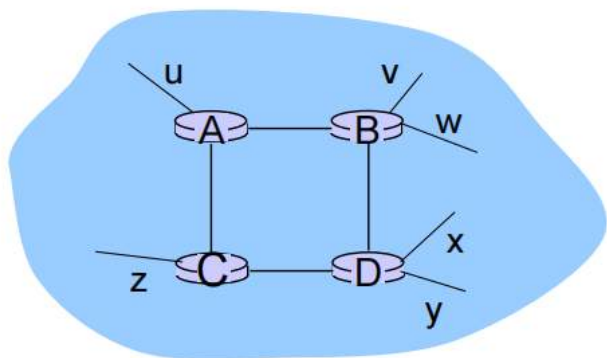
域内协议

- 亦称作**内部网关协议**
(Interior Gateway Protocols, IGP)
- 常见的域内路由协议有：
 - RIP：路由信息协议
 - OSPF：开放式最短路径优先
 - IGRP：内部网关路由协议（Cisco私有协议）

RIP

RIP是一个应用层协议，基于DV算法

- 在1982年随BSD-UNIX发行
- 距离矢量算法
 - 距离度量标准：跳数（最大值为15跳），意味着链路开销为1
 - 距离矢量信息每30秒通过响应报文（也称**通告**）与邻点交换信息
 - 每个通告报文：最多列出25个目的**子网**（从IP地址角度）

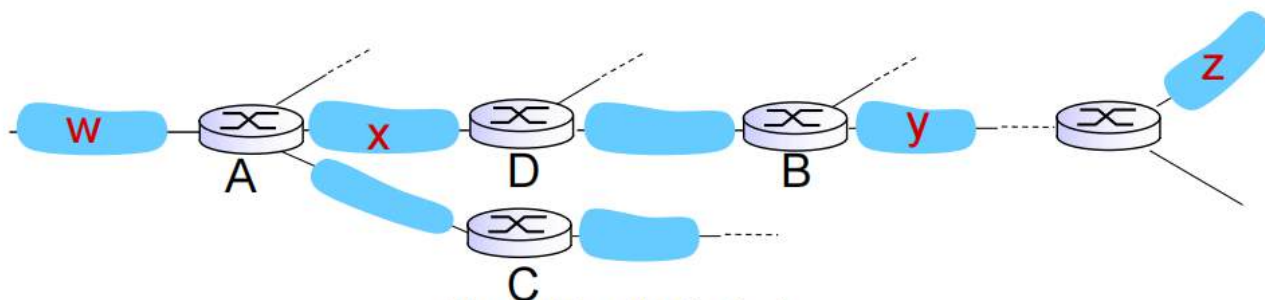


从路由器A到目的**子网**:

子网	跳数
u	1
v	2
w	2
x	3
y	3
z	2

该协议以**跳数**作为距离的衡量唯一标准，最大15跳因此适用于规模一般的网络，这可以避免**无穷问题**

示例



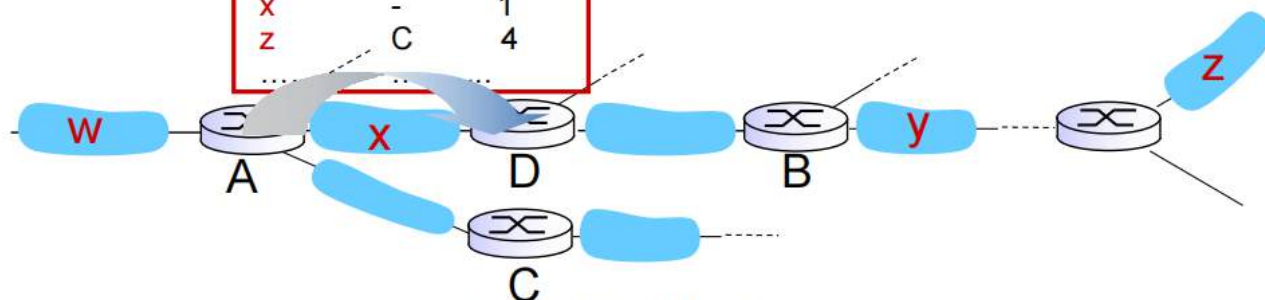
路由器D的路由表

目的子网	下一跳路由器	到目的地的跳数
W	A	2
y	B	2
Z	B	7
X	--	1
....		

假设A给D发送了一个通告

A到D的通告

目的地	下一跳	跳数
W	-	1
X	-	1
Z	C	4
....		



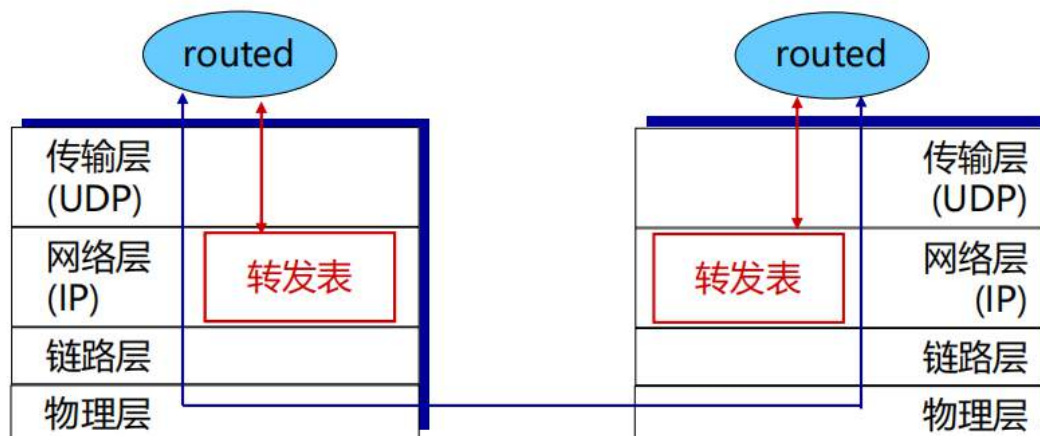
路由器D的路由表

目的子网	下一跳路由器	到目的地的跳数
W	A	2
y	B	2
Z	B A	7 5
X	--	1
....		

D发现从A走更快，于是更新路由表，这就是DV算法

当一个路由器有一定时间没有给自己通告，就认为和对方断开了

- RIP路由表，由routed应用进程管理
- 通告信息封装在UDP报文段中，进行周期性发送



由应用管理，是应用层协议

OSPF协议

基于LS算法

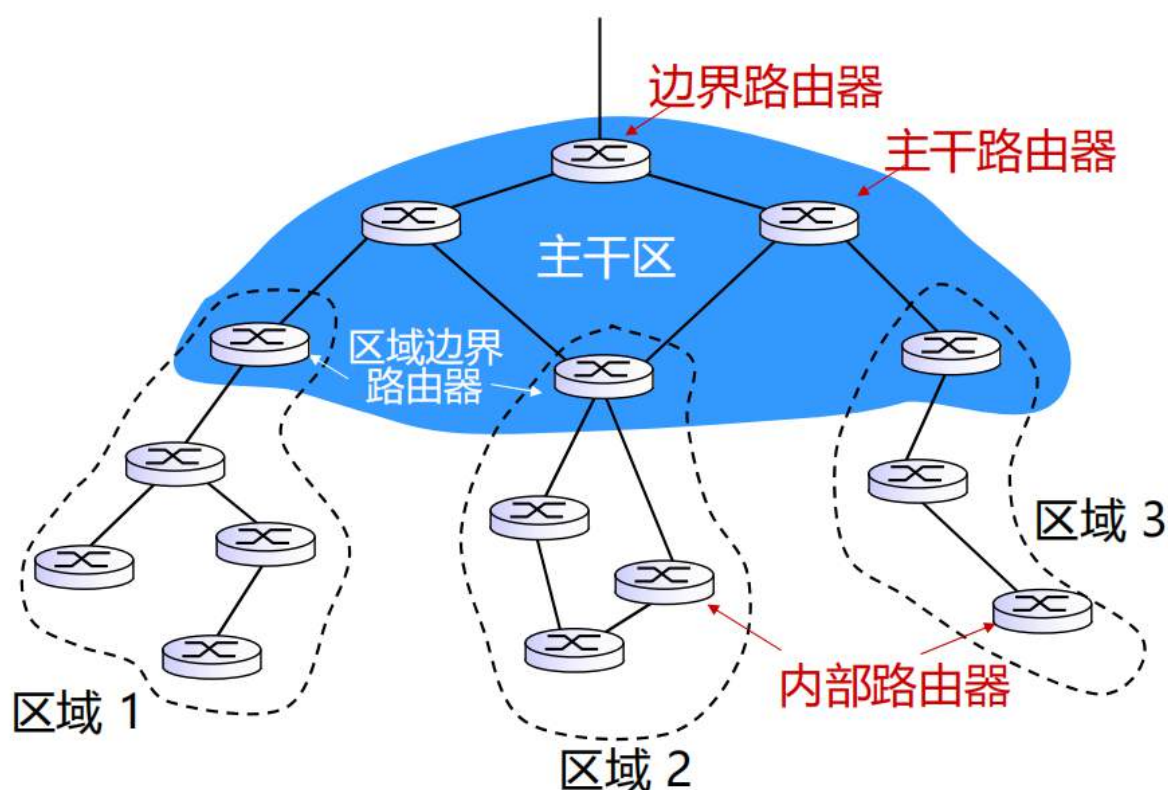
- “开放”：公开可用
- 采用链路状态算法
 - 泛洪链路状态信息分组
 - 每个节点构建拓扑图、运行Dijkstra算法
 - 设置开销
- OSPF通告针对每个邻点携带一条表项，最终传播至整个自治系统AS
 - OSPF报文直接采用IP协议（而不是TCP或UDP协议）
- IS-IS路由协议：与OSPF基本相同

为什么通告针对每个邻点携带一条表项，但是每个路由器知道全局的信息？

本质上只需要每个路由器都相互告诉对方自己邻居的信息，就可以补全完整地图信息

- **安全性**：所有OSPF报文均经身份验证（防止恶意入侵）
- 允许保留开销相同的**多条路径**（RIP中只允许一条路径）
- 每条链路针对不同的服务类型（**TOS**）可以设置多个开销（不同的度量标准）
 - 例如，卫星链路对于尽力而为的服务，开销设置为“低”；对于实时类型的服务，开销设置为“高”
- 支持单播和**多播**
 - 多播OSPF（MOSPF）与普通OSPF采用相同的拓扑数据库
- 在大型区域中，使用**分层OSPF**

分层OSPF



这里是一个AS内部的整体分层结构

下面讲AS之间的协议BGP

域间协议

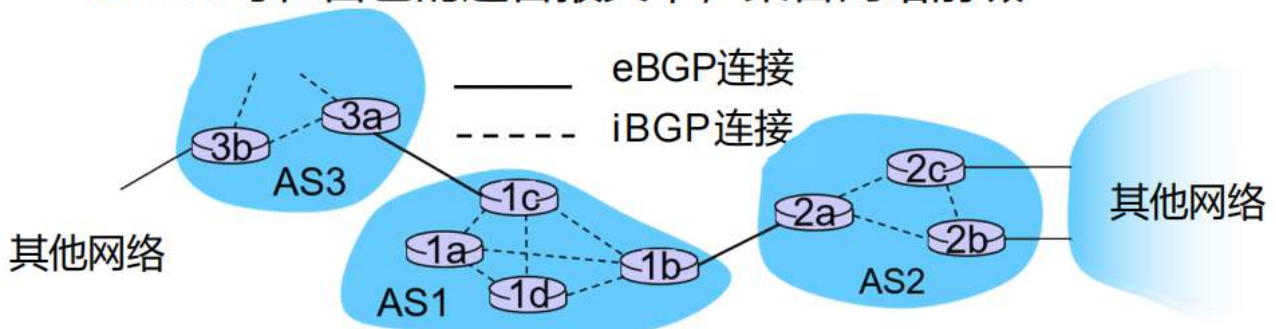
BGP

传输层协议，基于DV算法

- **BGP（边界网关协议）**：实际用的域间路由协议
 - “维持互联网一体化的粘合剂”
- BGP为每个自治系统，提供了一套方法：
 - **eBGP**：从相邻自治系统，获得子网可达性信息
 - **iBGP**：将可达性信息，传播至自治系统内所有的路由器
 - 基于可达信息和路由策略，确定到达其他网络的“最佳”路由
- 子网可以向互联网其余部分通告存在性：“**我在这里**”

AS之间不停相互通告自己的内部信息(eBGP)，其他AS内部再通告给自己的子网(iBGP)

- **BGP会话**：两个BGP路由器（“对等方”）交互BGP报文
 - 通告到达不同目的网络前缀的**路径**（“路径矢量”协议）
 - 通过TCP连接交互信息
- 当AS3向AS1通告一个网络前缀时
 - AS3**保证**自己能将数据报转发至该网络
 - AS3可在自己的通告报文中，聚合网络前缀



- 通告的信息中，包含BGP属性
 - 前缀 + 属性 = “路由”
- 两个重要的属性：
 - **AS-PATH**: 包括那些前缀通告报文，经过的自治系统，例如AS 67, AS 17
 - **NEXT-HOP**: 指示到下一自治系统的，特定域外路由器 (从当前AS到下一AS，可能存在多条链路)
- 网关路由器收到路由通告后，采用**导入策略**决定接受/拒绝
 - 例如，永远不经过AS x
 - **基于策略的路由**

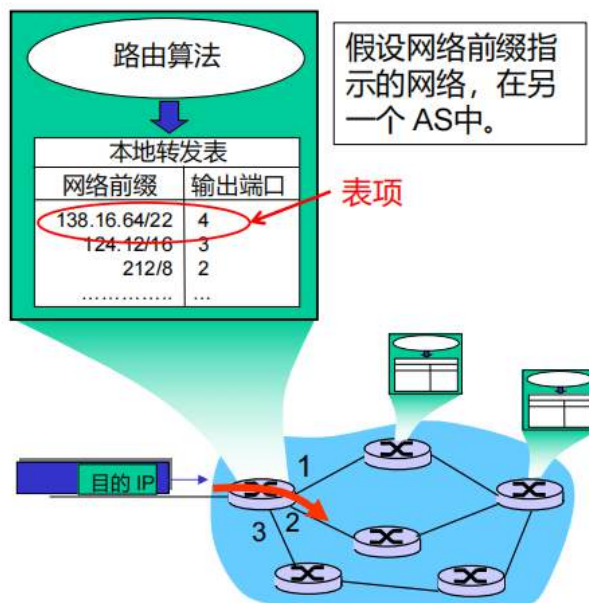
每个通告都包含网络前缀（即IP地址前缀）+ 两个特殊属性

next-hop指的就是两个AS之间连接的网关路由器，比如AS3通告给AS1的就是3a

- 路由器可能获知多条可以到达目的AS的路由，并基于以下因素选择路由：
 1. 本地偏好值属性：决策策略
 2. 最短的AS-PATH
 3. 最近的NEXT-HOP路由器：热土豆路由
 4. 其他准则

- BGP报文在对等方之间，通过TCP连接交互
- BGP报文：
 - **OPEN**：打开与对等方的TCP连接，并验证发送方的身份
 - **UPDATE**：通告新路径（或撤销旧路径）
 - **KEEPALIVE**：在没有UPDATE时保持连接；也用于确认OPEN请求
 - **NOTIFICATION**：报告之前报文的错误；也用于关闭连接

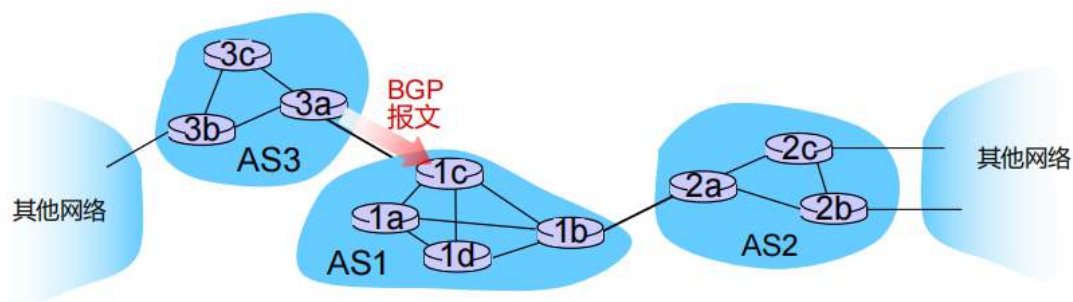
路由器怎么添加转发表项？



概述

- 路由器获知网络前缀
- 路由器为该前缀确认输出端口
- 路由器在转发表中添加“前缀-端口”表项

详细过程

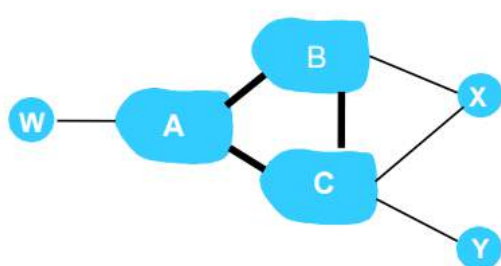


- ▣ BGP报文包含“路由”信息
- ▣ 路由信息由前缀和属性组成：AS-PATH, NEXT-HOP, ...
- ▣ 路由信息示例：
 - Prefix:138.16.64/22 ; AS-PATH: AS3 AS 131 ;
 - NEXT-HOP: 201.44.13.125

- **Prefix (138.16.64/22)**: 这就是目的地。相当于收件地址，代表一大片 IP 地址范围。
- **AS-PATH (AS3, AS131)**: 这是轨迹记录。它告诉接收者：这条路由信息是先从 AS131 出来的，然后经过了 AS3，现在传到了你这里。
 - **防环神器**：如果 AS1 发现这条路径里已经包含了自己的名字（AS1），它就会直接丢弃。这比 RIP 的毒性逆转高级得多，通过“看一眼路径里有没有我”就杜绝了环路。
- **NEXT-HOP (201.44.13.125)**: 这是下一站。告诉路由器，如果你想去这个目的地，先把包发给这个具体的 IP。

路由策略

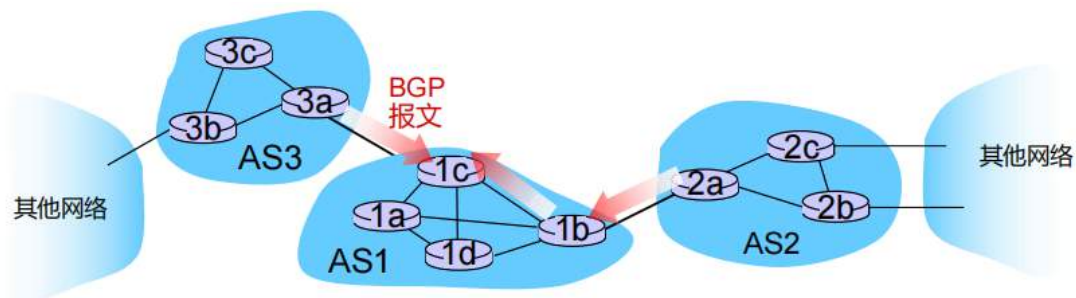
每个路由器可以有自己的偏好，可以拒绝通告别人等，这就叫做路由策略，是针对每个路由器的拒绝



图例：
 供应商网络
 消费者网络

- ▣ X 双宿
- ▣ **问题**：X 不希望存在从 B 经 X 到 C 的路径
 - X 不会向 B 通告一条能够通往 C 的路由
- ▣ B 向 X 通告路径 BAW
- ▣ **问题**：B 是否应该向 C 通告路径 BAW?
 - 因为 W 和 C 都不是 B 的客户，所以 B 无法从 CBAW 路由中获利

选择



- 路由器可能收到**相同**前缀的多条路由
- 必须选择一条路由
 - 偏好值属性
 - 最短 AS-PATH

示例:

AS2 AS17 to 138.16.64/22

AS3 AS131 AS201 to 138.16.64/22

选择

选择之后，假设确定了唯一一条路由，路由器怎么选择最佳路由？

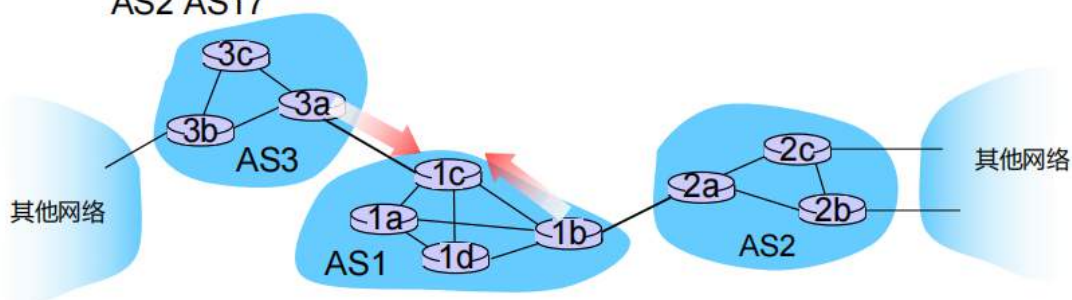
□ 采用所选路由的 NEXT-HOP 属性

- 路由的 NEXT-HOP 属性是 AS 路径起始处**路由器接口**的 IP 地址
- 示例:

AS-PATH: AS2 AS17 ; **NEXT-HOP: 111.99.86.55**

□ 路由器采用:

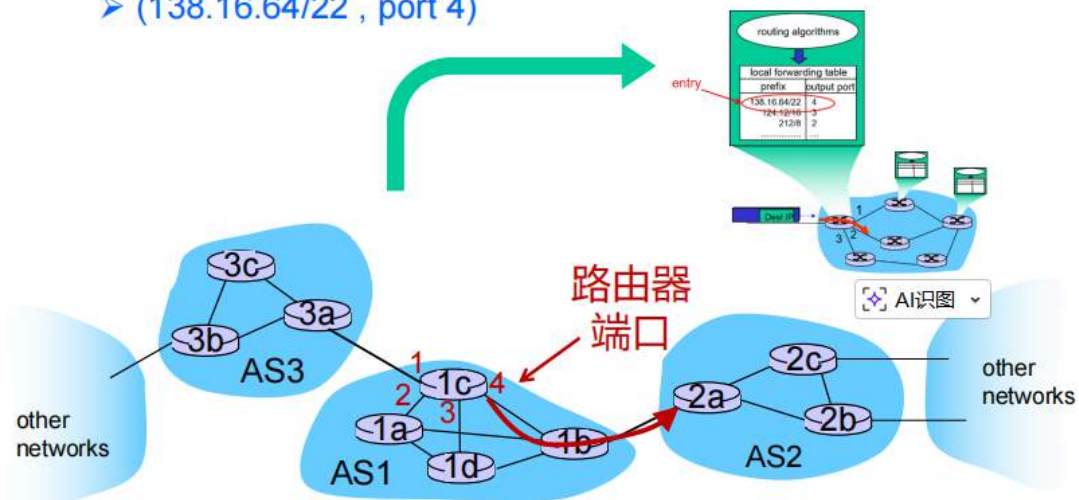
- **OSPF** 寻找从 1c 到 111.99.86.55的最短路径
- **热土豆路由**根据更近的距离，选择从 1c 到 AS3 AS131 而不是到 AS2 AS17



可以采用OSPF或者热土豆路由

路由器基于路由识别端口

- ▣ 识别指向 OSPF 最短路径的端口
- ▣ 添加“前缀-端口”表项至转发表中
 - (138.16.64/22 , port 4)



因此我们现在知道了，我们怎么让每个路由器自己更新转发表了：

总结

1. 路由器获知网络前缀
 - ▣ 通过来自其他路由器的 BGP 路由通告
2. 确定该前缀的路由器输出端口
 - ▣ 采用 BGP 路由协议，寻找最佳 AS 间路由
 - ▣ 采用 OSPF 路由协议，寻找通往最佳 AS 间路由的最佳 AS 内路由
 - ▣ 路由器为最佳路由，确定路由器端口
3. 在转发表中添加“前缀-端口”表项

AS内，AS间路由的对比

策略

- 域内：由一个管理员负责，无需策略选择
- 域间：不同AS的管理员，希望可以控制本网的路由流量，并知晓经过本网的用户

性能

- 域内：可以聚焦于性能
- 域间：策略可能比性能更加重要

规模

- 对域间路由更为重要，因为较大的AS总可以/也需要划分为多个较小的AS

ICMP和SNMP

ICMP

- ICMP：Internet Control Message Protocol，互联网控制报文协议

- 用于主机与路由器之间，实现网络级信息的通信

- 错误报告：主机不可达，网络不可达，端口不可达，协议不可达
- echo请求/应答（用于ping命令）

- 在网络层位于IP协议之上：

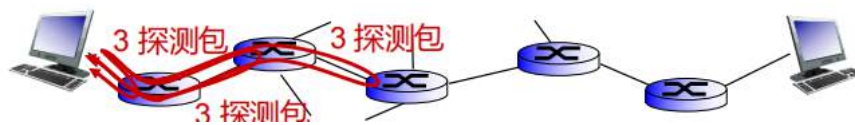
- ICMP报文以IP数据报承载

- ICMP报文：类型、代码，加上导致错误的IP数据报的前8个字节

类型	代码	描述
0	0	echo reply (ping)
3	0	dest network unreachable
3	1	dest host unreachable
3	2	dest protocol unreachable
3	3	dest port unreachable
3	6	dest network unknown
3	7	dest host unknown
4	0	source quench (congestion control - not used)
8	0	echo request (ping)
9	0	route advertisement
10	0	router discovery
11	0	TTL expired
12	0	bad IP header

Traceroute

- 源端向目的端发送系列UDP报文段
 - 第一组, TTL=1
 - 第二组, TTL= 2, ...
 - 不可达端口号
 - 当第n组数据报到达第n个路由器时:
 - 路由器丢弃数据报
 - 路由器向源端发送“TTL expired”的ICMP报文 (type 11, code 0)
 - 该ICMP报文有路由器名和IP地址
 - 源端收到ICMP报文, 记录RTT时间
- 迭代终止原则:**
- UDP报文段最终到达目的端
 - 目的端返回 “port unreachable” 的ICMP报文 (type 3, code 3)
 - 源端终止traceroute程序



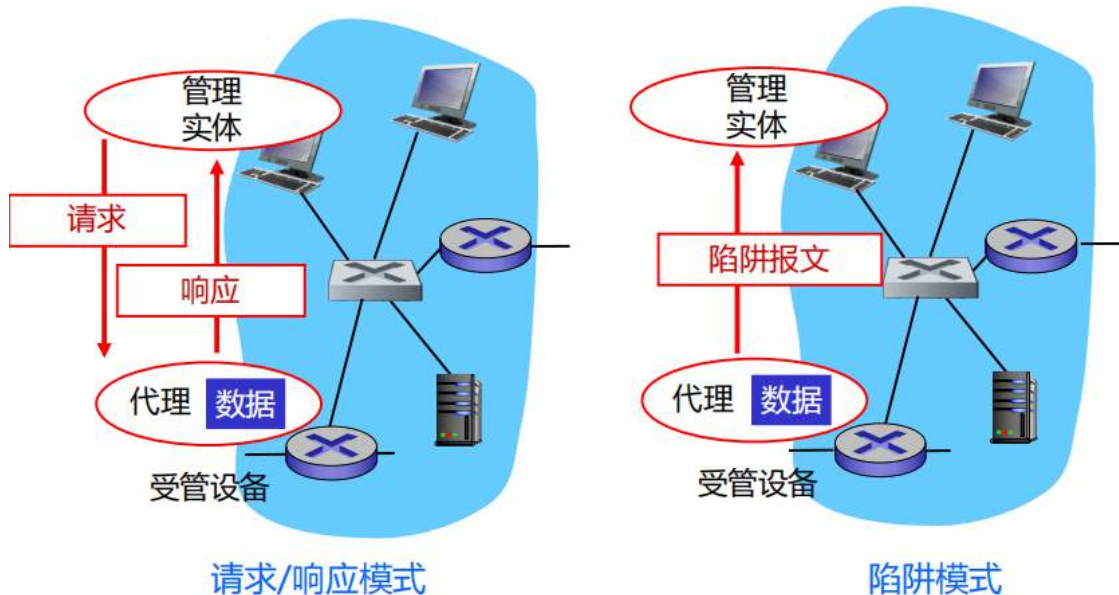
第一个包只经过一个就回来, 第二个包经过两个回来, ..., 以此类推, 直到到了目的端的包回来终止程序

SNMP

- **管理信息库(MIB):**
 - 分布式存储网络的管理数据
- **管理信息结构(SMI):**
 - MIB对象的数据定义语言
- **SNMP协议**
 - 在管理方和被管方之间, 传达对象信息和命令
- **安全、管理功能**
 - SNMPv3的主要新增功能

- 加密: DES-加密SNMP报文
- 认证: 计算、发送MIC(m,k), 即通过报文(m)和共享的密钥(k)计算哈希值(MIC)
- 防止回放: 使用nonce
- 基于视图的访问控制:
 - SNMP 实体维护多种用户关于访问权限、访问策略的数据库
 - 数据库本身作为受管对象, 可以被访问

传达MIB信息和命令的两种方式：



链路层

这里的数据单位是帧（封装网络层数据报），不再是数据报

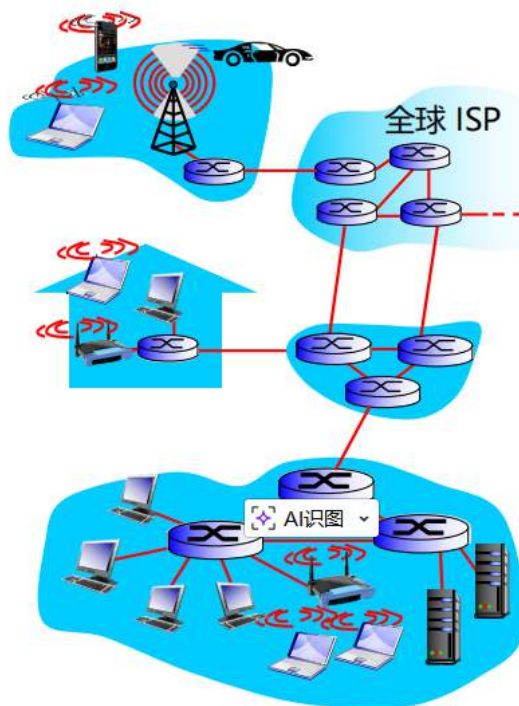
概述

术语:

- 主机、路由器、交换机：节点
- 在通信路径上，连接相邻节点的通信信道：链路
 - 有线链路
 - 无线链路
 - 局域网
- 第二层数据分组：封装网络层数据报的帧

数据链路层

负责将数据报通过链路，从一个节点传输到物理相邻的另一节点



- 数据报不同的链路上，基于不同的链路层协议传输：
 - 例如，第一段链路是以太网（802.3），中间段链路是帧中继，最后一段链路是无线传输（802.11）
- 每种链路层协议，提供与众不同的服务
 - 例如，可能或不能在链路上提供可靠数据传输

生活中的交通：

- 旅行：新校区→钟楼
 - 自行车：到新校区公交站
 - 公交车：到老校区公交站
 - 出租车：到达目的地钟楼
- 你 = 数据报
- 交通路段 = 通信链路
- 交通方式 = 链路层协议
- 旅行代理 = 路由算法

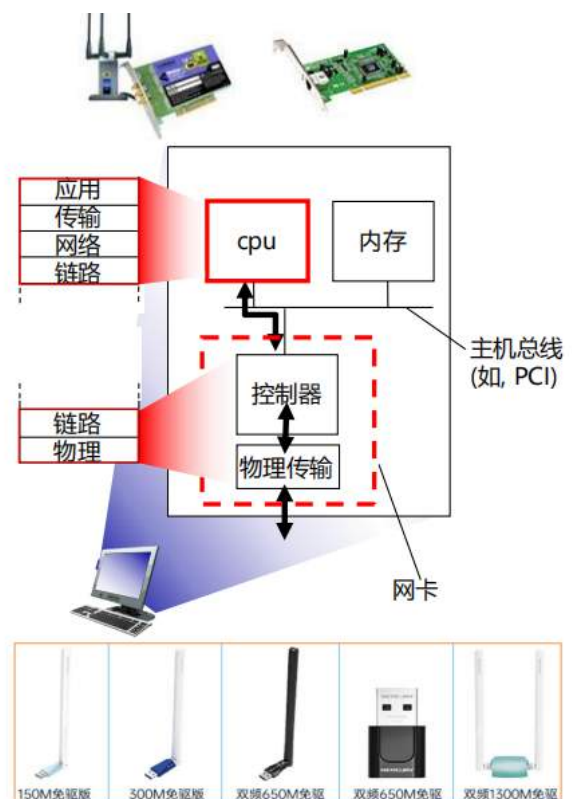
服务

- 成帧：
 - 数据报封装成帧，添加帧头和帧尾
- 链路访问：
 - 共享媒介时的信道访问
 - 帧头中的MAC地址，标识源点和目的点
 - 不同于IP地址！
- 相邻节点间的可靠交付：
 - 第三章传输层中已学习可靠数据传输协议
 - 误比特率低的链路上（光纤、某些双绞线）鲜少使用
 - 无线链路：错误率高

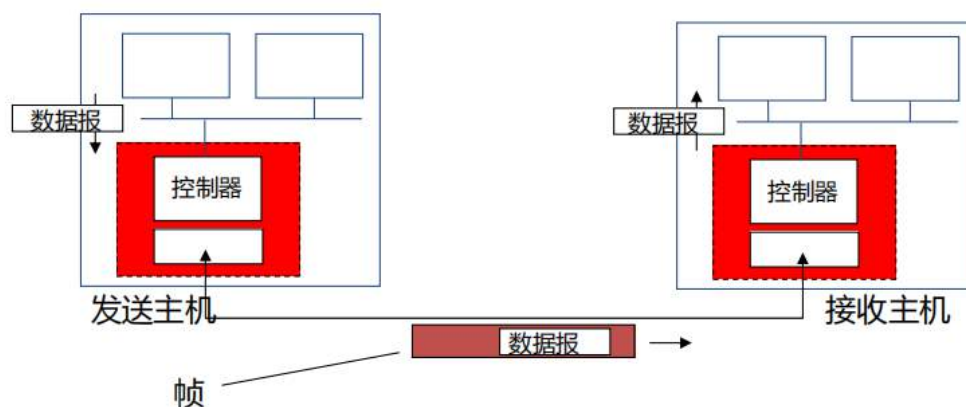
- **差错检测：**
 - 信号的衰减、噪声，都可能导致错误
 - 接收方检测到差错时，可能告知发送方重传或直接丢弃该帧
- **差错纠正：**
 - 接收方识别并纠正比特错误，而无需麻烦发送方重传
- **流量控制：**
 - 保持发送节点和接收节点的步调一致
- **半双工和全双工：**
 - 半双工指链路两端的节点，虽然均可传输数据，但是不能同时

实现方式

- 各个主机中都有链路层
- 通过适配器（又称网络接口卡NIC）或芯片实现
 - 802.3以太网卡、802.11无线网卡；以太网芯片组（一般集成至主板）
 - 实现了链路层和物理层功能
- 附在主机的系统总线上
- 软件、硬件、固件的集合



网卡本质上是实现链路层和物理层功能



□ 发送

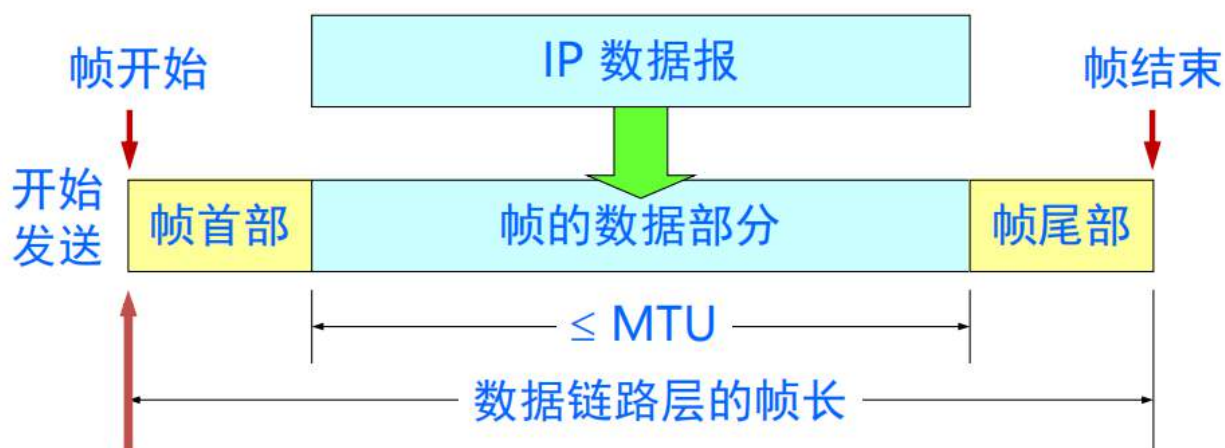
- 封装数据报为帧
- 添加差错检测、可靠数传、流量控制等功能

□ 接收

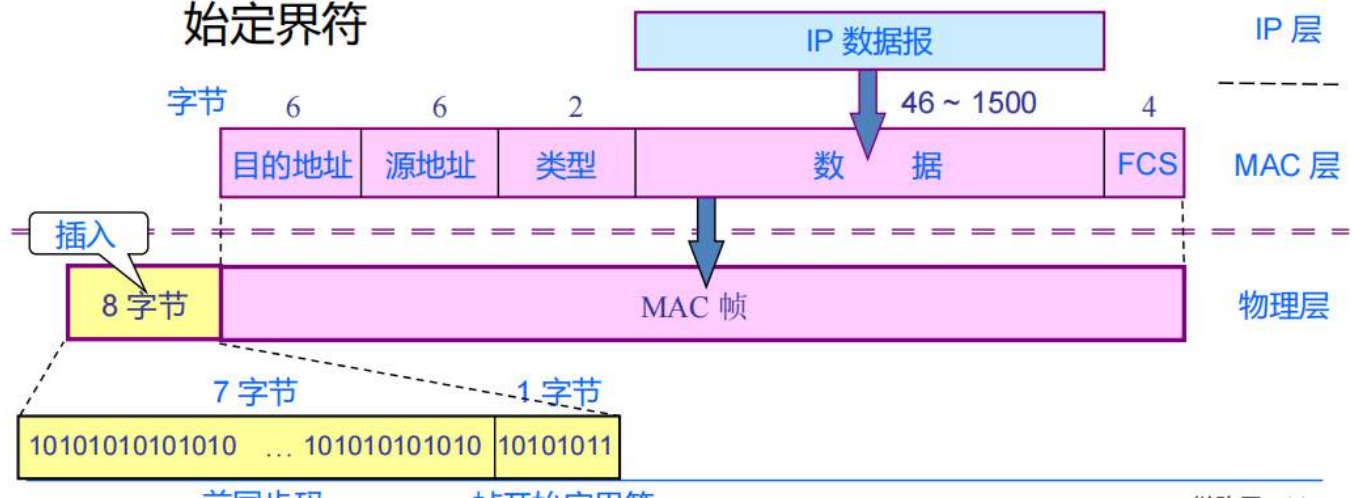
- 检测错误、可靠数传、流量控制等操作
- 解析数据报，交付给高层

封装成帧

- 封装成帧(framing)就是在一段数据的前后分别添加首部和尾部，以确定帧的界限，从而构成一个帧进行传输
- 其中，首部和尾部的一个重要作用就是进行**帧定界**



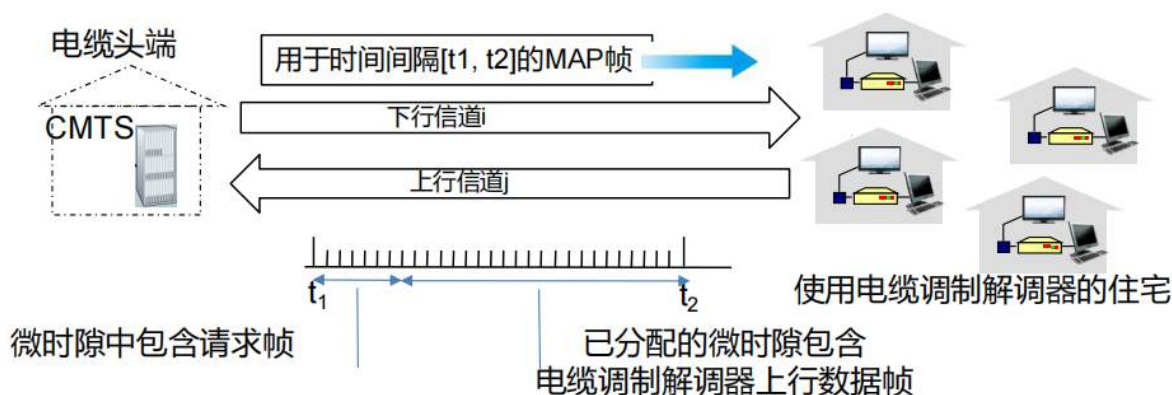
- 为了比特同步，实际传送的数据比 MAC 帧多 8 个字节
 - 第一个字段的 7 个字节：迅速实现 MAC 帧比特同步的前同步码
 - 第二个字段的 1 个字节：指示随后是 MAC 帧的帧开始定界符



透明传输

todo:需补充

MA表示多接入，以下的例子解释了实现原理



DOCSIS: 有线电视数据服务接口规范

- 在上行和下行频道上，采用频分复用（FDM）
- 上行采用时分复用（TDM）：部分时隙已分配，部分时隙存在争用
 - 下行MAP帧：分配上行时隙
 - 上行的时隙请求（以及数据）在选定的时隙中，以随机接入（二进制退避）方式传输

局域网LAN

Mac地址是设备的唯一地址，IP地址是该设备在不同网络下的地址，并不唯一

MAC地址 (Media Access Control Address): 物理地址，由网卡（NIC）在出厂时固化在硬件中。它是全球唯一的，用于标识局域网（LAN）中的特定设备。

IP地址 (Internet Protocol Address): 逻辑地址，是软件层面的标识。它反映了设备在当前网络拓扑结构中的位置，会随着你连接到不同的网络（如从家里换到咖啡馆）而改变。

特性	MAC 地址	IP 地址
全称	媒体访问控制地址 (物理地址)	互联网协议地址 (逻辑地址)
长度	48位 (6字节)，通常用十六进制表示	IPv4为32位；IPv6为128位
唯一性	全球硬件唯一，通常不可更改	在同一子网内唯一，随网络环境动态变化
OSI模型层级	数据链路层 (第2层)	网络层 (第3层)
主要功能	负责局域网内相邻节点间的数据传输	负责跨越不同网络进行端到端的寻址
格式示例	00-1A-2B-3C-4D-5E	192.168.1.1 或 2001:0db8:....

MAC地址和ARP

□ 32比特IP地址：

- 接口的网络层地址
- 用于第3层（网络层）转发

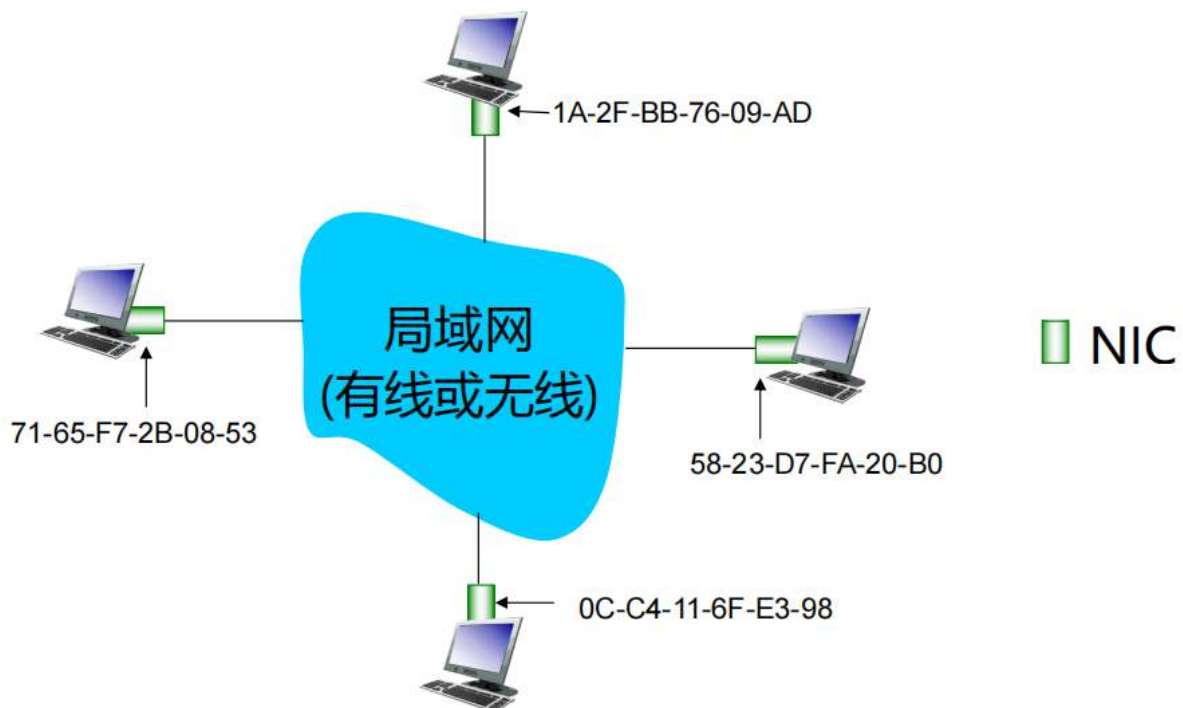
□ MAC（/局域网/物理层/以太网）地址：

- 功能：在“本地”从物理相连的一个接口向另一接口传输帧（在IP寻址意义上的同一网络）
- 48比特MAC地址（对于大多数局域网）烧录在NIC的ROM中，有时也可以通过软件进行设置
- 例如：1A-2F-BB-76-09-AD

十六进制表示法
(每个“数字”代表4比特)

MAC地址

局域网每个NIC都有唯一的局域网地址



NIC为网卡

□ MAC地址分配

- 由IEEE管理
- 制造商购买MAC地址空间的一部分（以24比特为单位）

□ IP与MAC

- IP地址，逻辑地址，层次化，不可携带
——地址取决于节点所连接的IP子网
- MAC地址，物理地址，扁平化，可以携带
——可以将LAN卡从一个LAN移动到另一个

ARP

ARP：地址解析协议

软件学院·计算机网络

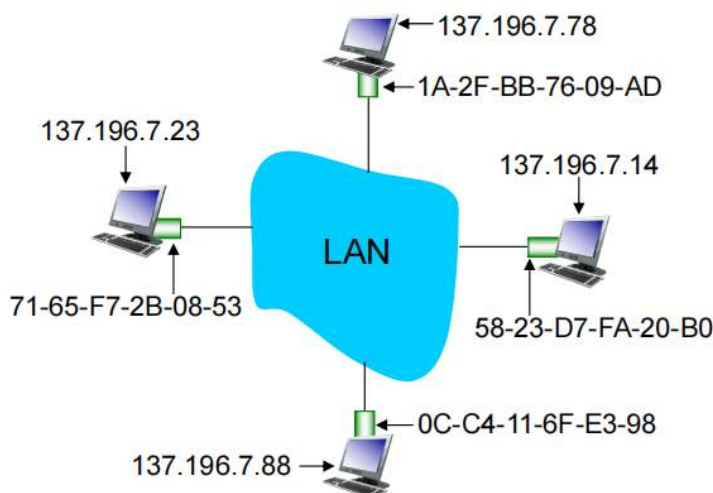
问题：已知接口的IP地址，如何确定其MAC地址？

□ **ARP表：**局域网每个IP节点（主机、路由器）都有这张表

- 表中包含一些局域网节点的IP/MAC地址映射：

<IP地址；MAC地址；TTL>

- TTL（生存时间）：地址映射将被遗忘的时间（典型值20分钟）



相同局域网下

- A想要向B发送数据报
 - B的MAC地址不在A的ARP表中
- A广播ARP查询分组，其中包含了B的IP地址
 - 目的MAC地址 = FF-FF-FF-FF-FF-FF
 - 局域网的所有节点都收到ARP查询分组
- B收到ARP分组，使用其（B的）MAC地址回复A
 - 数据帧单播到A的MAC地址
- A在ARP表中缓存IP-MAC地址映射，直到信息过时（超时）
 - 软状态：指示那些除非刷新，否则会因超时而消失的信息
- ARP是“即插即用”的：
 - 节点无需网络管理员的干预，可以自行创建ARP表

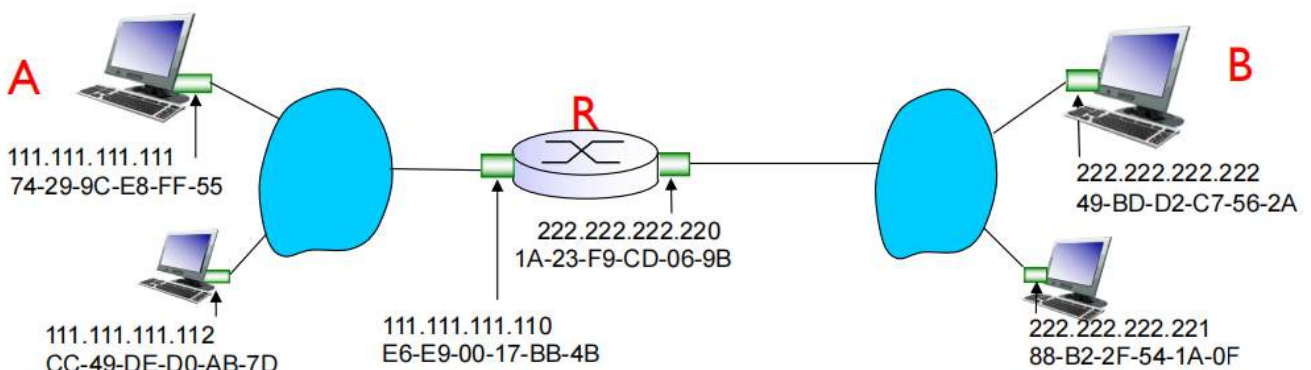
此时就会在A的ARP表中添加B的ip-mac-ttl的arp表项

不同局域网下

这时候需要寻址

示例：从A通过R向B发送数据报

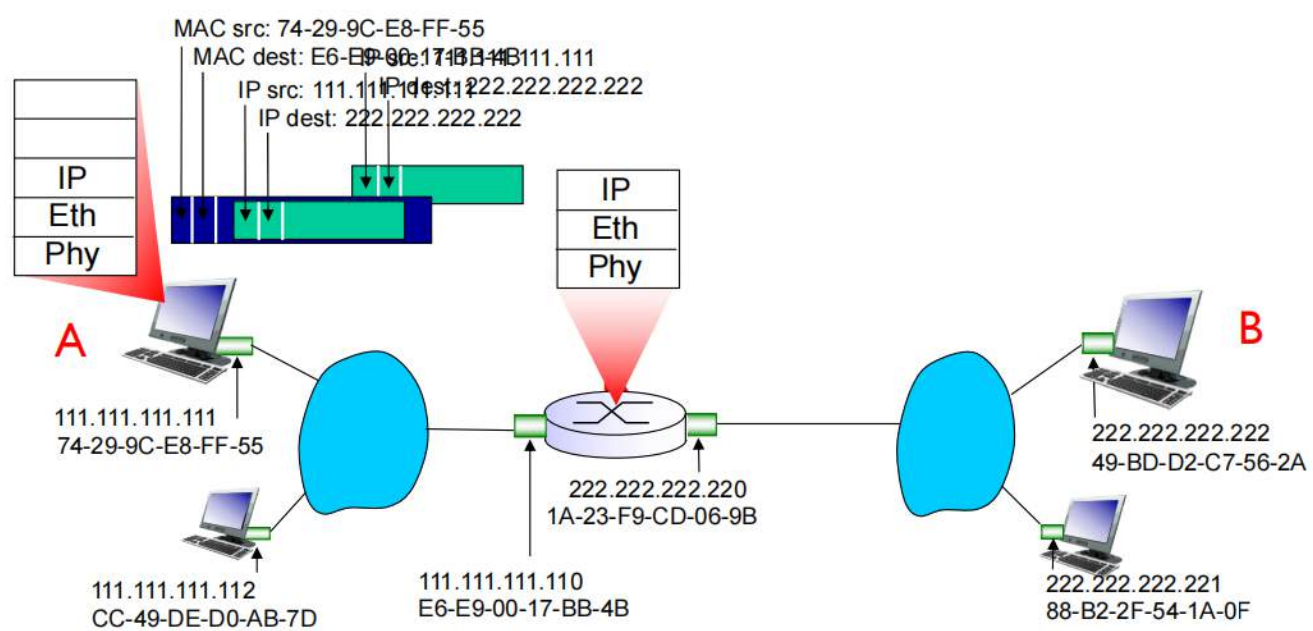
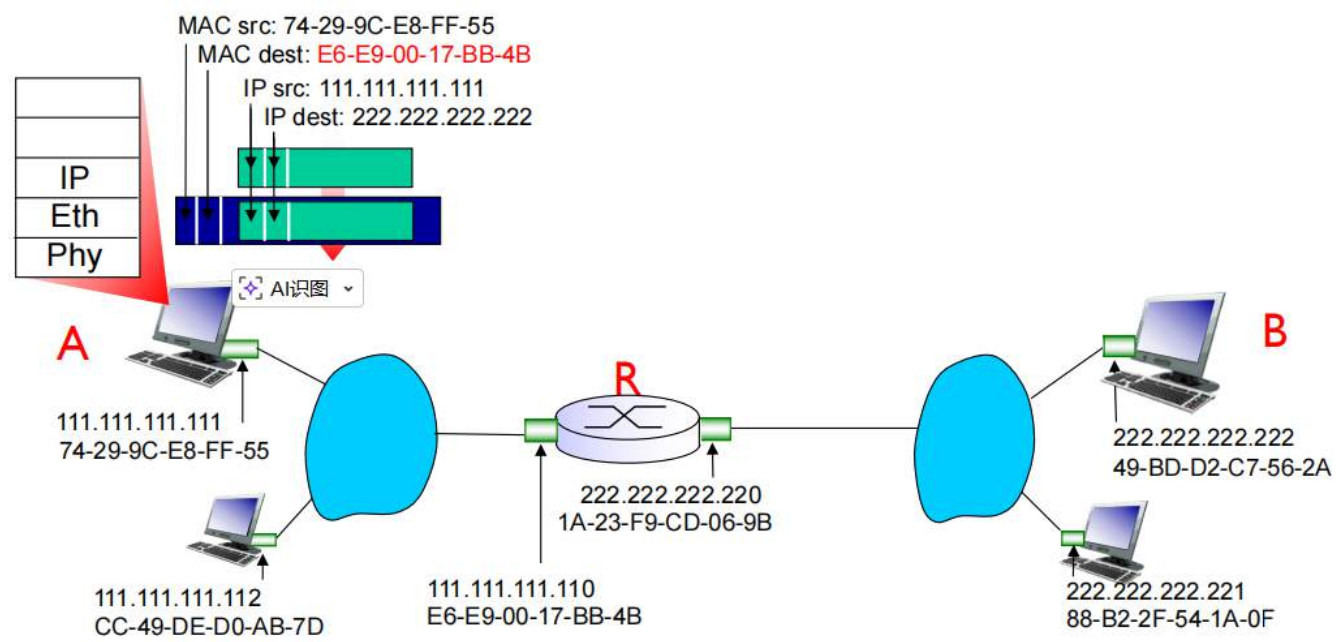
- 聚焦于寻址 – 看 IP 层数据报和 MAC 层帧
- 假设A知晓B的IP地址
- 假设A知晓第一跳路由器R的IP地址(怎么知道的？)
- 假设A知晓R的MAC地址（怎么知道的？）

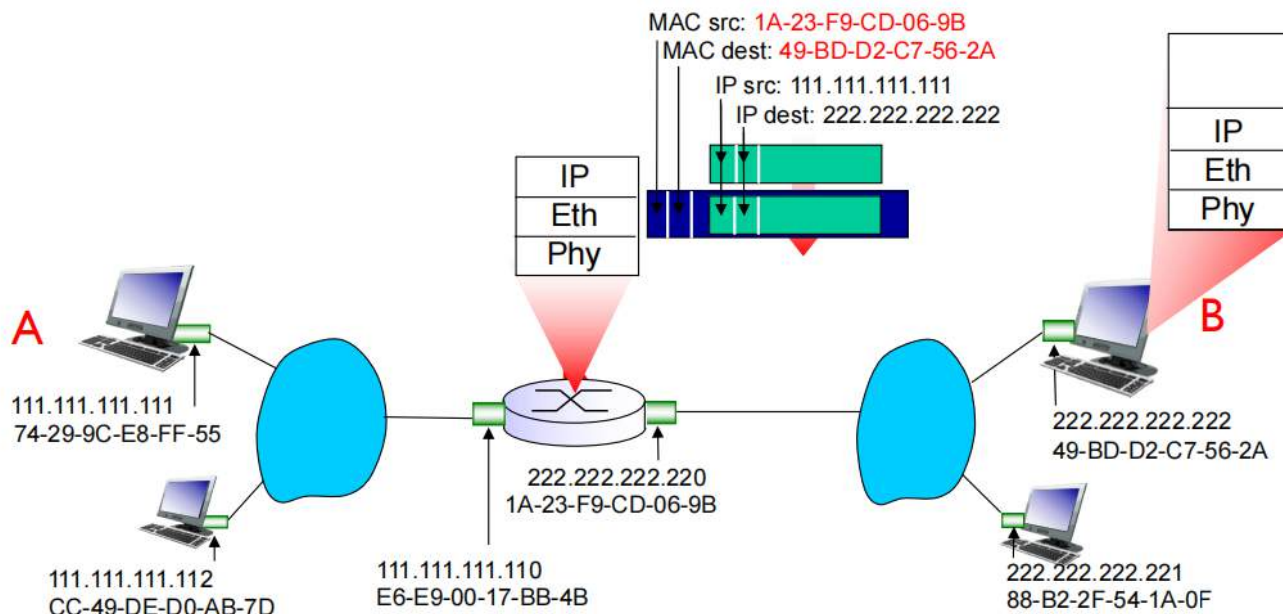


IP地址可以通过DNS解析获得

R的IP地址是网络层中的下一跳地址，我们也知道

R的MAC地址通过ARP协议获取





以太网协议

1. 以太网 (Ethernet): 规则与框架

以太网是目前应用最广泛的局域网 (LAN) 技术标准。它定义了数据如何在物理介质 (网线、光纤) 上进行传输, 位于 OSI 模型的**物理层**和**数据链路层**。

- **它的职责:** 规定了电信号频率、线缆类型、连接器形状, 以及如何将数据包装成“帧”(Frame) 进行发送。
- **传输方式:** 早期的以太网使用广播方式, 所有设备都能收到信号, 但只有目标设备会处理它。为了识别“谁是目标”, 它必须依赖 MAC 地址。

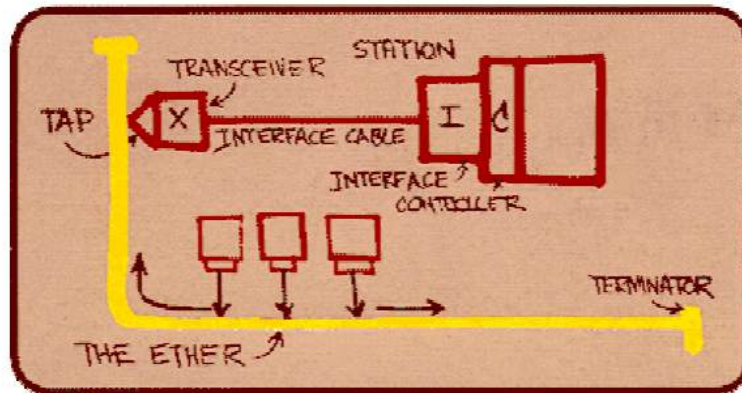
2. MAC 地址: 以太网中的“身份证”

MAC 地址全称是 **媒体访问控制地址**。在以太网协议中, 它被嵌在每个数据帧的头部。

- **寻址功能:** 在以太网这条“公路”上, MAC 地址就是车牌号。当网卡收到一个以太网帧时, 它首先检查帧头部的“目的 MAC 地址”。
 - 如果 MAC 地址与自己匹配, 或者是一个广播地址, 网卡就会接收数据并传给 CPU。
 - 如果不匹配, 网卡会在硬件层面直接丢弃该数据包, 不占用系统资源。

“统治” LAN的技术:

- 便宜
- 最早被广泛应用的 LAN 技术
- 较为简单, 比 token LANs 和ATM便宜
- 赶上了速率竞赛的步伐: 10, 100, 1000 Mbps



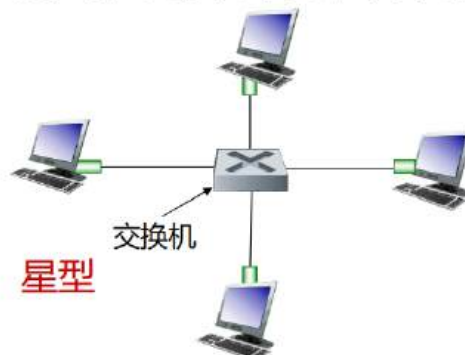
梅特卡夫的以太网草图

物理拓扑

- **总线型**: 90年代中期流行
 - 所有的结点在同一个冲突域中
- **星型**: 目前的主流
 - 交换机居中
 - 每一“辐”运行一个（单独的）以太网协议（节点之间不会相互冲突）



总线: 同轴电缆



冲突域: 共同竞争相同资源的所有用户的集合

广播域: 某一用户广播, 能接收到其广播的所有用户的集合

帧结构

- 发送适配器将IP分组封装在**以太网帧**中(或其他网络层协议分组)



preamble-前序:

- 7 个 10101010 字节尾随一个 10101011 字节
- 用来同步收发双方的时钟速率
- **address-地址:** 6 个字节, 帧为某个LAN上的所有适配器接收, 但只要地址不匹配就被丢弃
 - 如果适配器接收到具有匹配目标地址或广播地址 (例如 ARP) 的帧, 它将帧中的数据传递到网络层协议
 - 否则, 适配器将丢弃帧
- **type-类型:** 说明其上层协议 (大部分为 IP, 但其他协议如 Novell IPX和 AppleTalk也支持)
- **CRC:** 在接收端校验,如果出错, 则将该帧丢弃

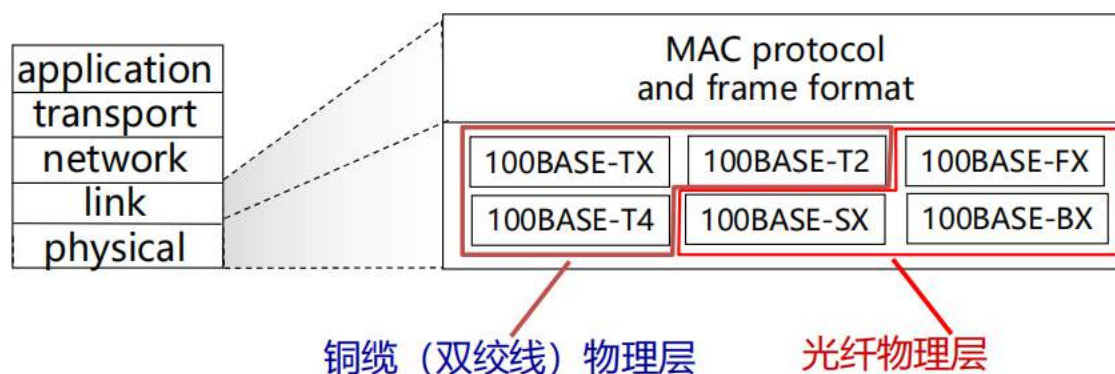


这里的结构要求和前面的MAC帧一致

以太网的标准是**IEEE 802.3**, 是**无连接、不可靠**的协议

□ 许多不同以太网标准

- 公用的MAC协议（介质访问控制）和帧格式
- 不同的传输速度：2Mbps，10Mbps，100Mbps，1Gbps，10Gbps
- 不同的物理层介质：光纤，同轴电缆，双绞线



交换机

1. 消除冲突，提升带宽利用率

为什么使用交换机，而不是全部串在一起？ 1. 消除冲突，提升带宽利用率

- **隔离冲突域**：交换机为每个接口提供独立的带宽。这意味着连接在不同端口上的设备可以同时发送数据，而不会像集线器那样发生“碰撞”。
- **全双工通信**：交换机支持双向同时传输数据（全双工），而集线器只能半双工。这直接使链路的有效吞吐量翻倍。

2. 实现数据的精准转发（按需转发）

- **避免盲目广播**：正如你提供的资料所描述，交换机会记录发送主机的 MAC 地址并构建地址表。
- **点对点传输**：当交换机知道目的 MAC 地址所在的接口时，它只会将帧转发到那个特定的端口，而不是发给所有人。这减少了网络中不必要的流量，保护了其他设备的带宽。

3. 网络安全与隐私保护

- **防止嗅探**：在集线器环境下，任何人都可以通过抓包工具看到局域网内的所有数据帧。
- **单播特性**：交换机的转发机制确保了数据帧只发送到目标设备，局域网内的其他主机无法轻易“偷听”非发送给自己的数据，提高了内网安全性。

4. 灵活的网络扩展与管理

- **自学习功能**：交换机即插即用，它通过接收到的流量自动学习网络拓扑，无需人工手动配置每个 MAC 地址对应的端口。

- **支持多种速率**：现代交换机通常符合 **802.3** 系列标准，可以同时支持 10Mbps、100Mbps 甚至 1000Mbps 的设备混合接入，并自动协商最佳速度。

5. 故障隔离

- **链路独立**：如果某个端口的网线短路或某个设备网卡损坏导致不断发送错误包，交换机可以将该端口的影响限制在最小范围，而不会导致整个局域网瘫痪。

□ 链路层设备

□ 存储、转发以太网帧

□ 过滤：检查进入设备的帧的MAC地址，选择性将帧转发至1个或多个输出链路，采用CSMA/CD策略

□ 透明性：主机和路由器感觉不到交换机的存在

□ 即插即用，自学习：交换机不需要配置

一般的交换机为二层设备（链路层，物理层），路由器为三层设备（网络层，链路层，物理层），但也有特殊的三层交换机

多点同时传输

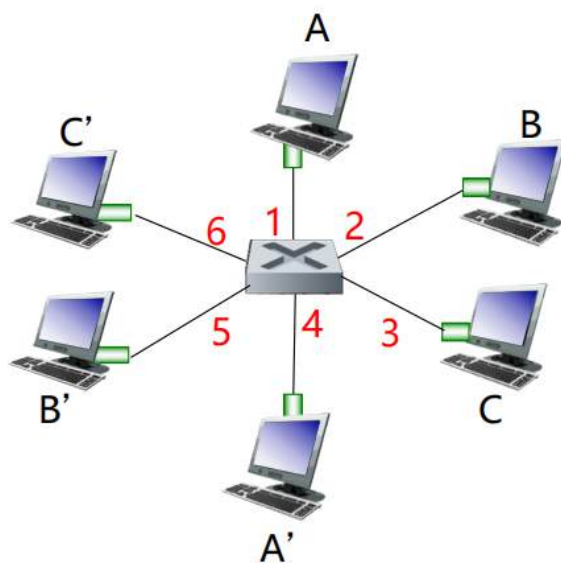
□ 主机直连至交换机

□ 交换机有缓存能力

□ 采用以太网协议的各相连链路（可能是异质的），彼此隔离，不会产生冲突，全双工

➤ 每条链路是一个冲突域

□ 交换：A-to-A' and B-to-B同时传输，无冲突



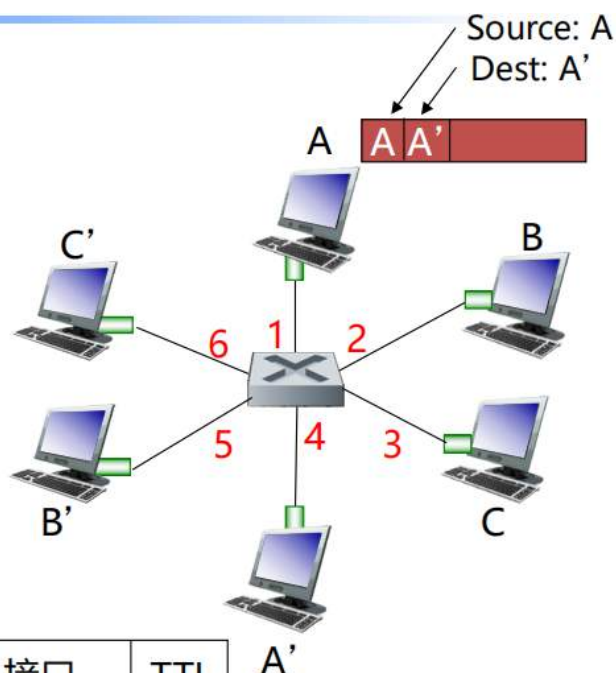
六个接口的交换机
(1,2,3,4,5,6)

每一个传输线路都独占两个口，因此不会产生冲突

比如说A-A'，这条链路独占交换机的1和4号口

□ 交换机了解可以通过哪些接口访问哪些主机

- 收到帧时，交换机 "学习" 发送方的位置：传入局域网段
- 在交换表中记录发送端/位置对



MAC地址	接口	TTL
A	1	60

转发表
(最初为空)

1. 学习阶段 (记录)

- **动作：**交换机接收到帧时，会记录该帧进入的端口（传入链路）以及发送该帧主机的 **源 MAC 地址**。
- **目的：**交换机通过这种方式构建“交换机表”（也叫 MAC 地址表），实现“自学习”，从而知道哪个 MAC 地址连接在哪个端口上。

2. 查表阶段 (索引)

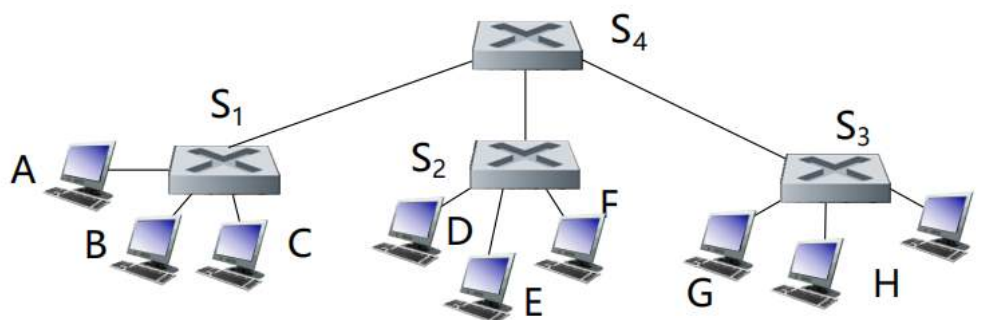
- **动作：**交换机提取帧中的 **目的 MAC 地址**，并在其维护的交换机表中进行搜索（索引）。

3. 决策阶段 (转发或过滤)

根据查表结果，交换机会采取不同的行动：

- **情况 A：表中找到了对应条目（单播转发/过滤）**
 - **过滤（丢弃帧）：**如果目的地址显示的接口就是数据帧“进来”的那个接口，说明目标和源在同一个网段，不需要通过交换机中转，直接丢弃以节省带宽。
 - **转发：**如果目的地址在另一个接口，交换机会精准地将帧转发到该条目所示的接口。
- **情况 B：表中没找到对应条目（泛洪 - Flood）**
 - **动作：**如果表里查不到这个目的地址，交换机会将该帧复制并发送给**除接收接口外的所有接口**。
 - **后续：**只有真正的目标主机接收该帧并回应，此时交换机又会通过步骤 1 学习到该目标的地址。

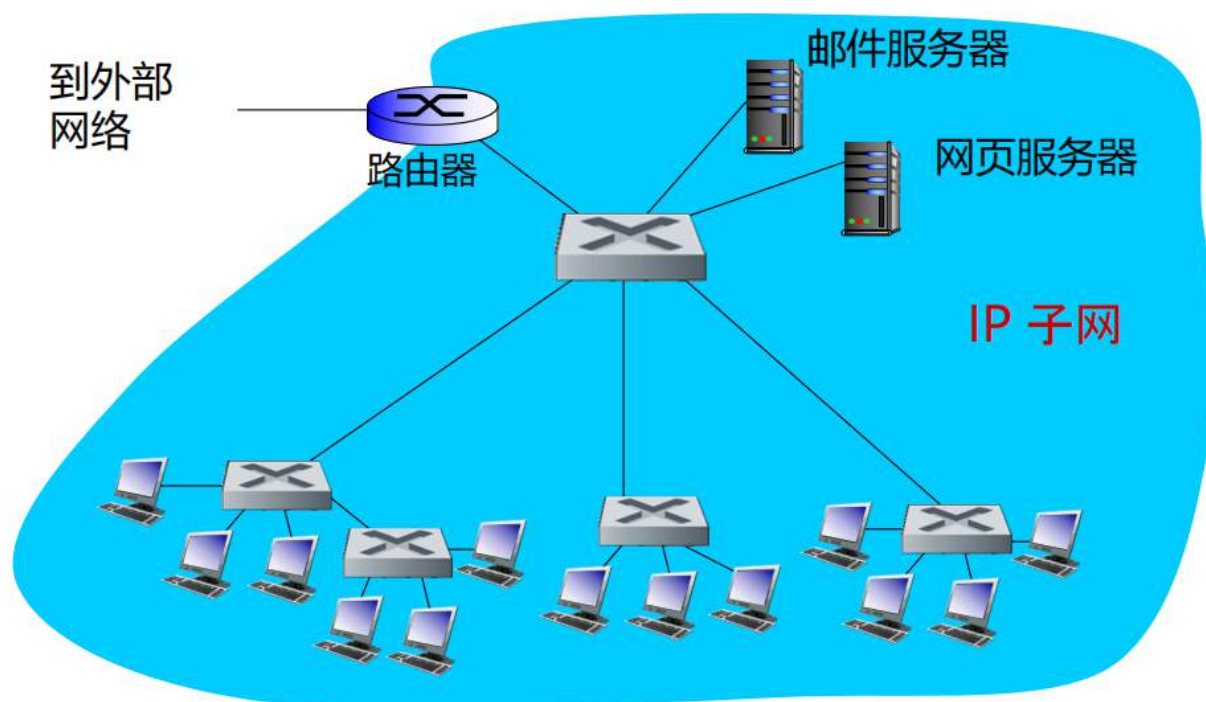
□ 交换机之间也能彼此互联



一旦泛洪，就会通知所有交换机及其主机

机构网络

软件学院·计算机网络

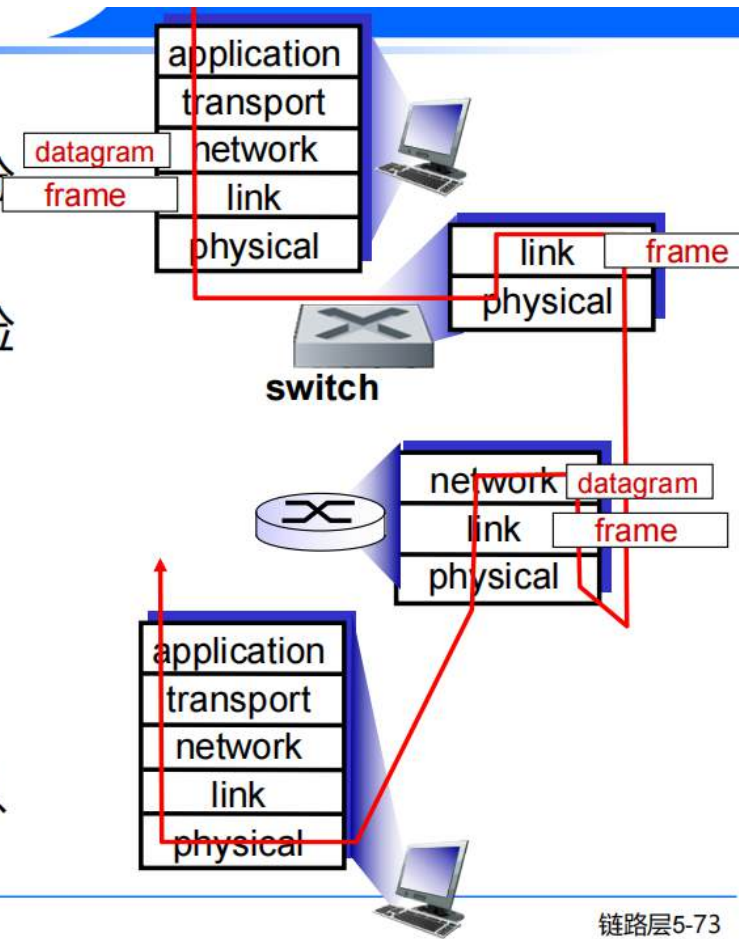


都是存储转发:

- ❑ 路由器：网络层设备（检查网络层头部）
- ❑ 交换机：链路层设备（检查链路层头部）

都有转发表:

- ❑ 路由器：使用路由算法、IP 地址计算转发表
- ❑ 交换机：使用泛洪、学习、MAC 地址学习转发表

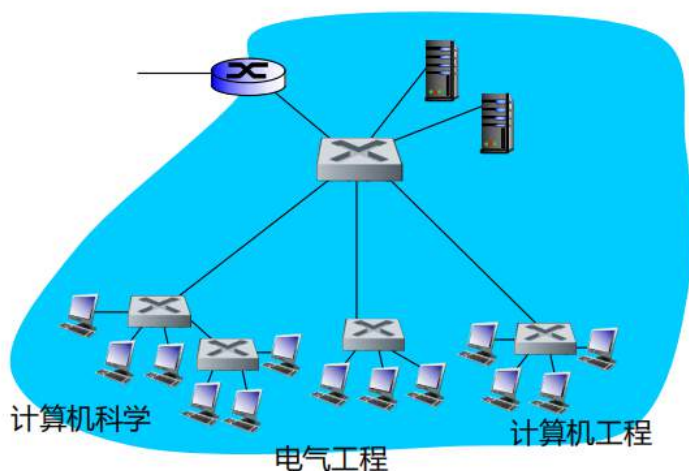


链路层5-73

	集线器	交换机	路由器
层次	1	2	3
隔离冲突域	否	是	是
隔离广播域	否	否	是
即插即用	是	是	否
路由优化	否	否	是

通常交换机相比路由器更靠近主机

虚拟局域网



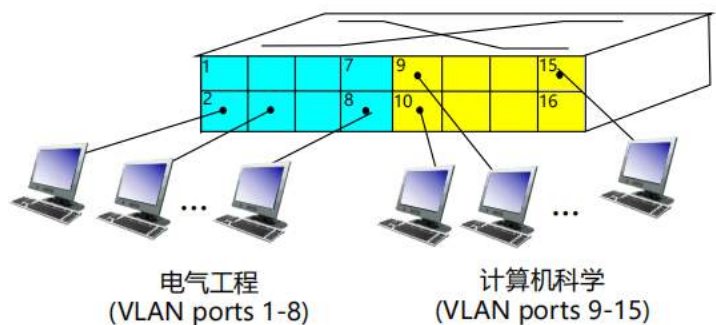
思考:

- 计算机科学用户将办公室迁移到 电气工程，但想要连接到 计算机科学交换机？
- 单个广播域：
 - 所有第 2 层广播流量（ARP、DHCP、目标 MAC 地址的未知位置）必须跨越整个 LAN
 - 安全/隐私、效率问题

虚拟本地局域网

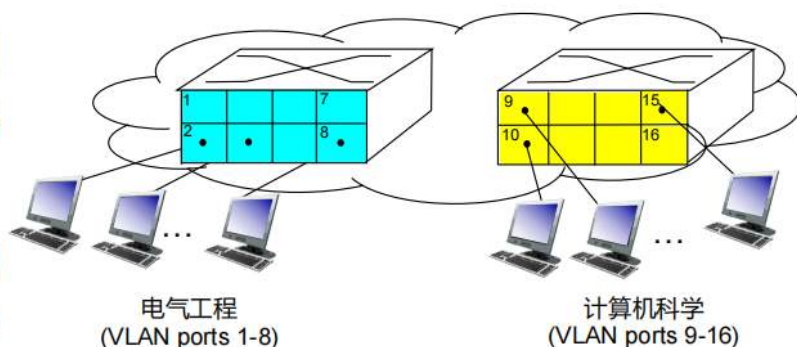
支持 VLAN 功能的交换机可配置为在单个物理 LAN 基础设施上定义多个虚拟 LANS。

基于端口的VLAN：交换机端口分组（通过交换机管理软件），使单个物理交换机作为多个虚拟交换机运行。

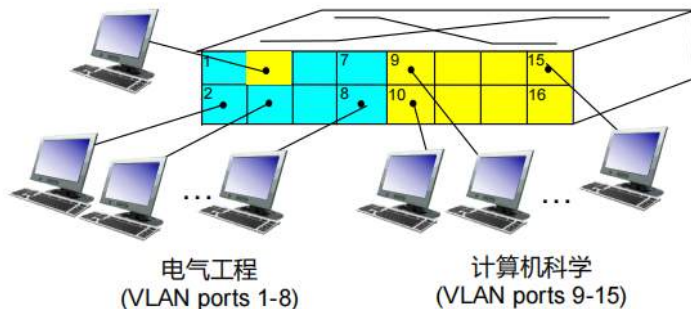


- ▣ **流量隔离**: 进出端口 1-8 的帧只能到达端口 1-8

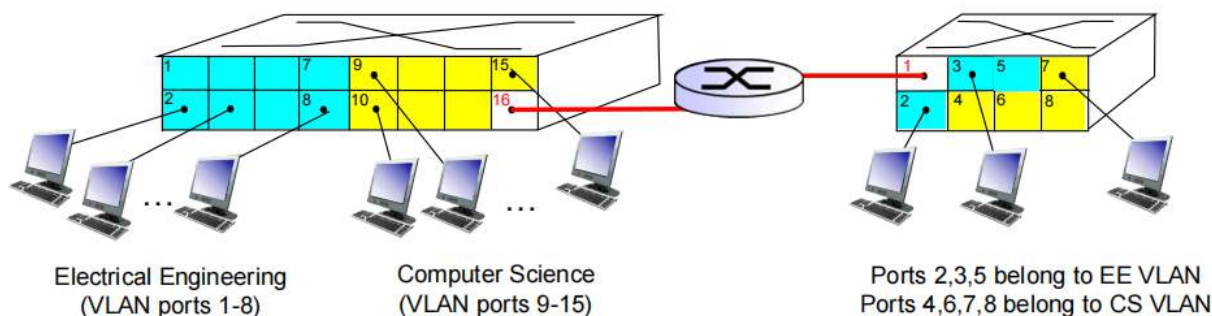
- 还可以根据端点的 MAC 地址而不是交换机端口来定义 VLAN



- ▣ **动态成员**: 可在 VLAN 之间动态分配端口

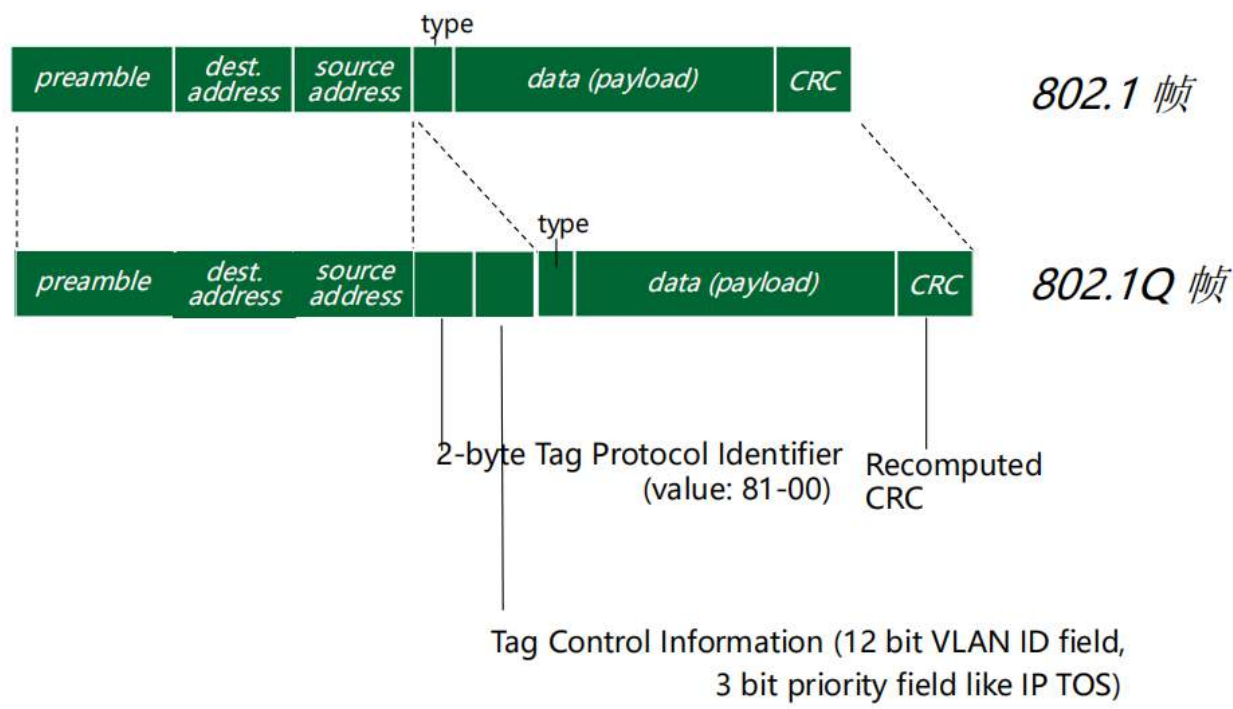


多个交换机组合后，也可以产生跨交换机的VLAN



- ▣ **汇聚端口**: 在多个物理交换机上定义的 VLANS 之间传输帧
 - 在VLAN内，交换机间转发帧时，因为需要携带VLAN ID信息，所以普通的802.1帧不再适用
 - 协议802.1q为汇聚端口间转发的帧，添加/删除额外的首部字段
- ▣ **VLAN之间**: 通过路由完成（就像独立交换机）
 - 实际上，供应商销售的是交换机和路由器的组合产品

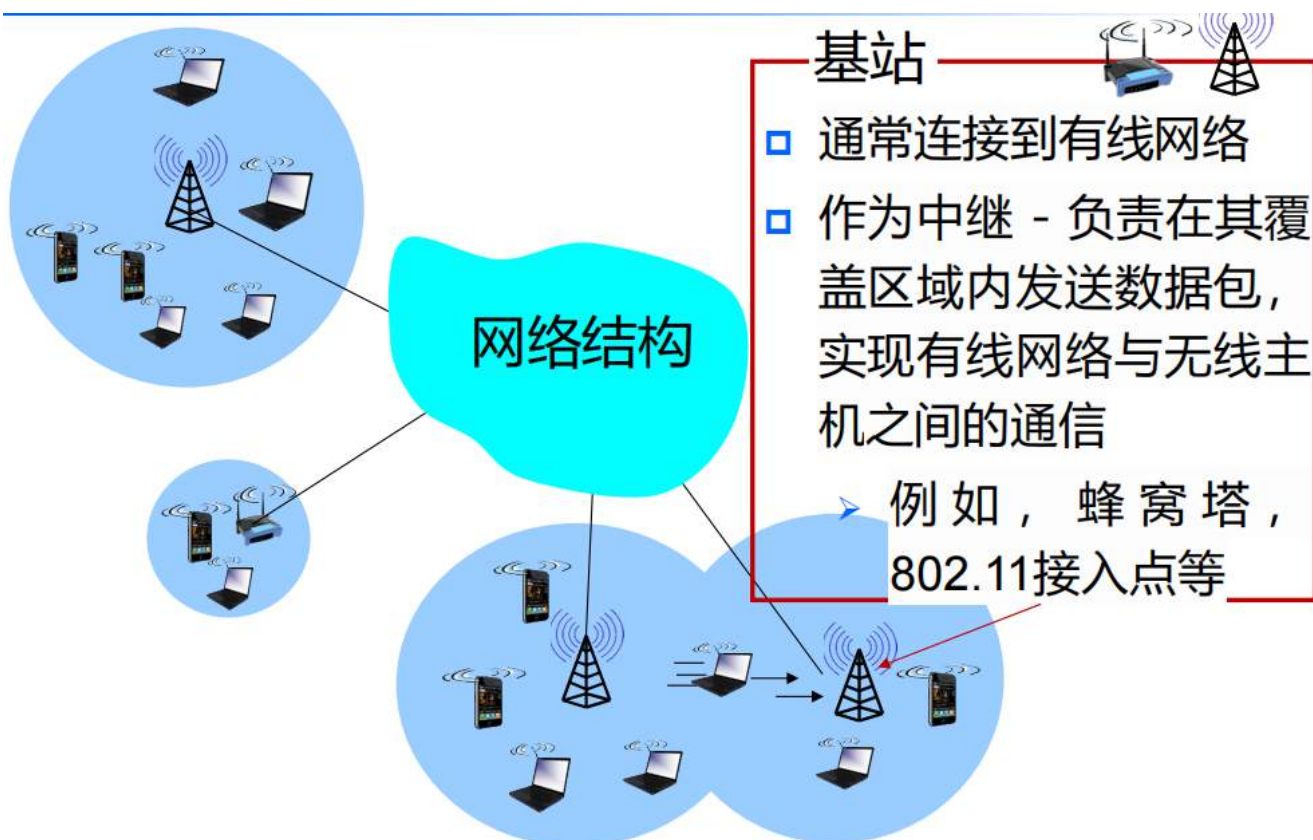
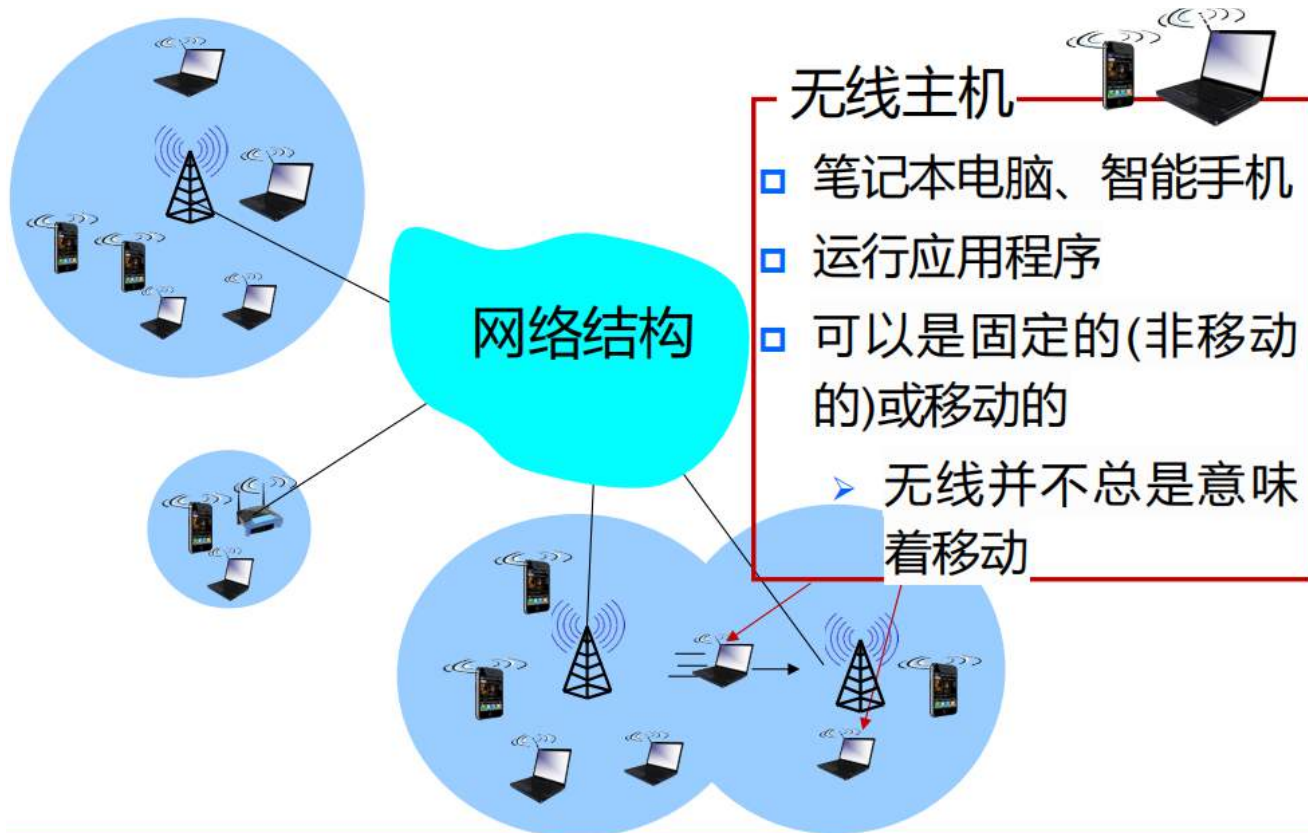
帧格式



无线网络

概述

组成部分



通常我们的家庭中的边缘路由器不称作基站



无线链路

- 通常用于连接移动设备和基站
- 也用于骨干链路
- 采用多路访问协议来协调链路访问
- 具有不同的数据速率和传输距离



两种模式

基础设置模式

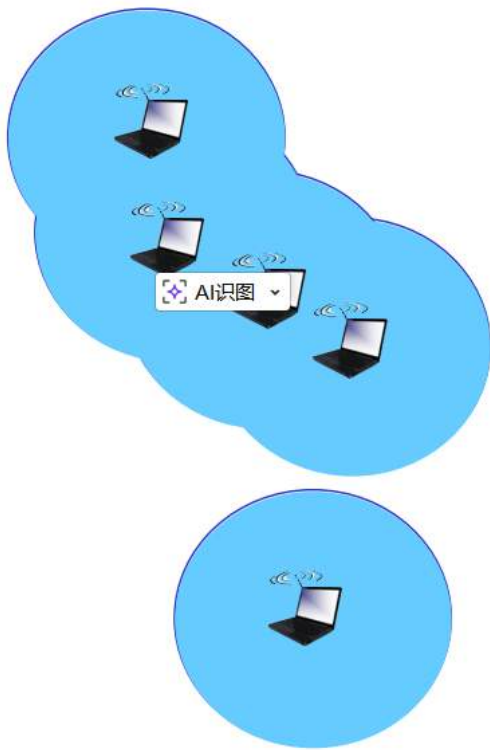
AI识别

基础设施模式

- 基站将移动设备连接到有线网络中
- 切换:移动设备更换为其提供有线网络连接的基站



自组织网络模式Ad Hoc



自组织网络模式(Ad Hoc Mode)

- 无需基站
- 节点只能在链路覆盖范围内向其他节点发送信息
- 节点自行组织成网络：在彼此之间进行路由选择

去中心化，每个节点在自己范围内向其他节点发送信息

特征

与有线链路相比的**重要**差异.....

- **信号强度衰落**：无线电信号在物质中传播时会衰减(路径损耗)。
 - 隐蔽站问题
- **来自其他源的干扰**：标准化的无线网络频率（例如，2.4 GHz）被其他设备（例如，电话）共享；设备（如电机）也会产生干扰。
 - 暴露站问题
- **多径传播**：无线电信号会在地面和其他物体上反射，以略微不同的时间到达目的地。

这些问题使跨无线链路（即使是点对点）的通信更加“困难”

□ 信噪比 (SINR)：信号与干扰、噪声的比值

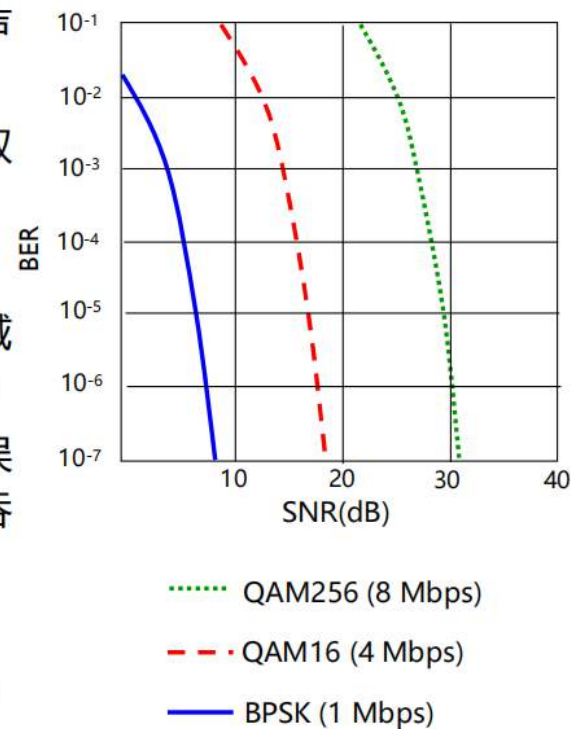
□ 信噪比越大，从干扰和噪声中提取信号就越容易（这是一件好事）

□ 信噪比与误码率 (BER) 的权衡

□ 在给定物理层的情况下：增加功率/减少干扰 -> 提高信噪比 -> 降低误码率

□ 在给定信噪比的情况下：选择满足误码率要求的物理层，以提供最高的吞吐量

- 信噪比可能随着移动性而改变：动态调整物理层(调制技术,速率)



码分多址(CDMA)

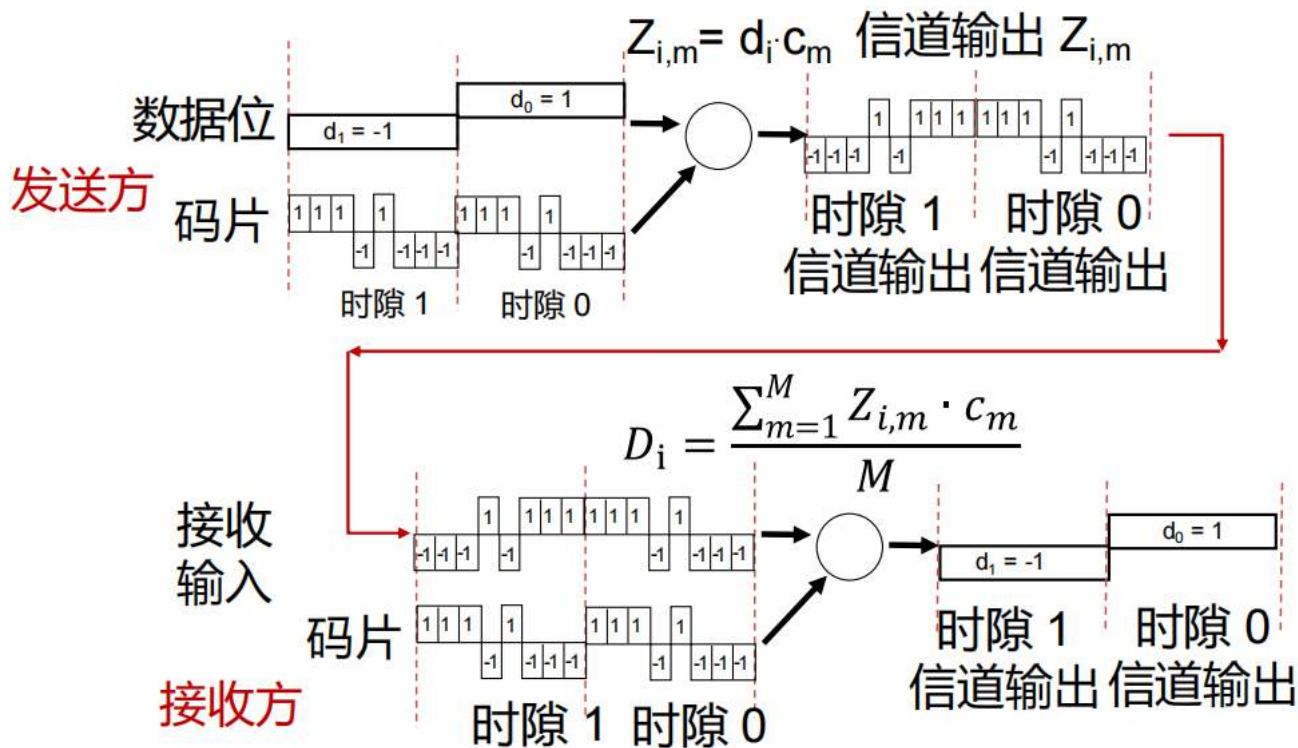
□ 每个用户都被分配了一个独特的“码片”，即码片集划分。

➢ 所有用户共享相同的频率，但每个用户都有自己的“切片”序列（即码片）来编码数据。

➢ 这允许多个用户“共存”并同时传输，且干扰最小（如果码是“正交”的）。

□ 编码信号 = (原始数据) × (切片序列)

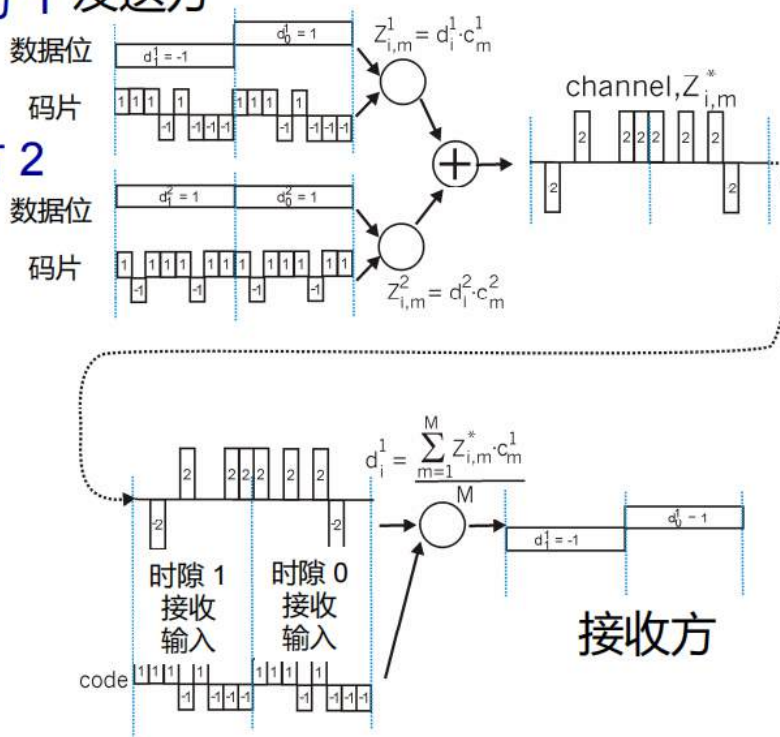
□ 解码：编码信号与切片序列的内积。



双发送方干扰

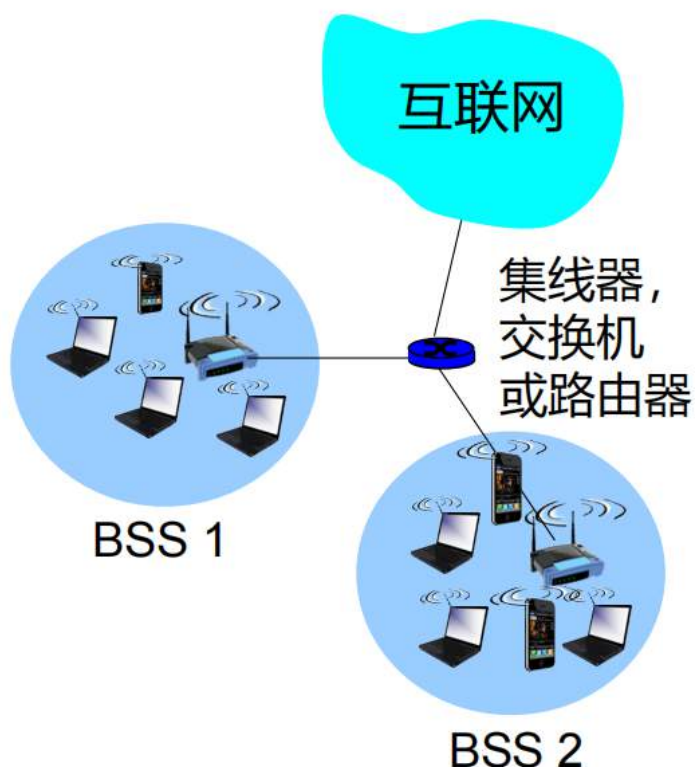
发送方 1 发送方

发送方 2



信道将发送方 1 和发送方 2 的传输信号进行叠加

通过使用与发送方 1 相同的编码, 接收器能够从叠加后的信道数据中恢复出发送方 1 的原始数据

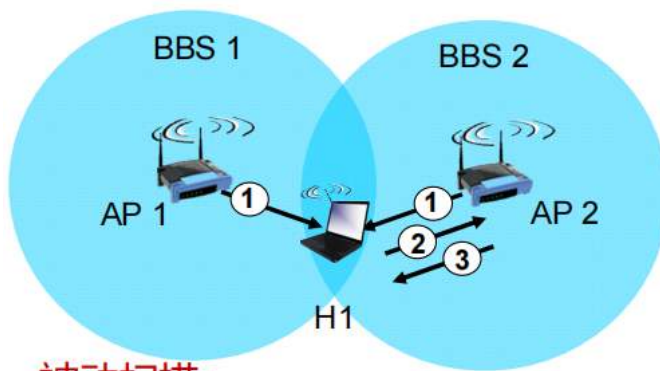


- 无线主机与基站通信
 - 基站 = 接入点 (AP)
- 在基础设施模式下, 基本服务集 (BSS) (也称为“单元”) 包含:
 - 无线主机
 - 接入点 (AP) : 基站
- 自组织模式: 仅限于主机

信道、关联

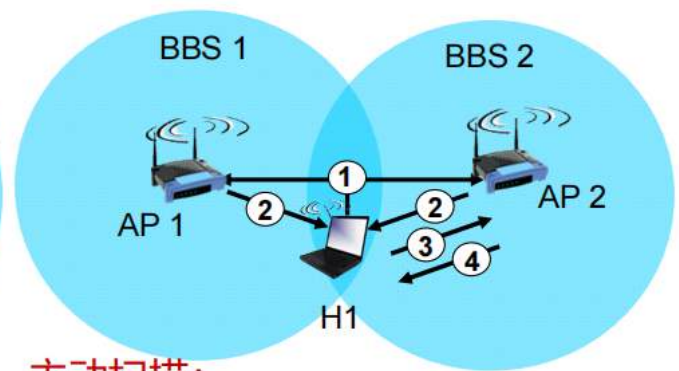
- 802.11b: 使用2.4GHz至2.485GHz频段, 并将其划分为11个不同频率的信道
 - 接入点 (AP) 管理员为接入点选择频率
 - 可能存在干扰: 可能与邻近接入点选择的信道相同!
- 主机: 必须与接入点**关联**
 - 扫描信道, 监听包含接入点名称 (SSID) 和MAC地址的信标帧
 - 选择要与其关联的接入点
 - 可能执行身份验证[第8章]
 - 通常会运行DHCP以在接入点的子网中获取IP地址

被动/主动扫描



被动扫描:

- (1)由接入点(APs)发送的信标帧
- (2)关联请求帧发送:H1向所选接入点发送
- (3)关联响应帧发送:由所选接入点向H1发送

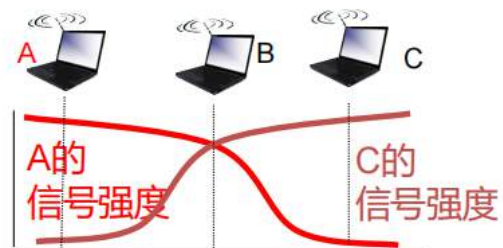
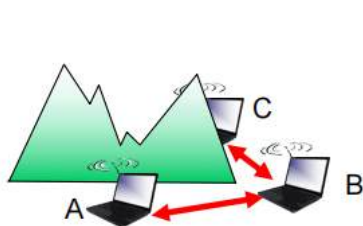


主动扫描:

- (1)H1广播探测请求帧
- (2)接入点(APs)发送探测响应帧
- (3)关联请求帧发送: H1向所选接入点发送
- (4)关联响应帧发送: 由所选接入点向H1发送

多路访问

- 避免碰撞: 两个或更多节点同时传输数据
- 802.11: CSMA - 传输前先检测
 - 避免与其他节点正在进行的传输发生碰撞
- 802.11: 没有碰撞检测!
 - 由于接收到的信号较弱(衰减), 在传输时难以接收(检测碰撞)
 - 无论如何都无法检测所有碰撞: 隐藏终端、衰减
 - 目标: **避免碰撞**: CSMA/CA (碰撞避免)



802.11 发送方

1. 如果检测到通道空闲时间达到DIFS

发送整个帧（无碰撞检测）

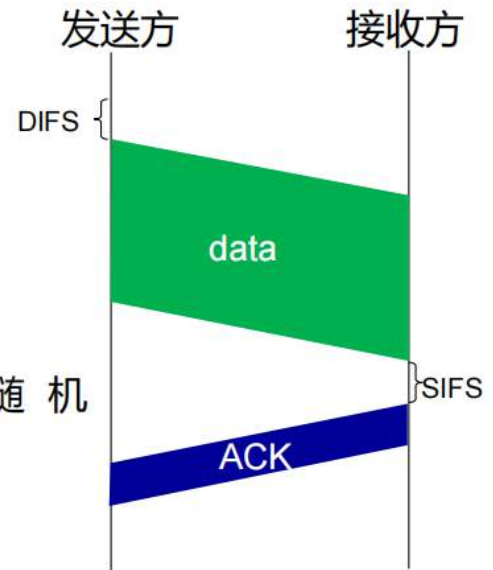
2. 如果检测到通道忙

开始随机退避时间

在通道空闲时,计时器倒计时

计时器到期时发送

如果没有收到ACK,则增加随机退避间隔,重复步骤2



802.11 接收方

— 如果帧接收正常

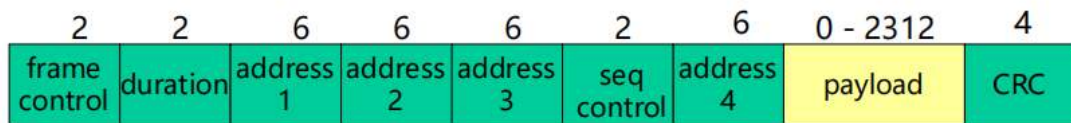
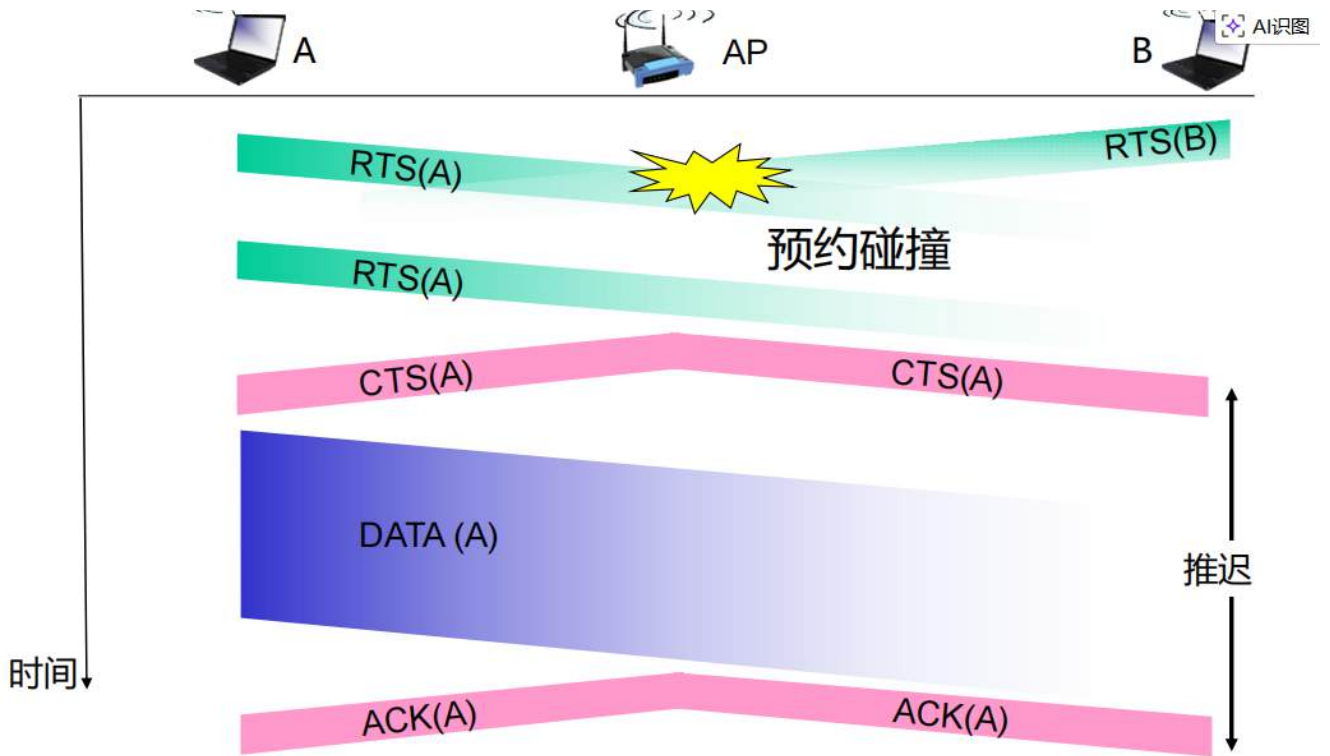
- 在SIFS后返回ACK(由于隐藏终端问题,需要ACK)

避免碰撞

设想：允许发送方“预留”通道，而不是随机访问数据帧：避免长数据帧发生碰撞

- 发送方首先使用CSMA向基站（BS）发送小的请求发送（RTS）数据包
 - RTS数据包之间仍然可能发生碰撞（但它们很短）
- 基站（BS）在接收到RTS后广播允许发送（CTS）
- 所有节点都能听到CTS
 - 发送方发送数据帧
 - 其他站点推迟传输

通过使用小型预约数据包来完全避免数据帧碰撞！



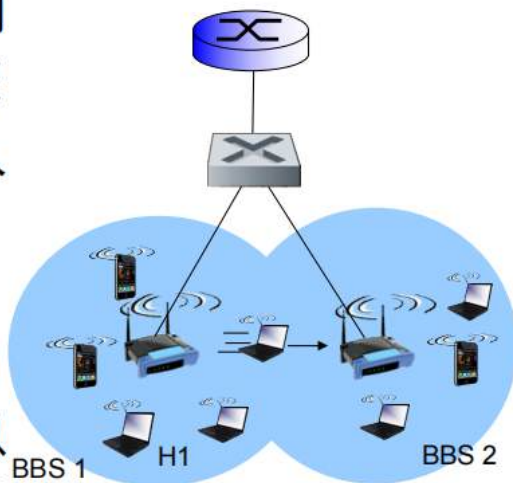
地址1:接收该帧的无线主机或AP的MAC地址

地址2:发送该帧的无线主机或AP的MAC地址

地址3: AP所连接的路由器接口的MAC地址

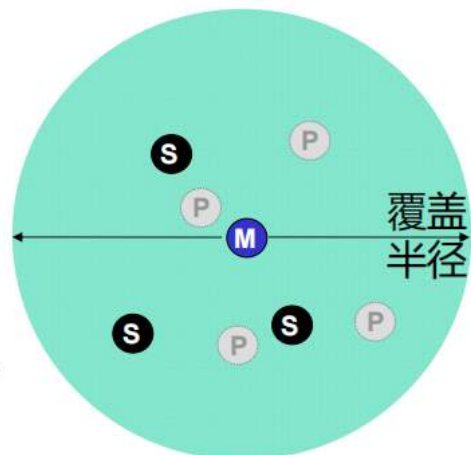
地址4: 仅用于自组织模式

- H1保持在相同的IP子网内：IP地址可以保持不变
- 交换机：H1与哪个接入点（AP）相关联？
 - 自学习（链路层）：交换机将看到来自H1的帧，并“记住”可以使用哪个交换机端口到达H1



802.11:个人局域网

- 直径小于10米
- 替代有线设备（鼠标、键盘、耳机）
- 自组织：无需基础设施
- 主/从设备：
 - 从设备请求发送权限（向主设备）
 - 主设备授予请求
- 802.15：从蓝牙规范发展而来
 - 2.4-2.5 GHz无线频段
 - 最高可达721 kbps

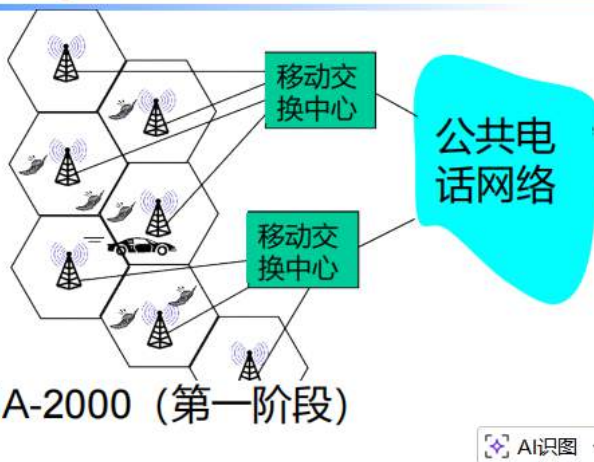


- M** 主设备
- S** 从设备
- P** 停放设备（非活动）

蜂窝网络的发展

软件学院·计算机网络

- 2G (语音)
 - GSM, IS-95
- 2.5G (语音+数据)
 - GPRS, EDGE, CDMA-2000 (第一阶段)
- 3G (语音+数据)
 - UMTS, TD-SCDMA, WCDMA, CDMA-2000
- 超4G (数据+语音)
 - LTE, LTE-A, 5G, B5G, 6G...



通过间接路由实现移动性

软件学院·计算机网络

